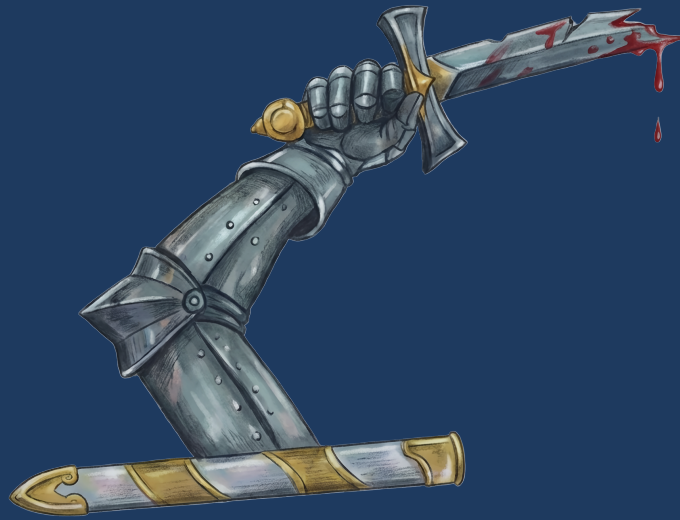




ToastScript

Language Specification & Complete Reference

Version 1.0 ■ The TōSh Shell

August 16, 2026

ToastScript Language Specification

© 2026 The TōSh Project. All rights reserved.

Typeset in L^AT_EX with Computer Modern and Source Code Pro.
Built on .NET 10 with the CLR runtime. Linux-first.

Contents

Preface	xv
I Language Core	1
1 Introduction to TōSh	2
1.1 What Is TōSh?	2
1.2 Quick Start	2
1.3 Design Philosophy	3
2 Syntax Fundamentals	4
2.1 Comments	4
2.1.1 When # Is a Comment	4
2.2 Identifiers	5
2.3 Keywords and Contextual Words	5
2.4 Strings	6
2.4.1 Escape Sequences	6
2.4.2 String Interpolation	7
2.5 Numeric Literals	7
2.5.1 Format and Alignment	7
2.6 Collection Literals	8
2.7 Special Literals	8
3 Type System	10
3.1 CLR Foundation	10
3.2 Built-in Type Aliases	10
3.3 Shell-Specific Types	13
3.3.1 StorageSize	13
3.3.2 TemporalAmount / TimeSpan	13
3.3.3 ToshRange	13
3.3.4 IShellRecordObject	13
3.3.5 IShellCallable	14
3.3.6 LazySequence	14
3.3.7 Vector	14
3.3.8 Matrix	14
3.3.9 Complex	15
3.3.10 Quantity (Unit System)	15
3.4 Type Conversion	18
3.5 Type Annotations	19
3.6 Truthiness	20
3.7 Refinement Types	21

4	Operators	24
4.1	Arithmetic Operators	24
4.2	Comparison Operators	25
4.2.1	Chained Comparison	25
4.3	Regex Operators	25
4.4	Logical Operators	26
4.5	Membership Operators	26
4.6	Type Operators	27
4.7	Null Coalescing	27
4.8	Ternary Operator	27
4.9	Assignment Operators	27
4.10	Operator Precedence	28
5	Variables & Scope	29
5.1	Variable Declarations	29
5.2	Constants	29
5.3	Destructuring	30
5.3.1	Record Destructuring	30
5.3.2	Array Destructuring	31
5.3.3	Tuple Destructuring	31
5.4	Spread Operator	31
5.4.1	Array Spread	31
5.4.2	Record Spread	31
5.4.3	Function Splatting	32
5.5	Computed Properties	32
5.6	Function References	32
5.7	Scoping	32
5.7.1	Visibility Modifiers	32
6	Control Flow	34
6.1	If / Else	34
6.2	For / In	34
6.3	While	35
6.4	Until	35
6.5	Switch / Case	35
6.6	Match Expressions	35
6.7	Type Narrowing	36
6.8	Try / Catch / Finally	37
6.9	Defer	37
6.10	Break, Continue, Return, Throw	38
6.11	Postfix Conditionals (if / unless)	39
7	Functions	41
7.1	Basic Functions	41
7.2	Arrow Functions (Wrappers)	41
7.3	Anonymous Functions (Lambdas)	41
7.4	Typed Parameters	42
7.5	Named Arguments and Callable Invocation	43
7.6	Return Types	43
7.7	Function Overloading	44
7.8	Recursion Depth	44
7.9	Operator Overloading	44

	7.9.1	Resolution order	45
	7.9.2	Limitations	45
7.10		Generator Functions	45
7.11		Asynchrony	46
7.12		Event Handlers	48
7.13		Doc-Comments	48
7.14		Visibility Modifiers	50
7.15		Functional Composition	50
8		Runes and Quoting	51
8.1		Rune Definitions	51
8.2		Rune Hygiene	51
8.3		Quote	52
8.4		Built-In Runes	52
9		Modules & User-Defined Types	53
9.1		Using Statements	53
9.2		Require Statements	53
9.3		Native Library Loading	54
	9.3.1	Parameter Modes	55
	9.3.2	Variadic Parameters	55
	9.3.3	Return Conventions	55
	9.3.4	Buffers and Arrays	56
	9.3.5	Calling Conventions	57
	9.3.6	Type Restrictions	57
9.4		Raw Structs	57
9.5		Raw Callbacks	58
	9.5.1	Exactly One Value	58
	9.5.2	Threading	58
	9.5.3	Restrictions	59
9.6		Module Definitions	59
9.7		Class Definitions	62
	9.7.1	Class Modifiers	65
	9.7.2	Member Modifiers	65
	9.7.3	Static Members	66
9.8		Generic Classes	67
9.9		Inheritance, Interfaces, and Traits	69
9.10		Interface Definitions	70
9.11		Trait Definitions	70
9.12		Union Definitions	71
9.13		Struct Definitions	71
9.14		Record Definitions	72
9.15		Nested Types	72
9.16		Enum Definitions	73
	9.16.1	Flag Enums	73
9.17		Extension Methods	74
9.18		Event Definitions	74
10		Pipelines & Special Syntax	76
10.1		The Pipeline	76
10.2		Command Substitution	76
10.3		Subexpressions	76

10.4	Ranges	76
10.5	Comprehensions	77
10.6	Live / Streaming Output	78
10.7	Background Jobs	79
10.8	Tilde Expansion	79
10.9	Glob Expansion	80
10.10	Redirection	80
10.11	Process Substitution	80
10.12	Static Method Calls	81
10.13	Object Construction	81
10.14	Nameof	81
10.15	Alloc (Native Buffer)	82
10.16	Picking Columns vs Rows	82
10.17	Structured Introspection	82
10.18	System Namespace	83
10.19	Binder Pass	83
11	Configuration	84
11.1	Config Home	84
11.2	Startup Order	84
11.3	config.tosh	84
11.4	profile.tosh	84
11.5	Autoload Directory	85
11.6	Configuration Sections	85
11.7	Config Commands	85
11.8	Special Variables	86
12	Scripts and Subcommand Dispatch	87
12.1	Subcommand Basics	87
12.2	Flags and Arguments	87
12.3	Nested Subcommands	88
12.4	Subcommand Modifiers	89
12.5	Auto-Help	89
12.6	Diagnostics	90
II	Compilation	91
13	Overview & Execution Model	92
13.1	When each mode runs	92
13.2	Compiler pipeline	92
13.3	Parity contract	92
14	Command-Line Interface	94
14.1	Flags	94
14.2	Examples	94
15	Profiles & the Tier Model	96
15.1	Tiers	96
15.2	Profiles	96
15.3	RequireTier semantics	97
15.4	Parity expectations	97

16	Type Discipline in Compiled Mode	98
16.1	The audit rules	98
16.2	Explicit dynamic opt-out	98
16.3	The <code>-allow-dynamic</code> flag	98
16.4	Implementation note	99
17	Public CLR Shape	100
17.1	Top-level statements	100
17.2	Overloads	100
17.3	Name mangling	100
17.4	User-defined types	100
17.5	Reference assemblies	101
18	Pipelines & Redirections Under Compile	102
18.1	Pipeline stages	102
18.2	Stream redirection	102
18.3	Input redirection	103
18.4	Nested redirection	103
19	Function References in Compiled Mode	104
19.1	Single-overload fast path	104
19.2	Overloaded references	104
19.3	Named arguments through references	104
19.4	Late-bound fallback	105
20	Other Bound-Shape Emitters	106
20.1	Records	106
20.2	Tuple destructuring	106
20.3	throw as expression	106
20.4	Other helpers	107
21	MSBuild Integration	108
21.1	Project shape	108
21.2	Tasks	108
21.3	Fallback path	109
21.4	NuGet consumption from C#	109
22	Limits Per Profile	110
23	Conformance Test Layer	111
23.1	Feature matrix	111
23.2	Conformance cases	111
23.3	Adding a row	111
24	Compiler Diagnostics	113
24.1	Compile-only diagnostic codes	113
24.2	Tier-violation diagnostics	113
24.3	Runtime callable diagnostics	113
III	Command Reference	115
25	Built-In Command Catalog	116
25.1	CLR	116

25.1.1	call	116
25.1.2	call-method	117
25.1.3	cast	117
25.1.4	clone	118
25.1.5	constructors	118
25.1.6	del-prop	118
25.1.7	describe-type	119
25.1.8	funcs	120
25.1.9	get-methods	120
25.1.10	get-prop	120
25.1.11	get-props	121
25.1.12	has-method	121
25.1.13	has-prop	122
25.1.14	load-assembly	122
25.1.15	members	123
25.1.16	methods	123
25.1.17	native-alloc	124
25.1.18	native-free	124
25.1.19	native-offsetof	124
25.1.20	native-read	125
25.1.21	native-sizeof	125
25.1.22	native-write	126
25.1.23	new	126
25.1.24	props	127
25.1.25	set-prop	127
25.1.26	type-of	128
25.1.27	types	128
25.2	Concurrency	129
25.2.1	async	129
25.2.2	await	129
25.2.3	channel	130
25.2.4	channel-close	130
25.2.5	channel-recv	131
25.2.6	channel-select	131
25.2.7	channel-send	132
25.2.8	race	132
25.2.9	scope	132
25.2.10	settle	133
25.2.11	spawn	133
25.2.12	timeout	134
25.3	Data	134
25.3.1	as-file	134
25.3.2	complex	135
25.3.3	from	135
25.3.4	hash	136
25.3.5	mat	136
25.3.6	parse	137
25.3.7	to	137
25.3.8	vec	138
25.4	Filesystem	138
25.4.1	append-file	138
25.4.2	back	139

25.4.3	basename	139
25.4.4	cat	140
25.4.5	cd	140
25.4.6	chmod	141
25.4.7	chown	141
25.4.8	close	142
25.4.9	copy-to	142
25.4.10	cp	143
25.4.11	df	144
25.4.12	dirname	145
25.4.13	dirs	145
25.4.14	du	146
25.4.15	exists	147
25.4.16	find	147
25.4.17	findmnt	148
25.4.18	flush	149
25.4.19	forward	150
25.4.20	glob	150
25.4.21	is-dir	151
25.4.22	is-file	151
25.4.23	is-link	151
25.4.24	length	152
25.4.25	ln	152
25.4.26	ls	153
25.4.27	lsblk	154
25.4.28	mkdir	156
25.4.29	mkdir-temp	156
25.4.30	mv	156
25.4.31	open-file	157
25.4.32	position	158
25.4.33	pwd	159
25.4.34	read-bytes	159
25.4.35	read-file	160
25.4.36	read-from	160
25.4.37	read-line-from	161
25.4.38	read-lines	161
25.4.39	read-to-end	162
25.4.40	readlink	162
25.4.41	realpath	163
25.4.42	rm	163
25.4.43	seek	164
25.4.44	stat	164
25.4.45	tempfile	165
25.4.46	touch	165
25.4.47	tree	166
25.4.48	write-bytes	166
25.4.49	write-file	167
25.4.50	write-line-to	168
25.4.51	write-to	168
25.5	Functional	169
25.5.1	compose	169
25.5.2	converge	169

25.5.3	curry	170
25.5.4	cycle	170
25.5.5	invoke	170
25.5.6	iterate	171
25.5.7	partial	172
25.5.8	recur	172
25.5.9	repeat	173
25.5.10	repeatedly	173
25.5.11	unfold	174
25.6	Math	174
25.6.1	round	174
25.7	Network	175
25.7.1	http	175
25.7.2	ip	177
25.7.3	ping	179
25.8	Pipeline	179
25.8.1	all	180
25.8.2	any	180
25.8.3	average	180
25.8.4	cartesian-product	181
25.8.5	chain	181
25.8.6	chunk	182
25.8.7	collect	182
25.8.8	combinations	183
25.8.9	count	183
25.8.10	dedup	183
25.8.11	describe	184
25.8.12	distinct	184
25.8.13	each	185
25.8.14	enumerate	185
25.8.15	filter	186
25.8.16	find-index	186
25.8.17	first	187
25.8.18	flat-map	187
25.8.19	flatten	187
25.8.20	frequencies	188
25.8.21	get	188
25.8.22	group-by	189
25.8.23	group-while	189
25.8.24	ignore	189
25.8.25	inspect	190
25.8.26	interleave	190
25.8.27	intersperse	191
25.8.28	join	191
25.8.29	last	192
25.8.30	map	192
25.8.31	max	193
25.8.32	median	193
25.8.33	min	193
25.8.34	none	194
25.8.35	parallel	194
25.8.36	partition	195

25.8.37	percentile	195
25.8.38	permutations	196
25.8.39	reduce	196
25.8.40	rename	197
25.8.41	reverse	197
25.8.42	row	197
25.8.43	scan	198
25.8.44	skip	198
25.8.45	skip-until	199
25.8.46	skip-while	199
25.8.47	sort	199
25.8.48	stdev	200
25.8.49	step-by	200
25.8.50	sum	201
25.8.51	summarize	201
25.8.52	take-until	202
25.8.53	take-while	203
25.8.54	tee	203
25.8.55	transpose	203
25.8.56	variance	204
25.8.57	where	204
25.8.58	window	205
25.8.59	xargs	205
25.8.60	zip	206
25.9	Process	206
25.9.1	bg	206
25.9.2	fg	207
25.9.3	jobs	207
25.9.4	kill	208
25.9.5	lsfd	208
25.9.6	ps	209
25.9.7	signal	210
25.9.8	wait-for	211
25.10	Prompt	211
25.10.1	prompt	211
25.10.2	prompt-dir	212
25.10.3	prompt-duration	212
25.10.4	prompt-exit	213
25.10.5	prompt-git	214
25.10.6	prompt-history	214
25.10.7	prompt-jobs	214
25.10.8	prompt-newline	215
25.10.9	prompt-text	215
25.10.10	prompt-time	216
25.10.11	prompt-userhost	216
25.10.12	styled	217
25.11	Scripting	217
25.11.1	assert	217
25.11.2	format	218
25.11.3	raise	218
25.11.4	undef	219
25.12	Shell	219

25.12.1	<code>apropos</code>	219
25.12.2	<code>benchmark</code>	220
25.12.3	<code>clear</code>	220
25.12.4	<code>config</code>	220
25.12.5	<code>dbg</code>	221
25.12.6	<code>debug</code>	221
25.12.7	<code>edit</code>	222
25.12.8	<code>eval</code>	222
25.12.9	<code>events</code>	223
25.12.10	<code>exec</code>	223
25.12.11	<code>exit</code>	224
25.12.12	<code>help</code>	224
25.12.13	<code>history</code>	225
25.12.14	<code>history-search</code>	226
25.12.15	<code>hush</code>	227
25.12.16	<code>read-line</code>	227
25.12.17	<code>source</code>	227
25.12.18	<code>time</code>	228
25.12.19	<code>tui</code>	228
25.12.20	<code>ulimit</code>	229
25.12.21	<code>umask</code>	230
25.12.22	<code>unless</code>	231
25.12.23	<code>view</code>	231
25.12.24	<code>which</code>	231
25.12.25	<code>with-retry</code>	232
25.13	System	232
25.13.1	<code>date</code>	232
25.13.2	<code>env</code>	233
25.13.3	<code>export</code>	233
25.13.4	<code>forget</code>	234
25.13.5	<code>free</code>	234
25.13.6	<code>guid</code>	235
25.13.7	<code>hostname</code>	235
25.13.8	<code>hostnamectl</code>	235
25.13.9	<code>id</code>	236
25.13.10	<code>journalctl</code>	236
25.13.11	<code>loginctl</code>	237
25.13.12	<code>lscpu</code>	238
25.13.13	<code>lsipc</code>	240
25.13.14	<code>networkctl</code>	241
25.13.15	<code>seq</code>	242
25.13.16	<code>sleep</code>	243
25.13.17	<code>systemctl</code>	243
25.13.18	<code>timespan</code>	244
25.13.19	<code>uname</code>	245
25.13.20	<code>uptime</code>	245
25.13.21	<code>vars</code>	246
25.13.22	<code>whoami</code>	246
25.14	Text	246
25.14.1	<code>cut</code>	247
25.14.2	<code>echo</code>	247
25.14.3	<code>grep</code>	247

25.14.4	head	248
25.14.5	join-lines	249
25.14.6	lines	249
25.14.7	match	250
25.14.8	raw	250
25.14.9	replace	251
25.14.10	split	252
25.14.11	tail	252
25.14.12	template	253
25.14.13	tr	253
25.14.14	uniq	254
25.14.15	wc	254
25.14.16	write	255
25.14.17	writeline	256
 IV Cookbook		257
26 Common Tasks		258
26.1	File System Operations	258
26.1.1	Find Large Files	258
26.1.2	Batch Rename	258
26.1.3	Disk Space Report	258
26.1.4	Watch for New Files	258
26.2	Text Processing	258
26.2.1	Log Analysis	258
26.2.2	Delimited Column Types	259
26.2.3	CSV Processing	259
26.2.4	Extract Unique Values	260
26.3	System Administration	260
26.3.1	Process Monitor	260
26.3.2	Service Health Check	260
26.3.3	Network Diagnostics	260
26.4	Data Transformation	260
26.4.1	JSON API Processing	260
26.4.2	Build a Report	260
26.4.3	Pivot Data	261
26.5	Functional Patterns	261
26.5.1	Pipeline Composition	261
26.5.2	Memoization with Records	261
26.5.3	Fibonacci with Unfold	261
26.5.4	Power Set	262
26.6	Configuration Recipes	262
26.6.1	Custom Prompt	262
26.6.2	Project-Specific Setup	262
26.7	TUI Applications	262
26.7.1	Interactive File Browser	262
26.7.2	Service Manager	263
 V Diagnostics		264
27 Diagnostic Code Reference		265

27.1	tosh.bind.*	266
27.2	tosh.command.*	266
27.3	tosh.compile.*	266
27.4	tosh.config.*	266
27.5	tosh.edit.*	266
27.6	tosh.format.*	266
27.7	tosh.get.*	267
27.8	tosh.help.*	267
27.9	tosh.history.*	267
27.10	tosh.naming.*	267
27.11	tosh.native.*	267
27.12	tosh.parser.*	267
27.13	tosh.row.*	275
27.14	tosh.runtime.*	275
27.15	tosh.tui.*	290
27.16	tosh.type.*	290
27.17	tosh.user.*	291
Index		292

List of Figures

Preface

Welcome to the **ToastScript Language Specification**—the definitive guide to every feature, command, operator, and idiom in the TōSh shell.

TōSh is a modern, object-oriented shell built on .NET 10 and the Common Language Runtime (CLR). It replaces traditional text-stream pipelines with *structured object pipelines*, giving you rich types, tab completion, and CLR interop right at the command line.

This document covers:

- The complete language syntax—variables, operators, control flow, functions, modules, classes, records, enums, and events.
- Every built-in command (over 200), with signatures, descriptions, and runnable examples.
- The type system, including CLR type aliases, shell records, callables, type conversion, and truthiness rules.
- Pipeline mechanics, redirection, process substitution, and background jobs.
- Configuration, startup files, and prompt customization.
- A *Cookbook* of practical recipes for everyday tasks.

Who Is This For?

Whether you are evaluating TōSh for the first time, writing your first `profile.tosh`, or building complex automation scripts, this specification is your single source of truth.

Conventions

monospace

Shell commands, code, and identifiers.

italic

Placeholder values you supply.

[brackets]

Optional arguments.

arg ...

One or more repetitions.

a | b

Choose one of the alternatives.

Part I

Language Core

Chapter 1

Introduction to TōSh

1.1 What Is TōSh?

TōSh (short for *ToastScript*) is a Linux-first interactive shell and scripting language. It is built on the .NET 10 CLR, meaning every value that flows through a pipeline is a **real object**—not a flat string.

- **Structured pipelines.** Commands emit lists, records, dates, file-system entries, and arbitrary CLR objects. Downstream commands filter, transform, and aggregate with full type awareness.
- **CLR integration.** Call any .NET method, inspect types, load assemblies, and even invoke native (unmanaged) libraries via built-in FFI.
- **Familiar surface.** If you know Bash, PowerShell, or Nushell, much of TōSh will feel natural: pipes, redirection, globs, \$variables, if/for/while, and function definitions all look close to what you expect.
- **Rich features.** Pattern matching with guards, destructuring, first-class functions, event handlers, module system, classes, records, enums, and a built-in TUI toolkit.

1.2 Quick Start

```
# List files, filter, and sort
ls -la | where _.Type == file | sort Size | reverse | first 5

# Variables and string interpolation
var name = "World"
echo $"Hello, {$name}!"

# Functions
func greet(who: string) {
    writeline $"Welcome, {$who}."
}
greet "Toast"

# Pipeline with structured data
ps -ef | where _.Cpu > 5.0 | sort Cpu | reverse | first 10
```

Your first TōSh session

1.3 Design Philosophy

1. **Objects, not text.** Parsing output into columns is a relic. Every TōSh command returns typed objects.
2. **Progressive disclosure.** Basic usage is simple; advanced features (modules, FFI, TUI) are there when you need them.
3. **Linux-native.** Built for Linux with first-class support for systemd, procfs, and POSIX signals.
4. **Correctness.** Strong typing with helpful error messages. `pipefail` by default.

Chapter 2

Syntax Fundamentals

2.1 Comments

TōSh supports regular comments, block comments, and documentation comments:

```
# This is a comment
echo hello # Inline comment

##{
This block comment may span lines.
}##

## Adds two numbers.
## @param a first value
## @param b second value
## @returns the sum
func add(a, b) { return ($a + $b) }
```

begins an ordinary single-line comment. ##{...}## is a block comment. Consecutive ## documentation comments attach to the next function, class, module, record, enum, event, or class member and feed hover, completion, signature-help, and help metadata.

2.1.1 When # Is a Comment

A lone # opens a comment only when it *stands alone as a word*: it must begin a word — be at the start of a line or preceded by whitespace — *and* be followed by whitespace or the end of the line. Anywhere else it is an ordinary bareword character:

# a comment	# '#' begins a word, space follows
echo hello # a comment	# same, mid-line
#	# a lone '#' at end of line is an empty comment
echo #ff0000	# bareword "#ff0000" -- no quoting needed
echo issue#42	# bareword "issue#42"
echo http://host/page#frag	# bareword, fragment intact
echo C#	# bareword "C#"

The word-start half of the rule is the POSIX shell rule: `bash` and `zsh` also refuse to start a comment mid-word, which is why `echo issue#42` prints `issue#42` in both. Requiring whitespace to follow as well is what TōSh adds, and it is what lets `#ff0000` go unquoted.

Note

`##` is unconditional. Doc comments and `##{...}##` blocks keep their meaning wherever they appear, so a divider line such as `#####` still reads as a doc comment. The one further exception is `#!` at byte offset zero, which is a shebang.

Migration note — July 31, 2026

Earlier builds treated `#` as a comment opener everywhere, so `echo issue#42` printed `issue` and silently discarded the rest of the line. Comments written `#comment` with no following space are now barewords; write `# comment` instead. The formatter normalises the spelling, and no comment in the TōSh tree needed changing.

2.2 Identifiers

Identifiers start with a letter or underscore and may contain letters, digits, underscores, and hyphens:

```
var my_count = 10
var total-items = 42
func process-batch() { ... }
```

Note

Hyphens are allowed in identifiers, which is unusual among programming languages but natural for shell commands (`read-file`, `group-by`).

2.3 Keywords and Contextual Words

TōSh has a small set of hard keywords and a larger set of contextual words. Hard keywords are always part of the language grammar. Contextual words are only special in the position where the grammar expects them, such as a type modifier before `class`, a member modifier before `prop`, or `where/let` inside a comprehension.

<code>abstract</code>	<code>enum</code>	<code>global</code>	<code>nameof</code>	<code>raw</code>
<code>alloc</code>	<code>event</code>	<code>guarded</code>	<code>native</code>	<code>readonly</code>
<code>arg</code>	<code>export</code>	<code>handles</code>	<code>new</code>	<code>record</code>
<code>as</code>	<code>extend</code>	<code>hermit</code>	<code>null</code>	<code>ref</code>
<code>bind</code>	<code>extends</code>	<code>hidden</code>	<code>obsolete</code>	<code>require</code>
<code>break</code>	<code>fading</code>	<code>hollow</code>	<code>once</code>	<code>required</code>
<code>callconv</code>	<code>false</code>	<code>if</code>	<code>out</code>	<code>return</code>
<code>case</code>	<code>finally</code>	<code>implements</code>	<code>override</code>	<code>rune</code>
<code>catch</code>	<code>fixed</code>	<code>in</code>	<code>overrule</code>	<code>sealed</code>
<code>class</code>	<code>flag</code>	<code>interface</code>	<code>partial</code>	<code>set</code>
<code>coerce</code>	<code>flags</code>	<code>lazy</code>	<code>priority</code>	<code>shared</code>
<code>const</code>	<code>fluid</code>	<code>leaky</code>	<code>private</code>	<code>shy</code>
<code>continue</code>	<code>for</code>	<code>let</code>	<code>prop</code>	<code>static</code>
<code>default</code>	<code>from</code>	<code>local</code>	<code>protected</code>	<code>strict</code>
<code>defer</code>	<code>fulfills</code>	<code>match</code>	<code>proud</code>	<code>struct</code>
<code>eager</code>	<code>func</code>	<code>module</code>	<code>public</code>	<code>subcmd</code>
<code>else</code>	<code>get</code>	<code>name-of</code>	<code>quote</code>	<code>subcommand</code>

switch	try	until	vital	yield
throw	type	uses	when	
trait	union	using	where	
true	unless	var	while	

Note

Most built-in command names are *not* reserved words; they are resolved by the command dispatcher. The words `and`, `or`, and `not` are operators, not keywords. They can technically appear inside identifiers (e.g. `band-name`) because TōSh allows hyphens in identifiers.

2.4 Strings

TōSh provides eight string forms to cover every quoting need:

Form	Escapes	Interpolation	Multi-line	Example
"..."	Yes	No	No	"Hello\n"
'...'	No	No	No	'raw text'
"""..."""	No	No	Yes	Raw triple-quoted block
"""..."""	ANSI C	No	Yes	ANSI C triple-quoted block
\$"..."	Yes	Yes	No	`\${name}`
\$\$\$"..."	No	Yes	Yes	Interpolated block
\$'...'	ANSI C	No	No	`\${name}`
\$\$\$'...'	ANSI C	Yes	Yes	Interpolated ANSI C block

Table 2.1: String literal varieties in TōSh

2.4.1 Escape Sequences

Double-quoted strings support these escapes:

Sequence	Meaning
\n	Newline
\t	Tab
\r	Carriage return
\\	Literal backslash
\"	Literal double quote
\0	Null character

An unrecognised escape is preserved verbatim, including its backslash: `"\d+"` contains the three characters `\`, `d`, and `+`. This makes common regex forms work without doubled slashes:

```
"a1" =~ "\d"      # true
"file.cs" =~ "\.cs$" # true
```

Single-quoted strings are raw: backslashes never introduce escapes and the next single quote closes the string. Use `$'...'` for ANSI-C escapes such as `\x1b`, or switch quote forms when the text itself contains a single quote.

Migration note — July 25, 2026

Earlier builds decoded escapes inside ordinary `'...'` strings and silently removed the backslash from unknown escapes in `"..."` and `$"..."`. Ordinary single quotes are now raw, and every escape-processing form preserves unknown escape pairs. Replace an ordinary single-quoted string with `$'...'` when the old escape processing was intentional.

2.4.2 String Interpolation

Interpolated strings (`$"..."`) embed expressions inside `{...}` delimiters:

```
var user = "Alice"
echo $"Hello, {$user}!"           # Variable
echo $"2 + 2 = {2 + 2}"           # Expression
echo $"Files: {ls | where _.Type == file | count}" # Pipeline
echo $"Today is {(date now).DayOfWeek.ToString()}" # Member access
```

```
var report = $"""
Name: {$name}
Items: {$items | count}
Total: {$items | sum Price}
"""
```

Multi-line interpolated string

2.5 Numeric Literals

```
42          # int (System.Int32)
3.14        # double (System.Double)
1_000_000   # underscores for readability
0xFF        # hexadecimal
0b1010      # binary
0o777       # octal
4i          # imaginary literal (Complex: 0 + 4i)
-2.5i       # imaginary literal (Complex: 0 - 2.5i)
```

An integer literal takes the narrowest type that holds it — `int`, then `long`, then `ulong` — and the rule is the same in every base, so `0xFFFFFFFFFFFFFFFF` is a `ulong` and not a truncated `int`. A literal too large for `ulong` is an error rather than a silently different number.

A suffix pins the type where inference is not what is wanted:

```
100u        # uint (ulong when too large for uint)
100L        # long
100UL       # ulong
```

Suffixes are case-insensitive, and `lu` is accepted alongside `ul`.

2.5.1 Format and Alignment

An interpolation hole may carry an alignment and a format clause, `{expr,align:format}`:

```
$"pi is {$n:F2}"           # pi is 3.14
$"hex {$i:X}"             # hex 2A
$"[$i,6]"                 # [ 42] right-aligned in six columns
```



```
$"[$i,-6]"           # [42 ] left-aligned
$"$when:yyyy-MM-dd"  # 2026-08-16
```

The clause is handed to the value's own formatter, so every .NET format string works. A clause a value cannot honour leaves it rendered plainly rather than failing.

Only separators at the top level of the hole are clauses: inside parentheses, brackets, braces or a string they belong to the expression. A conditional's colon is recognised as its own, so ``${x} > 0 ? "a" : "b"}`` needs no parentheses — though a conditional that wants a format clause does: ``${x} > 0 ? 1.5 : 2.5:F1}``.

2.6 Collection Literals

Record, dictionary, and set literals use the paired delimiters `{ | ... | }`, `{% ... %}`, and `{: ... :}`, respectively. The two characters in each opening or closing delimiter must be adjacent. Their empty spellings are `{ | }`, `{% %}`, and `{: :}`. Plain `{ ... }` remains block syntax.

```
# Lists
[1, 2, 3]
["a", "b", "c"]

# Records
{ | name = "Alice", age = 30 | }
{ | ($key) = "dynamic" | }      # computed property name
{ | ...$base, enabled = true | } # record spread

# Dictionaries with arbitrary keys
{% "left" => 1, "right" => 2 %}

# Sets
{: 1, 2, 2, 3 :}

# Tuples
("x", 42)

# Spread in list literals
[0, ...$items, 99]
```

2.7 Special Literals

```
true           # Boolean true
false          # Boolean false
null           # Null reference

# Durations (parsed as TimeSpan)
5s
100ms
2m30s

# ISO temporal instants (parsed as DateTimeOffset)
2026-03-27
2026-03-27T14:30
2026-03-27T14:30:45Z
2026-03-27T14:30:45.1234567-04:00
```

```
# IP addresses (parsed as System.Net.IPAddress)
192.168.1.1
::1
2001:db8::1

# Quantities with units (backtick syntax)
100`m      # 100 meters (Length)
9.8`m/s^2   # acceleration
5`kg        # 5 kilograms (Mass)
```

Intrinsic temporal literals deliberately accept only these ISO-style shapes: `yyyy-MM-dd`, `yyyy-MM-ddTHH:mm`, `yyyy-MM-ddTHH:mm:ss`, and the seconds form with one through seven fractional digits. Date-time forms may omit a zone or end in `Z` or a numeric `+HH:mm/-HH:mm` offset. Explicit commands such as `date parse` may accept additional human-oriented input, but that permissive parser is never used to guess the type of a bareword.

IPv4 literals likewise require four explicit decimal octets from 0 through 255, without leading zeroes. IPv6 uses its ordinary colon notation. Consequently, dotted numeric text such as `1.2.3` is neither a date nor an abbreviated IPv4 literal.

Migration note — July 25, 2026

Earlier builds let the CLR's culture-oriented date and abbreviated-IPv4 parsers guess the type of expression barewords. Text such as `1.2.3` could therefore become a date in 2003 or the address `1.2.0.3`. Intrinsic literals are now exact. Quote human-oriented input as text or pass it to an explicit parser such as `date parse`.

Chapter 3

Type System

3.1 CLR Foundation

Every value in TōSh is a .NET CLR object. There are no “untyped” values. The type system sits on top of the Common Language Runtime, giving you access to the full .NET ecosystem.

```
echo 42 | type-of      # System.Int32
echo "hello" | type-of # System.String
ls | first | type-of   # System.IO.FileSystemInfo-derived
date now | type-of     # System.DateTimeOffset
```

3.2 Built-in Type Aliases

TōSh provides over 40 short aliases for common CLR types. Use these in type annotations, casts, and new expressions.

Alias	CLR Type	Notes
string	System.String	Text
int	System.Int32	32-bit signed integer
long	System.Int64	64-bit signed integer
float	System.Single	32-bit IEEE float
double	System.Double	64-bit IEEE float
decimal	System.Decimal	128-bit decimal
bool	System.Boolean	true / false
byte	System.Byte	Unsigned 8-bit
sbyte	System.SByte	Signed 8-bit
short	System.Int16	16-bit signed
ushort	System.UInt16	16-bit unsigned
uint	System.UInt32	32-bit unsigned
ulong	System.UInt64	64-bit unsigned
char	System.Char	UTF-16 code unit
object	System.Object	Root CLR type
guid	System.Guid	UUID
uri	System.Uri	Uniform resource identifier
datetime	System.DateTime	Date + time
datetimeoffset	System.DateTimeOffset	Date + time + offset

Alias	CLR Type	Notes
dateonly	System.DateOnly	Date without time
timeonly	System.TimeOnly	Time without date
timespan	System.TimeSpan	Duration
duration	Tosh.Runtime.TemporalAmount	Calendar-aware duration
quantity	Tosh.Runtime.Units.Quantity	Any physical quantity
length, mass, temperature	Named *Quantity types	Base physical categories
timequantity / durationquantity	DurationQuantity	Fixed physical duration
datasize	DataSizeQuantity	Bit/byte physical data quantity
speed, area, volume	Named *Quantity types	Common geometry and motion categories
force, energy, power, pressure	Named *Quantity types	Common mechanics categories
frequency, angle, acceleration, density	Named *Quantity types	Derived physical categories
voltage, current, resistance, charge	Named *Quantity types	Electrical categories
torque, flowrate, capacitance, inductance	Named *Quantity types	Additional derived categories
substance, luminosity, angularvelocity	Named *Quantity types	SI and angular categories
regex	System.Text.RegularExpressions.Regex	Compiled pattern
list	System.Collections.Generic.List<object>	Growable ordered list
array	System.Object[]	Fixed-size array
T[]	T[]	Array of T — see below
dict	Dictionary<string, object>	Key-value dictionary
hashtable	System.Collections.Hashtable	Non-generic hash table
set	HashSet<object>	Unique value set
record	System.Dynamic.ExpandoObject	Anonymous dynamic record (aliases table, dynamicrecord)
tuple	System.Tuple types	Immutable positional tuple
file	System.IO.FileInfo	File reference
dir / directory	System.IO.DirectoryInfo	Directory reference
ptr	System.IntPtr	Native pointer
ptr	System.IntPtr	Native pointer (alias)

Alias	CLR Type	Notes
ip / ipaddress	System.Net.IPAddress	IP address
version	System.Version	Semantic version
Vector / vec	ToSh.Vector	Numeric vector shell type
Matrix / matrix / mat	ToSh.Matrix	Numeric matrix shell type
Complex / complex	ToSh.Complex	Complex number shell type

Table 3.1: Built-in type aliases

Aliases and CLR types of the same name

The aliases are matched without regard to case, so `INT` and `Int` both name `System.Int32`. Several aliases share a name with a different CLR type—`file` is `FileInfo` while `System.IO.File` is the static helper class, and `array` is `object[]` while `System.Array` is the class carrying `IndexOf` and `Sort`.

The spelling decides which one a *member access* reaches. An alias is canonically lower-case, so a capitalised head names the CLR type:

```
File.ReadAllText("/etc/hostname") # System.IO.File
Array.IndexOf([1, 2, 3], 2)       # System.Array
```

Everywhere a type is *named* rather than used—annotations, `cast`, and `new`—the alias wins regardless of case, so `var f: file` and `var f: File` both bind `FileInfo`:

```
var f: file = (new FileInfo("/etc/hostname"))
var a: array = [1, 2]                # object[], not System.Array
new array(1, 2, 3)                  # the shell's array type
```

Fully qualifying the name is always unambiguous, in either position.

Array types

A postfix `[]` names a CLR array of the element type, so `string[]` is `System.String[]`. It is accepted anywhere a type is: parameter and return annotations, variable and class-member declarations, and `cast`. The suffix repeats for jagged arrays (`string[][]`) and may be followed by `?`.

```
func names(paths: string[]) -> string[] {
    return $paths
}

var files: string[] = ["a.txt", "b.txt"]
var grid: int[][]   = [[1, 2], [3, 4]]
var typed = (cast string[] ["a", "b"])
```

Note

`T[]` is an array; `list<T>` is a `System.Collections.Generic.List<T>`. Both are available and they are different types — array remains the untyped spelling, equivalent to `object[]`.

Note

Only an *empty* bracket pair is a type suffix. In a native signature a bracket holds a fixed inline capacity instead — `buffer[256]`, `double[3]` — and those keep that meaning.

3.3 Shell-Specific Types

Beyond CLR types, TōSh introduces several shell-native types:

3.3.1 StorageSize

A value type representing byte quantities with human-friendly display:

```
var small = 10kb           # StorageSize of 10 kB (10,000 bytes)
var large = 2.5mb          # StorageSize of 2.5 MB (2,500,000 bytes)
echo ($large > $small)     # true
ls | where Size > 100kb    # filter files larger than 100 kB
ls | sort Size             # sort by StorageSize
```

Note

Storage-size suffixes `b`, `kb`, `mb`, `gb`, `tb`, `pb` use SI decimal factors (1 kb = 1,000 bytes). Binary suffixes `kib`, `mib`, `gib`, `tib`, `pib` use powers of 1,024. These suffixes are *expression-context literals*: they parse to `StorageSize` values in expressions (`var x = 10kb`, `(10kb > 5mb)`) but remain plain strings when passed as raw command arguments (`echo 10kb`). Suffix matching is case-insensitive and mandatory; a malformed or overflowing suffix form in expression position never silently falls back to a string.

3.3.2 TemporalAmount / TimeSpan

Duration values with natural syntax:

```
5s           # 5 seconds
100ms        # 100 milliseconds
2m30s        # 2 minutes 30 seconds
sleep 1s
```

3.3.3 ToshRange

Lazy sequences defined by start, optional step, and end:

```
1..10        # 1, 2, 3, ..., 10
1..2..10     # 1, 3, 5, 7, 9 (step of 2)
10..1        # 10, 9, 8, ..., 1 (auto-descending)
-2..2        # -2, -1, 0, 1, 2
```

Range starts, steps, and ends are signed 32-bit integers. A literal fractional bound such as `1.5..3` produces `tosh.parser.range_requires_integer`; negative integer bounds are ordinary ranges.

3.3.4 IShellRecordObject

The interface for structured records—produced by commands like `ls`, `ps`, `stat`, and record literals:

```
{ | name = "Alice", age = 30 | } # Anonymous record
ls | first | get-props          # List property names
```

3.3.5 IShellCallable

The interface for callable objects—lambdas, function references, partial applications, curried functions, and composed functions:

```
var fn = func(x) => ($x * 2)
invoke $fn 21                # 42
var callback = &greet         # Function reference
```

3.3.6 LazySequence

LazySequence is the runtime representation for lazy pipeline-friendly sequences. Infinite ranges, generator comprehensions, and iterator commands such as `iterate`, `recur`, `cycle`, `repeat`, and `repeatedly` can produce values without materialising the entire sequence. Items are evaluated on demand and memoised as the sequence is traversed.

```
1.. | first 5                # 1, 2, 3, 4, 5
0..2.. | first 5             # 0, 2, 4, 6, 8
iterate 1 func(x) => ($x * 2) | first 6
recur (0, 1) func(a, b) => ($a + $b) | first 8
($x * $x <| for x in 1..) | first 5 # generator comprehension
```

3.3.7 Vector

Vector is a shell-native dense vector of double values. Create vectors with the `vec` command, `new Vector(...)`, or static factory methods such as `Vector.zeros`, `Vector.ones`, `Vector.fill`, `Vector.range`, and `Vector.random`. Vectors render compactly as `[1, 2, 3]` even though they implement `IEnumerable<double>`.

```
var v = vec 1 2 3
var w = Vector.range(1, 3)
Vector.dot($v, $w)
Vector.cross((vec 1 0 0), (vec 0 1 0)) | raw
Vector.normalize($v) | raw
$v.Length
$v.Magnitude
```

3.3.8 Matrix

Matrix is a shell-native dense matrix of double values. Create matrices with `mat`, `new Matrix(...)`, or factories such as `Matrix.zeros`, `Matrix.ones`, `Matrix.fill`, `Matrix.identity`, and `Matrix.random`. Matrix multiplication uses `*`; element-wise multiplication is available as `Matrix.hadamard(left, right)`.

```
var a = mat [1, 2] [3, 4]
var b = Matrix.identity(2)
($a * $b) | raw
Matrix.transpose($a) | raw
Matrix.determinant($a)
Matrix.inverse($a) | raw
```

```
($a * (vec 1 1)) | raw
```

3.3.9 Complex

Complex wraps `System.Numerics.Complex` with shell type metadata, compact rendering, casts, imaginary literals, and static helpers. Use `complex`, `new Complex(...)`, numeric casts, string casts, or imaginary literals such as `4i`.

```
var z = complex 3 4
var w = 4i
($z + $w) | raw
Complex.conjugate($z) | raw
Complex.magnitude($z)
Complex.phase($z)
Complex.from-polar(2, Math.PI / 2) | raw
echo "3-4i" | cast complex | raw
```

3.3.10 Quantity (Unit System)

TōSh includes a dimensional-analysis unit system inspired by CAS calculators. A *quantity* is a numeric value paired with a physical unit. Unit literals use backtick syntax: the number is immediately followed by a backtick and the unit expression with no intervening whitespace. The conventional U+00B0 degree sign is the one adjacency shorthand and does not require a backtick.

```
100`m           # Length -- 100 meters
5`kg            # Mass -- 5 kilograms
9.8`m/s^2       # Acceleration
2.5`kWh         # Energy (SI prefix + unit)
90°            # Angle -- 90 degrees
20°C           # Absolute temperature -- 20 degrees Celsius
```

Syntax

```

⟨quantity-literal⟩ ::= ⟨decimal-real⟩`⟨unit-expr⟩ | ⟨decimal-real⟩°⟨angle-tail⟩? | ⟨decimal-real⟩°(C|F|R)
  ⟨unit-expr⟩      ::= ⟨unit-term⟩ ((*)|/) ⟨unit-term⟩* | 1/⟨unit-term⟩ ((*)|/) ⟨unit-term⟩*
  ⟨unit-term⟩      ::= ⟨unit⟩ (^ ⟨int⟩)?
  ⟨angle-tail⟩     ::= (*)|/ ⟨unit-term⟩ ((*)|/) ⟨unit-term⟩* | ^ ⟨int⟩
  ⟨unit⟩           ::= ⟨si-prefix⟩? ⟨base-unit⟩

```

Here `⟨decimal-real⟩` is a signed or unsigned decimal floating literal (including exponent notation and the ordinary digit-separator rules), not a radix integer, imaginary literal, or storage suffix. Unit expressions may contain multiplication (`*`), division (`/`), and integer exponents (`^`), but no grouping parentheses. SI prefixes from quecto through quetta are resolved only for unit definitions that opt into prefixes.

The degree shorthand uses the exact degree sign: `90°` is an angle, while `20°C`, `68°F`, and `540°R` are temperatures. Kelvin never takes a degree sign. Bare `c` and `F` retain their electrical meanings (coulomb and farad), so backtick temperature aliases are written `degC`, `degF`, and `degR`. Only the angle form may take a compound tail (for example `90°/s`); absolute scales such as `°C/s` are invalid.

Dimensions

Every unit decomposes into a product of **nine base dimensions** raised to integer exponents:

Dimension	SI Base Unit	Example
Length	meter (m)	100`m, 5`km, 12`in
Mass	kilogram (kg)	5`kg, 150`lb
Time	second (s)	30`s, 5`min, 2`hr
Electric Current	ampere (A)	3`A, 500`mA
Temperature	kelvin (K)	300`K, 20`°C, 72`°F
Amount of Substance	mole (mol)	2`mol
Luminous Intensity	candela (cd)	100`cd
Data	bit (bit)	1`GB, 500`kB
Angle	radian (rad)	3.14`rad, 90`deg

Named Quantity Types

When a quantity's dimension matches a known physical category, TōSh wraps it in a named type for clearer display and type checking:

Type	Dimension	Example Units
Length	L	m, km, mi, ft, in
Mass	M	kg, g, lb, oz
Duration	T	s, ms, min, hr
Temperature	Θ	K, degC, °C, degF, °F
DataSize	D	B, kB, MB, GB, TB
Speed	$L T^{-1}$	m/s, km/h, kph, mph
Area	L^2	m ² , km ² , acre
Volume	L^3	m ³ , L, gal
Acceleration	$L T^{-2}$	m/s ²
Force	$M L T^{-2}$	N, lbf
Energy	$M L^2 T^{-2}$	J, kJ, kWh, cal, BTU
Power	$M L^2 T^{-3}$	W, kW, hp
Pressure	$M L^{-1} T^{-2}$	Pa, bar, atm, psi
Frequency	T^{-1}	Hz, kHz, MHz, GHz
Voltage	$M L^2 T^{-3} I^{-1}$	V
Resistance	$M L^2 T^{-3} I^{-2}$	Ω
Capacitance	$I^2 T^4 M^{-1} L^{-2}$	F
Inductance	$M L^2 T^{-2} I^{-2}$	H
Charge	$I T$	C, Ah, mAh
Angle	A	rad, deg
Angular Velocity	$A T^{-1}$	rad/s, rpm
Torque	$M L^2 T^{-2}$	Nm
Density	$M L^{-3}$	kg/m ³
Flow Rate	$L^3 T^{-1}$	L/min, gal/min
Substance	N	mol
Luminosity	J	cd

Arithmetic

Quantities support the standard arithmetic operators. Dimensional analysis is applied automatically—adding incompatible dimensions is an error:

```

100'm + 50'm           # 150 m   (Length)
100'km + 500'm         # 100.5 km (auto-converts to larger unit)
100'm / 20's           # 5 m/s   (Speed)
5'kg * 9.8'm/s^2       # 49 N    (Force)
100'm + 5'kg           # ERROR: dimension mismatch

```

Multiplication and division use base values and return a canonical unit for the derived dimension; addition and subtraction preserve the left operand's display unit. Dividing equal dimensions yields an ordinary numeric scalar. Absolute Celsius, Fahrenheit, Rankine, and Kelvin values are temperature points; until temperature-difference units are introduced, arithmetic on those points is rejected rather than silently applying an affine offset as though it were a scale. Conversion and comparison remain supported.

Unit Conversion

The idiomatic conversion form reuses `as`; a leading backtick on the right marks a unit target rather than a type target:

```

2'mi as 'ft           # 10560 ft
2'hr as 's            # 7200 s
10'm/s as 'mph        # approximately 22.3694 mph
20°C as '°F          # 68 °F

```

The method forms `value.To("ft")` and `value.ConvertTo("ft")` are equivalent and are convenient for CLR interop or a dynamically selected target. The target must describe the same dimension.

Comparison operators also use base-value comparison with dimension checking:

```

1'km > 500'm          # true  (1000 m > 500 m)
5's == 5000'ms        # true

```

Bridge Promotion

When a plain `TimeSpan` or `StorageSize` is combined with a unit quantity, `TōSh` automatically promotes it:

```

5s + 10's             # 15 s   (TimeSpan promoted to Duration)

```

Typed script and function boundaries also parse external strings such as "5km", "2hr", and "10m/s" when the declared type is a quantity category. Friendly annotations include quantity, length, mass, temperature, datasize, speed, energy, power, and the other named categories. The existing duration alias continues to mean calendar-aware `TemporalAmount`; fixed physical time uses `DurationQuantity` or `timequantity`.

Members

Every quantity exposes the following properties (accessible via the record interface):

Property	Description
<code>Magnitude</code>	The numeric value in the original unit
<code>UnitSymbol</code>	The unit string as entered ("km", "m/s")
<code>BaseValue</code>	The value converted to SI base units
<code>CategoryName</code>	The type category ("Length", "Speed", ...)

```

var d = 5'km
echo $d.Magnitude      # 5
echo $d.UnitSymbol     # km
echo $d.BaseValue      # 5000
echo $d.CategoryName   # Length

```

3.4 Type Conversion

TōSh performs automatic type conversion in many contexts via its `TypeConversion` engine. Conversion is attempted in order:

1. **Identity.** Same type—no conversion needed.
2. **Null to nullable.** `null` converts to any nullable type.
3. **Numeric widening.** `int` $\rightarrow$ `long` $\rightarrow$ `double`, etc.
4. **String parsing.** Strings are parsed to the target type (`"42"` $\rightarrow$ `int`, `"true"` $\rightarrow$ `bool`).
5. **Collection coercion.** Lists can convert to arrays, sets, etc.
6. **CLR implicit/explicit operators.** If the target type defines conversion operators, they are used.

```

# Explicit cast
echo "42" | cast int      # String -> Int32

# Implicit in typed parameters
func double-it(n: int) => $n * 2
double-it "21"            # "21" auto-converts to 21

```

Narrowing is refused, not truncated

A conversion to an integral type succeeds when the value has nothing to lose and is refused when it does. Nothing narrows silently:

```

7.0 as int      # 7
"7.0" as int    # 7 - the string spells the same number
7.9 as int      # error: would discard its fractional part
"7.9" as int    # error: the same, for the same reason

```

The rule is about the *value*, not the type: a `double` narrows perfectly well when it holds a whole number. A numeric string follows whatever rule the number it spells would follow, so the spelling never decides the outcome.

Tip

The refusal names the fix. Round explicitly with `Math.Round`, `Math.Floor`, `Math.Ceiling` or `Math.Truncate`, and the conversion then has nothing to discard.

Casting to a declared type

`cast` also targets a type declared in ToastScript, including one nested inside a class. The target name is resolved against the CLR first and the declarations second, so an existing script that casts to a CLR type keeps doing so even if a declaration later takes the same name.

An **enum** converts from either a member name or a backing value, which is the reverse of the conversion that turns a member into its number:

```
enum Fuel : int { Mox = 3, Uranium = 8 }

cast Fuel 8           # Uranium
cast Fuel "Mox"       # Mox   (member names are matched case-insensitively)
cast int Fuel.Uranium # 8     (the round trip closes)
cast Fuel (cast int Fuel.Uranium) # Uranium

class Reactor { enum Fuel : int { Mox = 3 } }
cast Reactor.Fuel 3   # Mox
```

Every other declared kind—class, struct, record, union, interface, trait—converts only from a value that already *is* one, subclasses included. `cast` does not construct:

```
class Base { prop X = 7 }
class Derived extends Base { }

var d = new Derived()
(cast Base $d).X      # 7 - a Derived is a Base
cast Base 5           # error: cast converts only a value that already is one
```

Two failures are distinguished, because they call for different fixes. A name that resolves to no type at all raises `tosh.runtime.unknown_cast_target`—a misspelling, or a declaration that has not been required yet. A name that resolves but a value that will not convert raises `tosh.runtime.cast_failed`, and for an enum the message lists the members that do exist.

3.5 Type Annotations

An annotation names the type a value must have: on a variable (`var x: T`), a parameter (`func f(a: T)`), a return (`-> T`) or a class member (`prop P: T`). It is checked when the value is produced, and a value that does not already have the type is put through the conversions of §3.4 before being rejected.

Annotations differ from `cast` in when they run rather than in what they accept: `cast` converts at the point you write it, an annotation converts every value that arrives at the annotated position.

What satisfies an annotation

A subclass satisfies its base class. This holds in every annotated position, not only in variables:

```
class Shape { prop Kind: string = "shape" }
class Circle extends Shape { prop Kind: string = "circle" }

var s: Shape = new Circle()           # circle
func describe(v: Shape) -> string => $v.Kind
describe (new Circle())                # circle
func make() -> Shape => new Circle()   # a Circle is a Shape
```

The relation runs one way only. `var c: Circle = new Shape()` raises `tosh.type.mismatch`: every `Circle` is a `Shape`, and no `Shape` is necessarily a `Circle`.

An interface or trait names a contract, and an annotation naming one accepts any class that declares it with `or`. A contract declared on a base class counts for its subclasses:

```

interface Drawable { func Draw() -> string }
class Box fulfills Drawable { func Draw() -> string => "box" }
class Crate extends Box { }

func render(d: Drawable) -> string => $d.Draw()
render (new Box())                                # box
render (new Crate())                             # box - inherited from Box

trait Named { prop Name = "anon" }
class Widget uses Named { }
func label(n: Named) -> string => $n.Name
label (new Widget())                             # anon

```

The contract must be *declared*. A class with a structurally identical member but no clause is rejected with `tosh.type.mismatch`, because an annotation that accepted it would be duck typing with extra syntax, and the annotation would document nothing.

Where an annotation's name is resolved

A type name in an annotation is resolved against the scope that *declares* the annotation, not the scope that happens to be executing when the value arrives. A class inside a module therefore names itself, a sibling, or a module-local refinement type by its bare name, with no module path:

```

export partial module Geometry {
  export type Unit = double where (_ >= 0.0 and _ <= 1.0) coerce Math.Clamp(_,
    0.0, 1.0)
  export class Node { prop Label: string = "n" }

  export class Tree {
    prop Root : Node = new Node()           # a sibling, unqualified
    prop Alpha: Unit = 0.25                 # a module-local refinement type
    static func Empty() -> Tree => new Tree() # the declaring class itself
  }
}

(Geometry.Tree.Empty()).Root.Label        # n
var t = new Geometry.Tree()
$t.Alpha = 5.0
$t.Alpha                                  # 1 - the refinement still coerces

```

This matters because member annotations are checked when the member *runs*, by which time the module scope that declared the class is no longer on the engine's stack. Resolution against the declaring module is what makes the bare name mean what it reads as. Qualified names (§9.6) work from anywhere and are unaffected.

3.6 Truthiness

TōSh uses one truthiness protocol whenever a value is tested as a condition. The same rules apply to `if/while/until`, conditional expressions, comprehension and match guards, event when guards, `not/and/or`, and standard-library predicates such as `where`, `any`, `all`, and `assert`.

Type	Truthy	Falsy
bool	true	false
null	—	Always falsy
byte through UInt128, native integers, BigInteger	Non-zero	Zero
Half, float, double	Non-zero (including infinity)	Positive/negative zero, NaN
decimal	Non-zero	Zero
Complex	Non-zero with no NaN component	Zero or either component is NaN
string	Non-empty	Empty
ICollection, synchronous IEnumerable	Non-empty	Empty
Everything else	Always truthy	—

Table 3.2: Truthiness rules

Note

Count-bearing collections are tested through their count. A general synchronous `IEnumerable` is probed with one `MoveNext` call and the temporary enumerator is disposed. This can observably advance a deliberately single-pass enumerable; test such a value once and retain the result when that distinction matters.

Truthiness is not an explicit cast. For example, the non-empty string `"false"` is truthy, while converting that string to `bool` produces `false`. Logical operators always return a boolean and preserve short-circuit evaluation; they do not return one of their operands.

Refinement *where* predicates are the intentional exception: they are validation contracts and must return an actual `bool`. Refinement *if* `<guard>` *coerce* guards use the broad truthiness rules above.

3.7 Refinement Types

A *refinement type* constrains an existing type with one or more predicates. Values that fail a predicate are either repaired via a *coerce* clause or rejected at runtime.

Block Form

```
type PositiveInt = int {
    where _ > 0
}

type NormalizedPath = string {
    if (_ contains "\\") coerce (replace "\\\" "/" _)
    where _ starts-with "/"
    coerce "/"
}
```

The body may contain any combination of the three clause forms:

where `<predicate>` A constraint predicate. The value (bound to `_`) must evaluate to the boolean value `true` after all coercions have been applied; other truthy values are rejected. Multiple *where* clauses are evaluated as a conjunction.

if <guard> coerce <expr> A conditional normalisation that fires *before* predicate evaluation. When guard is truthy, expr replaces the value. Useful for always-on normalization such as trimming whitespace or canonicalizing path separators.

coerce <expr> (no guard) A fallback that fires only when one or more where predicates have already failed. The expression receives the failing value as `_` and produces a replacement. Predicates are then re-evaluated against the replacement; if they still fail the refinement is rejected.

Tip

Think of the three forms as three phases: *normalise* (if x coerce y) then *validate* (where) then *repair* (coerce).

Inline Form

```
type Port = int where (_ >= 1 and _ <= 65535)

# With an inline fallback: clamp to range if out of bounds
type Port = int where (_ >= 1 and _ <= 65535) coerce 80
```

Refinements on Parameters and Variables

Refinement types compose with the standard type annotation syntax:

```
# Function parameter
func serve(port: int where _ > 0) { ... }

# Variable declaration
var name: string where _ != ""

# Inline predicate (anonymous refinement)
var count: int where _ >= 0 = 0
```

Execution Order

When a value is checked against a refinement type:

1. All if <guard> coerce <expr> clauses fire in declaration order; when the guard is truthy, expr replaces the value.
2. All where predicates evaluate against the (possibly normalised) value.
3. If any predicate fails and an unconditional coerce clause exists, its expression runs and replaces the value.
4. Predicates are re-evaluated against the coerced value. Pass → success. Fail → runtime error.

Diagnostics

Code	Trigger
tosh.runtime.refinement_failed	Value did not satisfy a where predicate after all coercions.
tosh.runtime.refinement_requires_boolean	A where predicate returned a non-boolean value.

Limitation

Generic instantiation of refinement types (e.g. `list<PositiveInt>`) is not supported by the annotation resolver. For per-element validation, use `each _ as PositiveInt` in a pipeline instead.

Chapter 4

Operators

4.1 Arithmetic Operators

Operator	Symbol	Description
Addition	+	Numeric addition; also <code>DateTimeOffset</code> + <code>TimeSpan</code> , string concatenation, list concatenation
Subtraction	-	Numeric subtraction; also <code>DateTimeOffset</code> - <code>TimeSpan</code> , <code>DateTimeOffset</code> - <code>DateTimeOffset</code>
Multiplication	*	Numeric multiplication; <code>string</code> * <code>int</code> repeats the string
Division	/	Numeric division (integer when both operands are integral)
Integer division	//	Floor division; result is always rounded toward negative infinity
Modulo	%	Remainder after integer division
Exponentiation	**	Raises left operand to the power of the right ($2 ** 10 \rightarrow 1024$)
Unary negation	-x	Negates a numeric value

Table 4.1: Arithmetic operators

```
echo (2 + 3)           # 5
echo (10 / 3)          # 3 (integer division)
echo (10.0 / 3)        # 3.3333...
echo ("ha" * 3)        # "hahaha"
echo ([1, 2] + [3, 4]) # [1, 2, 3, 4]

# Date arithmetic
var now = date now
echo ($now + (timespan 1d)) # Tomorrow
echo ($now - (timespan 7d)) # One week ago
```

Arithmetic examples

4.2 Comparison Operators

Operator	Meaning
<code>==</code>	Equal
<code>!=</code>	Not equal
<code><</code>	Less than
<code>></code>	Greater than
<code><=</code>	Less than or equal
<code>>=</code>	Greater than or equal

Equality cascades through: `object.Equals`, `IComparable`, structural equality for records, and finally reference equality. Comparison operators attempt `IComparable` first, then fall back to string comparison.

```
echo (1 == 1)           # true
echo ("abc" < "def")    # true (lexicographic)
echo (10kb > 5kb)       # true (StorageSize comparison, both are literals)
echo (10kb > 5mb)       # false
```

4.2.1 Chained Comparison

Relational comparisons chain. Writing `a < b < c` means `(a < b)` and `(b < c)`, matching ordinary mathematical notation:

```
echo (1 < 2 < 3)        # true
echo (3 < 2 < 1)        # false
echo (1 <= 1 <= 1)      # true
echo (1 < 2 < 3 < 4)    # chains of any length
```

Two guarantees make a chain different from writing the conjunction out by hand:

- Each operand is evaluated *at most once*. In `1 < (next) < 3` the `next` call happens a single time, whereas the hand-written `(1 < (next))` and `((next) < 3)` would call it twice.
- Evaluation short-circuits. Once a pair compares false, the remaining operands are never evaluated.

Only `<`, `<=`, `>`, `>=`, `==`, and `!=` chain. A run containing `is`, `in`, `contains`, or a regex operator keeps the ordinary left-associative reading.

4.3 Regex Operators

Operator	Meaning
<code>=~</code>	Matches regex
<code>!~</code>	Does not match regex

```
echo ("hello" =~ "^h")  # true
echo ("world" !~ "^h")  # true

# In pipelines
ls | where _.Name =~ "\.cs$"
```

4.4 Logical Operators

Operator	Meaning
and / &&	Logical AND (short-circuit)
or /	Logical OR (short-circuit)
not	Logical NOT (unary prefix)

```

if ($age >= 18 and $hasId) { echo "Allowed" }
if ($age >= 18 && $hasId) { echo "Allowed" }    # equivalent
var hasValue = $value or "default"             # boolean result
var hasValue = $value || "default"             # equivalent
if (not $found) { echo "Not found" }

```

Both operands are tested with the truthiness rules in Table 3.2. and, or, and not return bool; use ?? when the desired result is an operand value rather than a boolean.

Note

TōSh prefers word-form logical operators (and, or, not) but also supports && and || as aliases for developers coming from C-family languages. The standalone & symbol remains reserved for background jobs, and ! only appears in != and !~.

4.5 Membership Operators

Operator	Meaning
in / is in / is-in	Value exists in collection
not in / is not in / not-in / is-not-in	Value does not exist in collection
contains	Collection contains value (also substring check for strings)
starts-with	String starts with the given prefix
ends-with	String ends with the given suffix

```

echo ("a" in ["a", "b", "c"])    # true
echo ("a" is in ["a", "b", "c"]) # equivalent
echo (5 not in [1, 2, 3])        # true
echo (5 is not in [1, 2, 3])     # equivalent
echo (5 not-in [1, 2, 3])        # hyphenated form
echo (5 is-not-in [1, 2, 3])     # hyphenated form
echo ("hello" contains "ell")    # true
echo ([1, 2, 3] contains 2)      # true
echo ({% "name" => "Ada" %} contains "name") # true: keys
echo ((1..5) contains 3)         # true
echo ("hello" starts-with "hel") # true
echo ("hello" ends-with "llo")   # true

```

Note

is in and in also perform a *substring check* when the right-hand operand is a string: ".is in "/foo/bar" returns true. String matching is ordinal (case-sensitive). For dictionaries, membership searches keys rather than values; other CLR and shell-native collections compare their elements with canonical equality. Non-collection left operands do not support contains and return false. collection contains value is exactly the operand-reversed spelling of value in collection.

4.6 Type Operators

Operator	Meaning
<code>is</code>	Type check (returns <code>bool</code>)
<code>is not</code>	Negated type check
<code>as</code>	Type cast (returns converted value or <code>null</code> on failure)

```
echo (42 is int)           # true
echo ("hi" is not int)    # true

# Type casting with 'as'
var n = "42" as int       # 42
var x = "hello" as int    # null (conversion failed)

# Pipeline cast (equivalent)
echo "42" | cast int      # 42
```

4.7 Null Coalescing

Operator	Meaning
<code>??</code>	Returns left side if non-null, otherwise right side
<code>?.</code>	Null-conditional member access

```
var name = $user ?? "Anonymous"
var len = $name?.Length ?? 0
```

4.8 Ternary Operator

```
var label = $count > 0 ? "items" : "empty"
echo ($x > 0 ? "positive" : ($x < 0 ? "negative" : "zero"))
```

4.9 Assignment Operators

Operator	Meaning
<code>=</code>	Simple assignment
<code>+=</code>	Add and assign
<code>-=</code>	Subtract and assign
<code>*=</code>	Multiply and assign
<code>**=</code>	Exponentiate and assign
<code>/=</code>	Divide and assign
<code>//=</code>	Floor-divide and assign
<code>%=</code>	Modulo and assign
<code>??=</code>	Assign if null

```
var x = 10
$x += 5      # 15
$x *= 2      # 30
$x ??= 100   # Still 30 (x isn't null)
```

4.10 Operator Precedence

Operators are listed from highest to lowest precedence.

Note that `as` and `is` sit at opposite ends of the table although both concern types. `as` is a cast: it yields a value, so it binds tighter than every binary operator and `x as int % 2` means `(x as int) % 2`. `is` is a test: it yields a boolean, so it binds loosely and `1 + 2 is int` tests the sum.

The bitwise operators are spelled as words because the symbols are taken: `&` runs a pipeline in the background and takes a function reference, and `|` separates pipeline stages. They also sit *tighter* than comparison, unlike C, so `flags band Mask == 0` groups as `(flags band Mask) == 0` — the way the line reads. Their order relative to one another follows C, which is not the part that surprises people.

Level	Operators	Description
1	<code>as</code>	Type cast
2	<code>**</code>	Exponentiation (right-associative)
3	<code>not</code> , <code>bnot</code> , <code>-</code> (unary)	Unary NOT, bitwise complement, negation
4	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Multiplicative
5	<code>+</code> , <code>-</code>	Additive
6	<code>shl</code> , <code>shr</code>	Bit shift
7	<code>band</code>	Bitwise AND
8	<code>bxor</code>	Bitwise XOR
9	<code>bor</code>	Bitwise OR
10	<code>..</code>	Range
11	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>~=</code> , <code>!~</code>	Comparison & regex
12	<code>is</code> , <code>is not</code>	Type testing
13	<code>in</code> , <code>not in</code> , <code>is in</code> , <code>is not in</code> , <code>contains</code> , <code>starts-with</code> , <code>ends-with</code> , <code>has</code>	Membership & flag testing
14	<code>and</code> / <code>&&</code>	Logical AND
15	<code>or</code> / <code> </code>	Logical OR
16	<code>?, :</code>	Ternary conditional
17	<code>??</code>	Null coalescing
18	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>**=</code> , <code>/=</code> , <code>//=</code> , <code>%=</code> , <code>??=</code>	Assignment

Table 4.2: Operator precedence (highest to lowest)

Chapter 5

Variables & Scope

5.1 Variable Declarations

```
var name = "Alice"           # Inferred type
var age: int = 30             # Explicit type annotation
var items = [1, 2, 3]         # List
var config = { | debug = true | } # Record
```

Note

Variables must be declared with `var` before use. Assigning to an undeclared name is an error.

Inferred types follow reassignment

A variable declared without an annotation takes its type from what it holds, and reassignment updates it. A use before the reassignment still sees the earlier type, so the rule is positional rather than whole-of-scope:

```
var x = "hello"
$x.Length           # 5 - a string
$x = [1, 2, 3]
$x.Length           # 3 - now an array
```

An *annotated* variable does not re-infer. The annotation is a contract, so an assignment that disagrees with it is an error to report rather than a new type to adopt:

```
var y: int = 10
$y = "nope"         # error: could not be converted to 'int'
```

A reassignment inside a conditional branch says nothing certain about the type after it, because the branch may not have run. The variable becomes dynamic there rather than adopting a type only one path produces.

5.2 Constants

The `const` keyword declares an *immutable binding*. The initialiser may be any expression, is evaluated once where the declaration appears, and the name is typed by the result. A constant cannot be reassigned, and cannot be redeclared in the scope that already binds it.

```

const MaxRetries = 3
const AppName = "Tosh"
const StartedAt = (date)      # any expression, evaluated once

echo $MaxRetries      # 3
# $MaxRetries = 5      # error: tosh.runtime.const_reassignment
# var MaxRetries = 5    # error: tosh.runtime.const_redeclaration

```

Constants are visible in the same scope and all nested scopes, exactly like `var`, and are resolved by variable lookup in the same way.

Note

`const` is deliberately *not* a compile-time constant. Earlier drafts of this specification described it as one, requiring a literal or constant expression and folding the value at use sites; the implementation never did either, and the binding form proved the more useful of the two — `const StartedAt = (date)` says exactly what it means. A compile-time constant remains a possible future addition under its own keyword, in which case the constant-expression rules would be literals, operators over them, and references to other such constants.

Immutability applies to the *binding*, not to the value it holds. A constant bound to a record or a list still permits its contents to be changed; what cannot change is which value the name refers to.

```

const Config = { | retries = 3 | }
$Config.retries = 5      # allowed --- the record is mutable
# $Config = { | retries = 5 | } # error --- the binding is not

```

Shadowing in a *nested* scope is not redeclaration and stays legal: an inner block or a function body may bind its own name without disturbing the constant outside it.

```

const X = 5
if (true) {
  const X = 6      # a different binding, inner scope
  echo $X          # 6
}
echo $X            # 5

```

5.3 Destructuring

5.3.1 Record Destructuring

```

var person = { | name = "Alice", age = 30, city = "Portland" | }

# Destructure specific fields
var { name, age } = $person
echo $name      # Alice
echo $age       # 30

```

Note

Record destructuring binds fields by name. Renaming and nested destructuring are not currently supported—extract fields individually when you need different variable names.

5.3.2 Array Destructuring

```
var items = [10, 20, 30, 40, 50]

var [first, second] = $items
echo $first          # 10
echo $second         # 20

# Skip elements with _
var [_, _, third] = $items
echo $third          # 30
```

5.3.3 Tuple Destructuring

Parentheses destructure positionally, exactly as brackets do, and read naturally beside the way a tuple is written:

```
var (x, y) = (1, 2)
echo $x          # 1

const (WIDTH, HEIGHT) = (80, 24)

func min-max() { return (3, 9) }
var (lo, hi) = min-max()

# Existing variables, no declaration
var a = 1
var b = 2
(a, b) = ($b, $a) # a swap: the value is read in full before either is written
```

A tuple's arity is checked; an array's is not. An array is a variable-length collection, so taking a prefix of one is a meaningful request and stays legal. A tuple has a fixed, declared shape, so naming the wrong number of targets for one is a miscount and raises `tosh.runtime.tuple_assignment_arity_mismatch`:

```
var [first, second] = [10, 20, 30, 40, 50] # fine - a prefix of an array
var (a, b) = (1, 2, 3)                     # error - the tuple has 3 elements
```

`const` applies to every name a destructuring binds, whichever bracket style is used, so `const (A, B) = (1, 2)` produces two constants and not two ordinary variables.

5.4 Spread Operator

The spread operator (`...`) works in three contexts:

5.4.1 Array Spread

```
var a = [1, 2, 3]
var b = [0, ...$a, 4] # [0, 1, 2, 3, 4]
```

5.4.2 Record Spread


```
var base = { | name = "Alice", role = "user" | }
var admin = { | ...$base, role = "admin", level = 5 | }
# { name = "Alice", role = "admin", level = 5 }
```

5.4.3 Function Splatting

```
var args = [1, 2, 3]
some-function ...$args      # Equivalent to some-function 1 2 3
```

5.5 Computed Properties

```
var key = "name"
var record = { | ($key) = "Alice", age = 30 | }
echo ($record.name)      # Alice

# Dynamic key from expression
var dynkey = "item-count"
var record2 = { | ($dynkey) = 42 | }
echo ($record2.item-count) # 42
```

5.6 Function References

Prefix a function name with & to get a reference to it:

```
func double(x) => $x * 2
var fn = &double
invoke $fn 21      # 42

# In pipelines
[1, 2, 3] | map &double # [2, 4, 6]
```

5.7 Scoping

TōSh uses **lexical scoping** with a scope stack. Each block (`{...}`) creates a new scope. Variables are visible in their declaring scope and all nested scopes.

5.7.1 Visibility Modifiers

Modifier	Behavior
(none)	Visible in current scope and nested scopes
shy	Private to the declaring scope (e.g. module-internal, class-private)
global	Visible across all scopes for the entire session
export	Visible when the containing module is imported

```
module Utils {
  shy var internal_counter = 0      # Only visible inside Utils
  export func get-count() => $internal_counter
}
```

```
global var DEBUG = false           # Visible everywhere  
}
```

Chapter 6

Control Flow

6.1 If / Else

```
if ($age >= 18) {  
    echo "Adult"  
} else if ($age >= 13) {  
    echo "Teenager"  
} else {  
    echo "Child"  
}
```

if can be used as an expression:

```
var label = if ($count == 1) { "item" } else { "items" }  
echo $"You have {$count} {$label}."
```

A condition may be a command call, written as it would be anywhere else. The call's value is what the condition tests:

```
if (is-dir $path) { echo "directory" } else { echo "not a directory" }  
  
func positive(n: int) -> bool => ($n > 0)  
while (positive $i) { ... }
```

The same holds for unless, while and until.

6.2 For / In

```
for item in [1, 2, 3, 4, 5] {  
    echo $item  
}  
  
for file in (ls | where _.Type == file) {  
    echo $"Processing: {$file.Name}"  
}  
  
# With ranges  
for i in 1..10 {  
    echo $"Step {$i}"  
}
```

6.3 While

```
var i = 0
while ($i < 10) {
  echo $i
  $i += 1
}
```

6.4 Until

until loops run *until* the condition becomes true (inverse of while):

```
var attempts = 0
until ($attempts >= 3) {
  echo "Attempt ${attempts + 1}"
  $attempts += 1
}
```

6.5 Switch / Case

```
switch ($status) {
  case "ok" { echo "All good" }
  case "warn" { echo "Warning" }
  case "error" { echo "Error!" }
  default { echo "Unknown status" }
}
```

Tip

switch does not fall through—each case is independent. No break is needed.

6.6 Match Expressions

match is a powerful pattern-matching expression. Arms use `pattern => result` syntax.

Pattern Forms

Form	Matches when
<code><literal></code>	Value equals the literal ("file", 42, true).
<code>_ <op> <value></code>	Comparison holds; <code>_</code> stands for the matched value. Supported: <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> , <code>==</code> , <code>!=</code> , <code>=~</code> , <code>!~</code> .
<code>_ is <type></code>	Runtime type check (<code>_ is string</code> , <code>_ is int</code>).
<code>_ (bare)</code>	Always matches; wildcard catch-all.
<code>default</code>	Always matches; conventional alternative to bare <code>_</code> .

Arm bodies may be a single pipeline expression (`pattern => expr`) or a block (`pattern => { ... }`). An optional guard clause `if ((<condition>))` after the pattern skips the arm when the guard is falsy even if the pattern matched.

The fat-arrow body also accepts the jump statements `return`, `yield`, `throw`, `break`, and `continue` directly, so generators and short-circuit returns can be expressed without an explicit block:

```
func emit-codes(events) {
  for $e in $events {
    match ($e.kind) {
      "ack" => yield $e
      "fatal" => throw $e.message
      default => continue
    }
  }
}
```

`match` is an expression: its value is the result of the matched arm. When used as a statement the result is discarded (side effects still apply).

Value patterns match by equality; comparison patterns use `_` as a placeholder for the matched value and support `>`, `>=`, `<`, `<=`, `=~`, and `is`:

```
# Value patterns (match by equality)
var kind = "file"
var label = match ($kind) {
  "file" => "it is a file"
  "dir"  => "it is a directory"
  default => "something else"
}

# Comparison patterns (use _ as a placeholder for the matched value)
var category = match ($score) {
  _ >= 90 => "A"
  _ >= 80 => "B"
  _ >= 70 => "C"
  default => "F"
}

# Type-check patterns
var typelabel = match ($x) {
  _ is string => "string value"
  _ is int    => "integer value"
  default    => "other type"
}
```

```
# Guards use 'if ((<condition>))' after the pattern
match ($n) {
  0          => echo "zero"
  _ > 0 if (($n is int)) => echo "positive integer"
  _ > 0      => echo "positive"
  default   => echo "negative"
}
```

Match with guards

6.7 Type Narrowing

Testing a value's type with *narrows* it: for the branch guarded by the test, the value is known to have the tested type, and its members are reachable without a cast. Both `if` and a `match` arm narrow, and they narrow identically:

```

class Node { prop Kind : string = "node" }
class Leaf extends Node { prop Value: int = 7 }

func describe(n: Node) -> string => match ($n) {
  _ is Leaf => $"leaf {$n.Value}"      # Value belongs to Leaf, not Node
  default  => $n.Kind
}

func viaIf(n: Node) -> string {
  if ($n is Leaf) { return $"leaf {$n.Value}" }
  return $n.Kind
}

```

Narrowing applies to the arm or branch only. Outside it the value has its declared type again, and a member that exists on neither the declared nor the narrowed type is still reported.

Note

This is what makes a visitor over a class hierarchy writable: every pass over an AST is a match whose arms test node types and then read the members those types add.

6.8 Try / Catch / Finally

```

# Plain catch - no variable bound
try {
  var data = read-file "config.json" | from json
  process $data
} catch {
  writeline "An error occurred while loading config"
} finally {
  writeline "Cleanup complete"
}

# Named catch - binds the thrown value or exception object
try {
  throw "something went wrong"
} catch (err) {
  writeline $"Caught: {$err}"
}

# Runtime errors expose a .Message field
try {
  var x: int = "not a number"
} catch (err) {
  writeline $"Unexpected: {$err.Message}"
}

```

catch accepts either no variable, catch { ... }, or one variable in parentheses, catch (err) { ... }. ToSh currently has one catch block per try statement; filter exception kinds inside the catch body with is, match, or regular conditionals.

6.9 Defer

defer schedules a block to run when the current scope exits, regardless of how it exits (normal completion, error, or early return). Multiple defers execute in LIFO (last-in, first-out) order:

```
func process-file(path) {
    var handle = open-file --read $path
    defer { close $handle }

    var data = read-to-end $handle
    # handle is automatically closed when function returns
    return $data
}
```

```
func example() {
    defer { writeline "third" }
    defer { writeline "second" }
    defer { writeline "first" }
}
example() # Output: first, second, third
```

Defer LIFO ordering

A defer is registered only when execution reaches its declaration. For example, a defer written after a return or an escaping throw is not registered. On scope exit, every registered cleanup is attempted exactly once in reverse registration order, even when a newer cleanup fails. Values yielded by cleanup pipelines are discarded; direct side effects such as file writes or explicit writer commands still occur. Values already produced by the ordinary scope body are preserved.

Failure handling during unwind is deterministic:

- If cleanup succeeds, the original completion, jump, cancellation, or failure resumes unchanged.
- If only one failure exists, ToSh rethrows that original exception without an aggregate wrapper.
- When failures compete, ToSh raises `ToshDeferAggregateException`. Its failure order is the scope-body failure first, when present, followed by cleanup failures in their actual LIFO execution order. Nested defer aggregates are flattened; unrelated aggregate and diagnostic exceptions are not.
- A cleanup failure supersedes a pending return, break, or continue. The displaced jump is control flow, not an entry in the failure list.

Cleanup runs in a cancellation-shielded context: cancellation that caused the scope to exit does not prevent a reached defer from receiving its one execution attempt. The original `OperationCanceledException` remains the outward cancellation result; any cleanup failures are retained in the public defer-failure metadata. A cleanup that may block should use an explicit timeout.

For compatibility, return, break, and continue executed by a deferred cleanup are local to that cleanup and suppressed. They neither replace the enclosing scope's pending exit nor prevent an earlier registered cleanup from running.

6.10 Break, Continue, Return, Throw

```
# break - exit a loop early
for item in $items {
    if ($item == "stop") { break }
    writeline $item
}
```

```

# continue - skip to next iteration
for item in $items {
    if ($item == "skip") { continue }
    writeline $item
}

# return - exit function with value
func find-first(items, predicate) {
    for item in $items {
        if ($predicate($item)) { return $item }
    }
    return null
}

# throw - raise an exception
func validate(value) {
    if ($value == null) {
        throw "Value cannot be null"
    }
}

```

throw also works as an *expression* in any value position—the false branch of a ternary, the right side of null-coalescing, or a match arm body. This lets you encode "this path must not be reached" inline:

```

# Throw in a ternary: one branch produces a value, the other throws
var x = ($valid ? $value : throw "validation failed")

# Throw in null-coalescing: fail immediately if a required value is absent
var conn = $connectionString ?? throw "Connection string is required"

# Throw in a match arm body
var label = match ($input) {
    _ is int    => $"int: {$input}"
    _ is string => $"str: {$input}"
    default    => throw $"Unexpected type: {typeof $input}"
}

```

6.11 Postfix Conditionals (if / unless)

A jump statement — return, yield, throw, break, or continue — may be followed by a *postfix conditional* that guards execution. The form is:

```

return 5 if $x == 9
return 5 if $x > 3 and $ready
return 8 unless $ready
yield $i if ($i % 2 == 0)
throw "bad" unless (validate $input)
break if $done
continue unless $found

```

Semantics:

- `stmt if cond` executes `stmt` only when `cond` is truthy. It is exactly equivalent to `if cond { stmt }`.

- `stmt unless cond` executes `stmt` only when `cond` is falsy. It is exactly equivalent to `if cond { } else { stmt }`; the original condition expression is preserved verbatim (no negation operator is inserted).

The condition is a full expression, so `return 5 if $x > 3` needs no parentheses; they remain available for clarity. The postfix form is recognised only when the `if` or `unless` keyword appears on the same logical line as the jump statement (no implicit statement boundary in between); this avoids ambiguity with a free-standing `if` block on the following line. Omitting the condition raises `tosh.parser.expected_postfix_condition`.

Postfix conditionals attach *only* to jump statements. Regular commands (`echo "hi" if $x`) and assignments (`var x = 1 if $y`) are not supported — use a block `if` instead.

Chapter 7

Functions

7.1 Basic Functions

```
func greet(name) {  
    writeline $"Hello, {$name}!"  
}  
greet "World"
```

7.2 Arrow Functions (Wrappers)

Arrow functions are single-expression functions. Inside arrow functions, \$1, \$2, etc. refer to positional parameters:

```
func double(x) => ($x * 2)  
  
# Positional parameters  
func add => ($1 + $2)  
add 3 4      # 7  
  
# As aliases  
func cls => clear  
func ll => ls -la
```

7.3 Anonymous Functions (Lambdas)

```
# Expression lambda  
var fn = func(x) => ($x * 2)  
  
# Block lambda  
var process = func(x) {  
    var result = ($x * 2)  
    return $result + 1  
}  
  
# Inline in pipelines  
[1, 2, 3] | map func(x) => ($x * 2)    # [2, 4, 6]
```

Implicit Pipeline Lambdas

In many pipeline commands, you can use block syntax with `_` as the current item instead of writing an explicit lambda:

```
[1, 2, 3] | map { _ * 2 }           # Equivalent to above
ls | where { _.Size > 1mb }       # Block predicate
ls | sort { _.Modified }          # Block key selector
```

7.4 Typed Parameters

```
func connect(host: string, port: int) {
    writeline $"Connecting to ${host}:${port}"
}

# Optional parameters (with ?)
func greet(name: string, title?: string = "friend") {
    writeline $"Hello, ${title} ${name}!"
}
greet "Alice"                # Hello, friend Alice!
greet "Alice" "Dr."          # Hello, Dr. Alice!

# Rest parameters (with ...)
func print-all(items...) {
    for item in $items {
        writeline $item
    }
}
print-all 1 2 3 4 5
```

Parameters may also have default values. Default expressions are evaluated when the argument is omitted:

```
func connect(host: string, port: int = 443) {
    echo $"${host}:${port}"
}

connect "example.com"          # port defaults to 443
connect "localhost" 8080
```

Default-Value Semantics

Defaults follow one protocol for functions, lambdas, methods, and constructors:

- A default is evaluated only when the caller supplies neither a positional nor a named argument for that parameter, and it is re-evaluated on every such call — never captured once at declaration time.
- Defaults evaluate in the callable's own lexical scope, at call time, so a default that reads an outer variable observes that variable's value when the call happens.
- Evaluation proceeds left to right, and a default may reference any *earlier* parameter that is already bound. Referring to the parameter's own name, or to a later parameter, raises `tosh.runtime.unknown_variable`.
- Overload resolution never evaluates a default; only the selected overload's defaults run, so a losing candidate produces no side effects.

- An evaluated default is converted through the parameter's annotation and refinement exactly like a supplied argument. A default that cannot be converted raises `tosh.runtime.parameter_default_conversion_failed`.

```
func step(a: int, b: int = $a + 1, c: int = $b + 1) -> string {
  return "${a},{b},{c}"
}

step 1                # 1,2,3  -- defaults chain left to right
step(1, c = 99)       # 1,2,99 -- named argument, b still defaults
```

Note

Because defaults evaluate at call time, a mutable default expression produces a fresh value per call. `func add(item, into = [])` therefore starts from a new empty list on every call rather than sharing one list across calls.

7.5 Named Arguments and Callable Invocation

Function calls can use command-style syntax, parenthesized call syntax, or named arguments. Named arguments use `name = value` inside the call and may be supplied out of order.

```
func deploy(host, port = 22, user = "root") {
  echo "${user}@${host}:${port}"
}

deploy "server"
deploy(host = "server", user = "alice")
deploy(user = "alice", port = 2222, host = "server")
```

Callable values can be invoked with `invoke` or postfix call syntax:

```
var double = func(x) => ($x * 2)
invoke $double 21
$double(21)

var add3 = func(a, b, c) => ($a + $b + $c)
(curry $add3)(1)(2)(39)    # 42
```

7.6 Return Types

```
func add(a: int, b: int) -> int {
  return $a + $b
}

func get-name() -> string { return "TōSh" }
```

Implicit Pipeline Return

A function does not need an explicit `return` to produce a value. All pipeline output emitted by the function body is collected and yielded as the function's result when the function exits normally. If a `return` is reached, its operands replace any buffered output:

```
func doubled-items(items) {
    $items | each { _ * 2 }    # output flows out; no 'return' needed
}

var results = (doubled-items [1, 2, 3])    # [2, 4, 6]
```

Generator functions (those containing `yield`) stream values progressively instead of buffering them—see Section 7.10.

7.7 Function Overloading

Functions can be overloaded by arity and typed parameters:

```
func greet() { echo "Hello!" }
func greet(name) { echo $"Hello, {$name}!" }
func kind(value: int) { echo "integer" }
func kind(value: string) { echo "string" }

greet                # Hello!
greet "World"        # Hello, World!
kind 42              # integer
kind "hi"            # string
```

7.8 Recursion Depth

TōSh protects the host process with a limit on active ToastScript execution frames. The default and highest supported limit are 128 frames. Functions, methods, lambdas, constructors, and nested `eval`/source execution participate in the same asynchronous-flow-local count; compiled ToastScript uses the same protocol.

Exceeding the limit raises the stable `tosh.runtime.recursion_limit_exceeded` diagnostic. It does not terminate the process, and an interactive session remains usable after the diagnostic is handled.

The limit may be lowered for a session or persisted in `config.tosh`:

```
$tosh.Config.Shell.MaxRecursionDepth = 64
```

Accepted values are 1 through 128. TōSh does not currently perform tail-call elimination, so deeply recursive algorithms should be rewritten as loops when practical.

7.9 Operator Overloading

Classes may overload binary operators by declaring methods whose names are the operator symbol itself. The method receives the right-hand operand as its single argument; `$this` refers to the left-hand operand.

```

class Vec {
  prop X: int = 0
  prop Y: int = 0

  func +(other) { var r = new Vec(); $r.X = $this.X + $other.X; $r.Y = $this.Y +
    $other.Y; return $r }
  func *(scalar) { var r = new Vec(); $r.X = $this.X * $scalar; $r.Y = $this.Y *
    $scalar; return $r }
  func ==(other) { return $this.X == $other.X and $this.Y == $other.Y }
}

var a = new Vec(); $a.X = 1; $a.Y = 2
var b = new Vec(); $b.X = 3; $b.Y = 4
$a + $b           # Vec { X: 4, Y: 6 }
$a * 3            # Vec { X: 3, Y: 6 }

```

The following operator symbols are accepted as method names:

Category	Symbols
Arithmetic	+, -, *, /, //, %, **
Comparison	==, !=, <, <=, >, >=
Match	=~, !~

7.9.1 Resolution order

When evaluating `a OP b` where either operand is a class instance:

1. If the left operand defines an `OP` method, it is invoked with `b` as the argument.
2. Otherwise, if the right operand defines an `OP` method, it is invoked with `a` as the argument (note: `$this` is still the right operand).
3. Otherwise, the standard built-in operator semantics apply, which typically raises a runtime error if neither side is a supported type.

7.9.2 Limitations

- Unary operators (prefix `-`, `not`) are not currently overloadable. Only the binary forms above are accepted as method names.
- Compound-assignment operators (`+=`, `-=`, etc.) are not overloadable directly; they desugar to the binary form, so defining `+` also covers `+=`.
- Indexer-style access (`a[i]`) is parser-level wired but not yet a spec-level guarantee; prefer named methods until indexer semantics stabilise.

7.10 Generator Functions

A function that contains a `yield` statement is a *generator function*. Each call returns a `LazySequence`—values are produced on demand as the sequence is consumed. A generator may yield any number of times and may mix `yield` with normal control flow.

```

func evens-up-to(n: int) {
  var i = 0
  while ($i <= $n) {
    yield $i
    $i += 2
  }
}

```

```

}

evens-up-to 10 | collect      # [0, 2, 4, 6, 8, 10]
evens-up-to 20 | first 4     # 0, 2, 4, 6

```

You can annotate the return type as `LazySequence` to make intent clear:

```

func fibonacci() -> LazySequence {
    var a = 0
    var b = 1
    while (true) {
        yield $a
        var next = $a + $b
        $a = $b
        $b = $next
    }
}

fibonacci | first 8          # 0, 1, 1, 2, 3, 5, 8, 13

```

Known Limitation

Generator functions with unbounded `while (true)` loops do not yet cooperate with early-termination commands such as `first`. The pipeline will run indefinitely. Use a finite upper bound in the loop condition, or reach for built-in lazy sources (`iterate`, `recur`, `cycle`, `repeat`, infinite ranges `1..`) which do support early termination.

Tip

Finite generator functions integrate naturally with the full pipeline command set: `first`, `skip`, `where`, `map`, `collect`, etc.

7.11 Asynchrony

`async` and `await` are *commands*, not keywords—they are resolved by the command dispatcher like any other builtin, and they do not appear in the keyword list for that reason.

`async` starts a callable or block in the background and returns a future handle; `await` waits for it and replays its outputs:

```

var f = async { sleep 0.2; echo done }
await $f                                # done

```

Awaiting CLR Tasks

`await` also accepts a CLR Task, `Task<T>`, `ValueTask`, or `ValueTask<T>`, so .NET asynchronous methods work the way a .NET reader expects. A task-returning call yields a task, and you await it—the same explicit model as C#:

```

using System.Net.NetworkInformation

var p = new Ping()
var reply = await ($p.SendPingAsync("8.8.8.8", 1000))
echo $reply.Status                    # Success

```

Because a task is a value, work can overlap—start several, then await each:

```
var a = $client.GetStringAsync($first)
var b = $client.GetStringAsync($second)

var one = await $a
var two = await $b           # both requests were already in flight
```

A method declared to return plain Task produces no value, so awaiting it emits nothing:

```
await (System.IO.File.WriteAllTextAsync($path, $text))
```

The Task helpers are reachable, because a generic method infers its type arguments from the arguments supplied, as C# does:

```
var a = Task.FromResult(1)
var b = Task.FromResult(2)
echo (await (Task.WhenAll($a, $b))) # both results, in order
```

Inference here is narrower than the compiler's: there is no best-common-type step, so a type parameter used in more than one position must bind to one type or a base of it. A call that supplies nothing to infer from—`Array.Empty()`, `Enumerable.Empty()`—reports `Cannot infer type argument 'T' naming the parameter it could not supply`, rather than claiming the method has no such overload.

Where inference cannot reach, write the type arguments:

```
Array.Empty<int>()           # an empty int[]
Enumerable.Empty<string>()
Task.FromResult<int>(7)
Tuple.Create<int, int>(1, 2)
```

An instance call takes them in the same position, `$x.m<int>(...)`.

The list must be glued to the method name and closed before the parenthesis, which is what tells it apart from a comparison: `$A < 5` is still an ordinary `<`.

Note

`await` unwraps as many layers as it finds, so a block whose last expression is an asynchronous call needs only one `await`:

```
var f = async { $p.SendPingAsync("8.8.8.8", 1000) }
var reply = await $f           # a PingReply, not a Task
```

This is a deliberate difference from C#, where the equivalent needs `Task.Unwrap`. A future whose value is a task has no use, and leaving it unflattened is a trap.

A failed task raises its *own* error, not the `AggregateException` wrapper the CLR would surface, so `$e.Message` inside `catch` is the message the operation actually failed with:

```
try {
    await (System.IO.File.ReadAllTextAsync("/nope.txt"))
} catch (e) {
    echo $e.Message           # Could not find a part of the path '/nope.txt'.
}
```


Warning

Reading `.Result` or calling `.GetAwaiter().GetResult()` on a task also works, and *blocks the calling thread*. Prefer `await`; the blocking forms can deadlock.

An un-awaited task displays as what it is, so a forgotten `await` is visible rather than mysterious:

```
Sp.SendPingAsync("8.8.8.8", 1000)    # Task<PingReply> (pending)
```

7.12 Event Handlers

Functions can be registered as event handlers:

```
func onDirChange(event) handles DirectoryChanged {
    writeline $"Changed to: {$event.NewDirectory}"
}

# With priority and guard
func onBigCommand(event) handles CommandCompleted priority 10
    when { $event.Duration > (timespan 1s) }
{
    writeline $"Slow command: {$event.Duration}"
}

# One-time handler
func onFirstStart(event) handles SessionStarted once {
    writeline "Welcome to your first session!"
}
```

The last value emitted by a `when` block is tested using the canonical truthiness rules in Table 3.2. A block that emits no values therefore has a falsy guard.

`priority` is an ordering rank, and *lower numbers run first*: a handler declared `priority 1` runs before one declared `priority 10`. Handlers that declare no priority run after every handler that declares one, in registration order.

Warning

The number is a position in a queue, not an importance. `priority 10` runs *later* than `priority 1`, which is the opposite of how the word reads in most systems — give the handler that must run first the smallest number.

7.13 Doc-Comments

Functions, classes, properties, and methods accept documentation comments immediately above their declaration. A doc-comment line begins with `##` (double hash). At runtime, `help <name>` renders the doc-comment:

```
## Adds two numbers together and returns the sum.
## @param=a The first operand.
## @param=b The second operand.
## @returns The sum as an int.
## @example
##   add 3 4    # 7
func add(a: int, b: int) -> int {
```

```
    return ($a + $b)
}
```

The supported tags are:

Tag	Purpose
@param=<name> <desc>	Document a named parameter. @param <name> <desc> is accepted as an equivalent spelling.
@returns <desc>	Describe the return value.
@example	Lines following this tag (still prefixed ##) are shown verbatim as examples in help output.
@deprecated <message>	Flag the item as deprecated; help renders a visible warning.
@see <ref>	Cross-reference another command or function.
@since <version>	First release in which the item appeared.
@throws <type> <desc>	Document an exception path the function may raise.

Tip

All tags are optional. A doc-comment containing only prose (no tags) is perfectly valid and will display as the item's description.

Warning

is a doc-comment, not an ordinary comment. Commenting code out with ## turns it into documentation, and it will appear in help and -help output. Use a single # to comment code out.

Naming an input

@param, @arg and @flag all document one named input and behave identically; they differ only in reading naturally beside the declaration they describe. A function takes parameters, while a script or a subcommand takes args and flags. Each accepts either separator — @arg=name <desc> and @arg name <desc> are the same tag.

Scripts and -help

A script that declares inputs with arg or flag answers -help (and -h) by describing itself, without running its body. The summary comes from the file-level doc-block, and each input is listed with the description written above it:

```
## Greets a person by name.

## Who to greet.
arg name: string

## Shout the greeting.
flag loud: bool = false

echo $"Hello, {$name}"
```

```
$ greet.tosh --help
Greets a person by name.
```

```
Usage: greet.tosh [options] <name>
```

Arguments:

```
name string Who to greet.
```

Options:

```
--loud bool Shout the greeting.
```

Required arguments are written `<name>`, optional ones `[name]`, and a rest argument `[name...]`. A script that declares its own help or `h` flag keeps it — the built-in answer is a default, never an override — and arguments after a bare `-` are data rather than a request for help.

Subcommands

A subcommand may document its inputs in its own doc-block, or leave each declaration to describe itself. Where both do, the subcommand's block wins, so one header can describe everything a subcommand takes:

```
## @summary Build, version-bump, and package.
## @arg=target What to build.
## @flag=fast Skip slow steps.
subcommand publish {
  arg target: string = "all"
  flag fast: bool = false
}
```

Overriding is per input rather than wholesale: an input the block does not mention keeps the description written above its declaration.

7.14 Visibility Modifiers

```
shy func internal-helper() { ... }      # Private to scope
global func available-everywhere() { ... } # Session-wide
export func public-api() { ... }        # Exported from module
```

7.15 Functional Composition

```
var inc = partial &add 1                # Partial application
var curried = curry &add3                # Currying
var composed = compose $inc &double     # Composition: double(inc(x))
invoke $composed 5                      # double(inc(5)) = 12
```

Chapter 8

Runes and Quoting

8.1 Rune Definitions

Runes are macro-like shell commands whose arguments are captured lazily as syntax thunks. They are useful for control-flow helpers that should decide when, whether, or how often to evaluate their arguments or body blocks.

```
rune do-twice(body) {  
    $body  
    $body  
}  
  
do-twice { writeline "hi" } # hi, hi
```

A rune can be invoked like any command, including with block arguments:

```
rune retry(count, body) {  
    for i in 1..$count {  
        try {  
            $body  
            return  
        } catch (err) {  
            if ($i == $count) { throw $err }  
        }  
    }  
}
```

8.2 Rune Hygiene

Runes are sealed by default. A sealed rune evaluates in a hygienic scope so variables declared inside the rune do not leak back to the caller. Mark a rune `leaky` when it intentionally writes bindings into the caller scope.

```
rune internal() {  
    var hidden = 42  
}  
  
leaky rune export-var() {  
    var visible = 42  
}
```

`fixed` and `lazy` are accepted rune modifiers. `lazy` is the default evaluation mode; `fixed` marks the captured argument behaviour as fixed for future expansion/runtime policy.

8.3 Quote

`quote { ... }` captures syntax as a first-class quoted value instead of evaluating it immediately. This is primarily useful inside runes for source inspection and AST-like metaprogramming.

```
rune show-source(expr) {  
  var src = (quote { $expr })  
  echo $src  
}  
  
show-source (1 + 2)           # contains "1 + 2"
```

8.4 Built-In Runes

The runtime currently provides `unless` as a built-in rune:

```
unless false { echo "runs" }  
unless true  { echo "skipped" }
```

Chapter 9

Modules & User-Defined Types

9.1 Using Statements

Import CLR namespaces for shorter type references:

```
using System.IO           # Import namespace
using System.IO = IO      # Aliased import
```

using is lexically scoped—it only applies in the block where declared.

9.2 Require Statements

Load scripts or modules:

```
# Import everything the file exports
require "./utils.tosh"

# Import specific module
require Inventory from "./toastlib.tosh"

# With alias
require Inventory from "./toastlib.tosh" as Inv

# Multiple modules
require { Reporting, Utilities } from "./toastlib.tosh"
```

require imports exports; it does not run the file in your scope. A required file is executed once per session in a scope of its own, and only declarations marked `export` are brought into the caller. Anything else stays private to the module—which is what makes `export` mean something.

```
# lib.tosh
func helper() { return "private to this file" }
export func shared() { return "importable" }
export var VERSION = 2

# main.tosh
require "./lib.tosh"
shared           # "importable"
echo $VERSION    # 2
helper           # error - not exported
```

Use `source` when you do want a file's every declaration in the current scope; that is the difference between the two. A file required for its exports that declares none raises `tosh.runtime.require_exports_nothing`, since importing nothing at all is never the intent and used to happen in silence.

How a relative path is resolved

Both `require` and `source` read a relative path as *beside the file doing the loading*, not beside the working directory, so a script that loads a sibling behaves the same however it is invoked:

```
# ~/tools/report.tosh
source "./format.tosh"      # always ~/tools/format.tosh
require "./stats.tosh"     # always ~/tools/stats.tosh
```

The rule follows the file being executed, so a sourced script reaches its *own* siblings rather than those of whatever sourced it.

The two differ in what happens when the path is not there. `require` stops. `source` then tries the working directory, so a path deliberately written relative to where the shell was invoked still resolves, and a file that is simply missing is reported against the directory the reader expects. Absolute paths and `~` paths are unaffected by either rule.

9.3 Native Library Loading

Load and bind native (unmanaged) libraries:

```
bind native "./libcrypto.so" as Crypto {
  func sha256(data: ptr, len: uint64) -> ptr
}
```

A bind block may also be written inside a module or a class body, so the library path is stated once and the bound functions hang off the type that wraps them:

```
export hermit class SystemFacts {
  shy bind native "libc.so.6" {
    func uname(out buf: UtsName) -> ok
    func sysconf(name: int) -> count
  }

  shared prop Kernel: string => SystemFacts.uname().release
  shared prop PageSize: long => SystemFacts.sysconf(30)
}
```

Native members inside a class are always static, and default to `shy` — the reverse of `func` — so the raw ABI surface stays hidden behind the typed members written over it. `proud` bind opts back out.

For a single binding that does not justify a block, `raw func` names its library inline with `from`:

```
raw func sysconf(name: int) -> count from "libc.so.6"
```

9.3.1 Parameter Modes

Parameters default to `in` (pass by value). `ref` passes a managed reference the callee both reads and writes. `out` is written by the callee and read by nobody, so it is *engine-allocated and does not appear at the call site*:

```
bind native "libc.so.6" as LibC {
  func gettimeofday(out tv: Timeval, nint) -> int
}

var r = LibC.gettimeofday(0)  # one argument, not two
$r.tv.tv_sec
```

`cstring` may be passed by reference, which is C's `char**`: the callee replaces the pointer rather than the characters.

```
bind native "libc.so.6" as LibC {
  func strtol(s: cstring, out endptr: cstring, base: int) -> long
  func strsep(ref stringp: cstring, delim: cstring) -> cstring
}

var r = LibC.strtol("42abc", 10)
$r.ReturnValue    # 42
$r.endptr         # "abc"
```

The pointer the callee leaves is *copied* into a string and the bytes behind it are left alone: `TōSh` frees only memory `TōSh` allocated, because the callee's allocator cannot be known from a shared object. A caller who must free the callee's buffer declares `ptr` and frees it explicitly. A managed string cannot be passed by reference, since writing one back would mean guessing the encoding as well as the allocator.

Use `out` when the callee fills a structure and reads nothing (`sysinfo`, `uname`, `gettimeofday`); use `ref` when it reads and writes the same memory (`poll`, `select`, `ioctl` with `TCSETS`). Passing a native buffer to a `ref` parameter writes the result back into unmanaged memory.

9.3.2 Variadic Parameters

A binding may end in `...`, C's variadic tail:

```
bind native "libc.so.6" as LibC {
  func snprintf(dst: ptr, n: long, fmt: cstring, ...) -> int
}

LibC.snprintf($buf, 256, "i=%d s=%s f=%.1f", 42, "hi", 2.5)
```

The tail is whatever the call site passes, so the call signature is built from the actual arguments and cached per shape. C's default argument promotions apply, because the callee reads them: anything narrower than `int` is passed as `int`, and `float` as `double`. Nothing may follow `...`, and the fixed parameters before it are still required.

9.3.3 Return Conventions

A native call declares how success is decided. This is orthogonal to what a successful call yields.

Return	Meaning
-> ok	int; 0 is success, anything else throws
-> count	>= 0 is success <i>and is the value</i> ; ssize_t
-> int count	the same, read at a stated width (int, long)
-> auto	no checking; project the out parameters
-> T where (_)	custom predicate over the return value
-> T	raw, unchecked

Bare count marshals the return as `ssize_t`, which is pointer-sized. Bound to a function that really returns `int`, a `-1` failure is read at the wrong width and can arrive as `4294967295` — non-negative, so the failure goes undetected. Whether that happens depends on what the callee left in the upper half of the return register, so it varies by library. State the width when the C function does not return `ssize_t`:

```
func read(fd: int, buf: ptr, n: long) -> count      # ssize_t
func open(path: cstring, flags: int) -> int count   # int32_t, -1 detected
```

where is the same keyword, placeholder and meaning as a refinement type, with `_` bound to the native return value. `ok` and `count` are that clause with its two commonest predicates given names — the convention is still stated explicitly at the declaration site.

A checked call spends its return value on the success decision, so the out parameter becomes the result: with exactly one, it is yielded directly; with several, a record keyed by parameter name; an unchecked call yields a record carrying `ReturnValue` alongside them. With no out parameters at all, a checked call yields *nothing* — the return value was spent on the success decision and there is no result left to project, so `LibC.chdir("/tmp")` emits no pipeline object. `count` is the exception in every position: its return value is a genuine count that happens to double as the error signal, so it is still yielded.

```
bind native "libc.so.6" as LibC {
  func sysinfo(out info: SysInfo) -> ok
}

var info = LibC.sysinfo()      # a SysInfo, not a record wrapping one
```

Failure throws a `NativeError` carrying `Errno`, `ErrnoName`, `Symbol`, `Library` and `Result`, rendered with the offending symbol and a `strerror` message.

9.3.4 Buffers and Arrays

`buffer[n]` collapses C's output-string idiom — always an adjacent (pointer, length) pair — into one parameter that expands to both ABI arguments and yields a decoded string:

```
bind native "libc.so.6" as LibC {
  func gethostname(out name: buffer[256])          -> ok
  func readlink(path: string, out buf: buffer[4096]) -> count
  func getloadavg(out avg: double[3], count: int)  -> int where (_ == 3)
}
```

With `-> count` the returned count supplies the buffer's true length rather than scanning for a NUL, which is what makes `readlink` safe — it does not terminate its output. A typed `T[n]` out-parameter is engine-allocated and comes back as an array.

9.3.5 Calling Conventions

`callconv` is a trailing clause on the function it applies to:

```
bind native "user32.dll" as User32 {
    func MessageBoxW(nint, string, string, uint) -> int callconv stdcall
}
```

It defaults to `cdecl`; `stdcall`, `thiscall`, `fastcall` and `winapi` are also accepted. An unrecognised name is an error rather than a silent fallback.

9.3.6 Type Restrictions

`cstring` is a borrowed NUL-terminated C string; `nint` / `nuint` (or `ptr` / `uptr`) are pointer-sized values. Plain `string` works as a parameter but not as a return type, which needs explicit ownership semantics. `out` and `ref` string parameters are rejected for the same reason and need an explicit pointer type.

9.4 Raw Structs

A raw struct declares a C memory layout. It is deliberately a separate declaration kind from `struct`: a `tosh struct` is a dictionary-backed object model with computed properties and refinement-typed fields, whereas a raw struct is a byte layout that becomes a real sequential-layout CLR type.

```
raw struct SysInfo {
    uptime:    long
    loads:     ulong[3]
    totalram:  ulong
    procs:     ushort
    totalhigh: ulong
    mem_unit:  uint
}
```

The emitted type is nameable anywhere a native interop type is expected, so `size-of`, `offset-of`, `alloc`, `read-buffer` and native signatures all accept it.

Padding is never declared. Sequential layout aligns each field naturally, exactly as a C compiler does, so a declaration transcribes the header and omits its `pad` / `__pad0` / `__glibc_reserved` members. In the example above `totalhigh` lands at offset 88, not 82, without any padding field being written.

Array counts are part of the ABI and cannot be inferred, since `char[65]` and `char[256]` are different layouts. `cstring[n]` is an inline character buffer; `τ[n]` is an inline array.

```
raw struct UtsName {
    sysname:    cstring[65]
    nodename:   cstring[65]
    release:    cstring[65]
    version:    cstring[65]
    machine:    cstring[65]
    domainname: cstring[65]
}
```

`bool` is rejected in favour of `byte`: default marshalling maps it to a four-byte Win32 `BOOL` rather than a C `_Bool`, which would shift every subsequent field.

Two optional header clauses, both rarely needed. `pack n` sets the packing; `size n` asserts the computed size, turning a mis-transcribed struct into a declaration-time error rather than silent memory corruption.

```
raw struct Packed pack 1 { a: byte; b: long }    # 9 bytes, not 16
raw struct Checked size 16 { a: long; b: long }
```

`raw union` places every field at offset zero.

Fields may carry defaults. These apply when `tosh` constructs a value and to `ref` parameters, but never to an `out` parameter, which arrives zero-filled — a CLR struct cannot carry an initializer the marshaller would run, and seeding one would mask a callee that failed to write.

```
raw struct PollFd {
  fd:      int    = -1
  events:  short  = 1
  revents: short
}
```

9.5 Raw Callbacks

A `raw callback` declares the C function-pointer type a native signature names when the library calls back into `TōSh`. Like `raw struct` it is a named declaration rather than an inline type, which is also how C spells these — nearly always a typedef.

```
raw callback Comparator(a: ptr, b: ptr) -> int

bind native "libc.so.6" as LibC {
  func qsort(base: ptr, count: nuint, size: nuint, compare: Comparator) -> void
}

func ascending(a, b) {
  return (read-buffer int32 $a) - (read-buffer int32 $b)
}

LibC.qsort($buf, 8, 4, &ascending)
```

The name resolves anywhere a native interop type is expected, and a function reference (`&name`) is what satisfies it. `callconv` applies as it does to a `func`.

9.5.1 Exactly One Value

A `TōSh` function yields a stream; a C callback returns a single value. A callback that produces anything other than exactly one value is an error rather than a silent choice, because the failure it would otherwise cause — a mis-sorted array from a comparator, an ignored event — is invisible at the call site. A `void` callback is the exception: it must produce nothing.

Note that `writeline` prints rather than yields, so ordinary tracing inside a callback does not trip this rule; bare expressions do.

9.5.2 Threading

The engine is single-threaded — its scope stack is a plain stack, and forks clone rather than lock — so a callback is only supported when the library invokes it on the same thread that

called in. That covers the common cases: comparators, GLFW's window and error callbacks, and anything dispatched from inside a call the script itself made.

A callback arriving on a library's own thread (SDL's audio callback, for example) does *not* run. It cannot throw either, because unwinding through native frames is undefined behaviour; instead the violation is recorded and rethrown once the native call returns, naming both threads. Failures raised inside a callback are carried the same way, so a `catch` around the native call sees them.

9.5.3 Restrictions

`ref` and `out` parameters are rejected: the value would have to be written back into the caller's memory after the body ran, and a silently ignored `out` is worse than a rejected one. Declare a pointer and use `read-buffer` / `write-buffer` instead. A `buffer[n]` parameter is likewise rejected — it is an inbound decoding convention with nothing to decode here. The success conventions `ok` and `count` decide whether a *call* failed, so they are rejected on a callback, whose return value is one it produces.

Callbacks are a control plane, not a data plane. Each invocation re-enters the engine, which is far more expensive than the C call around it — fine for window events and comparisons, unsuitable for per-sample or per-vertex work.

A thunk passed to a library is kept alive for as long as that library is loaded, so a callee that stores it (GLFW does) keeps working after the registering call returns.

9.6 Module Definitions

```
module Utilities {
  shy func internal-helper() { ... }
  export func banner(title: string) {
    writeline $"=== {$title} ==="
  }
  func default-accessible() { ... }
}

# Usage
Utilities.banner("Report")
```

Modules can contain nested types:

```
module Inventory {
  enum StockState { Healthy, Low, Out }
  record Item(Name: string, Quantity: int)
  class Shelf(label: string) { ... }

  func sample-items() {
    return [
      new Item("Widget", 100),
      new Item("Gadget", 5)
    ]
  }
}

# Access via dot notation
var items = Inventory.sample-items()
var state = Inventory.StockState.Healthy
```

Module-Qualified Command Dispatch

Functions exported from a module may also be invoked in command position using `module.function` syntax—no `$` prefix, arguments separated by whitespace like any other command:

```
module Geometry {
  func area(r) { return 3.14159 * $r * $r }
}

Geometry.area 5           # 78.53975
```

This works for any module-name casing, including identifiers that contain hyphens (`kebab-mod.hi`), underscores, or mixed case. Nested modules resolve segment-by-segment (`outer.inner.fn`).

Module-Qualified Type Names

A type declared inside a module may be named from outside it by its qualified path, in every position a type name is accepted—including a variable’s type annotation. Nested modules resolve segment-by-segment, the same rule as command dispatch:

```
module Outer {
  module Inner {
    export type SmallInt = int where (_ >= 0 and _ <= 10) coerce 10
    export class Widget { prop Name = "w" }
  }
}

var a: Outer.Inner.SmallInt = 5      # 5
var b: Outer.Inner.SmallInt = 99     # 10 --- the refinement's coercion runs
var w: Outer.Inner.Widget = (new Outer.Inner.Widget())
```

The qualified name resolves to the same definition the unqualified name would, so a refinement type used as a qualified annotation still applies its predicate and its coercion, and still rejects a value it cannot coerce.

Modules That Shadow CLR Types

A module may be named after a CLR type—`Math`, `Console`—and inside it that name refers to the module. Its own exports win, and the shadowed type stays reachable: a member that the module does not export is looked up on the CLR type of the same name.

```
module Math {
  export func Clamp(a, b, c) { return "module-wins" }
  export func widest() { return Math.Max(3, 7) }
}

Math.Clamp(1, 2, 3)      # module-wins --- the export
Math.widest()            # 7 --- System.Math.Max, not shadowed away
```

This matters most inside the module’s own body, where the shadowing is unavoidable: a refinement type declared in `module Math` whose coercion calls `Math.Clamp` would otherwise have no way to reach `System.Math`. A member found on neither the module nor a shadowed type is still an error rather than a null.

Partial Modules

A module marked `partial` may be declared more than once; the declarations merge into one module. Every declaration must carry `partial`—extending a module that was not declared `partial` raises `tosh.runtime.partial_mismatch`, matching the rule for classes, records, and structs.

```
partial module Sys {
  export func alpha() -> string { return "from-a" }
}

partial module Sys {
  export func beta() -> string { return "from-b" }
}

Sys.alpha()           # from-a
Sys.beta()            # from-b
```

Parts merge in declaration order, and each part sees what earlier parts exported, so a later part can build on an earlier one rather than merely sit beside it:

```
partial module Sys {
  export func base-name() -> string { return "core" }
}

partial module Sys {
  export func label() -> string { return $"[{"base-name()}]" }
}

Sys.label()           # [core]
```

Splitting a Partial Declaration Across Files

Partial declarations—modules, classes, records, and structs alike—may be split across separate files and assembled by `require`. Both the bare and the named import form work, in any order:

```
# sys-core.tosh
partial module Sys {
  export func alpha() -> string { return "from-core" }
}

# sys-extra.tosh
partial module Sys {
  export func beta() -> string { return "from-extra" }
}

# main.tosh
require Sys from "./sys-core.tosh"
require Sys from "./sys-extra.tosh"

Sys.alpha()           # from-core
Sys.beta()            # from-extra
```

The parts share one export table rather than being copied, so every reference to the module observes the merged state regardless of which file declared the member being used.

A declaration *without* `partial` does not merge—it replaces any previous declaration of that name, which is what makes redefining a module or class at the prompt behave as expected. Members of the replaced declaration are gone. This is the reason `partial` is required rather than inferred.

9.7 Class Definitions

```
class Point(x: double, y: double) {
  prop X: double = $x
  prop Y: double = $y

  # Computed property (evaluated on each access)
  prop Magnitude => (Math.Sqrt(($this.X * $this.X) + ($this.Y * $this.Y)))

  func Distance(other) -> double {
    var dx = $this.X - $other.X
    var dy = $this.Y - $other.Y
    return (Math.Sqrt(($dx * $dx) + ($dy * $dy)))
  }

  shared func Origin() { return new Point(0.0, 0.0) }
}
```

```
var p = new Point(3.0, 4.0)
echo $p.Magnitude           # 5
echo (Point.Origin())       # Point

var q = new Point(6.0, 8.0)
echo ($p.Distance($q))      # 5
```

Using classes

Class features:

- **Primary constructor**—parameters after the class name. They are in scope for property initializers.
- **Explicit constructors**—a member named after the class, `ClassName(params) { ... }`. A class may declare several, and they overload on their parameter types.
- **Stored properties**—`prop Name: Type = value`.
- **Computed properties**—`prop Name => (expr)` (type annotation is inferred).
- **Accessor blocks**—`prop Name { get ... set ... }`. Each accessor takes either an arrow body (`get => (expr)`) or a brace body (`get { ... }`), the latter holding as many statements as it needs. Inside `set`, the incoming value is `$value`.
- **Methods**—`func Name(params) { ... }`.
- **Static methods**—`shared func Name(params) { ... }` (also accepts `static`).
- **Self-reference**—`$this` inside methods.
- **Visibility**—`shy` for private members.
- **Instantiation**—`new ClassName(args)`.

Reaching Members from Inside the Class

A member is always reached through a qualifier, including from the class's own code. An instance member is reached through `$this.`, a static one through the class name:

```
class Counter {
```

```

prop Count: int = 0
shared prop Limit: int = 10

func Step() { $this.Count = $this.Count + 1 } # instance: $this.
prop AtLimit => ($this.Count >= Counter.Limit) # static: class name
}

```

A bare name in a class body is not a member reference. It is resolved the way any other command head is—against builtins, functions, and `$PATH`—so `Count` inside `Counter` names no member and reports `tosh.bind.unknown_command`. The rule is uniform: a bare name fails from a static position and an instance one alike, and the diagnostic names the qualified form to write instead.

This is what allows a class to run shell commands in its methods without its own members shadowing them: a class declaring `prop git` can still call `git status`.

```

class Point(seed: double) {
  prop X: double = $seed # in scope here: '$seed' or 'seed'
  func Show() { return $seed } # not in scope: initializers only
}

var p = new Point(3.0)
echo $p.X # 3
echo $p.seed # error: no member 'seed'

```

A primary-constructor parameter is not a member

A primary-constructor parameter is in scope in a *stored* property initializer and in a later parameter's default, and nowhere else in the body. It does not become a member. Declare a property from it, as `prop X: double = $seed` above does, to reach the value from the rest of the class.

```

class P(x: int, y: int = 2) {
  prop A = ($x + $y) # stored initializer, and a later default: in scope
  prop B => $x # computed: evaluated per access, after construction
  func g() { return $x } # method body: same
}

```

Where a primary-constructor parameter reaches

The reason is when each runs. A stored initializer runs *while* the value is being built, with the parameters still bound; a computed property, an accessor block, a method body and a static initializer all run afterwards, and a constructor parameter is a local of the construction that was never stored. Referring to one from those positions reports that it is a constructor parameter and is not in scope, rather than that the variable is undeclared.

Member access is case-insensitive, so `$p.x` above reaches `x`. A parameter that differs from a property only in case is therefore easy to mistake for a reachable one; the two are unrelated.

Chaining to the Primary Constructor

A class may declare a primary constructor and explicit constructors together. When it does, an explicit constructor reaches the primary one with `$this(...)`, which must be the first statement in its body:

```

class Point(x, y) {
  prop X = $x
}

```



```

prop Y = $y

## Construct from a single scalar: the same value on both axes.
Point(size) { $this($size, $size) }
}

(new Point(2, 3)).X      # 2
(new Point(4)).Y         # 4

```

The chain binds the primary parameters *before* property initializers run, which is what lets `prop X = $x` work no matter which constructor was called. Without it, an initializer that reads a primary parameter fails when construction comes in through an explicit constructor, because the primary parameters were never bound.

`$this(...)` mirrors `$super(...)`: same position rule, same once-only rule, pointed at this class's own primary constructor rather than its base class. The explicit constructor's parameters are visible to the chain arguments—`$this($a + $b)` is the usual shape—and the arguments are checked against the primary signature exactly as a direct `new` would be.

A chain does not replace the constructor body; the rest of the body runs after it. Note also that chaining does not permit an ambiguous pair: a primary and an explicit constructor of identical signature remain a declaration error, because a call would still have two candidates and no way to choose.

Constructor Overloads

Constructors overload on their parameter *types*, not on how many parameters they take. Two constructors may share an arity as long as a reader and the resolver can tell them apart:

```

class G {
  prop X: string = ""
  G(n: int) { $this.X = $"int:{n}" }
  G(s: string) { $this.X = $"str:{s}" }
}

(new G(7)).X      # int:7
(new G("hi")).X   # str:hi

```

The primary constructor participates as one more overload, so it may sit beside explicit constructors of a different signature:

```

class H(x: int) {
  prop X: string = "primary"
  H(s: string) { $this.X = $"str:{s}" }
}

(new H(7)).X      # primary
(new H("hi")).X   # str:hi

```

Two constructors with an *identical* signature are rejected where they are declared, with `tosh.runtime.duplicate_constructor`—including the case where an explicit constructor repeats the primary one:

```

class C(x: int) {
  C(y: int) { }      # error: same signature as the primary constructor
}

```

The check is a declaration rule rather than a call-site one because such a class can never be constructed at that signature: the error belongs where the mistake is, not at every `new`.

9.7.1 Class Modifiers

Class-level modifiers appear before the `class` keyword and control the class's overall behaviour.

sealed Prevents the class from being inherited. A sealed class cannot appear as a base class.

hollow Marks the class as *abstract*. It cannot be instantiated directly; subclasses must override all hollow methods.

hermit Marks the class as *static-only*. All members are auto-promoted to shared; constructors are not allowed; the class cannot be instantiated.

strict Makes every property in the class *read-only* after initialisation, as if each were individually marked `fixed`.

partial Allows the class definition to be split across multiple declarations, including declarations in separate files assembled by `require`. All declarations must carry the `partial` modifier. Properties and methods are merged; duplicate property names are silently skipped; methods support overloading.

Example:

```
sealed class Config {
  prop Host = "localhost"
  prop Port = 8080
}

hollow class Shape {
  hollow func area() -> double { }
}

hermit class MathHelper {
  func square(x) { writeline ($x * $x) }
}
MathHelper.square(5)           # 25

strict class Point {
  prop X = 0
  prop Y = 0
}

partial class User { prop Name = "" }
partial class User { func greet() { writeline $"Hi, {this.Name}" } }
```

9.7.2 Member Modifiers

Member-level modifiers appear before `prop` or `func` inside a class body. Several accept a second, C#-familiar spelling; both forms are accepted and mean exactly the same thing, so a reader arriving from C# is not stopped by vocabulary. The ToastScript word is listed first throughout.

shy / **private** Hides the member from public inspection and external access.

shared / **static** Member belongs to the class rather than an instance.

fixed / **readonly** Makes a property read-only after initialisation.

vital / **required** Marks a property as required—construction fails without a value.

guarded / **protected** Restricts access to the defining class and its subclasses.

overrule / **override** Overrides an inherited method from a parent class.

lazy Defers the property initialiser until first access; the result is cached.

fading / **obsolete** Marks the member as deprecated; emits a warning to stderr on use.

local Restricts visibility to the defining assembly (internal).

raw Marks a method for unsafe/native interop.

proud / **public** Explicitly marks a member as public (no-op, members default to public).

hollow / **abstract** (on a method) Declares an abstract method that subclasses must overrule.

An empty body { } is required: `hollow func name() { }`.

Canonical aliases. Every flavored modifier has an equally-valid canonical alias drawn from the wider .NET / object-oriented vocabulary; either form is accepted at parse time and produces identical semantics. Use whichever reads better in context. The pairs are: `shy/private`, `proud/public`, `guarded/protected`, `shared/static`, `fixed/readonly`, `vital/required`, `overrule/override`, `hollow/abstract`, `fading/obsolete`, `hermit/static` (class-level). The same canonical/flavored equivalence applies to class-level modifiers (`abstract/hollow`, `static/hermit`).

Example:

```
class Product {
    vital prop Name: string
    fixed prop Id = 0
    lazy prop Metadata = load_metadata($this.Id)
    shy prop _cache = []
    guarded func validate() { echo "checking" }
    fading prop OldName = "use NewName"
}
```

9.7.3 Static Members

A property or method marked `shared` / `static` belongs to the class rather than to any instance, and is reached through the class name:

```
class Counter {
    static prop Total = 0
    static func describe() { return "counted" }
}

echo Counter.Total      # 0
Counter.Total = 5
echo Counter.Total      # 5
```

A static path carries no \$. `$name` refers to a *variable*; a bare dotted name refers to a *type*. That rule already governed reading (`Math.PI`, `Reactor.Fuel.Mox`), and it governs assignment identically: `Counter.Total = 5` writes a static, while `$counter.Total = 5` writes a member of the variable `$counter`.

Because the two spellings are the same shape, the decision cannot be made before anything runs. A dotted target left of `=` is resolved when the statement executes, and a variable of that name wins: with `var person = { | Name = "ada" | }` in scope, `person.Name = "toast"` reports `tosh.runtime.variable_reference_requires_dollar` and suggests `$person.Name`, exactly as reading `person.Name` always has. A head that names neither a type nor a variable reports `tosh.runtime.unknown_static_assignment_target`.

One slot per declaration. A static declared on a base class is a single value shared with every subclass, not a copy per subclass. Reading and writing agree on where it lives:

```

class Base { static prop S = 1 }
class Derived extends Base { }

Derived.S = 7
echo Base.S           # 7
echo Derived.S        # 7

```

The instance rules apply unchanged. A static property with a set accessor runs it instead of storing; one with only a get accessor is read-only; and `fixed static` refuses assignment after the declaration's own initialiser has run. A static that answered differently from an instance property would be a second set of rules for no reason.

```

class Config {
  fixed static prop Version = "1.0"
  static prop Raw = 0
  static prop Level {
    get { return Config.Raw }
    set { Config.Raw = $value }
  }
}

Config.Level = 3
echo Config.Level           # 3
Config.Version = "2.0"      # error: fixed and cannot be reassigned

```

Assignment reaches nested types and CLR statics by the same path: `Outer.Inner.v = 9` and `System.Environment.ExitCode = 2` both work, the path being split at the longest prefix that names a type. A CLR `const` or `readonly` field is reported as read-only rather than as missing, since it can still be read.

9.8 Generic Classes

A class declaration may introduce one or more *type parameters* inside angle brackets after the class name. Type parameters can be used anywhere a type annotation is accepted: primary-constructor parameters, property types, method parameter and return types, and the `extends` clause of a derived class.

```

class Box<T>(initial: T) {
  prop value: T = $initial
  func unwrap() -> T { return $this.value }
  func set(v: T) { $this.value = $v }
}

var bi = new Box<int>(42)
echo $bi.unwrap()         # 42

var bs = new Box<string>("hi")
echo $bs.unwrap()         # hi

```

Instantiation

`new ClassName<arg1, arg2, ...>(ctorArgs)` creates an instance whose type parameters are bound to the supplied CLR types. The type-argument list must match the class's declared

arity exactly; mismatches — too few, too many, an empty `<>` list, or omitting the angle brackets entirely on a class that declares type parameters — raise an instantiation error.

Strict (no-coercion) bindings

When a constructor parameter, method parameter, method return value, or property type is declared as a class type parameter (e.g. `x: T`), the engine performs a *strict* `InstanceOfType` check at each use site against the bound CLR type — it does *not* run the standard value-conversion path that applies to ordinary annotations. In particular:

- `new Box<string>(42)` rejects with `tosh.runtime.annotation_conversion_failed` rather than silently stringifying `42` to `"42"`.
- `new Box<double>(9)` rejects rather than widening `int` to `double`.
- Reassignment to `$this.value` where `value: T` is bound goes through the same strict check.

Type-parameters whose binding does *not* resolve to a known CLR type (rare; usually a forward-declared user type) are treated nominally — any value is accepted, and the binding only participates in display.

Inheritance

A subclass can pass type arguments through to its base, propagate its own type parameters, or fix the base's parameters to concrete types:

```
class _Point<T1, T2> {
  hollow prop X: T1
  hollow prop Y: T2
}

class Point2D<T1, T2> extends _Point<T1, T2> {
  override prop X: T1
  override prop Y: T2

  Point2D<T1, T2>(x: T1, y: T2) {
    $this.X = $x
    $this.Y = $y
  }
}

var p = new Point2D<int, int>(3, 4)
echo $p.X                                # 3
```

Bindings flow through the inheritance chain: when an instance of `Point2D<int, int>` accesses an inherited member declared on `_Point`, the engine resolves `T1` and `T2` against the base's type-argument list using the child's bindings.

Generic annotations

Anywhere a type annotation is accepted, the form `ClassName<arg1, arg2, ...>` refers to the user-defined generic class `ClassName`. Annotations with this shape are recognised before falling through to the CLR type loader (where multi-arg names like `Foo<int, int>` would otherwise produce a `Type.GetType` error treating commas as type/assembly separators). A value satisfies the annotation when it is a `ToshClassInstance` whose definition (or any ancestor in the inheritance chain) names `ClassName`; the per-instance type-argument strings are not currently validated against the annotation strings.

Compiled mode

Generic classes do not emit a CLR shell type at compile time. Instead, the compiler registers the class via source-replay and emits `new`-expressions through a typed host overload

`ToshHost.NewObject(typeName, bareTypeName, string[] typeArgs, object?[] args)` which delegates to the engine's reification path. This means generic classes behave identically in REPL and compiled execution.

Type-Parameter Constraints

A `where T: <Constraint>[, <Constraint>...]` clause may follow the class header to restrict acceptable type arguments. Built-in constraints are summarised below; multiple `where` clauses may follow one another to constrain different type parameters. Unknown constraint names are accepted conservatively (reserved for future user-defined constraints).

Constraint	Satisfied by
Numeric / Number / INumber	int, long, short, byte, signed/unsigned variants, float, double, decimal, Half, BigInteger
Add / Sub / Mul / Div	Any numeric type, or any CLR type with a matching <code>op_*</code> static method
Comparable	Any type implementing <code>IComparable</code>
Eq	Always satisfied (placeholder)

```
class Box<T>(initial: T) where T: Numeric {
    prop value: T = $initial
    func bump(amount) { $this.value = $this.value + $amount }
}

var bi = new Box<int>(10);    $bi.bump(5)      # 15
var bf = new Box<double>(1.5); $bf.bump(2.5)    # 4.0
# new Box<string>("hi") -> rejected: 'string' does not satisfy 'Numeric'
```

The same constraint names also work as right-hand operands for the `is` / `is-not` operators on values, so runtime checks reuse the same registry as compile-time generic-parameter validation:

```
var x = 42
$x is Numeric           # true
$x is Comparable       # true
"hello" is Numeric     # false
$x is-not Numeric      # false
```

9.9 Inheritance, Interfaces, and Traits

Classes can extend another TōSh class, fulfill one or more interfaces, and use one or more traits. `implements` is accepted as an alias for `fulfills`. Parent constructors can be supplied in the `extends` clause or called from the constructor with `$super(...)`. Construction proceeds exactly once per class layer, from the root base class to the most-derived class. The preferred form is `extends Base(args)`. A direct `$super(args)` call is retained as an alternative for an explicit constructor, but it must be that constructor's first executable statement and must not be combined with arguments in the `extends` clause. If neither form is present, TōSh implicitly invokes a matching zero-argument base constructor; a base that requires arguments produces a diagnostic.

```
interface Printable {  
    func print()  
}  
  
trait Named {  
    prop Name = "anonymous"  
    func label() { return $this.Name }  
}  
  
class Report(title: string) fulfills Printable uses Named {  
    prop Title = $title  
    func print() { return $this.Title }  
}  
  
var r = new Report("Daily")  
echo ($r is Printable)      # true  
echo ($r is Named)         # true
```

9.10 Interface Definitions

Interfaces declare required method signatures. They cannot be instantiated. When a class fulfills an interface, TōSh validates that the class has each required method by name and arity.

```
interface Runnable {  
    func run()  
    func stop()  
}  
  
class Job fulfills Runnable {  
    func run() { echo "running" }  
    func stop() { echo "stopped" }  
}
```

9.11 Trait Definitions

Traits describe reusable requirements and optional defaults. A trait may declare required properties/methods, or provide default property values and method bodies. Classes using the trait must supply the required members; any defaults not already present on the class are injected into the class.

```
trait Timestamped {  
    prop CreatedAt = date now  
    func age() { return ((date now) - $this.CreatedAt) }  
}  
  
class Note uses Timestamped {  
    prop Text = ""  
}
```

9.12 Union Definitions

Unions define tagged variant types. Unit variants (no fields) are accessed as static members. Variants with fields are constructed as static calls. Every union instance exposes `Variant` (also aliased as `Tag`) plus any fields declared on its variant.

```
union Result {
    Ok(value)
    Error(message)
    None
}

var success = Result.Ok(42)
echo $success.Variant      # Ok
echo $success.value        # 42

var failure = Result.Error("not found")
echo $failure.message      # not found

var empty = Result.None
echo $empty.Variant        # None
```

Pattern Matching on Unions

Use `switch` on the `Variant` string to dispatch on the active variant:

```
func describe(r) {
    switch ($r.Variant) {
        case "Ok"      { echo $"Success: {$r.value}" }
        case "Error"   { echo $"Failed: {$r.message}" }
        case "None"    { echo "No value" }
    }
}

describe (Result.Ok 100)      # Success: 100
describe (Result.Error "oops") # Failed: oops
describe Result.None          # No value
```

Type Checking

Union instances respond to `is` checks against their union type:

```
echo ($success is Result)      # true
echo ($success is-not Result) # false
```

9.13 Struct Definitions

Structs are shell-defined value types with positional fields and optional class style members. By default, struct fields are read-only after construction. Mark a struct `fluid` to make fields writable. `partial` structs can be split across declarations; every declaration must be marked `partial`.

```
struct Point(x: int, y: int) {
    func sum() { return ($this.x + $this.y) }
```



```

}

var p = new Point(3, 4)
echo $p.x
echo ($p.sum())           # 7

fluid struct Counter(value: int)
var c = new Counter(0)
$c.value = 1

```

9.14 Record Definitions

Records are lightweight named types with positional fields:

```

record Item(
  Name: string,
  Quantity: int,
  Category?: string = "General",
  Tint?: string = "White"
)

var widget = new Item("Widget", 50)
var custom = new Item("Gadget", 10, "Electronics", "Blue")
echo $widget.Name           # Widget
echo $custom.Category       # Electronics

```

Records feature value equality, optional fields with `?`, and default values.

9.15 Nested Types

A class body may declare a type as well as members. `enum`, `class`, `struct`, `record`, `union`, `interface` and `trait` are all accepted, and are written exactly as they would be at the top level:

```

class Reactor {
  enum Fuel : int {
    Mox      = 3
    Uranium = 8
  }

  prop Loaded: Fuel = Fuel.Mox
}

var r = new Reactor()
echo $r.Loaded           # Mox
echo Reactor.Fuel.Uranium # Uranium

```

Inside the class the qualification is unnecessary — the code is already in `Reactor` — so `Fuel` and `Fuel.Mox` resolve directly in property initialisers, method bodies and constructors, as the example above does. Outside it, the type is reached through the class that declares it.

A nested type is a static member of the class that declares it, so it is reached through that class — `Reactor.Fuel` — and its own members follow by ordinary access: `Reactor.Fuel.Uranium`. The name does not appear in the scope surrounding the class, which is the point of nesting; `Fuel` alone resolves only inside `Reactor`.

A qualified name is also a type name, so a nested class can be constructed and annotated with it, at any depth:

```
class Outer {
  class Inner { prop V = 7 }
}

var i: Outer.Inner = new Outer.Inner()
echo $i.V # 7
```

shy applies as it does to any other member: a shy nested type is visible inside its declaring class and refused from outside.

9.16 Enum Definitions

```
enum StockState { Unknown, Healthy, Low, Out }

enum Permissions : int {
  None = 0
  Read = 4
  Write = 2
  Execute = 1
}

var state = StockState.Healthy
if ($state == StockState.Low) {
  echo "Reorder needed"
}
```

9.16.1 Flag Enums

An enum declared `flags` is one whose members are meant to be combined. The bitwise operators over its members yield a value of the same enum rather than a number, and that value names itself from the bits it carries:

```
flags enum Permissions : int {
  None = 0
  Execute = 1
  Write = 2
  Read = 4
}

var access = Permissions.Read bor Permissions.Write
echo $access # Write, Read
echo ($access as int) # 6
echo ($access has Permissions.Read) # true
```

The name is composed from the underlying value in declaration order, so `band` and `bxor` name their results too, and a value whose bits are not exactly covered by the members falls back to the number.

Without `flags` the same expression yields the underlying integer: combining members is a claim about the type that the author has not made, so the result is a number and not a member. Combining members of two different enums is an error either way.

has tests that *every* bit of the right operand is set in the left, so a composite flag answers true only when it is wholly present. A zero flag is vacuously present, matching `Enum.HasFlag`.

9.17 Extension Methods

`extend` adds methods to a type it does not own. `$this` is the receiver.

```
extend Color {  
    func ToGl() -> GlColor => Gfx.ToGl($this)  
    func Darken(by: double) -> Color => Gfx.Lerp($this, Black, $by)  
}  
  
$tint.ToGl()  
$tint.Darken(0.2)
```

An extension can only ever *add* a name. Dispatch reaches it last — after the receiver’s own methods, after inherited ones, and after a callable held in a property — so no call that already worked can change meaning.

Extensions are registered when the declaration runs, so a required module brings its extensions with it. An extension may declare only methods: it has nowhere to put state.

9.18 Event Definitions

Events are named message types with a fixed set of payload fields. Any code that can see the event name can raise it; any function in scope can subscribe with `handles` (see Section 7).

Defining events:

```
event BuildCompleted {  
    Project = ""  
    Duration = (timespan 0s)  
    Success = true  
}  
  
# Required event --- error if raised with no handlers registered  
required event CriticalError { Message = "", Code = 0 }  
  
# Local event --- automatically unregistered when the declaring scope exits  
local event ScopedNotification { Text = "" }
```

Raising events:

```
raise $BuildCompleted ({  
    Project = "MyApp",  
    Duration = (timespan 5s),  
    Success = true  
|})
```

Subscribing (in any visible scope):

```
func on-build(e) handles BuildCompleted {  
    if ($e.Success) {  
        writeline $"Build OK: {$e.Project} in {$e.Duration}"  
    } else {  
        writeline $"Build FAILED: {$e.Project}"  
    }  
}
```

```
    }  
}
```

Handler modifiers (`priority`, `when`, `once`) control dispatch order, conditional execution, and single-fire semantics—see Section 7 for the full syntax. Dispatch order runs from the lowest `priority` number to the highest, with unprioritised handlers last.

Fields not supplied in the `raise` payload keep their declared default values. Event fields are read-only inside the handler body.

Chapter 10

Pipelines & Special Syntax

10.1 The Pipeline

The pipe operator `|` connects command stages, sending the output of one as input to the next:

```
ls -la | where _.Type == file | sort Size | reverse | first 5
```

Tip

The current item in pipeline predicates and blocks is `_`. This is a special implicit variable—you do not need to declare it.

Each stage in a pipeline receives objects (not text). Commands like `where`, `sort`, and `map` operate on object properties natively.

10.2 Command Substitution

Capture command output into a variable:

```
var user = $(whoami)           # Capture as string
var count = $(ls | count)      # Capture pipeline result
var files = $(find . -name "*.cs" -type f)
```

10.3 Subexpressions

Parentheses group expressions and force evaluation:

```
var result = (2 + 3)
echo (String.Join(" ", ["a", "b"]))
echo ($items | count)
```

10.4 Ranges

```
1..10           # 1, 2, 3, ..., 10 (inclusive)
1..2..10        # 1, 3, 5, 7, 9 (step of 2)
0..5            # 0, 1, 2, 3, 4, 5
10..1           # 10, 9, 8, ..., 1 (auto-descending)
```

Ranges are lazy `ToshRange` objects implementing `IShellEnumerableObject`.

10.5 Comprehensions

Comprehensions build collections from sequences using a concise *body <| clause* syntax. The clause specifies the source, an optional filter, and optional intermediate bindings.

Collection Forms

The opening and closing delimiters determine the output type:

Syntax	Result type	Evaluation
<code>[body < clause]</code>	list (array)	eager
<code>{: body < clause :}</code>	set (HashSet)	eager
<code>{% key => value < clause %}</code>	dict	eager
<code>(body < clause)</code>	LazySequence	lazy / on demand

```
[$x * $x <| for x in 1..5]
# [1, 4, 9, 16, 25]

{: $x * $x <| for x in 1..5 :}
# {1, 4, 9, 16, 25} -- duplicates removed automatically

{% $x => $x * $x <| for x in 1..5 %}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

($x * $x <| for x in 1..) | first 5    # lazy -- safe with infinite source
# 1, 4, 9, 16, 25
```

Clause Keywords

for name in source Required. Iterates *source*; the current element is bound to *name* (no *\$* prefix). Inside the body and subsequent clause parts, refer to it as *\$name*.

where condition Optional filter. Only items for which *condition* is truthy are projected through to the body.

let alias = expr Optional intermediate binding. Evaluated once per item after the *where* filter. Multiple *let* bindings may appear; each is visible in subsequent bindings and in the body.

```
[$x <| for x in 1..20 where $x % 3 == 0]
# [3, 6, 9, 12, 15, 18]

[$label <| for x in 1..5 where $x % 2 != 0 let label = $"item-{x}"]
# ["item-1", "item-3", "item-5"]
```

Multi-Clause Forms

Three syntaxes extend a single *for* clause into a multi-variable iteration.

Cartesian product (comma ,) A comma after the source (or its modifiers) introduces a second clause. Every combination of elements is produced—equivalent to nesting two *for* loops:

```
[($x, $y) <| for x in 1..3, y in ["a", "b"]]
# [(1, "a"), (1, "b"), (2, "a"), (2, "b"), (3, "a"), (3, "b")]

[$x + $y <| for x in [1, 2], y in [10, 20]]
# [11, 21, 12, 22]
```

The explicit nested `for` keyword produces the same result:

```
[($x, $y) <| for x in 1..3 for y in ["a", "b"]] # identical
```

Parallel / zip (||) `||` between two clauses zips the sources together—one element from each source per step. Iteration terminates when the *shorter* source is exhausted:

```
[$x + $y <| for x in [1, 2, 3] || y in [10, 20, 30]]
# [11, 22, 33]

[$x + $y <| for x in [1, 2, 3] || y in [10, 20]] # shorter wins
# [11, 22]
```

Pattern destructuring ((a, b, ...)) When the source yields tuples or lists, use a parenthesised name list to bind each element to its own variable:

```
[$a + $b <| for (a, b) in [(1, 2), (3, 4)]]
# [3, 7]

[$a + $b + $c <| for (a, b, c) in [(1, 2, 3), (4, 5, 6)]]
# [6, 15]

# Pairs produced by 'entries' work too:
{% $k => $v * 2 <| for (k, v) in ($myDict | entries) %}
```

All four forms compose with `where`/`let` modifiers and work in every collection form (`[...]`, `{: :}`, `{% %}`, `( )`).

10.6 Live / Streaming Output

Long-running commands that emit objects progressively (via `yield` internally) are displayed incrementally in an interactive TTY session. The REPL automatically promotes a pipeline to *streaming mode* when it detects a gap of roughly 250 ms or more between successive output objects.

In streaming mode the REPL renders a live table:

- The top border, header row, and mid-border are drawn immediately after the first object arrives.
- Each subsequent object is appended as a new row in real time.
- If a later object causes a column to widen, the entire table is redrawn from the top (using cursor-up escape sequences) to keep widths consistent.
- The closing bottom border is drawn when the command completes or is interrupted with `ctrl-C`.

```
ping -c 5 www.google.com # rows appear one per second as replies arrive
tail -f /var/log/app.log # new rows stream in as lines are written
```

Commands that complete quickly (e.g. `ls`) fall back to the standard buffered renderer—the gap heuristic ensures short commands are never artificially slowed. Streaming is automatically disabled when stdout is redirected to a file or pipe (non-TTY).

10.7 Background Jobs

```
/bin/sleep 10 &           # Run in background
jobs                     # List background jobs
wait-for 1               # Wait for job 1
```

Warning

At the *start* of an expression, `&name` is a function reference, not a background operator. The background `&` only applies at the end of a pipeline.

10.8 Tilde Expansion

A *bareword* argument whose first character is `~` expands to a home directory before the command is invoked. Unlike glob expansion, this happens for **every** command—builtins, user functions, and external processes alike. Whether `~` means the home directory is a property of the language, not something each command decides for itself.

```
echo ~                   # /home/ada
echo ~/projects          # /home/ada/projects
echo ~root               # /root
cd ~                     # changes to the home directory
/bin/echo ~              # the external process receives /home/ada
```

Tilde expansion rules:

- `~` alone, and `~/path`, expand to the current user's home directory.
- `~name` expands to a configured directory alias if one exists, and otherwise to that user's home directory. An alias wins, because it was written down on purpose.
- A name matching neither is a diagnostic (`tosh.runtime.unknown_tilde_target`) rather than being passed through unchanged.
- Only **bareword** arguments are expanded. Quoted strings ("`~`") are never expanded, and neither is a variable that happens to hold a tilde—expansion is lexical, so it applies to the word as written.
- The tilde must be the *first* character of the word. `notes.txt~` and `--out=~ /x` are left alone.
- Tilde expansion runs before glob expansion, so `~/*.log` searches the home directory.

```
# Directory aliases give ~name spellings of your own
$tosh.Config.Shell.Dirs.work = "/srv/projects"
echo ~work/notes.md      # /srv/projects/notes.md

# Quoting is how a literal tilde is written
echo "~"                 # prints ~
```

At the start of a command, an expanded tilde is a path like any other: with `auto_cd` enabled a bare `~` changes to the home directory exactly as a bare `/tmp` changes to `/tmp`, and with it disabled both report that the target is a directory rather than a program.

10.9 Glob Expansion

When a *bareword* argument to an external process or one of the builtin commands `echo`, `write`, `writeline`, or `raw` contains the wildcard characters `*`, `?`, `[]`, or the extended glob prefix `@(`, TōSh expands it against the file system before the command is invoked.

```
ls *.txt           # expands to all .txt files in $PWD
echo src/**/*.*cs  # ** matches any number of path segments
rm build-?.log     # ? matches exactly one character
```

Glob expansion rules:

- Only **bareword** arguments are expanded. Quoted strings (`"*.txt"`, `'*.txt'`) and interpolated strings (`${*.txt}`) are *never* expanded.
- If the glob matches no files the literal pattern is passed to the command unchanged (no error).
- Only commands that implement the implicit-glob contract perform expansion. Pure shell built-ins such as `where`, `each`, and `sort` do not expand globs—use `glob` or `find` to produce a file list first.

```
# Use 'glob' to materialise a list from a pattern in a pipeline
glob "*.log" | where _.Size > 1mb | each { rm $_ }

# Quoted string - no expansion
echo "*.txt"      # prints the literal string *.txt
```

10.10 Redirection

TōSh uses named-stream redirection (since `<` and `>` are comparison operators):

Syntax	Description
<code>out> / o></code>	Redirect stdout (truncate)
<code>out» / o»</code>	Redirect stdout (append)
<code>err> / e></code>	Redirect stderr (truncate)
<code>err» / e»</code>	Redirect stderr (append)
<code>o+e> / out+err></code>	Redirect both (truncate)
<code>o+e» / out+err»</code>	Redirect both (append)
<code>e+o> / err+out></code>	Redirect stderr+stdout (truncate)
<code>e+o» / err+out»</code>	Redirect stderr+stdout (append)
<code>«<</code>	Here-string

Table 10.1: Redirection operators

```
echo "hello" out> ./hello.txt
echo "world" out» ./hello.txt      # Append
/bin/sh -c "echo err >&2" err> ./errors.txt
/bin/sh -c "output" o+e> ./combined.txt
```

10.11 Process Substitution

```
# Input process substitution
/bin/diff <(cat file1.txt) <(cat file2.txt)

# Output process substitution
command >(write-file output.txt)
```

10.12 Static Method Calls

Access CLR static methods and members directly:

```
String.Join(", ", ["a", "b"])      # "a, b"
Math.Sqrt(16)                     # 4
Path.GetExtension("./file.cs")    # ".cs"
System.Drawing.Color.Green        # Color object
```

10.13 Object Construction

```
new Point(3.0, 4.0)                # User-defined class
new list(1, 2, 3)                  # List<object>
new dict("name", "Alice")          # Dictionary
new set(1, 2, 2, 3)                 # HashSet (deduped)
new DateTime(2024, 1, 1)           # CLR type
```

10.14 Nameof

`nameof` reports a name as a string, without evaluating it.

```
var x = 42
echo (nameof($x))      # "x"
```

Given a member or type path, it reports the *last* segment—the name the expression asks about—as C# does:

```
var foo = { | Bar = { | Baz = 1 | } | }
echo (nameof($foo.Bar))      # "Bar"
echo (nameof($foo.Bar.Baz))  # "Baz"

class Counter { shared prop Limit: int = 10 }
echo (nameof(Counter.Limit)) # "Limit"
echo (nameof(Counter))       # "Counter"
```

The command form `name-of $foo.Bar` reduces its operand the same way.

A bare name that is also a variable in scope is rejected, since `nameof(x)` beside `var x` almost always means `nameof($x)`:

```
var x = 42
echo (nameof(x))      # error: tosh.runtime.nameof_requires_dollar
```

A member path is exempt from that check—its last segment is a member name, and a variable that happens to share it says nothing about what was written, so `nameof(Counter.Limit)` is valid beside a `var Limit`. An operand with no name to report, such as a trailing dot, raises `tosh.parser.nameof_expects_a_name`.

10.15 Alloc (Native Buffer)

```
alloc buffer = 1024      # Allocate 1024 bytes of native memory
alloc info = SysInfo     # Or size it from an interop type
```

Byte offsets on the buffer helpers are the `-at` flag, not a positional slot:

```
write-buffer $buffer 42 --at 8
read-buffer long $buffer --at 8
read-buffer bytes $buffer --count 16
forget $buffer           # unsets the variable and frees the buffer
```

The `native-*` names are canonical; `alloc`, `read-buffer`, `write-buffer`, `size-of` and `offset-of` are blessed aliases. `native-free` deliberately has no short alias, because `free` is the System memory command — use `forget`, which releases the variable and the buffer together.

With `raw struct` and `out` parameters, most code needs none of this: the engine allocates and marshals, and the buffer helpers remain for genuinely dynamic sizes and raw pointer arithmetic.

See Chapter 25 for full native interop commands.

10.16 Picking Columns vs Rows

`get` and `row` form the canonical pipeline-slicing cluster. `get` projects *members* (columns); `row` selects *positions* (rows). Both are variadic.

```
ls | get Name Size      # column picker -- variadic field projection
ls | get Name           # single column (yields scalars)
[10,20,30,40,50] | row 2  # row picker -- single index
[10,20,30,40,50] | row 4 0 2 # variadic, yields 50, 10, 30 in requested order
[10,20,30,40,50] | row [3,1,0] # list literal works too
[10,20,30,40,50] | row 1..3  # contiguous range
```

`select` and `pick` are soft aliases for `get`. Bad indices in `row` throw `tosh.row.index_out_of_range`. `row` only materialises the prefix of the pipeline necessary to satisfy the highest requested index, so it is safe on long but bounded streams; infinite ranges (`row 0..`) are rejected.

10.17 Structured Introspection

`members` and `methods` are the canonical introspection commands. Both accept first-argument keywords for filtering and querying:

Subcommand	Effect
<code>has <Name></code>	true/false for whether the named member exists.
<code>get <Name></code>	Yields each matching member descriptor (zero-or-more).
<code>props</code>	Filter to properties only (<i>members</i> only).
<code>fields</code>	Filter to fields only (<i>members</i> only).
<code>methods</code>	Filter to methods only (<i>members</i> only).
<code>events</code>	Filter to events only (<i>members</i> only; reserved).

The keyword `may` consume a trailing operand; any further arguments are parsed as type targets, so the piped form and the named-type form are interchangeable:

```

$obj | members has Length      # bool from a piped instance
$obj | members get FullName   # descriptor record(s)
$obj | members props          # all properties of $obj's type
$obj | methods has ToUpper    # bool

members has Length string      # equivalent: type as trailing arg
methods get ToUpper string    # descriptor for String.ToUpper

```

props and funcs are top-level shortcuts for members props and methods, respectively.

The verb-form CLR commands (call, call-method, get-prop, get-props, get-methods, has-prop, has-method, set-prop, del-prop) are deprecated since version 26.05.0.10; they remain registered for backward compatibility but emit deprecation diagnostics. Prefer the fluent member-access syntax (\$obj.Method(\$args), \$obj.Prop, \$obj.Prop = value) and the introspection cluster above.

10.18 System Namespace

The full System.* namespace tree is exposed via the CLR resolver without an import statement. This makes System.IO.* the canonical handle API for file streams, paths, and binary IO:

```

var fh = System.IO.File.OpenRead "/etc/hostname"
defer { $fh.Dispose() }
var bytes = System.IO.File.ReadAllBytes "/etc/hostname"
var leaf = System.IO.Path.GetFileName $path
var b64 = System.Convert.ToBase64String $bytes

```

10.19 Binder Pass

Between parse and evaluate, a static binder pass walks the AST and emits tosh.bind.unknown_command diagnostics for command names that look like typos for registered builtins or aliases (Levenshtein distance ≤ 1 for short names, ≤ 2 for longer). Same-source func declarations are recognised and never flagged.

Strictness depends on context:

Context	Default mode
Interactive REPL	Warn (stderr)
tosh -c "..."	Strict
tosh script.tosh / source	Strict
profile.tosh / autoload/*	Warn

To bypass the binder entirely (recovery escape hatch; undocumented in user-facing help), set TOSH_DISABLE_BINDER=1.

Chapter 11

Configuration

11.1 Config Home

The configuration directory is resolved in order:

1. TOSH_CONFIG_HOME environment variable
2. XDG_CONFIG_HOME/tosh
3. ~/.config/tosh

11.2 Startup Order

1. **config.tosh**—Shell configuration (runs first).
2. **profile.tosh**—User functions, aliases, event handlers.
3. **autoload/*.tosh**—Modular function libraries (sorted lexically).

Skip all startup files with `tosh -no-startup`.

11.3 config.tosh

Sets shell configuration properties:

```
$tosh.Config.Prompt.HeaderLeftLayout = "Time, Directory"
$tosh.Config.Prompt.NameText = "toast"
$tosh.Config.Shell.AutoCd = true
$tosh.Config.History.Persistent = true
$tosh.Config.History.MaxEntries = 10000
$tosh.Config.Display.TimeSpan.ScalarMode = "Long"
$tosh.Config.Shell.Pipefail = true
$tosh.Config.Theme.Tables.BoxStyle = "Double"
```

Example config.tosh

11.4 profile.tosh

User functions, aliases, and event handlers:

```
$tosh.Config.Shell.Dirs = {
  projects = "/home/user/projects",
  downloads = "/home/user/Downloads",
}
```

```

func cls => clear
func ll => ls -la
func recent(span: TimeSpan) {
    ls -la | where _.Modified > ((date now) - $span)
}

func onDirChanged(event) handles DirectoryChanged
    when { _.NewDirectory =~ "projects/" }
{
    writeline $"Entering project: {$event.NewDirectory.Name}"
}

```

Example profile.tosh

11.5 Autoload Directory

Files in `autoload/` are loaded in lexical order. Each can define modules, classes, and functions:

- `archive.tosh` — Archive management
- `collections.tosh` — Collection utilities
- `utils.tosh` — General utilities
- `sysinfo.tosh` — System information

11.6 Configuration Sections

Section	Purpose
Theme	Colors and styles for prompt, syntax, completions, diagnostics, tables, TUI
Display	Type rendering modes (DateTime, StorageSize, Permissions, Paging)
Repl	Continuation prompt, syntax highlighting, ghost text, completion
Prompt	Layout (HeaderLeft/Right, PromptLeft), modules (Time, Dir, Git, etc.)
Shell	Pipefail, AutoCd, Trace, Dirs
History	Persistent, FilePath, MaxEntries, Deduplication, IgnoreLeadingSpace
Startup	Startup path redirection

Table 11.1: Configuration sections

11.7 Config Commands

```

config                # Show full config
config browse         # Interactive config editor
config browse prompt  # Browse specific section
config get prompt.name-text # Get one value
config set prompt.name-text toast # Set one value
config reset prompt   # Reset section to defaults
config reload         # Re-execute all startup files
config init           # Scaffold missing startup files

```

11.8 Special Variables

Variable	Description
<i>Pipeline and control</i>	
-	Current pipeline item inside predicates and blocks.
\$this	Self-reference inside class and struct method bodies.
\$1, \$2, ...	Positional parameters in arrow functions.
<i>Last result</i>	
\$tosh.Last.Result	Pipeline output of the most recent statement.
\$tosh.Last.ExitCode	Exit code of the last external process (int).
\$tosh.Last.Duration	Wall time of the last command (TimeSpan).
<i>Current script</i>	
\$tosh.Script.Path	Absolute path of the running script file.
\$tosh.Script.Name	Filename portion of the running script.
\$tosh.Script.Directory	Directory containing the running script.
\$tosh.Script.Args	Arguments passed to the script (list).
<i>Current function</i>	
\$tosh.Function.Name	Name of the currently executing function.
\$tosh.Function.Args	Arguments passed to the current function (list).
\$tosh.Function.Input	Pipeline input to the current function.
<i>Session</i>	
\$tosh.Session.CurrentDirectory	Shell working directory (equivalent to pwd).
\$tosh.Session.JobCount	Number of running background jobs.
\$tosh.Session.OpenHandleCount	Number of open file/stream handles.
\$tosh.Session.NextHistoryId	ID that will be assigned to the next history entry.
\$tosh.IsLoginShell	true when running as a login shell.
<i>Host</i>	
\$tosh.Host.Version	TōSh version string.
\$tosh.Host.RuntimeId	.NET runtime identifier (e.g. linux-x64).
\$tosh.Host.Framework	Target framework moniker (e.g. net10.0).
\$tosh.Host.ProcessId	PID of the current shell process.
<i>Environment and configuration</i>	
\$env.NAME	Environment variable (case-insensitive lookup). Assigning to \$env.NAME = "value" routes through the same export path as export NAME = "value" and is also case-insensitive against existing variables.
\$tosh.Config	Live configuration object; see Chapter 11.

Table 11.2: Special variables

Chapter 12

Scripts and Subcommand Dispatch

TōSh has a built-in system for turning a script into a structured CLI tool. The `subcommand` block declares named commands with typed flags and positional arguments; TōSh handles parsing, validation, and auto-generated help text automatically.

12.1 Subcommand Basics

A `subcommand` block declares one named command. Its body runs when the user selects that command on the command line:

```
subcommand greet {  
  flag name: string = "World"  
  echo $"Hello, {$name}!"  
}
```

```
tosh myscript.tosh greet           # Hello, World!  
tosh myscript.tosh greet --name Alice # Hello, Alice!
```

Usage

Arrow form

A `subcommand` with parameters and a single-expression body may use the arrow form. Parameters in the arrow form are passed directly, not via flags:

```
subcommand greet(name: string) => echo $"Hello, {$name}!"
```

Warning

Parameters declared with `()` in the arrow form require the `=>` separator. Using a block body with `()` parameters is a parse error (`tosh.parser.subcommand_params_require_arrow`).

12.2 Flags and Arguments

Inside a `subcommand` body, `flag` declares an optional named option and `arg` declares a required positional argument:


```

subcommand deploy {
  arg host: string
  flag port: int    = 22
  flag user: string = "root"
  flag dry-run: bool = false

  if ($dry-run) {
    echo $"[dry-run] ssh {$user}@{$host}:{$port}"
  } else {
    ssh $"-p{$port}" $"{$user}@{$host}"
  }
}

```

```

tosh deploy.tosh deploy myserver
tosh deploy.tosh deploy myserver --port 2222 --user alice
tosh deploy.tosh deploy myserver --dry-run

```

Usage

- flag values are set with `-name value` or `-name=value`.
- The CLI flag name is exactly the identifier as declared—no case folding or kebab-from-snake conversion. A flag declared `flag dry-run` is invoked as `-dry-run`; flag `fooBar` as `-fooBar`; flag `MAX_RETRIES` as `-MAX_RETRIES`.
- Boolean flags may be set with `-name` (sets to true) or `-no-name` (sets to false).
- arg values are consumed from the remaining positional tokens in declaration order.
- `-` stops option parsing; all tokens after it are positional.

12.3 Nested Subcommands

Subcommands nest to build command trees. Dispatch descends into the child whose name matches the first positional token:

```

subcommand git {
  subcommand commit {
    flag message: string = ""
    flag all: bool = false
    echo $"committing: {$message}"
  }

  subcommand push {
    arg remote: string = "origin"
    flag force: bool = false
    echo $"pushing to {$remote}"
  }
}

```

```

tosh myscript.tosh git commit --message "initial"
tosh myscript.tosh git push origin --force

```

Usage

Flags are looked up leaf-to-root: a flag defined on a parent is available to all its descendants.

12.4 Subcommand Modifiers

Modifiers appear between subcommand and the name:

(none) Body runs only when this is the chosen leaf of dispatch.

eager Body runs even when execution descends into a child. Used for setup work (e.g. loading config before a child acts on it). May not be combined with **hollow**.

hidden Excluded from auto-generated help and “did you mean” suggestions. Still fully callable.

hollow Pure namespace—body must contain only nested subcommand declarations. Cannot be instantiated directly.

vital Throws a runtime error if children exist but the user did not choose one (instead of showing auto-help).

```
eager subcommand db {
  flag env: string = "production"
  writeline $"Connecting to {$env} database" # always runs

  subcommand migrate {
    flag dry-run: bool = false
    run-migrations --env $env --dry-run $dry-run
  }

  subcommand seed {
    run-seed-data --env $env
  }
}
```

eager parent

12.5 Auto-Help

Every subcommand automatically supports `-help` at any nesting level. The help output is generated from the `flag` and `arg` declarations at the current and ancestor levels. To opt out of automatic help, declare your own `flag help: bool = false`.

Doc-comments not yet included in auto-help

Although `##` doc-comments can be written above a subcommand block, the current runtime does not attach them to the auto-generated help output. Only the structural information (flag names, types, and which subcommands exist) is shown. If you need richer descriptions today, write them in the script body as `writeline` output guarded by `if ($tosh.Script.Help)`.

```
tosh myscript.tosh --help
tosh myscript.tosh git --help
tosh myscript.tosh git commit --help
```

Invoking auto-help

12.6 Diagnostics

Code	Trigger
tosh.runtime.missing_script_flag	Required flag not provided.
tosh.runtime.missing_script_argument	Required positional arg not provided.
tosh.runtime.unknown_script_flag	Unrecognised -flag on the command line.
tosh.runtime.script_option_requires_value	Flag that expects a value was given without one.
tosh.runtime.unexpected_script_argument	Extra positional token after all args are filled.
tosh.runtime.duplicate_subcommand	Two sibling subcommands share the same name.
tosh.parser.incompatible_subcommand_modifier	seager and hollow used together.
tosh.parser.subcommand_params_require_arrow	Block body used with () parameter form.
tosh.parser.hollow_subcommand_must_be_empty	hollow subcommand contains non-subcommand statements.
tosh.parser.duplicate_subcommand_modifier	Same modifier applied more than once.

Part II

Compilation

Chapter 13

Overview & Execution Model

TōSh runs in two modes: an *interpreter* (`ToshEngine`) and an *ahead-of-time compiler* (`Tosh.Compiler`) that emits standard .NET assemblies. The interpreter is the canonical semantics; the compiler is a faithfulness-preserving lowering of those semantics onto the CLR. Both modes share the same lexer, parser, binder, lowerer, and type checker — only the back end differs.

13.1 When each mode runs

Mode	Triggered by
Interpreter (REPL)	<code>tosh</code> , no positional source.
Interpreter (one-liner)	<code>tosh -c "..."</code> .
Interpreter (script)	<code>tosh path/to/script.tosh</code> .
Compiler (CLI)	<code>tosh -compile path.tosh -o out.dll</code> (§14).
Compiler (MSBuild)	<code><Project Sdk="Tosh.Sdk"></code> (§21).

13.2 Compiler pipeline

1. **Lex / parse.** Identical to the interpreter; produces a `ParseResult` with the same syntax tree.
2. **Bind / lower.** Identical to the interpreter; produces a `BoundUnit` containing `BoundStatements` and `BoundExpressions`.
3. **Type check.** The same `TypeChecker` runs, but in *compile mode* it emits an extra audit pass (`CheckCompileAnnotations`) for missing or implicit dynamic annotations (§16).
4. **Emit.** `BoundUnitEmitter` walks the bound tree and emits IL into a `PersistedAssemblyBuilder`. Each shape is gated by a profile/tier check (§15).
5. **Persist.** The assembly is written to disk; an optional reference assembly, apphost wrapper, and single-file bundle are produced as requested (§14).

13.3 Parity contract

A program that compiles cleanly must, when run, produce the same observable behaviour as the interpreter would for the same source. Observable here means: stdout, stderr, exit code, files written by redirection, and values returned to a hosting C# caller. Internal scheduling, allocation, and diagnostics-not-yet-promoted-to-errors are not part of the contract.

The conformance test layer (§23) is the machine-checked expression of this contract.

Where to look

The full pipeline lives under `src/Tosh.Compiler/` and `src/Tosh.Compiler.Runtime/`. The bound tree the emitter walks is defined in `src/Tosh.Language/Binding/BoundNodes/`.

Chapter 14

Command-Line Interface

The compiler is invoked through the same `tosh` binary that runs the shell. The first positional argument selects between shell and compile modes:

```
# Interpret a script.
tosh greet.tosh

# Compile a script to a managed assembly.
tosh --compile greet.tosh -o greet.dll
tosh -C greet.tosh -o greet.dll
```

14.1 Flags

Flag	Effect
<code>-o PATH, -output PATH</code>	Output assembly path. Required; the file extension does not have to be <code>.dll</code> .
<code>-profile NAME</code>	Compile profile (permissive, runtime, or pure). Default permissive. May also be written <code>-profile=NAME</code> .
<code>-compile-allow-dynamic,</code> <code>-allow-dynamic</code>	Suppress <code>tosh.compile.implicit_dynamic</code> diagnostics. Equivalent to opting out of strict-mode for variables whose type the inferer cannot pin down.
<code>-emit-refasm</code>	Also emit a sibling <code>*.ref.dll</code> stamped with <code>[ReferenceAssembly]</code> so <code>C#</code> / <code>F#</code> / Roslyn consumers can target the public surface without dragging in private members.
<code>-no-apphost</code>	Skip the native apphost wrapper (Windows: <code>.exe</code> ; Unix: bare). The <code>.dll</code> is still written and remains runnable via dotnet.
<code>-publish-single-file</code>	Bundle the .NET runtime and all dependencies into a single self-contained executable. Implies an apphost.

14.2 Examples

```
# Default profile -- accepts every shape the interpreter does.
tosh --compile script.tosh -o script.dll

# Strict -- reject source-replay (Tier 3) shapes.
tosh --compile script.tosh -o script.dll --profile=runtime

# Emit a refasm alongside the implementation assembly.
tosh --compile lib.tosh -o lib.dll --emit-refasm

# Single-file self-contained binary (~70 MB; bundles the runtime).
tosh --compile cli.tosh -o cli --publish-single-file
```

Errors

A `-compile` invocation with no source paths fails with “The ‘-compile’/’-C’ flag requires at least one script path.”. Unknown options inside the `-compile` arg list fail with “Unknown option ‘...’ for -compile.”.

Chapter 15

Profiles & the Tier Model

Every compiled shape lives at one of three tiers. The *profile* selects how strict the compiler is about which tiers it accepts.

15.1 Tiers

Tier	Definition
1 — native IL	Pure IL with no <code>ToshHost</code> call at runtime: literals, arithmetic, control flow, top-level user-function calls resolved at IL-emit time, real CLR module-type field reads.
2 — runtime-hosted	Real IL but calls into <code>Tosh.Compiler.Runtime.ToshHost</code> for builtin command dispatch, dynamic member access, qualified lookup, redirection scopes, etc.
3 — source-replay	The emitted assembly registers a span of the original tosh source and re-evaluates it through <code>ToshEngine</code> at runtime. Used for class / record / struct / union / enum / trait bodies, complex module bodies, and block-pipeline arguments.

15.2 Profiles

Profile	Max tier	Use case
permissive	3	Default. Accepts every shape the interpreter does. The compile is guaranteed to succeed if the program parses, binds, and type-checks.
runtime	2	Real, redistributable .NET assembly that doesn't carry its own source for re-evaluation. Rejects user-defined types, complex module bodies, and block arguments.
pure	1	No <code>ToshHost</code> calls at runtime, no source replay. To-day this is mostly aspirational — even <code>echo</code> is a Tier-2 builtin — but the profile exists as a forcing function for future “lift this builtin into IL” work.

15.3 RequireTier semantics

Internally, every shape emitter calls `RequireTier(int tier, string feature)` before lowering itself. If the requested tier exceeds the maximum allowed by the active profile, the emitter records a diagnostic of the form

```
profile 'pure' rejects tier 2 feature: builtin command dispatch (statement)
```

Diagnostics are deduplicated per (tier, feature) pair: a program with fifty calls to `ls` in pure profile reports the violation once, not fifty times.

15.4 Parity expectations

A profile narrowing is *always conservative*: any program that compiles under pure also compiles under runtime and permissive, and any program that compiles under runtime also compiles under permissive. Behaviour is identical across profiles when the program compiles under all three.

Choosing a profile

Pick `runtime` when you want a redistributable .NET library that does not need to ship its own source. Pick `permissive` during development or when the program uses `class` / `record` / `module` bodies the runtime profile rejects today. Pick `pure` only in tooling contexts (refasm-only builds, audit checks) where Tier-2 dispatch is itself the thing under test.

Chapter 16

Type Discipline in Compiled Mode

The type checker runs in every mode, but compile mode adds an *annotation audit* that the interpreter does not. The audit promotes weakly-typed shapes to errors so a redistributable assembly cannot expose a public surface that silently degrades to `dynamic`.

16.1 The audit rules

Diagnostic	When
<code>tosh.compile.missing_type_annotation</code>	A func has no return-type annotation, or a parameter has no type annotation.
<code>tosh.compile.implicit_dynamic</code>	A var declaration has no type annotation <i>and</i> the inferrer cannot pin down a concrete type from the initializer.

16.2 Explicit `dynamic` opt-out

The audit treats an explicit `: dynamic` annotation as an intentional opt-out. Both of the following are accepted by the audit:

```
func passthrough(x: dynamic) -> dynamic { return $x }

var f: dynamic = (ls)
```

The same source without the annotations would fail compile:

```
func passthrough(x) -> dynamic { return $x }
# tosh.compile.missing_type_annotation: parameter 'x' is missing an
# annotation. Use 'dynamic' to opt out explicitly.

var x = (ls)
# tosh.compile.implicit_dynamic: variable 'x' has no type annotation
# and the inferrer could not pin down a concrete type.
```

16.3 The `-allow-dynamic` flag

`-compile-allow-dynamic` (alias `-allow-dynamic`) suppresses `tosh.compile.implicit_dynamic`. It does not suppress `tosh.compile.missing_type_annotation`: function signatures must always be

either typed or explicitly `dynamic`, because they form the public CLR surface of the compiled assembly.

16.4 Implementation note

The `AnnotatedDynamic` flag on `BoundVariableDeclaration` records whether the source carried an explicit `: dynamic`. The `Lowerer` sets the flag (case-insensitive on the type name); the audit consults it in `CheckVarAnnotation`. Synonyms for explicit `dynamic` on parameters are `dynamic`, `any`, and `object`.

Chapter 17

Public CLR Shape

Compiled tosh source produces a single .NET assembly. The shape is designed to be consumable from C#, F#, and reflection-driven tooling without re-parsing the source.

17.1 Top-level statements

A top-level user function `func greet(name: string) -> string` becomes a public static method on a class named after the assembly's main type. Module bodies become nested static classes.

17.2 Overloads

When a tosh function name is overloaded, every overload is emitted as a separate static method under the same CLR name. Argument types come from the parameter annotations, so overload resolution from C# follows the ordinary CLR rules.

```
# tosh
func pick(a: int) -> string => $"one:{a}"
func pick(a: int, b: int) -> string => $"two:{a}+{b}"
```

```
// C# consumer
Greet.pick(1);           // "one:1"
Greet.pick(1, 2);        // "two:1+2"
```

17.3 Name mangling

Tosh identifiers may contain characters the CLR-targeted languages reject — hyphens are the common case (`func to-json`). The emitter rewrites each illegal character to `_` and prefixes leading digits, then stamps the original spelling on the resulting member with `[ToshOriginalName("to-json")]`. Tools such as the LSP, the MCP server, and editor extensions read the attribute to present the original identifier.

When two top-level types or functions mangle to the same CLR name a `tosh.compile.name_mangling_collision` diagnostic fails the build.

17.4 User-defined types

Compiled `class`, `struct`, `record`, `enum`, `interface`, `trait`, and `union` declarations are emitted as public CLR types stamped with `[ToshType(Kind, SpanStart, SpanLength)]`. The span

identifies the original declaration in the registered source so source-replay-backed types can rebind full interpreter semantics through `ToshHost.RegisterTypeFromSource` when needed.

17.5 Reference assemblies

`-emit-refasm` writes a sibling `*.ref.dll` stamped with `[ReferenceAssembly]`. The `refasm` exposes only the public surface (types, methods, signatures) and is the recommended target for downstream C# / F# / Roslyn consumers — it isolates them from private-member churn and reduces incremental rebuild scope.

Chapter 18

Pipelines & Redirections Under Compile

Pipeline construction and stream redirection are emitted as IL that calls into `Tosh.Compiler.Runtime.ToshHost` (Tier 2). The lowering preserves both the lazy-by-stage semantics of the interpreter and the deterministic disposal order of redirection sinks.

18.1 Pipeline stages

A multi-stage pipeline lowers to a chain of `RunStage` calls seeded by either `EmptyInput` (top-of-pipeline) or `SeedFromValue` (when a value-typed expression feeds the pipeline):

```
ls -la | where _.Type == "file" | sort-by Size | head 10
```

becomes, schematically:

```
var input = ToshHost.EmptyInput();
input = ToshHost.RunStage(ToshHost.ResolveCommand("ls"),    input, ["-la"]);
input = ToshHost.RunStage(ToshHost.ResolveCommand("where"), input, [_filter]);
input = ToshHost.RunStage(ToshHost.ResolveCommand("sort-by"),input, ["Size"]);
input = ToshHost.RunStage(ToshHost.ResolveCommand("head"), input, [10]);
ToshHost.DrainStatement(input);
```

Each stage is executed lazily: a stage's body does not run until the next stage pulls from its async enumerable. `DrainStatement` (statement context) and `DrainEnumerator` (value context) are the only points that force evaluation.

18.2 Stream redirection

Output redirections (`out>`, `err>`, `o+e>`, and the `»` append variants) and input redirection (`in<`) all lower to a single `ToshHost.BeginRedirection` call wrapping the pipeline body in a `using` (try/finally + `Dispose`):

```
echo hello out> "log.txt"
```

becomes, schematically:

```
using (ToshHost.BeginRedirection(
    streams: [1],           // stdout
    modes:   [0],           // truncate
    targets: ["log.txt"],
    body:    ...))
```

```

        inputPath: null))
    {
        /* pipeline body */
    }

```

Output sinks are flushed and closed in reverse order during `Dispose`; `Console.Out` / `Console.Error` / `Console.In` are restored to their pre-scope readers.

18.3 Input redirection

Input redirection installs *both* a `Console.SetIn(...)` reader (so commands that read `Console.In` — e.g. `read-line` — observe the redirected file) and a thread-static input path consulted by `EmptyInput()` (so commands that consume the pipeline input — e.g. `cat`, `wc`, `grep`, `read-lines` — observe the same file as a sequence of lines).

```

echo (read-line) in< "in.txt"           # reads via Console.In
cat in< "in.txt"                       # reads via pipeline input

```

18.4 Nested redirection

`RedirectionScope` tracks per-scope state for the streams and sinks it opened. Each scope only restores state it actually installed: an output-only nested scope cannot clobber an outer scope's input path on dispose, and vice versa.

Pre-existing footgun, now fixed

The original implementation restored the thread-static input path unconditionally on dispose. A nested output redirection inside an outer input redirection would therefore wipe the outer scope's input path, causing later pipeline-input consumers to see no input. The current implementation gates the restore on a `_trackedInputPath` flag set only when the inner scope itself opened an input redirection.

Chapter 19

Function References in Compiled Mode

The `&name` expression evaluates to an `IShellCallable`. In compiled mode the value is a `CompiledMethodReferenceValue` bound directly to one or more emitted `MethodInfos`, not a late-bound name lookup.

19.1 Single-overload fast path

When a function name has exactly one compiled overload the emitter loads the `MethodInfo` via `ldtoken / MethodBase.GetFromHandle` and calls `ToshHost.MakeFunctionReferenceFromMethod(method, name)`. Invocation reorders named arguments to match the parameter list and calls the method directly.

```
func inc(n: int) -> int => $n + 1
var f = &inc
$f(41)                                # 42
```

19.2 Overloaded references

When a function name has two or more compiled overloads the emitter emits a `MethodInfo[]` containing each overload and calls `MakeFunctionReferenceFromMethods(methods, name)`. Invocation routes through `InvokeUserOverload`, which scores each candidate against the supplied argument list and dispatches to the best match.

```
func pick(a: int) -> string => $"one:{$a}"
func pick(a: int, b: int) -> string => $"two:{$a}+{$b}"
var f = &pick
$f(1)                                # "one:1"
$f(1, 2)                             # "two:1+2"
```

19.3 Named arguments through references

Named-argument calls (`$f(value=42, label="answer")`) are reordered against the chosen overload's parameter list. Single-overload calls reorder in `BindNamedArguments`; overload-set calls inherit the existing `InvokeUserOverload` reorder pass. Missing optional parameters fall back to their declared default; missing required parameters surface as the ordinary arity-mismatch failure.

19.4 Late-bound fallback

For names that the compiler cannot resolve at emit time (e.g. a reference to a builtin command or to a function loaded later through a host) the emitter emits `MakeFunctionReference(name)` instead. The resulting callable consults the runtime command table at invocation time.

Chapter 20

Other Bound-Shape Emitters

Several language constructs lower to dedicated host helpers rather than inline IL. They are summarised here so the implementation surface is discoverable.

20.1 Records

Field shape	Lowering
name: value	Plain <code>Ldstr</code> key + value expression.
(\$key): value	Key expression evaluated, <code>ToString()</code> , then value (Tier 2).
...\$source	Call to <code>ToshHost.SpreadRecord</code> with the source on the stack (Tier 2).

```
var base = { | name: "alice", age: 30 | }
var ext  = { | ...$base, age: 31, role: "admin" | }
echo $ext.name $ext.age $ext.role      # alice 31 admin
```

20.2 Tuple destructuring

A statement of the form `($a, $b) = expr` lowers to one `DestructureArray(value, count)` call followed by `count` strongly-typed local stores. Missing slots collapse to `null`.

```
var a = 0
var b = 0
($a, $b) = [10, 20]
echo $a $b      # 10 20
```

20.3 `throw` as expression

A `throw` expr in expression position lowers to a `ToshHost.ThrowAsException(value)` call that returns object so the rest of the surrounding expression typechecks.

```
var port = $env.PORT ?? throw "PORT is required"
```

20.4 Other helpers

`ToArray`, `IndexOrNull`, `IsTruthy`, `AsRedirectionPath`, `MakeMemberProjection`, and `MakeFunctionReference[FromMethod[s]]` all live in `Tosh.Compiler.Runtime.ToshHost` and are called from the `BoundUnitEmitter` via cached `MethodInfos`.

Chapter 21

MSBuild Integration

The `Tosh.Sdk` MSBuild SDK turns a `.csproj`-style project into a `.NET` assembly built from `.tosh` sources, with the same flags the CLI exposes.

21.1 Project shape

```
<Project Sdk="Tosh.Sdk/1.0.0">

  <PropertyGroup>
    <OutputType>Exe</OutputType>          <!-- or Library -->
    <TargetFramework>net10.0</TargetFramework>
    <ToshProfile>permissive</ToshProfile>
    <ToshAllowDynamic>false</ToshAllowDynamic>
    <ToshEmitReferenceAssembly>true</ToshEmitReferenceAssembly>
    <PublishSingleFile>false</PublishSingleFile>
  </PropertyGroup>

  <ItemGroup>
    <ToshCompile Include="src/**/*.tosh" />
  </ItemGroup>

</Project>
```

21.2 Tasks

Task	Purpose
<code>Tosh.Sdk.Tasks.ToshCompile</code>	In-process MSBuild task. Mirrors the CLI compile pipeline (parse → bind → lower → type-check → emit) but runs inside the MSBuild worker, avoiding the per-build process-launch cost of the legacy <code><Exec></code> -based target. Inputs: <code>Sources</code> , <code>OutputPath</code> , <code>Profile</code> , <code>AllowDynamic</code> , <code>EmitReferenceAssembly</code> , <code>EmitAppHost</code> , <code>PublishSingleFile</code> , <code>AssemblyName</code> .
<code>Tosh.Sdk.Tasks.ToshStagePackageReferences</code>	Stages <code>PackageReference</code> content into the runtime's reference and analyzer paths so package types are visible during compile and at runtime.

21.3 Fallback path

If the in-process task assembly is unavailable (e.g. the SDK is consumed by a pinned older runtime), `Sdk.targets` falls back to launching the tosh CLI through `<Exec>`. Behaviour is identical; only the process model differs.

21.4 NuGet consumption from C#

A library compiled with `<ToshEmitReferenceAssembly>true</ToshEmitReferenceAssembly>` ships both `lib.dll` and `lib.ref.dll`. Standard .NET tooling selects the refasm at compile time and the implementation at runtime, so a C# consumer references the package with no special configuration:

```
<PackageReference Include="MyToshLibrary" Version="1.0.0" />
```

Chapter 22

Limits Per Profile

The following matrix lists the major shapes by tier and shows which profiles accept them. “✓” = compiles cleanly; “**R**” = rejected by that profile with a tier-violation diagnostic.

Shape	Tier	Permissive	Runtime	Pure
literals, arithmetic, control flow	1	✓	✓	✓
top-level user-function call (resolved at emit)	1	✓	✓	✓
real CLR module-type field read	1	✓	✓	✓
builtin command dispatch (statement / value)	2	✓	✓	R
builtin command dispatch (multi-stage)	2	✓	✓	R
user function dispatch via host (pipeline)	2	✓	✓	R
stream redirection (out>/err>/in</etc.)	2	✓	✓	R
qualified-name resolution (Foo.bar)	2	✓	✓	R
qualified-method invocation	2	✓	✓	R
dynamic member access	2	✓	✓	R
new T(...) object construction	2	✓	✓	R
record spread (...\$rec)	2	✓	✓	R
function reference (compiled binding)	2	✓	✓	R
function reference (late-bound)	2	✓	✓	R
subcommand-tree dispatch (compiled)	2	✓	✓	R
class / record / struct / union / enum body	3	✓	R	R
trait body	3	✓	R	R
complex module body	3	✓	R	R
top-level non-var/func declaration	3	✓	R	R
block-pipeline argument	3	✓	R	R
subcommand-tree dispatch (argv-driven entry)	3	✓	R	R
whole-script replay (rune expansion)	3	✓	R	R

Chapter 23

Conformance Test Layer

Two test surfaces in `tests/Tosh.Tests/CompilerFeatureMatrixTests.cs` turn the parity contract from §13.3 into machine-checked expectations.

23.1 Feature matrix

`FeatureCases()` yields one row per language shape with a flag per profile saying whether the shape should compile cleanly under that profile. The matrix is the executable form of §22.

```
yield return FeatureCase(  
    id: "redirection.out-file",  
    category: "redirection",  
    source: "echo hi out> \"x\"",  
    permissive: true, runtime: true, pure: false,  
    note: "stream redirection requires Tier 2");
```

23.2 Conformance cases

`ConformanceCases()` yields one row per shape that should produce a specific stdout (and optionally a specific file) under the default profile. The harness compiles the source, runs the resulting assembly, and asserts on the captured streams.

```
yield return ExecCase(  
    id: "redirection.in-pipeline",  
    source: "cat in< \"{TMPFILE}\"",  
    stdout: "alpha\\nbeta",  
    note: "input redirection seeds pipeline-input commands like cat",  
    seedFile: "alpha\\nbeta\\n");
```

23.3 Adding a row

Two-step recipe:

1. Add a `FeatureCase` or `ExecCase` row alongside the implementation change. `Runtime/Pure` flags should reflect the `RequireTier` call the emitter makes for the new shape.
2. Run `dotnet test tests/Tosh.Tests/Tosh.Tests.csproj -filter "FullyQualifiedName CompilerFeatureMatrixTests"` and confirm the row passes for every profile its flags claim.

Console serialisation

Tests that capture stdout/stderr or install `Console.SetIn` share the xUnit collection `ConsoleSerialCollection` so xUnit serialises them. Concurrent collections that touch `Console.{Out, Error, In}` otherwise interleave output and produce flaky equality checks.

Chapter 24

Compiler Diagnostics

The compiler emits two diagnostic families. The first is shared with the interpreter (parser, binder, type checker); the second is compile-only.

24.1 Compile-only diagnostic codes

Code	Meaning
<code>tosh.compile.implicit_dynamic</code>	A var has no annotation and the inferer cannot pin down a concrete type. Suppress with <code>-allow-dynamic</code> or annotate the variable.
<code>tosh.compile.missing_type_annotation</code>	A func parameter or return type is missing an annotation. Annotate or use <code>dynamic</code> . Cannot be suppressed by <code>-allow-dynamic</code> .
<code>tosh.compile.name_mangling_collision</code>	Two top-level types or functions mangle to the same CLR name. Rename one.

24.2 Tier-violation diagnostics

The deduped diagnostic emitted by `RequireTier` (§15.3) is not coded in the `tosh.compile.*` family today; it surfaces as a plain message of the form

```
profile 'NAME' rejects tier T feature: FEATURE
```

24.3 Runtime callable diagnostics

These are emitted at runtime by the host, not at compile time, but are included here because they are reachable only through compiled code paths (`IShellCallable.InvokeAsync`, `CompiledLambdaCallable`, `CompiledMethodReferenceValue`).

Code	Meaning
<code>tosh.runtime.callable_argument_count_mismatch</code>	A callable was invoked with too few or too many arguments for any matching overload.
<code>tosh.runtime.callable_named_argument</code>	A named argument did not match any parameter of the chosen overload.
<code>tosh.runtime.callable_invocation_requires_single_value</code>	A callable was invoked in a context that demands exactly one value but the callable produced none or more than one.

Full reference

The complete diagnostic-code reference for every namespace (`tosh.parser.*`, `tosh.runtime.*`, `tosh.compile.*`, `tosh.bind.*`, ...) lives in **Part V** and is regenerated from source by `python3 scripts/extract_diagnostic_codes.py`.

Part III

Command Reference

Chapter 25

Built-In Command Catalog

TōSh's built-in command catalog is generated from the runtime metadata exported by the shell itself. This keeps the PDF aligned with the installed command surface: names, aliases, categories, signatures, arguments, options, examples, pipeline input contracts, output descriptions, side-effect notes, and command categories all come from the same descriptors used by `help`, `MCP command_metadata`, and `tosh -dump-builtins`.

Tip

At runtime, use `help <name>` for one command, `apropos <text>` for search, `help search <text>` for structured help search, or `tosh -dump-builtins` for the full JSON catalog.

25.1 CLR

25.1.1 `call`

Library: *Tosh.Stdlib.Clr*

`call`

Invokes an instance or static CLR method.

Signature

`call <method-name> [args...] or call <type-name> <method-name> [args...]`

Argument	Description
method-name	The method to invoke (or a type name for static calls).
args	Arguments to pass to the method. (optional)

Pipeline input: scalar, record — Uses the piped object as the instance for method invocation.

Output: The return value of the invoked method.

```
"hello" | call ToUpper # Call an instance method on a piped string
call System.Math Sqrt 144 # Call a static method
```

Note

Deprecated. Prefer member-access syntax: `'$obj.Method($args)'` or `'$callable($args)'`.

Note

Prefer the fluent `'$obj.Method()'` expression syntax for instance calls and `'Type-Name.Method()'` for static calls. The `'call'` command form is a legacy fallback.

25.1.2 call-method

Library: *Tosh.Stdlib.Clr*

call-method

Invokes a method by dynamic name.

Signature

```
call-method [object] <method-name> [args...]
```

Output: Streams whatever the invoked method returns (single value, an enumeration of values, or nothing for void methods).

```
echo hello | call-method ToUpper
call-method $obj MethodName arg1
```

Note

Deprecated. Prefer member-access syntax: `'$obj.Method($args)'`.

25.1.3 cast

Library: *Tosh.Stdlib.Clr*

cast

Casts pipeline values to a CLR type or a type declared in ToastScript.

Signature

```
cast <type> [value ...]
```

Argument	Description
type	The target type to cast to: a CLR type including generic types like <code>list<int></code> , or a type declared in ToastScript.
value	Optional value(s) to cast. If omitted, reads from the pipeline. (optional)

Pipeline input: scalar, list — Casts each piped value to the target type.

Output: The cast values in the target type.

```
echo [1, 2, 3] | cast list<int> # Cast an array to a typed list
```

```
echo 42 | cast string # Cast a number to a string
cast Fuel 8 # Cast a backing value to a declared enum member
```

Note

Cast converts to CLR target types, including constructed generic collection types like ‘list<int>’, and to types declared in ToastScript. A declared enum converts from a member name or a backing value; any other declared type converts only from a value that already is one, because cast does not construct.

25.1.4 clone

Library: Tosh.Stdlib.Clr

clone

Creates a shallow copy of an object.

Signature

```
clone [object]
```

Output: A shallow copy of each input/object: the same record/dictionary/list shape with copied top-level entries.

```
$obj | clone
clone $obj
```

25.1.5 constructors

Library: Tosh.Stdlib.Clr

constructors

Lists constructors for CLR types, ToSh classes, and shell collection types.

Signature

```
constructors [type ...]
```

Output: Records describing each constructor: declaring type, parameter list, and a callable invocation example.

```
constructors System.String | first 5
constructors list<int>
constructors dict<string, int>
```

Note

Constructors works with CLR types, ToSh named types, and shell collection types like ‘list<int>’ and ‘dict<string, int>’.

25.1.6 del-prop

Library: Tosh.Stdlib.Clr

del-prop

Removes a property from a dynamic record.

Signature

```
del-prop [object] <name>
```

Argument	Description
object	The target dynamic record. If omitted, reads from the pipeline. (optional)
name	The property name to remove.

Pipeline input: scalar, record — Uses the piped object as the target when the object argument is omitted.

Output: The modified record with the property removed.

```
$obj | del-prop Name # Remove a property from a piped record
del-prop $obj Name # Remove by passing the object directly
```

Note

Deprecated. Prefer '\$obj.Prop = null', or 'forget' for dict keys.

Note

Only works on dictionary-backed dynamic records. CLR properties cannot be removed.

25.1.7 describe-type

Library: *Tosh.Stdlib.Clr*

describe-type

Describes CLR types, ToSh named types, or shell collection types.

Signature

```
describe-type [type ...]
```

Argument	Description
type	One or more type names or piped objects to describe. (optional)

Pipeline input: scalar, record — Describes the type of each piped object.

Output: Type projection objects with hierarchy, interfaces, properties, methods, and constructors.

```
describe-type string # Describe the String type
describe-type list dict table # Describe multiple shell types
42 | describe-type # Describe the type of a piped value
```


25.1.8 funcs

Library: *Tosh.Stdlib.Clr*

funcs

Lists public methods for CLR types or pipeline objects (shortcut for ‘methods’).

Signature

```
funcs [type ...]
```

Argument

Description

type

Optional type name. When omitted, uses piped objects. (optional)

Pipeline input: scalar, record — Inspects the type of each piped object.

Output: Method descriptor objects (alias for ‘methods’).

```
string | funcs # List public methods of String
$obj | funcs # List public methods of a piped object
```

25.1.9 get-methods

Library: *Tosh.Stdlib.Clr*

get-methods

Lists method names for an object or ToSh class.

Signature

```
get-methods [object]
```

Output: Records describing each method: name, return type, parameter list, and arity flags.

```
$obj | get-methods
get-methods $obj
```

Note

Deprecated. Prefer ‘methods’ (or ‘funcs’ shortcut).

25.1.10 get-prop

Library: *Tosh.Stdlib.Clr*

get-prop

Gets a property value by dynamic name.

Signature

```
get-prop [object] <name>
```

Argument	Description
object	The target object. If omitted, reads from the pipeline. (optional)
name	The property name to retrieve.

Pipeline input: scalar, record — Uses the piped object as the target when the object argument is omitted.

Output: The value of the named property.

```
$obj | get-prop $propName # Get a dynamically-named property from a piped
                           object
get-prop $obj Name # Get a property by passing the object as an argument
```

Note

Deprecated. Prefer member-access syntax: '\$obj.Prop'.

25.1.11 get-props

Library: *Tosh.Stdlib.Clr*

get-props

Lists property names for an object.

Signature

```
get-props [object]
```

Output: Records describing each property/field: name, declared type, accessibility, and current value when available.

```
$obj | get-props
get-props $obj
```

Note

Deprecated. Prefer 'members props' (or 'props' shortcut).

25.1.12 has-method

Library: *Tosh.Stdlib.Clr*

has-method

Checks whether an object or ToSh class has the named method.

Signature

```
has-method [object] <name>
```

Output: A bool — true when the target type/object exposes a method with the given name.

```
$obj | has-method ToString
```

```
has-method $obj ToString
```

Note

Deprecated. Prefer ‘methods has Name’.

25.1.13 has-prop

Library: Tosh.Stdlib.Clr

has-prop

Checks whether an object has the named property.

Signature

```
has-prop [object] <name>
```

Argument	Description
object	The target object. If omitted, reads from the pipeline. (optional)
name	The property name to check.

Pipeline input: scalar, record — Uses the piped object as the target when the object argument is omitted.

Output: A boolean: true if the property exists, false otherwise.

```
$obj | has-prop Name # Check if a piped object has a property
has-prop $obj Name # Check by passing the object directly
```

Note

Deprecated. Prefer ‘members has Name’.

25.1.14 load-assembly

Library: Tosh.Stdlib.Clr

load-assembly

Loads a .NET assembly from disk into the current process.

Signature

```
load-assembly <path> [path...]
```

Argument	Description
path	One or more file paths to .NET assemblies to load.

Output: Assembly objects for each loaded assembly.

```
load-assembly ./MyPlugin.dll # Load a plugin assembly
load-assembly Newtonsoft.Json.dll System.Data.dll # Load multiple assemblies
```

25.1.15 members

Library: *Tosh.Stdlib.Clr*

members

Lists or queries members for CLR types or pipeline objects.

Signature

```
members [has|get|props|fields|methods|events] [name] | members [type ...]
```

Argument	Description
subcommand-or-type	Optional subcommand keyword ('has', 'get', 'props', 'fields', 'methods', 'events') or a type name. (optional)
name	Operand for 'has'/'get': the member name to look up. (optional)

Pipeline input: scalar, record — Inspects the type of each piped object.

Output: Member descriptor objects with Name, Kind, MemberType, and other reflection metadata.

```
members string # List all members of String
DateTime.Now | members # List all members of a piped object
$obj | members has Name # Check whether $obj has a member named 'Name'
$obj | members get FullName # Return the descriptor for member 'FullName' (or null)
$obj | members props # Filter to properties only
$obj | members fields # Filter to fields only
$obj | members methods # Filter to methods only
```

25.1.16 methods

Library: *Tosh.Stdlib.Clr*

methods

Lists or queries public methods for CLR types or pipeline objects.

Signature

```
methods [has|get] [name] | methods [type ...]
```

Argument	Description
subcommand-or-type	Optional subcommand keyword ('has', 'get') or a type name. (optional)
name	Operand for 'has'/'get': the method name to look up. (optional)

Pipeline input: scalar, record — Lists or queries methods of each piped object's type.

Output: Method descriptor objects with Name, ReturnType, Parameters, and Signature properties.

```
methods string # List all methods of String
DateTime.Now | methods # List all methods of a piped object
$obj | methods has ToString # Check whether $obj has a method named 'ToString'
```

```
,
$obj | methods get ToString # Return the descriptor for method 'ToString' (or
null)
```

25.1.17 native-alloc

Library: Tosh.Stdlib.Clr

native-alloc

Allocates a native unmanaged buffer by byte size or interop type.

Aliases: alloc

Signature

```
native-alloc <bytes | type-name>
```

Argument	Description
bytes type-name	Byte count to allocate, or a native interop type name whose unmanaged size should be allocated.

Output: A native pointer (IntPtr) to the freshly allocated buffer.

```
alloc 64 # Allocate a 64-byte native buffer
alloc int32 # Allocate enough native memory for one Int32
```

25.1.18 native-free

Library: Tosh.Stdlib.Clr

native-free

Frees one or more native buffers allocated by alloc/native-alloc.

Signature

```
native-free [buffer ...]
```

Argument	Description
buffer ...	Native buffers to free. Buffers may also be supplied from the pipeline. (optional)

Output: Emits nothing; releases the native buffer as a side effect.

```
$buffer | native-free # Free a piped native buffer
native-free $a $b # Free explicit buffers
```

25.1.19 native-offsetof

Library: Tosh.Stdlib.Clr

native-offsetof

Returns the unmanaged field offset for a sequential or explicit-layout struct.

Aliases: offset-of

Signature

```
native-offsetof <type-name>[.<field-name>] [field-name]
```

Argument	Description
type-name[.field-name]	Sequential or explicit-layout struct type, optionally with a field suffix.
field-name	Public struct field name when it is not embedded in the first argument. (optional)

Output: An int — the byte offset of the named field within its containing struct.

```
offset-of System.Runtime.InteropServices.ComTypes.FILETIME dwLowDateTime #
  Offset by type and field
offset-of System.Runtime.InteropServices.ComTypes.FILETIME.dwHighDateTime #
  Offset by dotted path
```

25.1.20 native-read

Library: Tosh.Stdlib.Clr

native-read

Reads a C string, byte range, or native scalar/struct-layout value from native memory.

Aliases: read-buffer

Signature

```
native-read <cstring|bytes|type-name> [buffer|pointer] [-at <offset>]
[-count <bytes>]
```

Argument	Description
cstring bytes type-name	Read mode: a null-terminated C string, a byte array, or a supported native scalar/struct-layout type.
buffer pointer	NativeBuffer or pointer to read from. May be supplied from the pipeline. (optional)
-at	Byte offset from the buffer or pointer before reading. (optional) (<i>int</i>)
-count	Byte count. Required when the mode is 'bytes'; ignored otherwise. (optional) (<i>int</i>)

Output: The decoded value(s) read from the native buffer, in the requested format.

```
$buffer | native-read cstring # Read a C string from a native buffer
native-read bytes $buffer --count 16 # Read a byte range
native-read int32 $buffer --at 4 # Read an Int32 at an offset
```

25.1.21 native-sizeof

Library: Tosh.Stdlib.Clr

native-sizeof

Returns the unmanaged size of a supported native interop type.

Aliases: size-of

Signature

```
native-sizeof <type-name> [type-name ...]
```

Argument	Description
type-name ...	One or more supported native interop type names.

Output: An int — the size in bytes of the requested type or structure.

```
size-of int32 # Size of a primitive native type
size-of int32 double nint # Size several native types
```

25.1.22 native-write

Library: Tosh.Stdlib.Clr

native-write

Writes a C string, byte sequence, or struct-layout value into native memory.

Aliases: write-buffer

Signature

```
native-write <buffer|pointer> <value> [-at <offset>]
```

Argument	Description
buffer pointer	NativeBuffer or pointer to write into.
value	String, byte sequence, enum, primitive, pointer-sized value, or struct-layout value to write.
-at	Byte offset from the buffer or pointer before writing. (optional) (<i>int</i>)

Output: Emits nothing; writes the supplied value(s) into the native buffer as a side effect.

```
native-write $buffer "hello" # Write a C string
native-write $buffer [72 105 0] # Write explicit bytes
native-write $buffer 42 --at 8 # Write at an offset
```

25.1.23 new

Library: Tosh.Stdlib.Clr

new

Constructs a new CLR object, ToSh named type, or shell collection.

Signature

```
new <type-name> [ctor-args...]
```

Argument	Description
type-name	The CLR type name, ToSh named type, or shell collection type to construct.
ctor-args	Arguments to pass to the constructor. (optional)

Output: The newly constructed object.

```
var rng = new System.Random() # Construct a Random instance
var items = new list<String>("one", "two") # Generic collection construction
new System.Text.StringBuilder("hello").Append(" world").ToString() # Method
    chaining on a new object
```

Note

Tosh supports both the legacy ‘new <Type> ...’ command form and the newer C#-style ‘new Type(...)’ expression syntax. Shell collection types also support generic construction like ‘new list<String>(...)’.

25.1.24 props

Library: *Tosh.Stdlib.Clr*

props

Lists public properties for CLR types or pipeline objects (shortcut for ‘members props’).

Signature

props [type ...]

Argument	Description
type	Optional type name. When omitted, uses piped objects. (optional)

Pipeline input: scalar, record — Inspects the type of each piped object.

Output: Property descriptor objects (subset of ‘members’ output, filtered to Kind == Property).

```
string | props # List public properties of String
$obj | props # List public properties of a piped object
```

25.1.25 set-prop

Library: *Tosh.Stdlib.Clr*

set-prop

Sets or adds a property on a dynamic record.

Signature

set-prop [object] <name> <value>

Argument	Description
object	The target object. If omitted, reads from the pipeline. (optional)
name	The property name to set.
value	The value to assign.

Pipeline input: scalar, record — Uses the piped object as the target when the object argument is omitted.

Output: The modified object with the property set.

```
$obj | set-prop Name "value" # Set a property on a piped object
set-prop $obj Name "value" # Set a property by passing the object directly
```

Note

Deprecated. Prefer assignment syntax: '\$obj.Prop = value'.

25.1.26 type-of

Library: Tosh.Stdlib.Clr

type-of

Returns the CLR type or ToSh class type for each input object.

Signature

type-of [value...]

Argument	Description
value	One or more values to inspect. If omitted, reads from the pipeline. (optional)

Pipeline input: scalar, record — Returns the type of each piped object.

Output: The CLR Type or ToSh class descriptor for each input value.

```
type-of 42 # Get the type of a number
"hello" | type-of # Get the type of a piped value
ls | first | type-of # Get the type of a file system entry
```

25.1.27 types

Library: Tosh.Stdlib.Clr

types

Lists available CLR and ToSh shell types.

Signature

types [-a] [filter]

Argument	Description
filter	Optional name or pattern to filter types. (optional)
Flag / Option	Description
-a	Include all loaded assemblies, not just commonly used types.

Output: Type descriptor objects with Name, Namespace, and Assembly properties.

```
types System.String # Search for String type
types list # Search for list-like types
types map | where _.Namespace == ToSh # Filter to ToSh namespace
```

Note

Types searches both CLR types and ToSh shell types like 'list', 'array', 'dict', 'table', and 'tuple'.

25.2 Concurrency

25.2.1 async

Library: *Tosh.Stdlib.Concurrency*

async

Starts a callable or block and returns a future handle.

Signature

```
async <callable|block> [args ...]
```

Argument	Description
operation	A callable or block to run in the background.
args	Optional callable arguments. (optional)

Output: Returns a ShellFuture handle.

```
var f = async { sleep 0.2; echo done } # Start a deferred operation
```

25.2.2 await

Library: *Tosh.Stdlib.Concurrency*

await

Awaits a future from 'async' or a CLR Task, and replays its outputs.

Signature

```
await [awaitable]
```

Argument	Description
awaitable	A future from 'async', or a CLR Task/ValueTask. If omitted, read one from pipeline input. (optional)

Output: Replays the values produced by the operation, or the task's result.

```
var f = async { sleep 0.1; echo ok }; await $f # Await a future
var r = await ($p.SendPingAsync("8.8.8.8", 1000)); echo $r.Status # Await a
CLR task
```

25.2.3 channel

Library: *Tosh.Stdlib.Concurrency*

channel

Creates a new shell channel for sending and receiving values.

Signature

channel [capacity]

Argument	Description
capacity	Optional maximum number of items the channel can buffer. Omit for unbounded. (optional)

Output: Returns a ShellChannel value.

```
var ch = channel # Create an unbounded channel
var ch = channel 10 # Create a bounded channel that buffers up to 10 items
```

Note

Send values with channel-send, receive with channel-recv, and close with channel-close.

25.2.4 channel-close

Library: *Tosh.Stdlib.Concurrency*

channel-close

Closes a shell channel, signalling no further values will be sent.

Signature

channel-close <channel>

Argument	Description
channel	The channel to close. Omit to close channel(s) from pipeline input.

Output: Emits nothing; closes the channel(s) as a side effect.

```
channel-close $ch # Signal that no more values will be sent
```

Note

After closing, `channel-recv` will drain any buffered values and then complete. Closing an already-closed channel is a no-op.

25.2.5 `channel-recv`

Library: *Tosh.Stdlib.Concurrency*

`channel-recv`

Receives all values from a shell channel.

Signature

`channel-recv <channel>`

Argument	Description
<code>channel</code>	The channel to receive from. Omit to read from pipeline input.

Output: Streams every value from the channel until it is closed.

```
channel-recv $ch | each { |v| echo $v } # Stream all values from a channel
```

Note

Blocks until a value is available. Returns when the channel is closed and drained.

Note

A null payload is emitted as one value; a closed and drained channel emits no value.

25.2.6 `channel-select`

Library: *Tosh.Stdlib.Concurrency*

`channel-select`

Waits for the first available value from multiple channels.

Signature

`channel-select [channel ...]`

Argument	Description
<code>channels</code>	One or more channels to wait on. If omitted, reads channels from pipeline input. (optional)

Output: Returns a record with Index, Channel, and Value for the first available receive.

```
channel-select $ch1 $ch2 # Return the first channel with a value
```

25.2.7 channel-send

Library: *Tosh.Stdlib.Concurrency*

channel-send

Sends one or more values to a shell channel.

Signature

```
channel-send <channel> [values ...]
```

Argument	Description
channel	The channel to send to.
values	One or more values to send. Omit to send pipeline input instead. (optional)

Output: Emits nothing; sends each value to the channel as a side effect.

```
channel-send $ch hello # Send a single value to a channel
echo hello world | channel-send $ch # Forward pipeline items to a channel
```

Note

Blocks (asynchronously) when the channel is bounded and its buffer is full.

25.2.8 race

Library: *Tosh.Stdlib.Concurrency*

race

Executes operations concurrently and returns the first completion.

Signature

```
race <callable|block> <callable|block> [...]
```

Argument	Description
operation	A callable or block to execute. Provide two or more operations to race.

Output: Returns the first completed operation result.

```
race { sleep 0.1; echo slow } { sleep 0.01; echo fast } # Return the first
completed operation
```

25.2.9 scope

Library: *Tosh.Stdlib.Concurrency*

scope

Executes a block and automatically awaits all background jobs started inside it.

Signature

```
scope <block>
```

Argument	Description
block	A block that may start background jobs via spawn.

Output: Streams ShellJobCompletion records for every job started inside the scope.

```
scope { var j1 = spawn dotnet --version; var j2 = spawn dotnet --list-runtimes
      } # Start two background jobs and await both automatically
```

Note

All jobs registered during block execution are awaited on scope exit. If the block throws, every scope-owned job is killed before the exception propagates.

25.2.10 settle

Library: *Tosh.Stdlib.Concurrency*

settle

Executes operations concurrently and returns settled outcomes.

Signature

```
settle <callable|block> [...]
```

Argument	Description
operation	A callable or block to execute. Provide one or more operations to settle.

Output: Returns one outcome object per operation with Status, Value, and Error fields.

```
settle { echo ok } { throw boom } # Collect fulfilled and rejected outcomes
```

25.2.11 spawn

Library: *Tosh.Stdlib.Concurrency*

spawn

Starts an external command as a background job, or in the foreground with `-foreground`.

Signature

```
spawn [-f] <command> [arg ...]
```

Argument	Description
command	External command name or path to start as a background job.
args	Arguments to pass to the external command. (optional)

Flag / Option	Description
-f, -foreground	Run the process in the foreground (terminal passthrough) instead of as a background job.

Pipeline input: scalar, list — Optional pipeline input forwarded to the spawned process stdin.

Output: Returns a ShellJobInfo object for the started background job, or nothing when -foreground is used.

```
spawn dotnet --version # Start an external command in the background
echo hello | spawn cat # Feed pipeline input into a background process
spawn --foreground $bin_path --safe -c exit # Run a variable-path binary in
the foreground
```

Note

Use jobs to list active jobs and wait-for to await completion.

25.2.12 timeout

Library: Tosh.Stdlib.Concurrency

timeout

Runs an operation with a timeout.

Signature

```
timeout <seconds> <callable|block> [args ...]
```

Argument	Description
seconds	Timeout in seconds (supports fractional values).
operation	A callable or block to execute.
args	Optional callable arguments. (optional)

Output: Replays values from the operation when it completes in time.

```
timeout 2 { sleep 1; echo ok } # Complete within timeout
timeout 0.2 { sleep 1 } # Fail when elapsed
```

25.3 Data

25.3.1 as-file

Library: Tosh.Stdlib.Data

as-file

Materializes values into a temporary file and returns it as a file system entry.

Signature

```
as-file [text|json|csv] [value ...]
```

Argument	Description
format	Output format: text, json, or csv. Defaults to text. (optional)
value	Values to materialize. Can also come from pipeline. (optional)

Pipeline input: scalar, list, table — Materializes pipeline values into a temporary file.

Output: Returns a FileSystemEntry for the created temporary file.

```
ls | as-file # Materialize pipeline to temp file
as-file json $data # JSON format
```

Note

As-file materializes pipeline values into a temporary file and returns a file object you can pass to external executables.

25.3.2 complex

Library: *Tosh.Stdlib.Data*

complex

Builds a Complex value from numeric arguments or pipeline input.

Signature

```
complex [real [imaginary]]
```

Argument	Description
component	Zero, one, or two numeric components. If omitted, consumes pipeline input. (optional)

Pipeline input: scalar, list — Consumes scalar values or a single two-item collection and returns a Complex value.

Output: A Complex value.

```
complex 3 4 # Build a complex number from real and imaginary parts
echo [3, 4] | complex # Build a complex number from a single pair value
echo 3 4 | complex # Build a complex number from pipeline values
```

25.3.3 from

Library: *Tosh.Stdlib.Data*

from

Parses structured text into objects.

Signature

```
from <format> [options] [text]
```

Output: Parsed CLR objects from the text format.


```
echo "{\"name\":\"toast\"}" | from json
curl https://example/api | from json | flatten
cat data.toml | from toml
cat data.csv | from csv
```

Note

The ‘from’ and ‘to’ commands convert between text formats (json, csv, tsv, xml, toml) and CLR objects. Parsed values stay as CLR objects until you explicitly flatten them.

25.3.4 hash

Library: Tosh.Stdlib.Data

hash

Computes hashes for text input or files.

Signature

```
hash [algorithm] [path ...]
```

Argument	Description
algorithm	Hash algorithm: md5, sha1, sha256, sha384, or sha512. Defaults to sha256. (optional)
path ...	Files to hash. When omitted, hashes piped values as UTF-8 text. (optional) (<i>path-like</i>)

Output: ShellTextLine values containing the hex-encoded digest of each input.

```
echo hello | hash # Hash piped text with SHA-256
hash sha512 README.md # Hash a file with SHA-512
```

25.3.5 mat

Library: Tosh.Stdlib.Data

mat

Builds a Matrix from rows, scalars, or pipeline input.

Signature

```
mat [row ...]
```

Argument	Description
row	Zero or more scalar values or row sequences. If omitted, consumes pipeline input. (optional)

Pipeline input: scalar, list — Consumes scalar values or row sequences and returns a Matrix.

Output: A Matrix value.

```
mat [1, 2, 3] [4, 5, 6] # Build a matrix from explicit rows
echo [[1, 2], [3, 4]] | mat # Build a matrix from pipeline input
```

25.3.6 parse

Library: *Tosh.Stdlib.Data*

parse

Parses text input with a regular expression into shell record objects.

Signature

```
parse [-a] [-i] [-m] [-s] [-x] [-explicit-capture] <pattern|regex>
[text ...]
```

Argument	Description
pattern regex	The .NET regular expression pattern or Regex object to apply. (<i>string/regex</i>)
text ...	Optional explicit text values. When omitted, reads pipeline text. (optional)

Flag / Option	Description
-a, -all	Emit every match in each input value instead of only the first match.
-i, -ignore-case	Use case-insensitive matching.
-m, -multiline	Enable multiline mode so ^ and \$ match line boundaries.
-s, -singleline	Enable singleline mode so . matches newlines.
-x, -ignore-pattern-whitespace	Ignore unescaped whitespace and allow # comments in the regex pattern.
-explicit-capture	Only capture explicitly named or numbered groups.

Output: Structured records produced by the chosen parser (json/csv/yaml/etc.) — usually dictionaries, lists, or scalars.

```
ping -c 3 localhost | parse "time=(?<time_ms>[0-9.]+) ms"
echo "PID=42" | parse "PID=(?<Pid>[0-9]+)"
echo "first\nsecond" | parse -am "^(?<Value>\\w+)$" | get Value
```

Note

Parse and match use .NET regular expressions, including named groups and inline modifiers like '(?im)'.

25.3.7 to

Library: *Tosh.Stdlib.Data*

to

Serializes objects into structured text.

Signature

to <format> [options]

Output: Serialized text in the specified format.

```
ls | to json
ls | to csv
ls | to toml
ls | to json --compact
```

Note

The ‘from’ and ‘to’ commands convert between text formats (json, csv, tsv, xml, toml) and CLR objects. Parsed values stay as CLR objects until you explicitly flatten them.

25.3.8 vec

Library: Tosh.Stdlib.Data

vec

Builds a Vector from numeric arguments or pipeline input.

Signature

vec [item ...]

Argument	Description
item	Zero or more numeric items. If omitted, consumes pipeline input. (optional)

Pipeline input: scalar, list — Consumes numeric pipeline values or a single numeric collection and returns a Vector.

Output: A Vector value.

```
vec 1 2 3 # Build a vector from explicit items
echo 1 2 3 | vec # Build a vector from pipeline items
echo [1, 2, 3] | vec # Build a vector from a single list value
```

25.4 Filesystem**25.4.1 append-file**

Library: Tosh.Stdlib.Filesystem

append-file

Appends plain text to a file, creating it when needed.

Signature

```
append-file <path> [value...]
```

Argument	Description
path	The file path to append to. (<i>path-like</i>)
value ...	Optional explicit text values to append. When omitted, pipeline input becomes the appended text. (optional)

Pipeline input: scalar, record — When no explicit value arguments are supplied, pipeline values are rendered as plain text and appended to the file.

Output: Returns the resulting filesystem entry for the written file.

```
append-file ./notes.txt " more"
echo "tail line" | append-file ./notes.txt
```

Note

These commands use ToSh's plain-text serialization rules, not the rich table renderer, so they are safe for intentional file output.

25.4.2 back

Library: *Tosh.Stdlib.Shell*

[shell-only] — only available inside an interactive TōSh session.

back

Goes back to the previous directory in the stack.

Signature

```
back
```

Output: The FileSystemEntry for the previous directory.

```
back
```

25.4.3 basename

Library: *Tosh.Stdlib.Filesystem*

basename

Returns the file name portion of each path.

Signature

```
basename <path> [suffix]
```

Argument	Description
path	One or more paths to extract the filename from. (<i>path-like</i>)
suffix	Optional suffix to strip from the filename. (optional)

Pipeline input: scalar, list — Accepts piped path-like values.

Output: Returns the filename component of each path.

```
basename /home/user/file.txt
basename file.tar.gz .tar.gz # Strip suffix
```

25.4.4 cat

Library: *Tosh.Stdlib.Filesystem*

cat

Reads one or more files or piped text sources and emits their contents.

Signature

```
cat [-n|-b] [-svETAet] [path ...|-]
```

Argument	Description
path ... -	One or more file paths to concatenate, or '-' to read piped text input explicitly. (optional) (<i>path-like/string</i>)

Flag / Option	Description
-n	Number every emitted line.
-b	Number only non-blank lines.
-v	Display non-printing characters as '^X' or 'M-X'.
-s	Squeeze repeated blank lines into a single blank line.
-e	Equivalent to '-vE'.
-t	Equivalent to '-vT'.

Pipeline input: scalar, record, list — With no explicit paths, path-like pipeline values are treated as files when they all resolve to existing files; otherwise pipeline values are treated as text input. Use '-' explicitly when you want stdin-style text alongside file arguments.

Output: Returns plain text lines by default, or numbered record rows with 'Number' and 'Text' when '-n' or '-b' is used.

```
cat README.md
echo alpha beta | cat -
cat -n README.md
```

Note

With no explicit paths, 'cat' treats piped values as file paths only when every value resolves to an existing file. Otherwise it treats the pipeline as text input. Use '-' explicitly when mixing file paths with piped text.

25.4.5 cd

Library: *Tosh.Stdlib.Filesystem*

cd

Changes the current directory for the Tosh session.

Signature

```
cd [path | - | +]
```

Argument	Description
path	Absolute or relative directory path. Tilde expands to the home directory. (optional) (<i>path-like</i>)
-	Go back to the previous directory (pop from stack). (optional)
+	Go forward in the directory stack. (optional)

Output: The FileSystemEntry for the new working directory.

```
cd ~/projects # Absolute path
cd .. # Parent directory
cd - # Previous directory
cd # Home directory
```

25.4.6 chmod

Library: *Tosh.Stdlib.Filesystem*

chmod

Changes file permission bits.

Signature

```
chmod [-R] <mode> <path> [path...]
```

Argument	Description
mode	Permission mode string (e.g. 755, u+rw, a+x).
path	One or more files or directories. (<i>path-like</i>)

Flag / Option	Description
-R	Operate recursively on directories.

Output: Returns FileSystemEntry objects with long display for each changed path.

```
chmod +x script.sh
chmod 755 script.sh # Octal mode
chmod u+rw,go-w file.txt # Symbolic mode
chmod -R a+r ./docs # Recursive
```

25.4.7 chown

Library: *Tosh.Stdlib.Filesystem*

chown

Changes file owner and group.

Signature

```
chown [-R] <owner>[:group] <path> [path...]
```

Argument	Description
owner[:group]	Owner and optional group specification.
path	One or more files or directories. (<i>path-like</i>)

Flag / Option	Description
-R	Operate recursively on directories.

Output: Returns FileSystemEntry objects for each changed path.

```
chown user:group file.txt
chown -R www-data /var/www # Recursive change
```

25.4.8 close

Library: *Tosh.Stdlib.Filesystem*

close

Closes one or more managed file or server handles.

Signature

```
close <handle> [handle...]
```

Argument	Description
handle ...	One or more managed file handles to close. (optional)

Pipeline input: record — Consumes piped file handles when explicit handles are omitted.

Output: Closes the handles and does not emit pipeline output.

```
close $handle
echo $handle | close
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.9 copy-to

Library: *Tosh.Stdlib.Filesystem*

copy-to

Copies the remaining contents of one managed file handle into another compatible handle.

Signature

```
copy-to [source] <target>
```

Argument	Description
source	The readable managed file handle to copy from. (optional)
target	The writable managed file handle to copy into.

Pipeline input: record — Consumes a piped source handle when you only pass the target handle explicitly.

Output: Returns the number of bytes or text characters copied into the target handle.

```
copy-to $source $target
$source | copy-to $target
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.10 cp

Library: *Tosh.Stdlib.Filesystem*

cp

Copies a file or directory.

Signature

```
cp [-r] [-f] [-n] [-p] [-u] [-s] [-l] [-H] [-P] <source> [source ...]
<destination>
```

Argument	Description
source	One or more source files or directories. (<i>path-like</i>)
destination	The target path. (<i>path-like</i>)

Flag / Option	Description
-r	Copy directories recursively.
-f	Force overwrite of existing files.
-n	Do not overwrite existing files.
-p	Preserve file attributes such as timestamps.
-u	Only copy when the source is newer than the destination.
-s	Create symbolic links instead of copying files.
-l	Create hard links instead of copying files.
-H	Follow symbolic links on the command line (copy target, not link).
-P	Never follow symbolic links (copy the link itself). This is the default.

Output: Returns FileInfo or DirectoryInfo objects for each copied target.

```
cp file.txt backup.txt
cp -r src/ dst/ # Recursive directory copy
cp -s file.txt link.txt # Create a symbolic link
cp -l file.txt hardlink.txt # Create a hard link
```

25.4.11 df

Library: *Tosh.Stdlib.Filesystem*

df

Returns mounted file system usage information.

Aliases: mounts

Signature

```
df [-hTli] [-t type[,type...]] [-x type[,type...]] [-total] [-output
columns] [-show columns] [-hide columns] [-show-all] [path ...]
```

Argument	Description
path ...	Optional paths used to resolve the containing mounted filesystem. (optional) (<i>path-like</i>)

Flag / Option	Description
-h	Accepts the familiar human-readable flag; ToSh sizes are already typed and human-friendly.
-T	Ensures the filesystem type column is visible.
-l	Restricts the output to local filesystems.
-t <type[,type...]>	Includes only matching filesystem types.
-x <type[,type...]>	Excludes matching filesystem types.
-total	Appends a typed aggregate total row.
-i	Displays inode usage instead of block usage.
-output <columns>	Selects which filesystem properties are rendered.
-show <columns>	Select which properties are rendered (display-only column selection).
-hide <columns>	Hide specific properties from the output.
-show-all	Display every available column.

Pipeline input: list — Uses piped path-like values when explicit paths are omitted. Without path input, ‘df’ lists the full mounted filesystem set.

Output: Produces typed filesystem usage objects, with optional aggregate totals.

```
df --total
df -t ext4 --show FileSystem,Type,UsePercent,MountedOn
df . | get { FileSystem, MountedOn }
```

25.4.12 dirname

Library: *Tosh.Stdlib.Filesystem*

dirname

Returns the directory portion of each path.

Signature

dirname <path> [path...]

Argument	Description
path	One or more paths to extract the directory from. (<i>path-like</i>)

Pipeline input: scalar, list — Accepts piped path-like values.

Output: Returns the directory component of each path.

```
dirname /home/user/file.txt
```

25.4.13 dirs

Library: *Tosh.Stdlib.Shell*

[**shell-only**] — only available inside an interactive TōSh session.

dirs

Shows or manages the directory stack.

Signature

```
dirs [goto <index> | remove <index> | clear]
```

Argument	Description
subcommand	goto <index>, remove <index>, or clear. (optional)

Output: Lists the current directory stack.

```
dirs
dirs goto 2 # Navigate to stack entry
dirs clear # Clear the directory stack
```

25.4.14 du

Library: Tosh.Stdlib.Filesystem

du

Returns disk usage information for files and directories.

Aliases: disk-usage, usage

Signature

```
du [-a] [-s] [-d depth] [-h] [-c] [-x] [-time] [-t size] [-exclude
pattern] [-show columns] [-hide columns] [-show-all] [path ...]
```

Argument	Description
path ...	Optional roots to measure. (optional) (<i>path-like</i>)

Flag / Option	Description
-a	Include file rows as well as directory summaries.
-s	Summarize each root instead of emitting recursive rows.
-d <depth>	Limit recursion depth.
-h	Accepts the familiar human-readable flag; ToSh sizes are already typed and human-friendly.
-c	Appends a typed grand total row.
-x	Stay on the same filesystem as each requested root.
-time	Include the latest modified timestamp for each emitted row.
-t <size>	Exclude entries smaller than size (or with - prefix, larger than absolute size).
-threshold <size>	Alias for -t.
-exclude <pattern>	Exclude files matching the shell glob pattern.
-show <columns>	Select which properties are rendered (display-only column selection).
-hide <columns>	Hide specific properties from the output.
-show-all	Display every available column.

Pipeline input: list — Uses piped path-like roots when explicit paths are omitted. Falls back to the current directory when neither are present.

Output: Produces typed path-usage objects with optional modified-time metadata and aggregate totals.

```
du -s .
du -a -c --time
du -x ./projects | get { Name, Size, Modified }
```

25.4.15 exists

Library: *Tosh.Stdlib.Filesystem*

exists

Checks whether each path exists.

Signature

```
exists <path> [path...]
```

Argument	Description
path ...	One or more filesystem paths to test. Paths may also be supplied from the pipeline. (<i>path-like</i>)

Output: Structured records with Path and Exists fields.

```
exists README.md # Check a single path
glob "src/**/*.cs" | exists | where _.Exists # Filter path-test results
```

25.4.16 find

Library: *Tosh.Stdlib.Filesystem*

find

Recursively finds file system entries.

Signature

```
find [path ...] [-name pattern] [-iname pattern] [-path pattern]
[-ipath
pattern] [-regex pattern] [-iregex pattern] [-type f|d|l] [-perm mode]
[-maxdepth n] [-mindepth n] [-size +/-size] [-mtime +/-days]
[-newer-than duration] [-older-than duration] [-empty]
```

Argument	Description
root ...	One or more filesystem roots to search. (optional) (<i>path-like</i>)

Flag / Option	Description
-name <pattern>	Filter by shell-style name pattern.
-iname <pattern>	Case-insensitive name pattern.
-path <pattern>	Filter by shell-style path pattern against the root-relative path.
-ipath <pattern>	Case-insensitive path pattern.
-regex <pattern>	Filter by .NET regex against the root-relative path.
-iregex <pattern>	Case-insensitive regex filter against the root-relative path.
-type <file dir link>	Filter by filesystem entry kind.
-perm <mode>	Filter by Unix permission bits (octal). Prefix with - for 'at least' or / for 'any of'.
-maxdepth <n>	Limit recursion to at most N directory levels below the starting path.
-mindepth <n>	Skip results shallower than N directory levels.
-empty	Match only empty files and empty directories.
-size <expr>	Filter by size; accepts +N/-N suffixes c/k/M/G (e.g. -size +1M).
-mtime <expr>	Filter by modification age in days (+N for older, -N for newer).
-days <expr>	Alias for -mtime.
-newer-than <path-or-time>	Match entries modified after the given file or timestamp.
-older-than <path-or-time>	Match entries modified before the given file or timestamp.

Pipeline input: list — Uses piped path-like roots when explicit roots are omitted. Falls back to the current directory when neither are present.

Output: Returns typed filesystem entries with rich metadata that flow naturally through the object pipeline.

```
find . -name *.tosh
find . -path */src/*.cs # Match on relative path
find . -perm -755 # Files with at least rwxr-xr-x
find . -regex ".*\\.tosh$"
find . -iregex ".*readme.*"
```

25.4.17 findmnt

Library: Tosh.Stdlib.Filesystem

findmnt

Wraps the system findmnt utility, returning typed mounted-filesystem objects for JSON-backed invocations.

Signature

```
findmnt [-AflbR] [-S source] [-T target] [-M mountpoint] [-t types] [-O options] [-o columns] [path-or-device ...]
```

Argument	Description
[path-or-device ...]	Optional mountpoints or source devices to match. (optional) (<i>path-like/string</i>)
Flag / Option	Description
-S <source>	Match a source device or source specification.
-T <target>	Match the filesystem that contains a target path.
-M <mountpoint>	Match a specific mountpoint.
-t <types>	Limit the result set to specific filesystem types.
-O <options>	Limit the result set to mounts with matching options.
-R	Include submounts for matching filesystems.
-U	Drop duplicate targets.
-l	Return a flattened list instead of a hierarchy.
-A	Disable built-in filters and include everything findmnt normally hides.
-b	Render size-oriented columns in raw bytes.
-D	Use a 'df'-style display preset.
-I	Use a 'df -i'-style inode display preset.
-o <columns>	Select findmnt-style output columns such as 'TARGET,SOURCE,FSTYPE,OPTIONS'.
-output-all	Expose every selectable structured findmnt column.

Pipeline input: none — The structured 'findmnt' builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns reusable mounted-filesystem objects with nested child mounts when the underlying 'findmnt -json' output is hierarchical.

```
findmnt # Browse the mounted-filesystem tree as typed objects
findmnt -l | where _.Target.StartsWith("/run") # Flatten the mount tree and
filter by target path
findmnt -o TARGET,SOURCE,FSTYPE # Pick explicit findmnt-style display columns
without changing the underlying objects
```

Note

ToSh wraps 'findmnt -json -bytes -output-all' so mounted filesystems stay as typed objects in the pipeline while the default renderer can still show them as a tree with columns. The default result is hierarchical, so use '-l' when you want flat filtering in a pipeline, and shell-facing aliases like 'FsType' and 'FsRoot' match the visible column names. Output-format-only modes like '-pairs', '-raw', '-noheadings', polling, and verification currently fall back to the external 'findmnt' utility unchanged.

25.4.18 flush

Library: Tosh.Stdlib.Filesystem

flush

Flushes one or more managed file handles.

Signature

```
flush <handle> [handle...]
```

Argument	Description
handle ...	One or more managed file handles to flush. (optional)

Pipeline input: record — Consumes piped file handles when explicit handles are omitted.

Output: Flushes the handles and does not emit pipeline output.

```
flush $handle
echo $handle | flush
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.19 forward

Library: Tosh.Stdlib.Shell

[shell-only] — only available inside an interactive TōSh session.

forward

Goes forward to the next directory in the stack.

Signature

```
forward
```

Output: The FileSystemEntry for the next directory.

```
forward
```

25.4.20 glob

Library: Tosh.Stdlib.Filesystem

glob

Expands filesystem glob patterns.

Signature

```
glob [-a] <pattern> [pattern ...]
```

Argument	Description
pattern	One or more glob patterns to expand.

Flag / Option	Description
-a	Include hidden entries in results.

Pipeline input: list — Uses piped patterns when no arguments are given.

Output: Returns FileSystemEntry objects for each matched path.

```
glob *.txt
glob -a **/*.cs # Recursive with hidden files
```

25.4.21 is-dir

Library: Tosh.Stdlib.Filesystem

is-dir

Checks whether each path is a directory.

Signature

```
is-dir <path> [path...]
```

Argument	Description
path ...	One or more filesystem paths to test. Paths may also be supplied from the pipeline. (<i>path-like</i>)

Output: Structured records with Path and IsDirectory fields.

```
is-dir README.md # Check a single path
glob "src/**/*.cs" | is-dir | where _.IsDirectory # Filter path-test results
```

25.4.22 is-file

Library: Tosh.Stdlib.Filesystem

is-file

Checks whether each path is a file.

Signature

```
is-file <path> [path...]
```

Argument	Description
path ...	One or more filesystem paths to test. Paths may also be supplied from the pipeline. (<i>path-like</i>)

Output: Structured records with Path and IsFile fields.

```
is-file README.md # Check a single path
glob "src/**/*.cs" | is-file | where _.IsFile # Filter path-test results
```

25.4.23 is-link

Library: Tosh.Stdlib.Filesystem

is-link

Checks whether each path is a symbolic link.

Signature

```
is-link <path> [path...]
```

Argument	Description
path ...	One or more filesystem paths to test. Paths may also be supplied from the pipeline. <i>(path-like)</i>

Output: Structured records with Path and IsLink fields.

```
is-link README.md # Check a single path
glob "src/**/*.cs" | is-link | where _.IsLink # Filter path-test results
```

25.4.24 length

Library: *Tosh.Stdlib.Filesystem*

length

Returns the current stream length for one or more managed file handles.

Signature

```
length [handle ...]
```

Argument	Description
handle ...	One or more managed file handles whose current stream length should be reported. <i>(optional)</i>

Pipeline input: record — Consumes piped file handles when explicit handles are omitted.

Output: Returns the current underlying stream length for each supplied handle.

```
length $handle
echo $handle | length
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.25 ln

Library: *Tosh.Stdlib.Filesystem*

ln

Creates hard links or symbolic links.

Signature

```
ln [-s] [-f] <target> <link-path>
```

Argument	Description
target	The target path that the link will point to. <i>(path-like)</i>
link-path	The path where the link will be created. <i>(path-like)</i>
Flag / Option	Description
-s	Create a symbolic link instead of a hard link.
-f	Remove existing destination files.

Output: Returns a FileSystemEntry for the created link.

```
ln original.txt hardlink.txt
ln -s /usr/bin/python3 ./python # Create symbolic link
```

25.4.26 ls

Library: *Tosh.Stdlib.Filesystem*

ls

Lists file system entries.

Signature

```
ls [-aAldRFihrSt] [-T [depth]] [-sort name|size|time] [-time
modified|access|created] [-group-directories-first] [-tree [depth]]
[-show columns] [-hide columns] [-show-all] [path ...]
```

Argument	Description
path ...	Optional directories or files to list. (optional) <i>(path-like)</i>

Flag / Option	Description
-a	Include hidden entries.
-A	Include hidden entries while matching standard almost-all ls behavior.
-l	Use the long listing view.
-d	List directory arguments themselves instead of their contents.
-R	Traverse directories recursively.
-F	Classify names with shell-style suffixes like '*' and '@'.
-i	Include inode metadata in the compact table view.
-r	Reverse the current sort order.
-S	Sort by size descending.
-t	Sort by the active time field descending.
-sort <name size time>	Choose the primary listing sort field.
-time <modified access created>	Choose which time field long listings and time sorts use.
-group-directories-first	Group directories ahead of files before applying the primary sort.
-la	Combine hidden and long listing output.
-T <depth>	Render entries as a tree (optional max depth).
-tree	Render entries as a tree.
-show <columns>	Select which properties are rendered (display-only column selection).
-hide <columns>	Hide specific properties from the output.
-show-all	Display every available column.

Pipeline input: none — Ls is still explicit-arg-first; path input is not yet consumed from the pipeline.

Output: Produces typed filesystem entries that the display layer renders as shell tables by default.

```
ls -la
ls -R --group-directories-first
ls -l --time access | where _.Type == file | get { Name, Accessed }
```

Note

Filesystem metadata stays typed in the pipeline, even when Tosh renders it like a shell table.

25.4.27 lsblk

Library: *Tosh.Stdlib.Filesystem*

lsblk

Wraps the system lsblk utility, returning typed block-device objects for JSON-backed invocations.

Signature

```
lsblk [-aAdbflmpStzDNv] [-I list] [-e list] [-x column] [-o columns]
```

[device ...]

Argument	Description
[device ...]	Optional device paths or names to scope the query to. (optional) <i>(path-like/string)</i>
Flag / Option	Description
-a	Include empty devices.
-A	Hide empty devices.
-d	Suppress dependencies and child devices.
-b	Render size-oriented columns in raw bytes.
-f	Use the filesystem-oriented display preset.
-m	Use the permissions-oriented display preset.
-t	Use the topology-oriented display preset.
-D	Use the discard-oriented display preset.
-z	Use the zoned-device display preset.
-M	Use the model/serial/vendor display preset.
-L	Use the label/UUID/partition display preset.
-p	Show full device paths.
-S	Restrict the query to SCSI devices.
-N	Restrict the query to NVMe devices.
-v	Restrict the query to virtio devices.
-I <majors>	Include only the specified major numbers.
-e <majors>	Exclude the specified major numbers.
-x <column>	Sort by an lsblk column name such as 'NAME', 'SIZE', or 'PATH'.
-o <columns>	Select lsblk-style output columns such as 'NAME,SIZE,FSTYPE,MOUNTPOINTS'.
-0	Expose every selectable structured lsblk column.

Pipeline input: none — The structured 'lsblk' builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns reusable block-device objects with nested child devices when the underlying 'lsblk -json' output is hierarchical.

```
lsblk # Browse block devices as a tree-with-columns object table
lsblk -l -f | where _.FsType == "ntfs" # Flatten the block-device view and
    filter by filesystem type
lsblk -o NAME,PATH,SIZE # Pick explicit lsblk-style display columns without
    changing the underlying objects
```

Note

ToSh wraps 'lsblk -json -bytes -output-all' so block devices stay as typed objects in the pipeline while the default renderer can still show them as a tree with columns. The default result is hierarchical, so use '-l' when you want flat filtering in a pipeline, and shell-facing aliases like 'FsType', 'FsVer', and 'FsAvail' match the visible column names. Output-format-only flags like '-pairs', '-raw', and '-noheadings' currently fall back to the external 'lsblk' utility unchanged.

25.4.28 `mkdir`

Library: Tosh.Stdlib.Filesystem

`mkdir`

Creates one or more directories.

Signature

```
mkdir [-pv] [-m mode] <path> [path...]
```

Argument	Description
path	One or more directory paths to create. (<i>path-like</i>)

Flag / Option	Description
-p	Create parent directories as needed; no error if existing.
-v	Print a message for each created directory.
-m <mode>	Set file mode (as in <code>chmod</code>), e.g. 755.

Pipeline input: list — Accepts piped path-like values.

Output: Returns `DirectoryInfo` objects for each created directory.

```
mkdir newdir
mkdir -p a/b/c # Create nested directories
```

25.4.29 `mkdir-temp`

Library: Tosh.Stdlib.Filesystem

`mkdir-temp`

Creates a temporary directory and returns it as a file system entry.

Signature

```
mkdir-temp [prefix]
```

Argument	Description
prefix	Optional prefix for the directory name. Defaults to 'tosh'. (optional)

Output: Returns a `FileSystemEntry` for the created temporary directory.

```
mkdir-temp
mkdir-temp myapp # Custom prefix
```

25.4.30 `mv`

Library: Tosh.Stdlib.Filesystem

mv

Moves or renames files and directories.

Signature

```
mv [-nufTiv] [-t directory] <source> [source ...] <destination>
```

Argument	Description
source ...	One or more source paths to move. (<i>path-like</i>)
destination	The destination path or directory. (optional) (<i>path-like</i>)

Flag / Option	Description
-n	Do not overwrite existing files.
-u	Only move when the source is newer than the destination.
-f	Force overwrite for file targets, clearing a previous '-n'.
-t <directory>	Use an explicit destination directory.
-T	Treat the destination as a normal path, not a target directory.
-v	Explain what is being done.

Pipeline input: none — The current 'mv' implementation is explicit-arg-first and does not consume pipeline input.

Output: Returns FileInfo or DirectoryInfo objects for the moved targets.

```
mv old.txt new.txt
mv -n *.txt archive/
mv -t archive alpha.txt beta.txt
```

Note

Mv now overwrites existing file targets by default, closer to Unix 'mv'. '-n', '-u', '-t', and '-T' are available to control that behavior explicitly.

25.4.31 open-file

Library: *Tosh.Stdlib.Filesystem*

open-file

Opens one or more files as managed text or binary handles.

Signature

```
open-file [-read|-write|-append] [-binary] [-encoding <name>]
<path> [path...]
```

Argument	Description
path ...	One or more file paths to open. (optional) (<i>path-like</i>)

Flag / Option	Description
-read, -r	Open for reading. This is the default.
-write, -w	Open for writing and replace any previous contents.
-append, -a	Open for writing at the end of the file.
-binary, -b	Open a binary handle instead of a text handle.
-encoding <name>	Use a specific text encoding for text writers.

Pipeline input: list — Consumes piped path-like input when explicit file paths are omitted.

Output: Returns managed file-handle objects that can be passed to ‘read-from’, ‘read-line-from’, ‘read-to-end’, ‘write-to’, ‘write-line-to’, ‘flush’, ‘close’, ‘seek’, ‘position’, ‘length’, and ‘copy-to’.

```
open-file ./notes.txt
open-file --write ./notes.txt
open-file --binary --append ./data.bin
```

Note

Open-file is the start of ToSh’s managed stream system. It returns explicit text or binary handle objects instead of hiding file resources behind implicit properties. Active handles are also visible through ‘\$tosh.Session.OpenHandles’ and ‘\$tosh.Session.OpenHandleCount’.

25.4.32 position

Library: Tosh.Stdlib.Filesystem

position

Returns the current stream position for one or more managed file handles.

Signature

position [handle ...]

Argument	Description
handle ...	One or more managed file handles whose current position should be reported. (optional)

Pipeline input: record — Consumes piped file handles when explicit handles are omitted.

Output: Returns the current byte position for each supplied handle when that position can be reported safely.

```
position $handle
echo $handle | position
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.33 pwd

Library: *Tosh.Stdlib.Filesystem*

pwd

Returns the current directory as a DirectoryInfo object.

Signature

pwd

Output: Returns the current working directory as a DirectoryInfo object.

```
pwd # Print the current working directory
pwd | get .FullName # Get the full path as a string
```

25.4.34 read-bytes

Library: *Tosh.Stdlib.Filesystem*

read-bytes

Reads one or more files and returns each file as a byte array.

Signature

read-bytes <path> [path...]

Argument	Description
path ...	One or more file paths to read as raw byte arrays. (optional) (<i>path-like</i>)

Pipeline input: list — Consumes piped path-like input when explicit file paths are omitted.

Output: Returns one byte-array value per file.

```
read-bytes ./image.bin
read-bytes ./image.bin | type-of
```

Note

These commands accept normal path-like values, including strings, FileInfo, and ToSh FileSystemEntry objects.

25.4.35 read-file

Library: *Tosh.Stdlib.Filesystem*

read-file

Reads one or more files and returns each file as a single string value.

Signature

```
read-file <path> [path...]
```

Argument	Description
path ...	One or more file paths to read as whole-text values. (optional) (<i>path-like</i>)

Pipeline input: list — Consumes piped path-like input when explicit file paths are omitted.

Output: Returns one string value per file.

```
read-file ./notes.txt
ls *.md | first | read-file
```

Note

These commands accept normal path-like values, including strings, `FileInfo`, and `ToSh FileSystemEntry` objects.

25.4.36 read-from

Library: *Tosh.Stdlib.Filesystem*

read-from

Reads a text or binary chunk from an open managed file handle.

Signature

```
read-from [handle] [count]
```

Argument	Description
handle	The managed file handle to read from. (optional)
count	Optional chunk size. Defaults to <code>{StreamCommandUtilities.DefaultReadChunkSize}</code> . (optional) (<i>int</i>)

Pipeline input: record — Consumes a piped file handle when no explicit handle argument is supplied.

Output: Returns a string chunk for text handles or a byte array chunk for binary handles.

```
$handle | read-from 64
read-from $handle 4096
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.37 read-line-from

Library: *Tosh.Stdlib.Filesystem*

read-line-from

Reads the next text line from an open managed file handle.

Signature

`read-line-from [handle]`

Argument**Description**

handle

The managed text file handle to read from. (optional)

Pipeline input: record — Consumes a piped file handle when no explicit handle argument is supplied.

Output: Returns the next line as a `ShellTextLine` value, or nothing at end-of-file.

```
$handle | read-line-from
read-line-from $handle
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.38 read-lines

Library: *Tosh.Stdlib.Filesystem*

read-lines

Reads one or more files and emits their contents line-by-line.

Signature

`read-lines <path> [path...]`

Argument**Description**

path ...

One or more file paths to read line-by-line. (optional)
(*path-like*)

Pipeline input: list — Consumes piped path-like input when explicit file paths are omitted.

Output: Returns ShellTextLine values, one per line across the supplied files.

```
read-lines ./notes.txt
read-lines ./notes.txt | grep error
```

Note

These commands accept normal path-like values, including strings, FileInfo, and ToSh FileSystemEntry objects.

25.4.39 read-to-end

Library: Tosh.Stdlib.Filesystem

read-to-end

Reads the remainder of an open managed file handle.

Signature

```
read-to-end [handle]
```

Argument	Description
handle	The managed file handle to read from. (optional)

Pipeline input: record — Consumes a piped file handle when no explicit handle argument is supplied.

Output: Returns the remaining text or bytes from the handle.

```
$handle | read-to-end
read-to-end $handle
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.40 readlink

Library: Tosh.Stdlib.Filesystem

readlink

Returns symbolic link targets.

Signature

```
readlink [-f] <path> [path...]
```

Argument	Description
path	One or more symbolic link paths. (<i>path-like</i>)

Flag / Option	Description
-f	Resolve the final target, following chains of symbolic links.

Output: Returns the target path of each symbolic link.

```
readlink ./link
readlink -f ./chain # Canonicalize
```

25.4.41 realpath

Library: *Tosh.Stdlib.Filesystem*

realpath

Returns fully resolved absolute paths.

Signature

```
realpath <path> [path...]
```

Argument	Description
path	One or more paths to resolve. (<i>path-like</i>)

Output: Returns fully resolved absolute path strings.

```
realpath ./relative/path
```

25.4.42 rm

Library: *Tosh.Stdlib.Filesystem*

rm

Removes files or directories.

Signature

```
rm [-rfiv] <path> [path...]
```

Argument	Description
path	One or more files or directories to remove. (<i>path-like</i>)

Flag / Option	Description
-r	Remove directories and their contents recursively.
-f	Ignore nonexistent files; never prompt.
-i	Prompt before each removal.
-v	Explain what is being done.

Pipeline input: list — Accepts piped path-like values.

Output: Returns a RemovalResult with total size and descendant tree.

```
rm temp.txt
rm -rf build/ # Force recursive delete
```

25.4.43 seek

Library: *Tosh.Stdlib.Filesystem*

seek

Moves an open managed file handle to a new position and returns the handle for continued piping.

Signature

```
seek [handle] <offset> [begin|current|end]
```

Argument	Description
handle	The managed file handle to reposition. (optional)
offset	The byte offset to seek to or by. (<i>long</i>)
origin	The seek origin: begin, current, or end. Defaults to begin. (optional)

Pipeline input: record — Consumes a piped file handle when no explicit handle argument is supplied.

Output: Moves the handle and returns it so you can continue piping into ‘read-from’, ‘read-line-from’, ‘read-to-end’, or ‘copy-to’.

```
seek $handle 0 begin
$handle | seek 128 current
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.44 stat

Library: *Tosh.Stdlib.Filesystem*

stat

Returns detailed metadata for one or more paths.

Signature

```
stat [-L] [-f|-filesystem] [--show columns] [--hide columns]
[--show-all] <path> [path...]
```

Argument	Description
path	One or more paths to inspect. (<i>path-like</i>)

Flag / Option	Description
-L	Dereference symlinks before reading metadata.
-f, -filesystem	Return filesystem usage information for the containing mount instead of file-entry metadata.
-show <columns>	Select which properties are rendered.
-hide <columns>	Hide specific properties from the output.
-show-all	Display every available column.

Pipeline input: list — Uses piped path-like values when explicit paths are omitted.

Output: Typed filesystem-entry metadata, or filesystem-usage objects in -f mode.

```
stat ./README.md
stat -L ./link.txt # Dereference symlink
stat -f . | get { RequestedPath, FileSystem, MountedOn } # Filesystem mode
```

25.4.45 tempfile

Library: Tosh.Stdlib.Filesystem

tempfile

Creates a temporary file and returns it as a file system entry.

Signature

```
tempfile [prefix] [extension]
```

Argument	Description
prefix	Optional prefix for the file name. Defaults to 'tosh'. (optional)
extension	Optional file extension. (optional)

Output: Returns a FileSystemEntry for the created temporary file.

```
tempfile
tempfile data .csv # Custom prefix and extension
```

25.4.46 touch

Library: Tosh.Stdlib.Filesystem

touch

Creates files or updates access and modification timestamps.

Signature

```
touch [-acm] [-d time|-r file] [-c] <path> [path...]
```

Argument	Description
path ...	One or more filesystem paths to create or timestamp. (path-like)

Flag / Option	Description
-a	Update only the access time.
-m	Update only the modification time.
-c	Do not create missing files.
-d <time>	Use an explicit timestamp value.
-r <path>	Copy the timestamp from another file or directory.

Pipeline input: list — Consumes piped path-like input when explicit paths are omitted.

Output: Returns updated FileInfo or DirectoryInfo objects for the paths it touched.

```
touch notes.txt
touch -c -a notes.txt
touch -d 2026-03-28T12:00:00 notes.txt
```

Note

Touch now supports ‘-a’, ‘-m’, ‘-c’, ‘-d’, and ‘-r’, plus grouped short flags like ‘-am’.

25.4.47 tree

Library: Toshi.Stdlib.Filesystem

tree

Wraps the system tree utility, returning typed tree-entry objects.

Signature

```
tree [-adfl level] [-show columns] [-hide columns] [-show-all] [path
...]
```

Argument	Description
path	Directory paths to tree. Defaults to current directory. (optional) (<i>path-like</i>)

Flag / Option	Description
-a	Include hidden files.
-d	List directories only.
-f	Print full path for each entry.
-L <level>	Limit recursion depth.
-show <columns>	Select which properties are rendered.
-hide <columns>	Hide specific properties from the output.
-show-all	Display every available column.

Output: Returns typed tree-entry objects.

```
tree
tree -L 2 -d # Directories only, 2 levels
```

25.4.48 write-bytes

Library: Toshi.Stdlib.Filesystem

write-bytes

Writes byte-oriented content to a file, replacing any previous contents.

Signature

```
write-bytes <path> [bytes...]
```

Argument	Description
path	The file path to create or replace. (<i>path-like</i>)
bytes ...	Optional explicit byte-oriented values. When omitted, pipeline input becomes the byte payload. (optional)

Pipeline input: scalar, record — When no explicit byte values are supplied, pipeline input is converted into a byte payload.

Output: Returns the resulting filesystem entry for the written file.

```
write-bytes ./data.bin [1, 2, 3, 255]
read-bytes ./source.bin | write-bytes ./copy.bin
```

Note

Write-bytes accepts raw byte arrays, byte-like collections, and byte-convertible scalar values. Strings are encoded as UTF-8 in this first slice.

25.4.49 write-file

Library: *Tosh.Stdlib.Filesystem*

write-file

Writes plain text to a file, replacing any previous contents.

Signature

```
write-file <path> [value...]
```

Argument	Description
path	The file path to create or replace. (<i>path-like</i>)
value ...	Optional explicit text values to write. When omitted, pipeline input becomes the file body. (optional)

Pipeline input: scalar, record — When no explicit value arguments are supplied, pipeline values are rendered as plain text and written to the file.

Output: Returns the resulting filesystem entry for the written file.

```
write-file ./notes.txt "hello world"
echo alpha beta | write-file ./notes.txt
```

Note

These commands use ToSh's plain-text serialization rules, not the rich table renderer, so they are safe for intentional file output.

25.4.50 write-line-to

Library: Tosh.Stdlib.Filesystem

write-line-to

Writes one or more text lines to an open managed text file handle.

Signature

```
write-line-to <handle> [value...]
```

Argument	Description
handle	The managed text file handle to write into.
value ...	Optional explicit values to write as one line. When omitted, each pipeline value becomes its own line. (optional)

Pipeline input: scalar, record — When no explicit values are supplied, each pipeline value is written as its own line.

Output: Writes line-oriented text into the handle and does not emit pipeline output.

```
write-line-to $handle hello world
echo alpha beta | write-line-to $handle
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.4.51 write-to

Library: Tosh.Stdlib.Filesystem

write-to

Writes plain text or bytes to an open managed file handle.

Signature

```
write-to <handle> [value...]
```

Argument	Description
handle	The managed file handle to write into.
value ...	Optional explicit values to write. When omitted, pipeline input becomes the written payload. (optional)

Pipeline input: scalar, record — When no explicit values are supplied, pipeline values are written into the handle.

Output: Writes into the handle and does not emit pipeline output.

```
write-to $handle hello world
echo alpha beta | write-to $handle
```

Note

These commands work with managed file handles returned by ‘open-file’ or by ‘FileSystemEntry’ methods like ‘OpenText()’ and ‘OpenRead()’. ‘seek’ returns the handle so you can keep piping through the stream workflow, while ‘copy-to’ copies from one compatible handle into another.

25.5 Functional

25.5.1 compose

Library: *Tosh.Stdlib.Functional*

compose

Composes two or more callables into a single callable that chains them left-to-right.

Signature

```
compose <callable1> <callable2> [callable ...]
```

Argument	Description
callable1	The first callable in the chain.
callable2	The second callable in the chain.
callable	Additional callables to chain. (optional)

Output: A single callable that applies all given callables in left-to-right order.

```
compose func(x) => ($x + 1) func(x) => ($x * 2) # Compose increment and
double
$f = compose $parse $validate $transform; $f $input # Build a processing
pipeline
```

25.5.2 converge

Library: *Tosh.Stdlib.Functional*

converge

Applies a callable to the seed repeatedly until two consecutive results are equal, then yields that stable value.

Signature

```
converge <seed> <callable|block>
```

Argument	Description
seed	The initial value to start iterating from.
callable block	A function applied repeatedly until two consecutive results are equal.

Output: The first stable (fixed-point) value where consecutive applications produce equal results.

```
converge 100 func(x) => ($x / 2 + 50 / $x) # Newton's method for square root
of 100
```

25.5.3 curry

Library: *Tosh.Stdlib.Functional*

curry

Converts a fixed-arity callable into a curried callable value.

Signature

```
curry <callable>
```

Argument	Description
callable	A fixed-arity callable value to curry.

Pipeline input: none — ‘curry’ returns a callable that accumulates arguments until the original callable is saturated.

Output: Returns a curried callable. Calling it with too few arguments returns another callable; calling it with enough arguments runs the original callable.

```
var add3 = func(a, b, c) => ($a + $b + $c); var curried = curry $add3 #
Create a curried callable
var step1 = invoke $curried 1; var step2 = invoke $step1 2; invoke $step2 39
# Invoke a curried callable one step at a time
```

25.5.4 cycle

Library: *Tosh.Stdlib.Functional*

cycle

Infinitely repeats the pipeline items in order.

Signature

```
... | cycle
```

Pipeline input: scalar, record — Items to repeat cyclically.

Output: The pipeline items repeated in order, infinitely.

```
echo 1 2 3 | cycle | first 9 # Cycle a sequence
[a b c] | cycle | first 7 # Cycle an array
```

Note

Produces an infinite sequence. Always pair with ‘first’, ‘take-while’, or ‘take-until’ to bound the output.

25.5.5 invoke

Library: *Tosh.Stdlib.Functional*

invoke

Invokes a callable value such as a lambda or function object.

Signature

invoke <callable> [arg ...]

Argument	Description
callable	A callable value such as an anonymous 'func(...)' expression or other shell callable object.
arg ...	Optional positional arguments passed to the callable. (optional)

Pipeline input: none — 'invoke' is explicit-argument based for now. It does not automatically feed the current pipeline into the callable.

Output: Returns whatever values the callable yields.

```
invoke (func(x) => ($x * 2)) 21 # Invoke an inline anonymous function
var add = func(x, y) => ($x + $y); invoke $add 3 4 # Store a lambda and
invoke it later
var factor = 3; var scale = func(x) => ($x * $factor); invoke $scale 7 #
Invoke a closure that captures lexical state
```

25.5.6 iterate

Library: Tosh.Stdlib.Functional

iterate

Generates an infinite sequence by repeatedly applying a callable to the previous result, starting from a seed. Pair with 'take' or 'take-while' to bound the output.

Signature

iterate <seed> <callable|block>

Argument	Description
seed	The initial value of the sequence.
callable block	A function applied to the previous value to produce the next.

Output: An infinite sequence: seed, f(seed), f(f(seed)), ...

```
iterate 1 func(x) => ($x * 2) | first 10 # Powers of 2
iterate 0 { _ + 1 } | take-until { _ > 5 } # Counting sequence bounded by
take-until
```

Note

Produces an infinite sequence. Always pair with 'first', 'take-until', or 'take-while' to bound the output.

25.5.7 partial

Library: *Tosh.Stdlib.Functional*

partial

Binds leading arguments to a callable and returns a new callable value.

Signature

```
partial <callable> [arg ...]
```

Argument	Description
callable	A callable value to partially apply.
arg ...	Leading positional arguments to bind ahead of time. (optional)

Pipeline input: none — ‘partial’ is explicit-argument based and returns a new callable value.

Output: Returns a callable value that prepends the bound arguments before invoking the original callable.

```
var add = func(x, y) => ($x + $y); var inc = partial $add 1; invoke $inc 41 #
    Bind the first argument of a callable
invoke (partial (func(a, b, c) => ("{$a}-{$b}-{$c}")) alpha beta) gamma #
    Bind multiple leading arguments
```

25.5.8 recur

Library: *Tosh.Stdlib.Functional*

recur

Generates an infinite sequence from seed values using a recurrence relation. The callable receives the last N values (matching seed count) and returns the next value.

Signature

```
recur <seeds> <callable|block>
```

Argument	Description
seeds	The initial values of the recurrence, e.g. (0, 1) for Fibonacci.
callable block	A function applied to the last N values to produce the next. N matches the seed count.

Output: An infinite sequence: seeds..., f(seeds), f(seed[1..], f(seeds)), ...

```
recur (0, 1) func(a, b) => ($a + $b) | first 10 # Fibonacci sequence
recur (1, 1, 1) func(a, b, c) => ($a + $b + $c) | first 10 # Tribonacci
sequence
```

Note

Produces an infinite sequence. Always pair with ‘first’, ‘take-while’, or ‘take-until’ to bound the output.

25.5.9 repeat

Library: *Tosh.Stdlib.Functional*

repeat

Produces a sequence of the same value repeated. Without a count argument, repeats infinitely.

Signature

```
repeat <value> [count]
```

Argument	Description
value	The value to repeat infinitely.
count	Optional maximum number of repetitions. (optional)

Output: The same value repeated.

```
repeat 0 | first 5 # Five zeros
repeat hello 3 # Three hellos
```

Note

Without a count, produces an infinite sequence. Pair with ‘first’, ‘take-while’, or ‘take-until’ to bound.

25.5.10 repeatedly

Library: *Tosh.Stdlib.Functional*

repeatedly

Produces an infinite sequence by re-evaluating a block for each item. The block receives the 0-based index.

Signature

```
repeatedly <callable|block>
```

Argument	Description
callable block	A block or function to evaluate each time. Receives the current index as <code>\$__</code> and as the first argument.

Output: Infinite sequence of evaluated block results.

```
repeatedly { random int 1 100 } | first 5 # Five random numbers
repeatedly func(i) => ($i * $i) | first 6 # Squares via index
```

Note

Produces an infinite sequence. Always pair with ‘first’, ‘take-while’, or ‘take-until’ to bound the output.

Note

Unlike ‘repeat’, this re-evaluates the block for each item, so side effects and randomness work.

25.5.11 unfold

Library: Tosh.Stdlib.Functional

unfold

Generates values from a seed by repeatedly applying a callable. The callable receives the current state and must return a [value, next-state] pair, or null to stop.

Signature

```
unfold <seed> <callable|block>
```

Argument**Description**

seed

The initial state passed to the callable.

callable|block

A function that receives state and returns [value, next-state] or null to stop.

Output: A sequence of values produced by the callable until it returns null.

```
unfold 1 func(n) => if ($n <= 5) { [$n ($n + 1)] } else { null } # Generate 1
    through 5
unfold [0 1] func(s) => [($s[0]) [($s[1]) ($s[0] + $s[1])]] # Fibonacci
    sequence
```

25.6 Math**25.6.1 round**

Library: Tosh.Stdlib.Maths

round

Rounds a number to the specified number of decimal places.

Signature

```
round [value] [decimals]
```

Argument**Description**

value-or-decimals

A number to round (with optional decimals), or just the decimal count when input is piped.

decimals

Number of decimal places to round to. Defaults to 0. (optional)

Pipeline input: scalar — Accepts a numeric value to round when only the decimal count is passed as an argument.

Output: The rounded value as a double.

```
round 3.14159 2 # Round a literal to 2 decimal places
echo 3.14159 | round 2 # Round a piped value to 2 decimal places
echo 3.7 | round # Round a piped value to nearest integer
```

25.7 Network

25.7.1 http

Library: *Tosh.Stdlib.Net*

http

Sends HTTP requests and returns structured response objects or decoded bodies.

Signature

```
http
<get|post|put|patch|delete|head|options|request|send|serve|host|
servers|stop>
...
```

Argument	Description
<get post put patch delete head options> <url>	Send an HTTP request immediately. (optional)
request <method> <url>	Build an immutable HTTP request definition without sending it yet. (optional)
send [request]	Send an <code>HttpRequestDefinition</code> or <code>HttpRequestMessage</code> from an argument or the pipeline. (optional)
serve host <dir>	Start a temporary HTTP file server rooted at a directory and return a live server handle. (optional)
servers	List open HTTP file server handles. (optional)
stop [handle id ...]	Stop one or more HTTP file servers, or all of them when no target is provided. (optional)

Flag / Option	Description
-header <name> <value>	Add a request header. Repeatable.
-json <value>	Serialize a value as JSON request content.
-body <text>	Send plain text request content.
-file <path>	Send request content from a file.
-form <record>	Send application/x-www-form-urlencoded content from a record-like value.
-content-type <value>	Override the request content type.
-timeout <duration>	Set the per-request timeout.
-bearer <token>	Add a Bearer authorization header.
-auth basic <user> <pass>	Add a Basic authorization header.
-follow -no-follow	Control redirect following for the request.
-as <response json text bytes lines>	Choose how the response should be materialized.
-out <path>	Write the raw response body bytes to a file.
-fail	Turn HTTP non-success status codes into diagnostics instead of returning a response object/body.
-browse	For 'http serve', render a lightweight browser page with directory listings and share metadata.
-upload	For 'http serve', accept uploads. Browser uploads and raw PUT/POST uploads are both supported.
-once	For 'http serve', close the server after the first handled request.
-bind <address>	Bind 'http serve' to a specific address.
-lan	Bind 'http serve' to all interfaces and advertise LAN-friendly share URLs for other devices.
-port <port>	Bind 'http serve' to a specific port. Use '0' to request an ephemeral port.
-index <file>	Serve this index file for directories before directory listings.
-token <token>	Protect 'http serve' with a fixed token. The returned ShareUrl includes it automatically.
-generate-token	Generate a random token for 'http serve' and return it on the server handle.

Pipeline input: scalar — 'http send' accepts a single HttpRequestDefinition or HttpResponseMessage from the pipeline.

Output: Returns either a structured HttpResponseInfo object or a decoded body, depending on '-as'. 'http serve' returns live HttpFileServerHandle objects.

```

http get https://example.com --as text # Fetch a text response
http post https://example.com/api --json { | Name = "Toast" | } --as response #
    Send JSON and keep the structured response
http request GET https://example.com | http send --as response # Build then
    send a request object
http serve ./share --browse # Start a temporary file server with a
    lightweight share page
http serve ./share --browse --lan # Share a directory with other devices on
    the same network
http serve ./dropbox --upload --generate-token # Start a temporary upload
    server with a generated access token
http servers | get { Id, ShareUrl, Upload } # Inspect open temporary file

```

`servers`

Note

The native ‘http’ builtin is object-first and backed by .NET HttpClient. Use ‘-as response’ for a structured response object, or ‘-as json|text|bytes|lines’ to project the body directly. ‘http serve <dir>’ starts a temporary file server and returns a live server handle; ‘-browse’, ‘-upload’, ‘-token’, ‘-generate-token’, and ‘-lan’ turn it into a lightweight cross-platform sharing tool. Use ‘http servers’, ‘http stop’, or ‘close’ to manage it.

25.7.2 ip

Library: Tosh.Stdlib.Net

ip

Wraps the system ip utility, returning typed objects for supported JSON-backed subcommands.

Signature

```
ip
[addr|link|route|neigh|rule|netns|tunnel|tuntap|vrf|maddr|mroute|token|
ntable]
[verb] [filter ...] | ip <other-subcommand ...>
```

Argument	Description
addr address a [show add del flush] [filter ...]	Query or mutate network addresses. Queries return typed objects; mutations pass through to the system ‘ip’. (optional)
link l [show set] [filter ...]	Query or mutate network links. Queries return typed objects; mutations (set up/down, etc.) pass through. (optional)
route r [show add del change replace] [filter ...]	Query or mutate routing table entries. Queries return typed objects; mutations pass through. (optional)
neigh neighbour n [show add del flush] [filter ...]	Query or mutate ARP/neighbor table. Queries return typed objects; mutations pass through. (optional)
rule ru [show add del] [filter ...]	Query or mutate routing policy rules. Queries return typed objects; mutations pass through. (optional)
netns [list add del exec ...]	Query or manage network namespaces. ‘list’ returns typed objects; other operations pass through. (optional)
tunnel tun [show add del change] [filter ...]	Query or mutate IP tunnels. Queries return typed objects; mutations pass through. (optional)
tuntap tap [show add del] [filter ...]	Query or mutate TUN/TAP devices. Queries return typed objects; mutations pass through. (optional)
vrf [show exec ...] [filter ...]	Query VRF devices or execute commands in a VRF context. ‘show’ returns typed objects; ‘exec’ passes through. (optional)
maddr maddress [show] [filter ...]	Query multicast addresses per interface. Returns typed objects. (optional)
mroutemr [show] [filter ...]	Query multicast routing table entries. Returns typed objects. (optional)
token [list set del] [filter ...]	Query or mutate tokenized interface identifiers. ‘list’ returns typed objects; mutations pass through. (optional)
ntable [show change] [filter ...]	Query or mutate the neighbor table parameters. ‘show’ returns typed objects; mutations pass through. (optional)
<other-subcommand ...>	Unsupported subcommands fall back to the system ‘ip’ utility unchanged. (optional)

Pipeline input: none — The structured ‘ip’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Query subcommands return typed objects (interfaces, routes, neighbors, rules, namespaces). Mutation subcommands (add, del, set, etc.) pass through to the system ‘ip’ utility and return its text output.

```
ip addr # List interfaces as structured objects
ip link | where _.IsUp # Filter active interfaces in the pipeline
ip route | where _.IsDefault # Show the default route as a typed object
ip addr | each { _.Addresses } | flatten | get Address # Project nested typed
IP addresses
ip addr add 192.168.1.100/24 dev eth0 # Add an IP address to an interface
ip link set eth0 up # Bring an interface up
ip route del default # Delete the default route
```

```
ip netns # List network namespaces as structured objects
ip tunnel # List IP tunnels as structured objects
ip tuntap # List TUN/TAP devices as structured objects
ip maddr # List multicast addresses per interface
ip ntable # Show neighbor table parameters
```

Note

ToSh wraps 'ip addr', 'ip link', 'ip route', 'ip neigh', 'ip rule', 'ip netns', 'ip tunnel', 'ip tuntap', 'ip vrf', 'ip maddr', 'ip mroute', 'ip token', and 'ip ntable' around the JSON-capable system utility so queries flow through the pipeline as typed objects. Mutation verbs (add, del, set, change, replace, flush) pass through to the system 'ip' utility directly. Other subcommands fall back to the external 'ip' utility unchanged.

25.7.3 ping

Library: Tosh.Stdlib.Net

ping

Pings a host and returns typed reply objects.

Signature

```
ping [-c count] [-W timeout-ms] [-s size] [-i interval] [-t ttl]
    [-4|-6]
    [-D] <host>
```

Argument	Description
host	Host name or IP address to ping.
Flag / Option	Description
-c, -count <count>	Number of echo requests to send. Defaults to 4.
-W, -timeout <milliseconds>	Per-request timeout in milliseconds. Defaults to 4000.
-s, -size <bytes>	Payload size in bytes. Defaults to 32.
-i, -interval <seconds>	Delay between requests, in seconds. Defaults to 1.
-t, -ttl <hops>	Set the IP time-to-live value for requests.
-4	Resolve and use an IPv4 address.
-6	Resolve and use an IPv6 address.
-D, -dont-fragment	Set the don't-fragment flag where supported.

Output: Per-reply records: address, round-trip time, and status flag.

```
ping -c 3 localhost # Ping localhost three times
ping -4 -W 1000 example.com | get { Host, Sequence, Status, RoundtripTime } #
    Project typed ping replies
```

25.8 Pipeline

25.8.1 all

all

Returns true if every pipeline value matches the predicate.

Signature

all <callable|block>

Argument	Description
callable block	A lambda or block predicate evaluated with ToastScript truthiness.

Pipeline input: scalar, record — Consumes the current pipeline and stops at the first non-matching item.

Output: Returns ‘true’ if every item matches the predicate; otherwise ‘false’.

```
echo 2 4 6 | all func(x) => (((x % 2) == 0)) # Check whether all items match
ls | all { _.Exists }
```

25.8.2 any

any

Returns true if any pipeline value matches the predicate.

Signature

any <callable|block>

Argument	Description
callable block	A lambda or block predicate evaluated with ToastScript truthiness.

Pipeline input: scalar, record — Consumes the current pipeline and stops at the first matching item.

Output: Returns ‘true’ if any item matches the predicate; otherwise ‘false’.

```
ps | any func(p) => ($p.Name == "sshd")
echo 1 2 3 | any func(x) => (x == 2) # Check whether any item matches
```

25.8.3 average

Library: Tosh.Stdlib.Pipeline

average

Averages numeric, quantity, storage size, or timespan values.

Aliases: avg

Signature

average [member-path]

Argument	Description
member-path	Optional member path to extract numeric values from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the arithmetic mean.

Output: The arithmetic mean of the pipeline values. Supports numeric, Quantity, StorageSize, and TimeSpan types.

```
echo 10 20 30 | average # Average a list of numbers
echo 1'km 500'm | average # Average compatible quantities
ls | average .Length # Average file sizes
```

25.8.4 cartesian-product

Library: *Tosh.Stdlib.Pipeline*

cartesian-product

Produces the Cartesian product of two sequences. For infinite sources, uses diagonal enumeration.

Signature

```
... | cartesian-product <other-sequence> [callable|block]
```

Argument	Description
other-sequence	A second sequence to form pairs with.
callable block	Optional combiner. Receives each pair as arguments. Defaults to [a, b] arrays. (optional)

Pipeline input: scalar, record — First sequence for the Cartesian product.

Output: All combinations of items from the pipeline and the other sequence.

```
[1 2] | cartesian-product [a b] # All pairs: [1,a],[1,b],[2,a],[2,b]
1.. | cartesian-product (1..) | first 10 # Diagonal enumeration of infinite
sources
echo 1 2 3 | cartesian-product [10 20] func(a, b) => ($a * $b) # With
combiner
```

Note

For infinite sources, uses diagonal (Cantor) enumeration so every pair is reached in finite time.

25.8.5 chain

Library: *Tosh.Stdlib.Pipeline*

chain

Lazily concatenates the pipeline with one or more sequences.

Signature

```
... | chain <sequence> [sequence ...]
```

Argument	Description
sequences	One or more arrays, lists, or ranges to concatenate after the pipeline.

Pipeline input: scalar, record — Pipeline items are yielded first, followed by each argument sequence.

Output: All items from the pipeline (if any) followed by all items from each argument sequence.

```
echo 1 2 | chain [3 4] [5 6] # Concatenate multiple sequences
echo a b c | chain [d e f] # Append to pipeline
chain [1 2] [3 4] [5 6] # Concatenate without pipeline input
```

25.8.6 chunk

Library: Tosh.Stdlib.Pipeline

chunk

Groups pipeline items into fixed-size batches.

Signature

```
chunk <size>
```

Argument	Description
size	The number of items per chunk.

Pipeline input: scalar, record — Collects pipeline items into fixed-size batches.

Output: Arrays of up to ‘size’ items. The last chunk may be smaller.

```
echo 1 2 3 4 5 | chunk 2 # Group into pairs
1..10 | chunk 3 | map { count } # Chunk then count each group
```

25.8.7 collect

Library: Tosh.Stdlib.Pipeline

collect

Collects all pipeline items into a single list.

Signature

```
collect
```

Pipeline input: scalar, record, list, table — Consumes the current pipeline and buffers every incoming item into one array result.

Output: Returns a single array containing the pipeline items in order.

```
echo 1 2 3 | collect # Collect scalar pipeline items into one array
ls *.cs | collect # Capture a multi-item file listing as one value
findmnt -l | where _.FsType == ext4 | collect # Buffer filtered structured
rows into one array
```

25.8.8 combinations

Library: *Tosh.Stdlib.Pipeline*

combinations

Yields all k-element combinations (subsets) of the pipeline items.

Signature

```
... | combinations <k>
```

Argument	Description
k	The number of elements in each combination.

Pipeline input: scalar, record — Items to select combinations from.

Output: Arrays of k elements from the pipeline in lexicographic order.

```
[1 2 3] | combinations 2 # All 2-element subsets
echo a b c d | combinations 3 # All 3-element subsets
```

25.8.9 count

Library: *Tosh.Stdlib.Pipeline*

count

Counts the number of objects in the current pipeline.

Signature

```
count
```

Pipeline input: scalar, record — Consumes the entire pipeline and returns the count.

Output: An integer representing the total number of pipeline objects.

```
echo a b c | count # Count items in a pipeline
ls | count # Count files in the current directory
```

25.8.10 dedup

Library: *Tosh.Stdlib.Pipeline*

dedup

Removes consecutive duplicate values from the pipeline. Unlike 'distinct', preserves non-adjacent duplicates.

Signature

```
... | dedup [member-path]
```

Argument**Description**

member-path

Optional member path to extract the comparison key from each object. (optional)

Pipeline input: scalar, record — Yields each item only if different from the previous.**Output:** Pipeline items with consecutive duplicate values removed.

```
echo 1 1 2 2 3 1 1 | dedup # Remove consecutive duplicates
ls | sort-by .Extension | dedup .Extension # Dedup by member path
```

Note

Unlike ‘distinct’, only removes adjacent duplicates. Preserves non-consecutive repeats.

25.8.11 describe*Library: Tosh.Stdlib.Pipeline***describe**

Returns descriptive statistics for numeric pipeline values.

Signature

```
describe [member-path]
```

Argument**Description**

member-path

Optional member path to extract numeric values from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns descriptive statistics.**Output:** A table of summary statistics: Count, Mean, Median, StdDev, Min, Max, Q1, Q3.

```
echo 23 45 12 67 34 89 11 55 | describe # Summary statistics
```

25.8.12 distinct*Library: Tosh.Stdlib.Pipeline***distinct**

Removes duplicate pipeline values.

Signature

```
distinct [member-path]
```

Argument	Description
member-path	Optional member path to extract the comparison key from each object. (optional)

Pipeline input: scalar, record — Yields each pipeline object only the first time its value (or keyed value) is seen.

Output: Pipeline items with duplicate values removed. Order is preserved.

```
echo 1 2 2 3 1 | distinct # Remove duplicate values
ls | distinct .Extension # Distinct by a member path
```

25.8.13 each

Library: *Tosh.Stdlib.Pipeline*

each

Executes a block or callable once for each input object.

Aliases: foreach

Signature

```
each <callable|block>
```

Argument	Description
callable block	A lambda or block executed once per input item.

Pipeline input: scalar, record — Consumes the current pipeline and executes the callable or block once per input item.

Output: Returns whatever values the callable or block emits for each input item.

```
echo one two | each { _.ToUpper() } # Transform each item to uppercase
DriveInfo.GetDrives() | each func(d) => ($d.Name) # Extract drive names
```

Note

Collections stay intact until you explicitly expand them with ‘each’ or ‘flatten’.

25.8.14 enumerate

Library: *Tosh.Stdlib.Pipeline*

enumerate

Pairs each pipeline item with its index, yielding [index, item] arrays.

Signature

```
... | enumerate [start]
```

Argument	Description
start	Optional starting index (default: 0). (optional)

Pipeline input: scalar, record — Each item is paired with its index.

Output: Two-element arrays [index, item] for each pipeline item.

```
echo a b c | enumerate # Default 0-based indexing
echo a b c | enumerate 1 # 1-based indexing
```

25.8.15 filter

Library: *Tosh.Stdlib.Pipeline*

filter

Filters pipeline values with a callable value or block predicate.

Signature

```
filter <callable|block>
```

Argument	Description
callable block	A lambda or block predicate evaluated with ToastScript truthiness.

Pipeline input: scalar, record — Consumes the current pipeline and keeps only items whose predicate is truthy. Tree-shaped inputs are pruned like ‘where’.

Output: Returns the input items that matched the predicate.

```
echo 1 2 3 4 | filter func(x) => (((x % 2) == 0)) # Filter with a lambda
ls | filter { _.Type == file } # Filter with a block
```

25.8.16 find-index

Library: *Tosh.Stdlib.Pipeline*

find-index

Returns the 0-based index of the first pipeline value matching the predicate, or -1 if none match.

Signature

```
find-index <callable|block>
```

Argument	Description
callable block	Predicate callable or block evaluated for each pipeline value.

Output: An int — the zero-based index of the first matching item, or -1 if none matches.

```
echo 10 20 30 | find-index { _ == 20 } # Find the first matching index
ls | find-index func(f) => $f.Name.EndsWith(".slnx") # Find an index with a
lambda
```

25.8.17 first

Library: *Tosh.Stdlib.Pipeline*

first

Returns the first object or first N objects from the pipeline.

Signature

first [count]

Argument	Description
count	The number of objects to return. Defaults to 1. (optional)

Pipeline input: scalar, record — Yields the first N items then stops consuming.

Output: The first N pipeline objects.

```
echo 1 2 3 | first # Get the first item
ls | first 5 # Get the first five items
```

25.8.18 flat-map

Library: *Tosh.Stdlib.Pipeline*

flat-map

Transforms each pipeline value with a callable or block then flattens the results.

Signature

flat-map <callable|block>

Argument	Description
callable block	A transform that returns a sequence for each input item.

Pipeline input: scalar, record — Transforms each item and flattens the resulting sequences into one stream.

Output: The flattened results of applying the transform to each pipeline item.

```
echo 1 2 3 | flat-map { [_ (_ * 10)] } # Expand each item into two
ls | flat-map { ls $_.FullName } # List contents of each subdirectory
```

25.8.19 flatten

Library: *Tosh.Stdlib.Pipeline*

flatten

Explicitly expands enumerable pipeline values by one level.

Signature

flatten

Pipeline input: scalar, list — Expands each enumerable in the pipeline by one level.

Output: Individual items from each expanded enumerable in the pipeline.

```
echo [1 2] [3 4] | flatten # Flatten nested arrays
ls /tmp /var | flatten # Flatten directory listings into one stream
```

25.8.20 frequencies

Library: Tosh.Stdlib.Pipeline

frequencies

Counts occurrences of each distinct value in the pipeline.

Signature

```
frequencies [member-path]
```

Argument	Description
member-path	Optional member path to count frequencies of, instead of the whole item. (optional)

Output: Records of the form { Value, Count } describing how often each distinct input value occurred.

```
echo red blue red green blue red | frequencies # Count distinct values
ls | frequencies Extension # Count file extensions
```

25.8.21 get

Library: Tosh.Stdlib.Pipeline

get

Gets a member or projects fields from each pipeline item.

Aliases: pick, select

Signature

```
get <member-path> or get <m1> <m2> ... or get { <member-path>, ... }
```

Output: The selected member value(s) — one item per requested path, in input order.

```
ls -la | get Name # Pluck a single field
ls | get Name Size Modified # Project multiple fields (variadic)
ps | get { Name, PID, Memory } # Project multiple fields with brace syntax
echo 1 2 3 | get func(x) => ($x * 2) # Project with a function
```

Note

‘get’ is the column-picker (member values). For row picking by index/range/list, use ‘row’.

25.8.22 group-by

Library: *Tosh.Stdlib.Pipeline*

group-by

Groups pipeline values by a member path, block, or callable.

Signature

```
group-by <member-path|callable|block>
```

Output: Group records of the form { Key, Items } — one per distinct key produced by the projection.

```
ls | group-by Extension
ps | group-by func(p) => ($p.Name.Substring(0, 1))
```

25.8.23 group-while

Library: *Tosh.Stdlib.Pipeline*

group-while

Groups consecutive items while the predicate holds.

Signature

```
group-while <callable|block>
```

Argument	Description
callable block	A predicate that receives each item. A new group starts when its result is falsy.

Pipeline input: scalar, record — Groups consecutive pipeline items while the predicate is truthy.

Output: Arrays of consecutive items grouped while the predicate holds.

```
echo 1 1 2 2 3 | group-while { _ == $prev } # Group equal consecutive values
echo 1 2 3 10 11 12 | group-while func(x, prev) => ($x - $prev <= 1) # Group
runs of nearly consecutive numbers
```

25.8.24 ignore

Library: *Tosh.Stdlib.Pipeline*

ignore

Consumes and discards all pipeline input.

Signature

```
... | ignore
```

Output: Emits nothing; drains the input pipeline silently.

```
build-project | ignore # Run for side effects and discard output
http post https://example.com/hooks --json $payload | ignore # Suppress
```

```
response output
```

25.8.25 inspect

Library: *Tosh.Stdlib.Pipeline*

inspect

Inspects piped CLR objects inline, or returns legacy flat output with `-flat`.

Signature

```
inspect [-a] [-flat]
```

Flag / Option	Description
<code>-a</code>	Include non-public and static members in the inspection.
<code>-flat</code>	Use flat text output instead of the interactive tree browser.

Pipeline input: scalar, record — Inspects each piped object.

Output: An interactive tree view or flat member-value records depending on mode.

```
ls -la | first | inspect # Inspect a file entry interactively
new System.Random() | inspect -a # Inspect all members including private
new System.Random() | inspect --flat # Flat output for scripting
```

Note

Inspect opens an inline tree browser for CLR values in interactive sessions. Use `'-a'` for non-public/static members, `'-flat'` for the legacy static inspection object output, and `'i'` inside the browser to insert the selected member text into the active REPL line at the cursor (or queue it for the next prompt when no line is active). In the REPL, `'F2'` tries to inspect the reference under the cursor, and `'Alt+I'` is available as a fallback on terminals that do not expose function keys cleanly.

25.8.26 interleave

Library: *Tosh.Stdlib.Pipeline*

interleave

Alternates items from the pipeline with items from another sequence.

Signature

```
interleave <other-sequence>
```

Argument	Description
<code>other-sequence</code>	An array or list to alternate with the pipeline items.

Pipeline input: scalar, record — Alternates items from the pipeline with the other sequence.

Output: Items from the pipeline and the other sequence in alternating order.

```
echo 1 2 3 | interleave [a b c] # Alternate numbers and letters
```

25.8.27 intersperse

Library: *Tosh.Stdlib.Pipeline*

intersperse

Inserts a separator value between each pair of pipeline items.

Signature

```
... | intersperse <separator>
```

Argument	Description
separator	The value to insert between each pipeline item.

Pipeline input: scalar, record — Items are yielded with the separator between each pair.

Output: The original items with the separator inserted between each pair.

```
echo 1 2 3 | intersperse 0 # Insert zeros between items
echo a b c | intersperse "-" # Insert dashes
```

25.8.28 join

Library: *Tosh.Stdlib.Pipeline*

join

Joins pipeline items into a single string, or combines path segments with -p/-path.

Signature

```
... | join [-p|-path] [separator-or-segment ...]
```

Argument	Description
separator-or-segment	In default mode: the string placed between items. In path mode (-p/-path): an additional path segment appended after any piped input. (optional) (<i>string</i>)

Flag / Option	Description
-p	Path mode: join segments with the platform path separator using System.IO.Path.Join semantics. Piped input is prepended; positional args are additional segments.
-path	Alias for -p.

Pipeline input: scalar, record, list, table — Consumes every pipeline item, converts it to a string, and joins them. In path mode, piped items become the leading path segments.

Output: Default mode: a single string of pipeline items separated by the given separator.
Path mode: a single path string.

```
echo a b c | join "-" # Join scalar items with '-'
[1, 2, 3] | join ", " # Stringify and join array elements
cat words.txt | lines | join " " # Collapse lines into a single space-
    separated string
join -p "etc" "tosh" "config.toml" # Build a path from arguments
pwd | join -p "logs" "today.log" # Build a path from piped input plus more
    segments
```

25.8.29 last

Library: Tosh.Stdlib.Pipeline

last

Returns the last object or last N objects from the pipeline.

Signature

```
last [count]
```

Argument	Description
count	The number of objects to return from the end. Defaults to 1. (optional)

Pipeline input: scalar, record — Buffers the pipeline and returns the final N items.

Output: The last N pipeline objects.

```
echo 1 2 3 | last # Get the last item
ls | last 3 # Get the last three items
```

25.8.30 map

Library: Tosh.Stdlib.Pipeline

map

Transforms each pipeline value with a callable value or block.

Signature

```
map <callable|block>
```

Argument	Description
callable block	A lambda or block that transforms each input item into exactly one output value.

Pipeline input: scalar, record — Consumes the current pipeline and emits one transformed value per input item.

Output: Returns one transformed value for each input item.

```
echo 1 2 3 | map func(x) => ($x * 2) # Transform values with a lambda
ls | map { _.Name } # Transform values with a block
```

25.8.31 max

Library: *Tosh.Stdlib.Pipeline*

max

Returns the maximum pipeline value.

Signature

max [member-path]

Argument	Description
member-path	Optional member path to extract the comparison value from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the maximum value.

Output: The maximum value from the pipeline.

```
echo 3 1 4 1 5 | max # Find the maximum value
ls | max .Length # Find the largest file by size
```

25.8.32 median

Library: *Tosh.Stdlib.Pipeline*

median

Returns the median of numeric pipeline values.

Signature

median [member-path]

Argument	Description
member-path	Optional member path to extract numeric values from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the median.

Output: The median value of the pipeline. For even-length lists, returns the average of the two middle values.

```
echo 1 2 3 4 5 | median # Median of a list
ls | median .Length # Median file size
```

25.8.33 min

Library: *Tosh.Stdlib.Pipeline*

min

Returns the minimum pipeline value.

Signature

min [member-path]

Argument**Description**

member-path

Optional member path to extract the comparison value from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the minimum value.

Output: The minimum value from the pipeline.

```
echo 3 1 4 1 5 | min # Find the minimum value
ls | min .Length # Find the smallest file by size
```

25.8.34 none**none**

Returns true if no pipeline values match the predicate.

Signature

none <callable|block>

Argument**Description**

callable|block

A lambda or block predicate evaluated with ToastScript truthiness.

Pipeline input: scalar, record — Consumes the current pipeline and stops at the first matching item.

Output: Returns ‘true’ if no items match the predicate; otherwise ‘false’.

```
echo 1 2 3 | none func(x) => ($x > 10) # Check whether no items match
ls | none { _.Type == link }
```

25.8.35 parallel

Library: Tosh.Stdlib.Pipeline

parallel

Executes a block or callable concurrently for each input object.

Signature

parallel [-threads <n>] <callable|block>

Argument**Description**

callable|block

A lambda or block executed once per input item, concurrently.

Flag / Option	Description
<code>-threads <n></code>	Maximum degree of parallelism (default: processor count).

Pipeline input: scalar, record — Consumes the current pipeline and executes the callable or block in parallel for each input item.

Output: Returns whatever values the callable or block emits for each input item, in input order.

```
echo 1 2 3 | parallel { echo ( _ * 2 ) } # Double each item in parallel
```

25.8.36 partition

Library: *Tosh.Stdlib.Pipeline*

partition	
Splits pipeline values into two lists based on a predicate: [matches, non-matches].	
Signature	
partition <callable block>	
Argument	Description
callable block	A predicate evaluated for each item using ToastScript truthiness.
Pipeline input: scalar, record — Consumes the pipeline and splits items into two groups by predicate.	
Output: A two-element array: [items-where-true, items-where-false].	
<pre>echo 1 2 3 4 5 partition { _ > 3 } # Partition by a condition ls partition { _.Extension == ".cs" } # Separate C# files from others</pre>	

25.8.37 percentile

Library: *Tosh.Stdlib.Pipeline*

percentile	
Returns the Nth percentile of numeric pipeline values.	
Signature	
percentile <N> [member-path]	
Argument	Description
percentile	The percentile to compute (0–100).
member-path	Optional member path to extract numeric values from each object. (optional)
Pipeline input: scalar, record — Consumes the pipeline and returns the percentile value.	

Output: The value at the requested percentile using linear interpolation.

```
echo 1 2 3 4 5 6 7 8 9 10 | percentile 95 # 95th percentile
```

25.8.38 permutations

Library: Tosh.Stdlib.Pipeline

permutations

Yields all k-element permutations (orderings) of the pipeline items. Defaults to full-length permutations.

Signature

```
... | permutations [k]
```

Argument	Description
k	The number of elements in each permutation. Defaults to the full length if omitted. (optional)

Pipeline input: scalar, record — Items to permute.

Output: Arrays of k elements representing each permutation.

```
[1 2 3] | permutations # All 6 permutations of 3 elements
echo a b c | permutations 2 # All 2-element orderings
```

25.8.39 reduce

Library: Tosh.Stdlib.Pipeline

reduce

Folds the current pipeline into one value using a seed and callable value or block.

Signature

```
reduce <seed> <callable|block>
```

Argument	Description
seed	The initial accumulator value.
callable block	A lambda or block that combines the current accumulator with each input item and returns the next accumulator.

Pipeline input: scalar, record — Consumes the current pipeline from left to right and folds it into one final value.

Output: Returns the final accumulator value. On an empty input stream, ‘reduce’ returns the seed unchanged.

```
echo 1 2 3 4 | reduce 0 func(acc, x) => ($acc + $x) # Fold numeric values
echo one two three | reduce "" { $acc + _.Substring(0, 1) } # Fold with a
block
```

25.8.40 rename

Library: *Tosh.Stdlib.Pipeline*

rename

Renames fields on record-like objects.

Signature

```
rename <old> <new> [old2 new2 ...]
```

Argument

old new ...

Description

One or more old/new field-name pairs to apply to each record-like input value.

Output: Records describing each rename operation, or nothing when used purely for its side effect.

```
ls | get { Name, Length } | rename Length Size # Rename one field
ps | get { Id, Name } | rename Id Pid Name Command # Rename several fields
```

25.8.41 reverse

Library: *Tosh.Stdlib.Pipeline*

reverse

Reverses the order of the current pipeline objects.

Signature

```
reverse
```

Pipeline input: scalar, record — Buffers the pipeline then yields items in reverse order.

Output: All pipeline objects in reversed order.

```
echo 1 2 3 | reverse # Reverse a sequence
ls | sort .Name | reverse # Reverse a sorted listing
```

25.8.42 row

Library: *Tosh.Stdlib.Pipeline*

row

Picks one or more rows from the pipeline by index, range, or list of indices.

Signature

```
row <index ...> | row <range> | row <list>
```

Argument

indices

Description

One or more zero-based indices, a range, or a list/array of indices.

Pipeline input: scalar, record — Reads input items by their zero-based position.

Output: The pipeline rows at the requested indices, in the order they were requested.

```
ls | row 3 # Pick the row at index 3
ls | row 7 8 9 # Pick multiple rows by index (variadic)
ls | row [3, 1, 0] # Pick rows from a list literal (preserves the listed
order)
ls | row 2..5 # Pick a contiguous slice
```

25.8.43 scan

Library: Tosh.Stdlib.Pipeline

scan

Like reduce but yields every intermediate accumulator value.

Signature

```
scan <seed> <callable|block>
```

Argument	Description
seed	The initial accumulator value.
callable block	A function that takes (accumulator, current-item) and returns the new accumulator.

Pipeline input: scalar, record — Folds the pipeline from left to right, yielding each intermediate value.

Output: Every intermediate accumulator value (one per input item).

```
echo 1 2 3 4 | scan 0 func(acc, x) => ($acc + $x) # Running total
echo a b c | scan "" { $acc + _ } # Running string concatenation
```

25.8.44 skip

Library: Tosh.Stdlib.Pipeline

skip

Skips the first object or first N objects from the pipeline.

Signature

```
skip [count]
```

Argument	Description
count	The number of objects to skip. Defaults to 1. (optional)

Pipeline input: scalar, record — Consumes and discards the first N items, then yields the rest.

Output: The remaining pipeline objects after skipping the first N.

```
echo 1 2 3 4 5 | skip 2 # Skip the first two items
echo a b c | skip # Skip the first item
```

25.8.45 skip-until

Library: *Tosh.Stdlib.Pipeline*

skip-until

Skips input values until the predicate becomes true.

Signature

```
skip-until <predicate-expression|callable>
```

Argument	Description
predicate	A callable or block evaluated for each item. Starts yielding when its result is truthy.

Pipeline input: scalar, record — Discards items until the predicate is truthy, then yields the rest.

Output: Pipeline items starting from the first that satisfies the predicate.

```
echo 1 2 3 4 5 | skip-until { _ >= 3 } # Skip until a condition is met
```

25.8.46 skip-while

Library: *Tosh.Stdlib.Pipeline*

skip-while

Skips input values while the predicate remains true.

Signature

```
skip-while <predicate-expression|callable>
```

Output: All input items starting from (and including) the first one for which the predicate result was falsy.

```
echo 1 2 3 4 | skip-while { _ < 3 }
echo 1 2 3 4 | skip-while func(x) => ($x < 3)
```

25.8.47 sort

Library: *Tosh.Stdlib.Pipeline*

sort

Sorts the current pipeline objects.

Aliases: sort-by

Signature

```
sort [-r|-reverse] [-n|-numeric] [-u|-unique] [-h|-human-numeric]
[-o|-ordinal] [member-path|callable|block]
```


Argument	Description
key	A member path, callable, or block to extract the sort key. (optional) (<i>member-path/callable/block</i>)
Flag / Option	Description
-r, -reverse	Reverse the sort order.
-d, -desc, -descending	Alias for -reverse: sort in descending order.
-n, -numeric	Use numeric comparison.
-u, -unique	Remove duplicate values after sorting.
-h, -human-numeric	Human-numeric sort (understands storage sizes).
-o, -ordinal	Compare by code point, case-sensitively.

Pipeline input: scalar — Collects all pipeline objects, sorts them, then re-emits.

Output: The input pipeline objects in sorted order.

```
ps | sort Memory # Sort by property
ls -la | sort Modified | reverse # Sort then reverse
ps | sort func(p) => ($p.Name.Length) # Lambda sort key
$codes | sort --ordinal # Code-point order, for generated output
```

25.8.48 stdev

Library: Tosh.Stdlib.Pipeline

stdev

Returns the population standard deviation of numeric pipeline values.

Aliases: stddev

Signature

stdev [member-path]

Argument	Description
member-path	Optional member path to extract numeric values from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the standard deviation.

Output: The population standard deviation of the pipeline values.

```
echo 2 4 4 4 5 5 7 9 | stdev # Standard deviation
```

25.8.49 step-by

Library: Tosh.Stdlib.Pipeline

step-by

Takes every Nth item from the pipeline.

Signature

```
... | step-by <n>
```

Argument**Description**

n

Take every Nth item (must be ≥ 1).**Pipeline input:** scalar, record — Yields every Nth item from the pipeline.**Output:** Every Nth item from the pipeline.

```
1.. | step-by 3 | first 5 # Every 3rd integer
echo a b c d e f g | step-by 2 # Every other item
```

25.8.50 sum*Library: Tosh.Stdlib.Pipeline***sum**

Sums numeric, quantity, storage size, or timespan values.

Signature

```
sum [member-path]
```

Argument**Description**

member-path

Optional member path to extract numeric values from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the total.**Output:** The sum of the pipeline values. Supports numeric, Quantity, StorageSize, and TimeSpan types.

```
echo 1 2 3 4 | sum # Sum a list of numbers
echo 1'km 500'm | sum # Sum compatible quantities
ls | sum .Length # Total file sizes
```

25.8.51 summarize*Library: Tosh.Stdlib.Pipeline***summarize**

Computes structured summary objects for requested aggregates over the pipeline.

Aliases: summary**Signature**

```
summarize [-sum [columns]] [-avg [columns]] [-min [columns]] [-max
[columns]] [-count [columns]]
```

Argument	Description
[column member-path] [-sum [columns]] [-avg [columns]] [-min [columns]] [-max [columns]] [-count [columns]]	With no arguments, infer every sensible aggregate for every summarizable column. A single bare column or member path such as ‘Size’ or ‘_.Used’ narrows auto mode to that one target. Flags request explicit operations. (optional)

Flag / Option	Description
-sum [columns]	Compute sums for scalar input or the named columns.
-avg [columns], -average [columns]	Compute averages for scalar input or the named columns.
-min [columns]	Compute minima for scalar input or the named columns.
-max [columns]	Compute maxima for scalar input or the named columns.
-count [columns]	Count input rows when no columns are supplied, or non-null values for the named columns.

Pipeline input: scalar, record, list, table — Consumes the current pipeline rows and returns one structured ColumnSummary object per requested or inferred scalar target or member path.

Output: Returns ColumnSummary objects describing the requested or inferred aggregates. The original input rows are not appended back into the result.

```
df | summarize # Infer every sensible aggregate for every summarizable df
column
df | summarize _.Used # Infer every sensible aggregate for a single member
path target
seq 5 | summarize --sum --avg --min --max --count # Summarize a scalar
numeric pipeline explicitly
ps | summarize --avg Memory --max Memory # Compute multiple aggregates over
one column
```

25.8.52 take-until

Library: Tosh.Stdlib.Pipeline

take-until

Yields input values until the predicate becomes true.

Signature

```
take-until <predicate-expression|callable>
```

Argument	Description
predicate	A callable or block evaluated for each item. Stops when its result is truthy.

Pipeline input: scalar, record — Yields items until the predicate is truthy.

Output: Pipeline items up to (but not including) the first that satisfies the predicate.

```
echo 1 2 3 4 5 | take-until { _ >= 4 } # Take until a condition is met
1..100 | take-until func(x) => ($x > 10) # Take until value exceeds 10
```

25.8.53 take-while

Library: *Tosh.Stdlib.Pipeline*

take-while

Yields input values while the predicate remains true.

Signature

take-while <predicate-expression|callable>

Output: Input items from the front of the stream up to (but not including) the first one for which the predicate result was falsy.

```
echo 1 2 3 4 | take-while { _ < 3 }
echo 1 2 3 4 | take-while func(x) => ($x < 3)
```

25.8.54 tee

Library: *Tosh.Stdlib.Pipeline*

tee

Passes values through while also writing them out or capturing them.

Signature

tee [-a] [path] or tee -v <name>

Argument	Description
path	Optional file path to write a copy of the stream to. (optional) <i>(path-like)</i>
Flag / Option	Description
-a, -append	Append to the output file instead of replacing it.
-v, -variable <name>	Capture the stream into a shell variable while still passing values through.

Output: Re-emits each input item unchanged, while also writing it to the configured destination as a side effect.

```
ls | tee snapshot.txt | where _.IsDirectory # Write a copy to a file and keep piping
ps | tee -v processes | count # Capture a pipeline into a variable
```

25.8.55 transpose

Library: *Tosh.Stdlib.Pipeline*

transpose

Pivots rows into columns. Each input record's keys become column headers and values become rows.

Signature

transpose

Pipeline input: record, table — Reads records from the pipeline and transposes them.

Output: Pivoted records where original keys become headers and values become rows.

```
echo { |a=1, b=2| } { |a=3, b=4| } | transpose # Pivot rows into columns
```

25.8.56 variance

Library: Tosh.Stdlib.Pipeline

variance

Returns the population variance of numeric pipeline values.

Signature

variance [member-path]

Argument	Description
member-path	Optional member path to extract numeric values from each object. (optional)

Pipeline input: scalar, record — Consumes the pipeline and returns the variance.

Output: The population variance of the pipeline values.

```
echo 2 4 4 4 5 5 7 9 | variance # Population variance
```

25.8.57 where

Library: Tosh.Stdlib.Pipeline

where

Filters pipeline objects with a predicate block or callable.

Signature

where <predicate-expression|callable>

Argument	Description
predicate	A predicate block or callable evaluated with ToastScript truthiness. (block/callable)

Pipeline input: scalar — Consumes any pipeline objects and tests each against the predicate.

Output: Pipeline objects for which the predicate result was truthy.

```
ls -la | where _.Type == file # Filter by property
```

```
ls -la | where func(item) => ($item.Name.ToLower().EndsWith(".md")) # Lambda
predicate
```

Note

Inside predicate expressions, bare member access resolves against the current pipeline object.

25.8.58 window

Library: *Tosh.Stdlib.Pipeline*

window

Yields sliding windows of a given size over the pipeline.

Signature

```
window <size> [callable|block]
```

Argument	Description
size	The number of items in each sliding window.
callable block	Optional transform applied to each window. (optional)

Pipeline input: scalar, record — Yields overlapping windows of the specified size as the pipeline flows.

Output: Arrays (or transformed results) for each sliding window position.

```
echo 1 2 3 4 5 | window 3 # Sliding windows of size 3
1..10 | window 2 func(a, b) => ($a + $b) # Pairwise sums
```

25.8.59 xargs

Library: *Tosh.Stdlib.Pipeline*

xargs

Builds command invocations from text input.

Signature

```
xargs [-n count] <command> [fixed-arg ...]
```

Argument	Description
command	Command to invoke for the generated arguments.
fixed-arg ...	Arguments that are always passed before piped arguments. (optional)

Flag / Option	Description
-n, -max-args <count>	Invoke the command repeatedly with at most this many piped arguments per invocation.

Output: Streams whatever the invoked sub-command produces, once per batch of input arguments.

```
echo README.md AGENTS.md | xargs wc -l # Pass piped words as command
arguments
glob "*.log" | xargs -n 1 rm # Run one invocation per input argument
```

25.8.60 zip

Library: Tosh.Stdlib.Pipeline

zip

Merges two sequences pairwise. The second sequence is the first argument (an array).

Signature

```
zip <other-sequence> [callable|block]
```

Argument	Description
other-sequence	An array or list to merge pairwise with the pipeline.
callable block	Optional combiner. Receives each pair as arguments. Defaults to creating two-element arrays. (optional)

Pipeline input: scalar, record — Merges the pipeline with another sequence pairwise.

Output: One result per pair. Without a combiner, yields two-element arrays.

```
echo a b c | zip [1 2 3] # Pair pipeline with an array
echo 1 2 3 | zip [10 20 30] func(a, b) => ($a + $b) # Zip with a combiner
```

25.9 Process

25.9.1 bg

Library: Tosh.Stdlib.Processes

bg

Resumes a suspended job in the background.

Signature

```
bg [id]
```

Argument	Description
id	The id of the suspended job to resume in the background. (optional) (int)

Output: Emits nothing; resumes the targeted job in the background as a side effect.

```
bg # Resume the most recently suspended job in the background.
bg 2 # Resume suspended job 2 in the background.
```

Note

Resumes a suspended job in the background. The process receives SIGCONT but does not get the terminal, so it runs without interactive I/O. If no id is given, the most recently suspended job is used.

25.9.2 fg

Library: *Tosh.Stdlib.Processes*

fg

Resumes a suspended job in the foreground.

Signature

`fg [id]`

Argument**Description**

id

The id of the suspended job to bring to the foreground. (optional) (*int*)

Output: Emits nothing; brings the targeted job into the foreground as a side effect.

```
fg # Resume the most recently suspended job.
fg 2 # Resume suspended job 2.
```

Note

Resumes a suspended job (stopped with Ctrl+Z) in the foreground. If no id is given, the most recently suspended job is used.

25.9.3 jobs

Library: *Tosh.Stdlib.Processes*

jobs

Lists ToSh background jobs.

Signature

`jobs`

Output: Job records describing each tracked background job: id, status, command line, and pid.

```
jobs # List active background jobs
jobs | get { Id, Status, CommandLine } # Project job fields
```


Note

Jobs lists ToSh background jobs started with a trailing '&'. A background launch updates '\$tosh.Last.Result' with the started job info, while 'jobs' and 'wait-for' are the primary inspection commands.

25.9.4 kill

Library: Tosh.Stdlib.Processes

kill

Stops a ToSh background job or operating-system process.

Signature

```
kill <job-id|pid> [job-id|pid ...]
```

Argument	Description
job-id pid ...	One or more ToSh background job ids, ShellJobInfo values, ProcessInfo values, or native process ids.

Output: Emits nothing; sends the requested signal to the targeted process(es) as a side effect.

```
sleep 60 &; jobs | first | kill # Kill a background job from the pipeline
kill 12345 # Kill a native process id
```

Note

Kill can stop either a ToSh background job or a native operating-system process by pid.

25.9.5 lsfd

Library: Tosh.Stdlib.Processes

lsfd

Wraps the system lsfd utility, returning typed open-file-descriptor objects for JSON-backed invocations.

Signature

```
lsfd [-l] [-p pid[,pid...]] [-i[4|6]] [-o columns]
      [-summary[=only|append|never]]
```

Argument	Description
[-p pid[,pid...]]	Optionally restrict the query to specific processes. (optional)

Flag / Option	Description
<code>-l</code>	Include thread-level rows with a ‘TID’ column.
<code>-p <pid(s)></code>	Restrict the query to one or more process ids.
<code>-i[4 6]</code>	Restrict the result set to IPv4 and/or IPv6 sockets.
<code>-o <columns></code>	Select lsfd-style columns such as ‘COMMAND,PID,FD,TYPE,NAME’.
<code>-summary[=only append never]</code>	Include or isolate lsfd summary counters alongside structured descriptor rows.
<code>-show <columns></code>	Use ToSh display-only column selection without changing the underlying ‘FileDescriptorInfo’ objects.
<code>-hide <columns></code>	Hide display columns while keeping the full typed rows in the pipeline.
<code>-show-all</code>	Expose every selectable structured lsfd column discoverable from the local ‘lsfd -H’ catalog.

Pipeline input: none — The structured ‘lsfd’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns typed open-file-descriptor rows and, when summary mode is enabled, typed counter rows describing totals such as open files and sockets.

```
lsfd # Browse open file descriptors as typed rows
lsfd -p 1 -o COMMAND,PID,ASSOC,TYPE,NAME # Inspect a specific process with
explicit lsfd columns
lsfd --summary=only # Show typed summary counters only
```

Note

ToSh wraps ‘lsfd -json’ so open-file-descriptor rows stay typed in the pipeline. ‘-summary=only’ yields typed counters, and ‘-summary=append’ returns both row and summary objects. Text-format-only modes like ‘-raw’, ‘-noheadings’, filter expressions, and custom counters currently fall back to the external ‘lsfd’ utility unchanged.

25.9.6 ps

Library: *Tosh.Stdlib.Processes*

ps

Lists running processes as Tosh process objects.

Signature

```
ps [-eAf] [-p pid[,pid...]] [-ppid pid[,pid...]] [-u user[,user...]]
[-t tty[,tty...]] [-o columns] [-sort field[,field...]] [-show
columns] [-hide columns] [-show-all] [name-or-id ...]
```

Argument	Description
<code>name-or-id</code>	Optional process names or PIDs to filter. (optional)

Flag / Option	Description
-e	Show every process.
-A	Show every process (alias for -e).
-f	Full listing with extra columns.
-p <pid,...>	Select processes by PID.
-ppid <pid,...>	Select by parent PID.
-u <user,...>	Select by owning user.
-t <tty,...>	Select by terminal.
-o <columns>	Specify output columns.
-sort <field,...>	Sort by the given fields.
-tree	Display as a process hierarchy tree.
-forest	Display as a process hierarchy tree (alias for -tree).
-show <columns>	Select which properties are rendered.
-hide <columns>	Hide specific properties from the output.
-show-all	Display every available column.

Output: Typed process objects with properties like Name, Id, Memory, CPU, User.

```
ps --tree # Process tree showing parent-child hierarchy
ps -f --sort -Id # Full listing sorted by ID descending
ps -u root | get { Name, Id, User } # Filter by user
ps | sort Memory | reverse | first 5 # Top 5 by memory
```

Note

Process memory values are surfaced as Tosh StorageSize objects, not raw strings.

25.9.7 signal

Library: Tosh.Stdlib.Processes

signal

Sends a signal to a ToSh background job or operating-system process.

Signature

```
signal <signal> <job-id|pid> [job-id|pid ...]
```

Argument	Description
signal	Signal name or number, such as TERM, INT, HUP, or 15.
job-id pid ...	One or more ToSh background job ids, ShellJobInfo values, ProcessInfo values, or native process ids.

Output: Emits nothing; delivers the requested signal as a side effect.

```
signal TERM 12345 # Send SIGTERM to a process
jobs | where _.Status == "Running" | signal INT # Signal jobs from the
pipeline
```

Note

Signal sends a named or numeric signal to a ToSh job or a native process id.

25.9.8 wait-for

Library: `Tosh.Stdlib.Processes`

wait-for

Waits for one or more ToSh background jobs to finish.

Signature

`wait-for [job-id ...]`

Argument	Description
job-id ...	Optional ToSh background job ids or job objects to await. With no targets, waits for all current jobs. (optional)

Output: Emits nothing on success; throws when the awaited condition does not become true within the timeout.

```
wait-for # Wait for all background jobs
spawn git status | wait-for # Wait for a job supplied by the pipeline
```

Note

Wait-for blocks until one or more background jobs finish and returns structured completion objects.

25.10 Prompt**25.10.1 prompt**

Library: `Tosh.Stdlib.Shell`

[shell-only] — only available inside an interactive TōSh session.

prompt

Returns a styled prompt segment. Dispatches to the named segment builder (time, dir, git, userhost, history, jobs, duration, exit, text, newline).

Signature

`prompt <segment> [args ...]`

Argument	Description
segment	Segment kind. One of: time, dir, git, userhost, history, jobs, duration, exit, text, newline.
args ...	Remaining arguments forwarded to the chosen segment builder (segment-specific options, e.g. <code>-fg</code> , <code>-bg</code> , <code>-bold</code> , <code>-dim</code> , plus per-segment flags). (optional)

Output: Styled prompt segment(s) produced by the selected segment builder.

```
prompt time --format HH:mm --dim
prompt dir --depth 2 --fg cyan
prompt git --bold
prompt text " » " --fg gray
```

Note

Each subcommand corresponds to a legacy ‘prompt-`<segment>`’ command. The legacy names remain available for now; use ‘help prompt-`<segment>`’ (e.g. ‘help prompt-time’) for the per-segment option reference.

25.10.2 prompt-dir

Library: `Tosh.Stdlib.Shell`

[shell-only] — only available inside an interactive TōSh session.

prompt-dir

Returns the current directory as a styled prompt segment.

Signature

```
prompt-dir [-fg color] [-bg color] [-bold] [-depth n]
```

Flag / Option	Description
<code>-fg <color></code>	Foreground color name or ANSI-style color token.
<code>-bg <color></code>	Background color name or ANSI-style color token.
<code>-bold</code>	Render the segment in bold.
<code>-depth <n></code>	Show only the last <code>n</code> path components.

Output: Styled prompt segment(s) describing the current directory.

```
prompt-dir --fg blue --bold
prompt-dir --fg yellow --depth 2
```

25.10.3 prompt-duration

Library: `Tosh.Stdlib.Shell`

[shell-only] — only available inside an interactive TōSh session.

prompt-duration

Returns the last command duration as a styled prompt segment.

Signature

```
prompt-duration [duration] [-fg color] [-bg color] [-bold] [-dim]
[-threshold-ms value]
```

Argument	Description
duration	Duration to render instead of the last command duration. (optional)

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.
-threshold-ms <value>	Suppress the segment unless duration is at least this many milliseconds.

Output: Styled prompt segment(s) describing the duration of the previous command.

```
prompt-duration
prompt-duration 2.5s --threshold-ms 250
```

25.10.4 prompt-exit

Library: `Tosh.Stdlib.Shell`

[**shell-only**] — only available inside an interactive TōSh session.

prompt-exit

Returns the last non-zero exit code as a styled prompt segment.

Signature

```
prompt-exit [code] [-fg color] [-bg color] [-bold] [-dim]
```

Argument	Description
code	Exit code to render instead of the shell's last exit code. (optional) (<i>int</i>)

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.

Output: Styled prompt segment(s) describing the previous command's exit status.

```
prompt-exit
prompt-exit 7 --fg red --bold
```

25.10.5 `prompt-git`

Library: `Tosh.Stdlib.Shell`

[shell-only] — only available inside an interactive TōSh session.

`prompt-git`

Returns git branch and status as styled prompt segments.

Signature

```
prompt-git [-fg color] [-bg color] [-bold]
```

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.

Output: Styled prompt segment(s) describing the current git branch / status.

```
prompt-git
prompt-git --fg bright-green --bold
```

25.10.6 `prompt-history`

Library: `Tosh.Stdlib.Shell`

[shell-only] — only available inside an interactive TōSh session.

`prompt-history`

Returns the next history id as a styled prompt segment.

Signature

```
prompt-history [id] [-fg color] [-bg color] [-bold] [-dim]
```

Argument	Description
id	History id to render instead of the next live history id. (optional) (<i>long</i>)

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.

Output: Styled prompt segment(s) showing the current history index.

```
prompt-history
prompt-history 432 --fg gray --dim
```

25.10.7 `prompt-jobs`

Library: `Tosh.Stdlib.Shell`

[**shell-only**] — only available inside an interactive TōSh session.

prompt-jobs

Returns the current background job count as a styled prompt segment.

Signature

```
prompt-jobs [count] [-fg color] [-bg color] [-bold] [-dim]
```

Argument	Description
count	Job count to render instead of the shell's live background-job count. (optional) (<i>int</i>)

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.

Output: Styled prompt segment(s) summarising background-job counts.

```
prompt-jobs
prompt-jobs 3 --fg yellow --bold
```

25.10.8 prompt-newline

Library: Tosh.Stdlib.Shell

[**shell-only**] — only available inside an interactive TōSh session.

prompt-newline

Inserts a line break in the prompt.

Signature

```
prompt-newline
```

Output: A styled newline segment used to break long prompts onto multiple lines.

```
prompt-newline
```

25.10.9 prompt-text

Library: Tosh.Stdlib.Shell

[**shell-only**] — only available inside an interactive TōSh session.

prompt-text

Returns literal text as a styled prompt segment.

Signature

```
prompt-text <text> [-fg color] [-bg color] [-bold] [-dim]
```


Argument	Description
text	Literal text to render as a styled prompt segment.
Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.

Output: A styled text segment with the supplied literal content.

```
prompt-text "> " --fg cyan
prompt-text "::" --fg gray --dim
```

25.10.10 `prompt-time`

Library: `Tosh.Stdlib.Shell`

[**shell-only**] — only available inside an interactive TōSh session.

`prompt-time`

Returns the current time as a styled prompt segment.

Signature

```
prompt-time [-fg color] [-bg color] [-bold] [-dim] [-format
pattern]
```

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.
-format <pattern>	Date/time format string used to render the current time.

Output: Styled prompt segment(s) showing the current wall-clock time.

```
prompt-time --dim
prompt-time --format "HH:mm:ss" --fg gray
```

25.10.11 `prompt-userhost`

Library: `Tosh.Stdlib.Shell`

[**shell-only**] — only available inside an interactive TōSh session.

`prompt-userhost`

Returns the current user and host as a styled prompt segment.

Signature

```
prompt-userhost [-fg color] [-bg color] [-bold] [-dim]
```

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-dim	Render the segment dimmed.

Output: Styled prompt segment(s) describing the current user@host.

```
prompt-userhost --dim
prompt-userhost --fg gray
```

25.10.12 styled

Library: *Tosh.Stdlib.Display*

styled

Creates a styled text segment with color and formatting.

Signature

```
styled <text> [-fg color] [-bg color] [-bold] [-italic]
[-underline] [-dim]
```

Argument	Description
text	Text to wrap in a StyledText value.

Flag / Option	Description
-fg <color>	Foreground color name or ANSI-style color token.
-bg <color>	Background color name or ANSI-style color token.
-bold	Render the segment in bold.
-italic	Render the segment in italic.
-underline	Render the segment underlined.
-dim	Render the segment dimmed.

Output: Styled-text values that carry inline color/format markup for downstream prompts and renderers.

```
styled "hello" --fg cyan --bold
styled "warning" --fg yellow --bg red
```

25.11 Scripting

25.11.1 assert

Library: *Tosh.Stdlib.Scripting*

assert

Asserts that a condition is true; throws a diagnostic error if it is false.

Signature

```
assert <predicate> [message]
```

Argument	Description
predicate	Predicate callable or block that must evaluate truthy.
message	Optional diagnostic message emitted when the assertion fails. (optional)

Output: Emits nothing on success; throws a diagnostic when the predicate is falsy.

```
assert { (2 + 2) == 4 } # Assert an invariant
assert { $env.HOME != null } "HOME must be set" # Assertion with a custom
message
```

25.11.2 format

Library: Tosh.Stdlib.Scripting

format

Pretty-prints tosh source files.

Signature

```
format [-check] [-stdout] [-diff] <path>...
```

Argument	Description
path	One or more tosh source files. Use '-' to read stdin. (path-like)

Flag / Option	Description
-check	Do not write; exit non-zero if any file would change.
-stdout	Print formatted output to stdout instead of rewriting the file.
-diff	Print a unified-style diff of the changes that would be applied.

Output: Yields one record per file with Path, Changed, and (in -check mode) Match status.

```
format script.tosh # Rewrite a single file in place
format --check src/**/*.tosh # Verify a tree without writing
cat script.tosh | format - # Format stdin and emit to stdout
```

25.11.3 raise

Library: Tosh.Stdlib.Scripting

raise

Raises an event, invoking all registered handlers.

Signature

```
raise <event> | <event> | raise
```

Argument	Description
event	Event object, event factory, or event type to raise. May also be supplied from the pipeline. (optional)
fields	Optional record of field overrides when raising from an event factory. (optional)

Output: Never returns normally — throws a diagnostic / runtime error built from its arguments.

```
$event | raise # Raise a piped event
raise $factory {| Name: "deploy", Status: "ok" |} # Raise from an event
factory with fields
```

25.11.4 undef

Library: *Tosh.Stdlib.Scripting*

undef

Removes user-defined functions.

Signature

```
undef <name> [name...]
```

Argument	Description
name	One or more function names to remove.

Output: Records with Name and Removed properties indicating which functions were removed.

```
undef myfunction # Remove a user-defined function
undef greet parse transform # Remove multiple functions at once
```

25.12 Shell**25.12.1 apropos**

Library: *Tosh.Stdlib.Shell*

apropos

Searches Tosh help topics with fuzzy matching.

Signature

```
apropos <query>
```

Argument	Description
query	The search term to match against help topics.

Pipeline input: scalar — Reads the query from the pipeline if not given as an argument.

Output: Matching help topic summaries with relevance scores.

```
apropos json # Search help for JSON-related topics
apropos loop # Search help for loop constructs
```

Note

Apropos performs fuzzy help search across commands and Tosh language topics.

25.12.2 benchmark

benchmark

User-defined rune (macro).

Signature

```
benchmark <label> <body>
```

25.12.3 clear

Library: Tosh.Stdlib.Shell

clear

Clears the terminal display.

Signature

```
clear
```

Output: Emits nothing; clears the terminal display as a side effect.

```
clear # Clear the terminal
```

25.12.4 config

Library: Tosh.Stdlib.Shell

config

Gets or changes shell configuration.

Signature

```
config [browse [query]]|get <path>|set <path> [value]|reset [path]|init
[path]|reload]
```

Argument	Description
browse [query]	Opens the full-screen config browser, optionally filtered by an initial query, with staged editing, subtree diffs, structured section and collection editors, reusable confirmation and validation surfaces, filesystem browsing, apply/save flows, startup reload/init actions, live prompt/style/theme previews, and raw text editing for advanced cases. (optional)
get <path>	Reads one config value. (optional)
set <path> [value]	Sets one config value. (optional)
reset [section]	Resets a section or the whole config object. (optional)
reload	Replays startup config files into the current session. (optional)
init [directory]	Scaffolds a new config directory. (optional)

Pipeline input: scalar — Only ‘config set’ consumes piped scalar input, using it as the new value when no explicit value argument is present.

Output: Produces the live config object, one config value, status rows, or an interactive browser request depending on the form.

```
config get Shell.Prompt # Read a config value
config set Shell.Prompt.NameText toast # Set a config value
config browse prompt # Open the interactive config browser
config init ~/.config/tosh # Scaffold a config directory
```

25.12.5 dbg

dbg

User-defined rune (macro).

Signature

dbg <expr>

25.12.6 debug

Library: *Tosh.Stdlib.Shell*

debug

Runs a Tosh script with interactive step-through debugging.

Signature

debug <path> [args...]

Argument	Description
path	Path to the Tosh script to debug. (<i>string</i>)

Output: Streams whatever the wrapped pipeline produces; emits diagnostic events as a side effect.

```
debug ./script.tosh # Debug a script with step-through execution.
debug ./script.tosh arg1 arg2 # Debug a script passing arguments.
```

Note

Use 'n' or 'next' to step, 'c' or 'continue' to resume, 'q' or 'quit' to abort, 'vars' to inspect variables.

25.12.7 edit

Library: *Tosh.Stdlib.Shell*

edit

Open a file in the configured terminal editor.

Signature

```
edit [file]
```

Argument**Description**

file

Path of the file to edit. (optional)

Output: No output — the editor takes over the terminal until it exits.

```
edit ~/.config/tosh/profile.tosh # Open a file in the configured editor
edit notes.md # Open a local file
```

Note

Resolves the editor from \$EDITOR, \$VISUAL, then falls back to 'tome', then 'vi', then 'nano'. The child process inherits the terminal, so it can run a full-screen UI.

25.12.8 eval

Library: *Tosh.Stdlib.Shell*

eval

Parses and evaluates a string as Tosh source in the current session.

Signature

```
eval <source> [source...]
```

Argument**Description**

source

Tosh source text to parse and evaluate in the current session.

Output: Streams whatever values the evaluated source emits.

```
eval "1 + 2" # Evaluate a literal expression
eval $"System.Drawing.Color.{$_}" # Build and evaluate code at runtime
read-lines colors.txt | each { eval $"System.Drawing.Color.{$_}" } # Resolve
named members per line
```

25.12.9 events

Library: *Tosh.Stdlib.Shell*

events

Lists, inspects, and manages event handlers.

Signature

```
events [list|names|handlers <event>|remove <event> <handler>|clear
<event>]
```

Argument	Description
action	The action to perform: list, names, handlers, remove, or clear. (optional)
event-name	The event name (for handlers, remove, clear). (optional)
handler	The handler to remove (for remove action). (optional)

Output: Event descriptors, names, or handler objects depending on the action.

```
events # List all registered events
events names # List event names only
events handlers OnPrompt # List handlers for an event
events clear OnPrompt # Remove all handlers for an event
```

25.12.10 exec

Library: *Tosh.Stdlib.Shell*

exec

Replace the current ToSh process with an external command.

Signature

```
exec [-] <command> [arg ...]
```

Argument	Description
command	The external command to execute.
arg	Arguments to pass to the command. (optional)

Flag / Option	Description
-	Signals the end of options; everything after is treated as the command and arguments.

Output: No output — the current process is replaced.

```
exec tosh # Replace the current shell with a new tosh instance
exec zsh # Switch to zsh
exec /bin/sh -c "echo hi" # Exec a shell one-liner
```


Note

Exec replaces the current ToSh process with an external command. On Unix-like systems it uses native process replacement, so ‘exec tosh’ or ‘exec zsh’ behaves like the shell built-in you may know from zsh.

25.12.11 exit

Library: Tosh.Stdlib.Shell

exit

Requests the current Tosh session to exit.

Aliases: logout

Signature

exit [code]

Pipeline input: none — Not applicable.

Output: No output. The session terminates.

```
exit # Exit the current session
exit 1 # Exit with a specific exit code
```

Note

In a login shell, ‘exit’ behaves like ‘logout’: it warns about running background jobs (use ‘exit’ again to force), and sources ~/.config/tosh/logout.tosh before terminating.

25.12.12 help

Library: Tosh.Stdlib.Shell

help

Shows searchable Tosh help for commands, language topics, CLR types, and externals.

Signature

```
help [-cli] [topic ... | browse [query] | search <query> | related
<topic> | categories]
```

Argument	Description
topic	The command, language topic, type, or external executable to describe. (optional)
-cli [query]	Opens the inline fuzzy tree browser, optionally seeded with an initial query or topic. (optional)
browse [query]	Opens the full-screen help browser, optionally filtered by an initial query. (optional)
search <query>	Searches help topics by name, alias, category, and description. (optional)
related <topic>	Shows related topics for the given command or language feature. (optional)
categories	Lists help categories and topic counts. (optional)

Flag / Option	Description
-cli	Open the inline fuzzy help browser instead of returning help objects.

Pipeline input: scalar — With no explicit args, piped scalar values are treated as help topics. ‘search’ and ‘related’ also consume piped query/topic text.

Output: Produces help summaries, full help topics, search results, category rows, launches the inline browser with ‘-cli’, or returns an interactive browser request for ‘help browse’ depending on the form.

```
help ls # Describe a command
help search regex # Search help
help --cli regex # Open the inline fuzzy help browser
help browse grep # Open the help browser
```

Note

Use ‘help search <query>’ or ‘apropos <query>’ to find commands and language topics quickly, ‘help -cli’ for the inline fuzzy tree browser, or ‘help browse’ for the fullscreen split-pane browser. In the REPL, ‘F1’ opens the inline help browser seeded from the token under the cursor, and ‘Alt+H’ is available as a fallback on terminals that do not expose function keys cleanly.

25.12.13 history

Library: *Tosh.Stdlib.Shell*

[shell-only] — only available inside an interactive TōSh session.

history

Shows, searches, expands, runs, deletes, saves, reloads, or clears shell history.

Signature

```
history [status|path|search <text>|expand <spec>|run <spec>|delete
<spec>|save|reload|clear]
```

Argument	Description
search <text>	Searches history entries by text. (optional)
expand <spec>	Expands a history event specification without running it. (optional)
run <spec>	Resolves and executes a history event specification. (optional)
delete <spec>	Deletes one or more history entries by id or spec. (optional)
path save reload clear	History maintenance subcommands. (optional)

Pipeline input: none — History is producer-oriented; replay and expansion are explicit subcommands, while ‘j syntax remains REPL-only sugar.

Output: Produces structured history entries, file paths, expanded command text, or replay results depending on the chosen subcommand.

```
history # List history entries
history search build # Search history
history expand "!!!" # Preview a history expansion
history delete 42 # Delete a history entry
```

Note

History is file-backed in normal interactive sessions and each entry now has a stable id. Use ‘history path’, ‘history search <text>’, ‘history delete <spec>’, ‘history save’, ‘history reload’, ‘history clear’, ‘history expand 237’, or ‘history run 237’ to inspect or replay it. In the REPL, ‘Ctrl+R’, ‘!’, ‘!237’, ‘!-2’, ‘!prefix’, ‘!?text’, ‘!\$', ‘!^’, ‘!*’, and ‘^old^new^’ also work as interactive history features.

25.12.14 history-search

Library: Tosh.Stdlib.Shell

[shell-only] — only available inside an interactive TōSh session.

history-search

Searches shell history entries by text.

Signature

```
history-search <text>
```

Argument	Description
text	Case-insensitive text to search for. May also be supplied from the pipeline.

Output: History records matching the query: index, command line, and timestamp.

```
history-search git # Search history by argument
echo build | history-search # Search history from the pipeline
```

25.12.15 hush

Library: *Tosh.Stdlib.Shell*

hush

Suppresses one or more diagnostic codes within the current scope.

Signature

```
hush <code> [code...]
```

Argument

Description

code ...

Diagnostic codes (e.g. `tosh.runtime.fading_member`) to suppress. (*string*)

Output: Emits nothing; mutates the current scope's hush set as a side effect.

```
hush tosh.runtime.fading_member # Suppress a deprecation warning in the
    current scope
$toosh.Config.Diagnostics.Hushed.Add("tosh.naming.shadowed_builtin") #
    Suppress globally from a profile.tosh entry
```

25.12.16 read-line

Library: *Tosh.Stdlib.Shell*

read-line

Reads a line of text from standard input.

Signature

```
read-line [prompt]
```

Argument

Description

prompt

Optional prompt text to print before reading. (optional)

Output: A single string containing the line read from stdin (or null on EOF).

```
read-line "Name: " # Prompt for one line
var answer = (read-line "Continue? ") # Capture input in a variable
```

25.12.17 source

Library: *Tosh.Stdlib.Shell*

source

Executes a Tosh script file in the current session and lets it affect the caller scope.

Signature

```
source <path> [arg...]
```

Argument	Description
path	Path to a .tosh script file to execute in the current session. (<i>path-like</i>)
args	Arguments forwarded to the script. (optional)

Output: Streams whatever values the sourced script emits.

```
source ~/.config/tosh/profile.tosh # Re-run your profile in the current
session
source ./setup.tosh prod 8080 # Run a script with arguments
```

25.12.18 time

Library: *Tosh.Stdlib.Time*

time

Measures the execution time and resource usage of a command or block.

Signature

```
time <command|block> [args...]
```

Argument	Description
command block [args...]	A block like '{ expr }' or a command followed by its arguments.

Output: Returns a CommandTimingInfo record with timing and resource metrics.

```
time { seq 1000000 | reduce 0 { $acc + _ } }
time ls /usr/lib
```

Note

Metrics include wall time, user/system CPU, memory, and page faults.

25.12.19 tui

Library: *Tosh.Stdlib.Shell*

[**shell-only**] — only available inside an interactive TōSh session.

tui

Interactive TUI components for scripts. Provides list pickers, confirmations, text input, file pickers, and custom screens. Use `-cli` for inline (non-fullscreen) prompts.

Signature

```
tui
pick|confirm|input|file|filter|screen|add-list|add-text|add-input|
add-picker|layout|run
[options]
```

Argument	Description
<code>pick [items...]</code>	Pick one or more values from arguments or pipeline input. (optional)
<code>confirm <message></code>	Ask for a yes/no confirmation. (optional)
<code>input [prompt]</code>	Read text input, optionally multiline or password-style. (optional)
<code>file</code>	Open a file or directory picker. (optional)
<code>filter [items...]</code>	Open a fuzzy filter picker. (optional)
<code>screen add-* layout run</code>	Build and run composed TUI screens from pipeline-carried screen definitions. (optional)

Flag / Option	Description
<code>-cli</code>	Use inline terminal prompts instead of returning fullscreen TUI request objects where supported.
<code>-multi, -m</code>	Allow multiple selections for 'pick', 'filter', and list widgets.
<code>-result</code>	Return a structured outcome object instead of only the selected value/result.
<code>-prompt <text></code>	Prompt text for picker, filter, input, and widget subcommands.
<code>-display <property></code>	Property name used as the display label for object items.
<code>-page-size <n></code>	Number of visible entries in pick/filter lists.
<code>-default <value yes no></code>	Default input value or confirmation default.
<code>-multiline</code>	Allow multiline input for 'input' and 'add-input'.
<code>-password</code>	Mask text for 'input'.
<code>-path <start></code>	Initial path for 'file'.
<code>-filter <glob></code>	File picker filter such as '*.tosh'.
<code>-directory, -d</code>	Choose directories instead of files for 'file'.
<code>-id <id></code>	Stable widget id for screen-builder subcommands.
<code>-searchable, -s</code>	Make a list widget searchable.
<code>-bind <widget.property></code>	Bind a text widget to another widget property.
<code>-no-wrap</code>	Disable text wrapping for 'add-text'.
<code>-ratio <a:b></code>	Layout split ratio for 'layout'.
<code>-gap <n></code>	Gap between layout regions.

Output: Emits nothing; drives the interactive TUI session as a side effect.

```
tui confirm "Deploy now?" --cli # Inline confirmation
ls | tui pick --display Name --result # Pick from pipeline values
tui input "Project name:" --default demo --cli # Inline text input
```

25.12.20 ulimit

Library: *Tosh.Stdlib.Shell*

ulimit

Display or set resource limits.

Signature

```
ulimit [-H] [-a] [resource [value]]
```

Argument	Description
resource	The resource limit to query or set (e.g. 'nofile', 'nproc', 'stack', 'core', 'fsize'). (optional)
value	The new soft limit value to set, or 'unlimited'. (optional)
Flag / Option	Description
-H	Show or set the hard limit instead of the soft limit.
-a	Show all resource limits.

Output: A table of resource limits when no arguments given, or a single value when querying a specific resource.

```
ulimit # Show all resource limits
ulimit -a # Show all resource limits
ulimit nofile # Show the soft file descriptor limit
ulimit nofile 4096 # Set the soft file descriptor limit to 4096
ulimit -H nofile # Show the hard file descriptor limit
```

Note

Displays or sets POSIX resource limits (via `getrlimit/setrlimit`). Only available on Unix-like systems. Setting a hard limit requires root privileges. Use 'unlimited' to remove a soft limit (set to hard limit).

25.12.21 umask

Library: *Tosh.Stdlib.Shell*

umask

Display or set the file creation mask.

Signature

```
umask [mask]
```

Argument	Description
mask	The octal file creation mask to set (e.g. 022, 077). (optional)

Output: The current umask as a zero-padded octal string (e.g. '0022'), or nothing when setting.

```
umask # Display the current umask
umask 022 # Set umask to 022 (owner: all, group/other: no write)
umask 077 # Set umask to 077 (owner: all, group/other: no access)
```

Note

Controls the default permission bits removed from newly created files and directories. Without arguments, displays the current mask in octal. Only available on Unix-like systems.

25.12.22 unless**unless**

User-defined rune (macro).

Signature

```
unless <condition> <body>
```

25.12.23 view

Library: `Tosh.Stdlib.Shell`

view

Gets or sets shell display preferences.

Signature

```
view
[compact|detail|datetime|datetimeoffset|dateonly|timeonly|timespan|
size|permissions|attributes|columns]
```

Output: Emits nothing; opens the supplied content in the configured pager/viewer as a side effect.

```
view dateonly scalar relative
view timeonly table 24h
view duration table seconds
```

25.12.24 which

Library: `Tosh.Stdlib.Shell`

which

Resolves built-in commands and external executables.

Aliases: whence

Signature

```
which <name ...>
```

Argument**Description**

name

One or more command names to resolve.

Output: Command resolution objects showing Kind (Builtin/External/Alias/Function), Name, and Path.


```
which ls # Find the ls command
which git python node # Resolve multiple commands
```

25.12.25 with-retry

with-retry

User-defined rune (macro).

Signature

```
with-retry <attempts> <body>
```

25.13 System

25.13.1 date

Library: Tosh.Stdlib.Time

date

Creates, parses, and adjusts date/time values.

Signature

```
date [-d|-date-only] [-t|-time-only]
<now|utc-now|today|tomorrow|yesterday|parse|from-unix|from-unix-ms|
date-only|time-only|<iso-date>
... or <date> | date [-d] [-t] <add|sub|date-only|time-only> ...
```

Argument	Description
mode operation	Creation mode ('now', 'utc-now', 'today', 'tomorrow', 'yesterday', 'parse', 'from-unix', 'from-unix-ms', 'date-only', 'time-only', or an ISO value) or pipeline operation ('add', 'sub', 'date-only', 'time-only'). (optional)
value ...	Values required by the chosen mode or operation, such as a parse string, Unix timestamp, or duration. (optional)

Flag / Option	Description
-d, -date-only, -dateonly	Project results to DateOnly values.
-t, -time-only, -timeonly	Project results to TimeOnly values.

Output: DateTime, DateOnly, or TimeOnly values depending on the requested mode/operation and projection flags.

```
date now -d -t
date parse 2026-03-29T12:34:56Z -d
date parse 2026-03-29T12:34:56Z | cast timeonly
```

25.13.2 env

Library: *Tosh.Stdlib.Sys*

env

Lists, queries, sets, or unsets environment variables. Runs nested commands with temporary environment changes when a command follows.

Signature

```
env [name ...] | env [-u name] [name=value ...] | env [-u name]
[name=value ...] - <command ...>
```

Argument	Description
name ...	With no assignments, query one or more environment variable names. (optional)
name=value ...	Temporary environment assignments for ‘env’ output or a nested command. (optional)
command ...	Optional nested command to run under the temporary environment snapshot. (optional)

Flag / Option	Description
-u <name>	Unset a variable in the temporary environment snapshot.
-	Treat the remaining arguments as the nested command even when there are no assignments.

Pipeline input: scalar, record — With no explicit names, piped scalar values are treated as queried environment-variable names.

Output: Returns typed environment-variable entries, or the nested command’s output when assignments/unsets are used with a command. Use ‘\$env.NAME’ for direct value-only access when you want the variable value instead of the structured ‘env’ entry object. Missing ‘\$env’ members resolve to ‘null’ rather than raising a missing-member error.

```
env PATH # Inspect the structured PATH entry
echo $env.PATH # Read just the PATH value
echo $env.EDITOR # Read another environment variable directly
```

Note

Env keeps its object-returning query mode, but it can also build a temporary environment snapshot with ‘name=value’ and ‘-u name’, optionally running a nested command under that snapshot.

25.13.3 export

Library: *Tosh.Stdlib.Sys*

export

Exports a Tosh value to the process environment.

Signature

```
export <name> = <value>
```

Argument	Description
name	The environment variable name.
=	Assignment operator. (optional)
value	The value to assign. If omitted, exports the current Tosh variable of that name. (optional)

Output: No output. The environment variable is set in the current process.

```
export PATH = "/usr/local/bin:$env.PATH" # Prepend to PATH
export MY_VAR = "hello" # Set a new environment variable
export MY_VAR # Export an existing Tosh variable
```

25.13.4 forget

Library: Tosh.Stdlib.Sys

forget

Removes Tosh variables, functions, and exported environment names.

Aliases: unset

Signature

```
forget <name> [name...]
```

Argument	Description
name ...	Variables, functions, or exported environment names to remove. Values can also be piped in. (<i>string</i>)

Output: Emits nothing; removes the named bindings from the current scope as a side effect.

```
unset TEMP_VALUE # Remove a name with the unset alias
forget old-helper # Remove a function or variable
```

25.13.5 free

Library: Tosh.Stdlib.Sys

free

Returns system memory and swap usage.

Signature

```
free
```

Output: A record with TotalMemory, UsedMemory, FreeMemory, TotalSwap, UsedSwap, and FreeSwap as StorageSize values.

```
free # Show memory and swap usage
free | get .UsedMemory # Get the used memory value
```

25.13.6 guid

Library: *Tosh.Stdlib.Sys*

guid

Creates, parses, formats, and inspects GUID values.

Signature

```
guid [new [v4|v7]|empty|parse|format <d|n|b|p|x>|info] [value ...]
```

Output: A System.Guid value (or the requested string projection).

```
guid
guid new v7
guid info 550e8400-e29b-41d4-a716-446655440000
```

25.13.7 hostname

Library: *Tosh.Stdlib.Sys*

hostname

Returns the current host name.

Signature

```
hostname
```

Output: A single string containing the host name.

```
hostname # Show the current host name
```

25.13.8 hostnectl

Library: *Tosh.Stdlib.Sys*

hostnectl

Wraps `hostnamectl`, returning a structured host-status object for supported JSON-backed status queries.

Signature

```
hostnectl [status | <other-command ...>]
```

Argument	Description
[status]	With no subcommand, ToSh treats 'hostnectl' as a structured status query. Explicit 'status' behaves the same way. (optional)
<other-command ...>	Mutating or unsupported commands fall back to the native 'hostnamectl' utility unchanged. (optional)

Flag / Option	Description
-show <columns>	Use ToSh display-only column selection on the structured host-status object.
-hide <columns>	Hide display columns while preserving the underlying typed host object.
-show-all	Expose every selectable structured host-status display column.

Pipeline input: none — The structured ‘hostnamectl’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns a structured host-status object for supported JSON-backed ‘hostnamectl status’ queries. Other commands currently fall back to the native ‘hostnamectl’ output.

```
hostnamectl # Inspect structured host status
hostnamectl --show Hostname,OperatingSystem,KernelRelease # Render a focused
host summary
hostnamectl | get { Hostname, MachineID, BootID } # Project host identity
properties in the pipeline
```

25.13.9 id

Library: Tosh.Stdlib.Sys

id

Returns current user and group identity information.

Signature

```
id [-u|-g|-G] [-n]
```

Flag / Option	Description
-u	Print only the effective user ID.
-g	Print only the effective group ID.
-G	Print all group IDs.
-n	Print names instead of numeric IDs (with -u, -g, or -G).

Output: An identity record with Uid, Gid, User, Group, and Groups properties, or a single value with flags.

```
id # Show full identity information
id -un # Show the current username
```

25.13.10 journalctl

Library: Tosh.Stdlib.Sys

journalctl

Wraps journalctl, returning typed journal-entry objects for supported JSON-backed query invocations.

Signature

```
journalctl [-n count] [-u unit] [--since when] [--until when] [query ...]
```

Argument	Description
[query ...]	Structured journal queries accept common ‘journalctl’ filters such as match expressions, unit filters, priorities, boot selectors, and time windows. (optional)

Flag / Option	Description
-n <count> -lines <count>	Limit the structured result set to the most recent entries.
-u <unit> -unit <unit>	Restrict structured output to a specific systemd unit.
--since <when>	Restrict entries to those at or after the supplied time.
--until <when>	Restrict entries to those at or before the supplied time.
-p <range> -priority <range>	Restrict entries to a priority or priority range.
-b [id] -boot [id]	Restrict entries to the current boot or a selected boot.
-g <pattern> -grep <pattern>	Restrict structured output to entries whose messages match a pattern.
-user	Query the user journal instead of the system journal.
-system	Query the system journal explicitly.
-show <columns>	Use ToSh display-only column selection on the structured journal-entry rows.
-hide <columns>	Hide display columns while preserving the underlying structured journal entries.
-show-all	Expose every selectable structured journal-entry display column.

Pipeline input: none — The structured ‘journalctl’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Streams structured journal-entry objects from ‘journalctl -o json’ for supported non-follow query paths. Unsupported or text-oriented modes fall back to the native utility.

```
journalctl -n 20 # Stream the newest journal entries as structured objects
journalctl -u sshd.service | first 10 # Inspect a unit's logs in the pipeline
journalctl --since yesterday | where _.Priority <= 3 # Filter recent high-
priority entries
```

25.13.11 loginctl

Library: Tosh.Stdlib.Sys

loginctl

Wraps loginctl, returning typed session/user/seat rows and structured login property sets for supported queries.

Signature

```
loginctl [list-sessions | list-users | list-seats | show-session <id
...> | show-user <user ...> | show-seat <seat ...> | <other-subcommand
...>]
```

Argument	Description
[list-sessions list-users list-seats]	With no subcommand, ToSh treats ‘loginctl’ as a structured ‘list-sessions’ query. The explicit list subcommands return typed rows. (optional)
show-session <id ...> show-user <user ...> show-seat <seat ...> <other-subcommand ...>	Returns structured login property sets for supported ‘show-*’ queries. (optional) Unsupported subcommands fall back to the native ‘loginctl’ utility unchanged. (optional)

Flag / Option	Description
-p <property[,property...]> -property <property[,property...]>	Restrict ‘show-*’ to specific fetched properties. ToSh still injects the relevant identity property internally so multiple-result output stays structured.
-all	Include empty properties in supported ‘show-*’ queries when the underlying ‘loginctl’ invocation supports it.
-show <columns>	Use ToSh display-only column selection on structured list rows or property sets.
-hide <columns>	Hide display columns while preserving the underlying typed objects.
-show-all	Expose every selectable structured display column for the current output shape.

Pipeline input: none — The structured ‘loginctl’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns typed session, user, or seat rows for supported list queries, and structured property-set objects for supported ‘show-*’ queries. Other subcommands currently fall back to the native ‘loginctl’ output.

```
loginctl # List sessions as typed rows
loginctl list-users | where _.State == active # Filter active login users in
the pipeline
loginctl show-user 1000 | get { UID, Name, State, Sessions } # Inspect
structured user-login properties
```

25.13.12 lscpu

Library: Tosh.Stdlib.Sys

lscpu

Wraps the system `lscpu` utility, returning typed CPU summary, topology, and cache objects for JSON-backed invocations.

Signature

```
lscpu [-B] [-hierarchic when] | lscpu -e[-all|-online|-offline] [-x]
[-y] [-o columns] | lscpu -C [-B] [-o columns]
```

Argument	Description
<code>[-e -C]</code>	Without a mode flag, ‘lscpu’ returns a structured CPU summary. ‘-e’ switches to per-CPU topology rows and ‘-C’ switches to CPU cache rows. (optional)

Flag / Option	Description
<code>-B</code>	Render cache sizes in raw bytes for summary and cache views.
<code>-e</code>	Return per-CPU topology rows from ‘lscpu -extended -json’.
<code>-C</code>	Return CPU cache rows from ‘lscpu -caches -json’.
<code>-a</code>	Include both online and offline CPUs in extended mode.
<code>-b</code>	Restrict extended mode to online CPUs.
<code>-c</code>	Restrict extended mode to offline CPUs.
<code>-x</code>	Request hexadecimal CPU masks where the underlying ‘lscpu’ mode supports them.
<code>-y</code>	Request physical IDs instead of logical IDs in extended mode.
<code>-o <columns></code>	Select lscpu-style columns such as ‘CPU,NODE,SOCKET,CORE,ONLINE,MHZ’ for ‘-e’ or ‘NAME,LEVEL,TYPE,ONE-SIZE’ for ‘-C’.
<code>-output-all</code>	Expose every selectable structured column for ‘-e’ or ‘-C’.
<code>-hierarchic <when></code>	Pass through ‘auto’, ‘always’, or ‘never’ for the summary view.

Pipeline input: none — The structured ‘lscpu’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns a structured CPU summary by default, typed per-CPU topology rows with ‘-e’, or typed CPU cache rows with ‘-C’.

```
lscpu # Show the structured CPU summary
lscpu -e | where _.Online == true | first 8 # Browse structured per-CPU
      topology rows
lscpu -C -B | get { Name, Level, OneSize, AllSize } # Inspect cache metadata
      with byte-oriented sizes
```


Note

ToSh wraps the JSON-capable ‘lscpu’ modes instead of scraping text output. The default command yields a structured CPU summary, ‘-e’ yields per-CPU topology rows, and ‘-C’ yields cache rows. ‘-parse’, raw-only, help, version, and sysroot modes currently fall back to the external ‘lscpu’ utility unchanged.

25.13.13 lsipc

Library: Tosh.Stdlib.Sys

lsipc

Wraps the system lsipc utility, returning structured IPC resource and limit records for JSON-backed invocations.

Signature

```
lsipc [-m|-M|-q|-Q|-s|-S] [-g] [-i id|-N name] [-c] [-t] [-b] [-P] [-o
columns]
```

Argument**Description**

[-m|-M|-q|-Q|-s|-S]

Select a specific IPC resource family. With no resource flag, ‘lsipc’ returns the global IPC limits and usage summary. (optional)

Flag / Option	Description
-m	Return System V shared-memory rows.
-M	Return POSIX shared-memory rows.
-q	Return System V message-queue rows.
-Q	Return POSIX message-queue rows.
-s	Return System V semaphore rows.
-S	Return POSIX semaphore rows.
-g	Return global usage/limit rows, optionally scoped by 'm', 'q', or 's'.
-i <id>	Restrict the query to a specific System V IPC id.
-N <name>	Restrict the query to a specific POSIX IPC name.
-c	Include creator-related fields such as creator uid, user, and group.
-t	Include time-oriented fields such as attach, detach, change, or last-operation timestamps.
-b	Request byte-oriented numeric sizes from the underlying 'lsipc' command.
-P	Render permissions numerically instead of symbolically.
-l	Force list output shape where the underlying 'lsipc' mode supports it.
-o <columns>	Select lsipc-style columns such as 'KEY,ID,OWNER,SIZE,NATTCH' or 'RESOURCE,LIMIT,USED,USE%':
-show <columns>	Use ToSh display-only column selection on the structured rows after parsing.
-hide <columns>	Hide display columns while keeping the full structured rows in the pipeline.

Pipeline input: none — The structured 'lsipc' builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns structured IPC resource rows or global IPC limit/usage rows, with typed sizes, counts, and ISO-normalized timestamps where the underlying data supports them.

```
lsipc # Browse global IPC limits and current usage as structured rows
lsipc -m | first 5 # Inspect System V shared-memory rows
lsipc -g -m | get { Resource, Limit, Used, UsePercent } # Show the global
shared-memory limits and utilization summary
```

Note

ToSh wraps 'lsipc -J' so IPC resources and global IPC limits flow through the pipeline as structured rows instead of terminal-shaped text. Text-format-only modes like '-raw', '-export', '-newline', and shell-variable output currently fall back to the external 'lsipc' utility unchanged.

25.13.14 networkctl

Library: Tosh.Stdlib.Sys

networkctl

Wraps networkctl, returning typed network-link rows for supported list queries.

Signature

```
networkctl [list [pattern ...] | <other-command ...>]
```

Argument	Description
[list [pattern ...]]	With no subcommand, ToSh treats ‘networkctl’ as a structured ‘list’ query. Explicit ‘list’ behaves the same way. (optional)
<other-command ...>	Unsupported, detail, and mutating commands currently fall back to the native ‘networkctl’ utility unchanged. (optional)

Flag / Option	Description
-a -all	Pass through to the structured ‘networkctl list’ query to include all visible links.
-l -full	Pass through to the structured ‘networkctl list’ query.
-show <columns>	Use ToSh display-only column selection on the structured network-link rows.
-hide <columns>	Hide display columns while preserving the underlying typed link objects.
-show-all	Expose every selectable structured network-link display column.

Pipeline input: none — The structured ‘networkctl’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns typed network-link rows for supported ‘networkctl list’ queries. Other commands currently fall back to the native ‘networkctl’ output.

```
networkctl # List network links as typed rows
networkctl | where _.Setup == unmanaged # Filter links by setup state in the
    pipeline
networkctl --show Link,Operational,Setup,Managed # Render a focused network-
    link summary
```

25.13.15 seq

Library: Tosh.Stdlib.Sys

seq

Generates a numeric sequence.

Signature

```
seq <stop> | seq <start> <stop> | seq <start> <step> <stop>
```

Argument	Description
stop	Final value when one argument is provided.
start	Initial value when two or three arguments are provided. (optional)
step	Step size when three arguments are provided. Cannot be zero. (optional)

Output: An int (or numeric) sequence — one value per element of the requested range.

```
seq 5 # Generate 1 through 5
seq 0 2 10 # Generate an stepped sequence
seq 5 -1 1 # Generate a descending sequence
```

25.13.16 sleep

Library: *Tosh.Stdlib.Time*

sleep

Pauses execution for a duration.

Signature

```
sleep <duration> [duration...]
```

Argument	Description
duration	A duration: integer (seconds), decimal, TimeSpan, quantity (e.g. 500‘ms), or string like ‘1s’, ‘2m’.

Output: No output. Resumes after the specified duration.

```
sleep 2 # Sleep for 2 seconds
sleep 500‘ms # Sleep for 500 milliseconds using a unit literal
sleep 1 2 3 # Sleep for a total of 6 seconds
```

25.13.17 systemctl

Library: *Tosh.Stdlib.Sys*

systemctl

Wraps systemctl, returning typed unit rows for supported list queries and structured unit property sets for supported show and status queries.

Signature

```
systemctl [list-units [pattern ...] | list-unit-files [pattern ...] |
show <unit ...> [options] | status <unit ...> | <other-subcommand
...>]
```

Argument	Description
[list-units [pattern ...]]	With no subcommand, ToSh treats ‘systemctl’ as a structured ‘list-units’ query. Explicit ‘list-units’ behaves the same way. (optional)
list-unit-files [pattern ...]	Returns typed unit-file state rows for installed unit files. (optional)
show <unit ...>	Returns structured systemd unit property sets for one or more units. (optional)
status <unit ...>	Returns structured unit status objects for one or more units, including recent logs when available. (optional)
<other-subcommand ...>	Unsupported subcommands fall back to the native ‘systemctl’ utility unchanged. (optional)

Flag / Option	Description
-type <type[,type...]> -t <type[,type...]>	Restrict structured ‘list-units’ or ‘list-unit-files’ output to specific unit types such as ‘service’ or ‘socket’.
-state <state[,state...]>	Restrict structured ‘list-units’ output to specific load/active states.
-all	Include inactive and unloaded units in the structured listing.
-failed	Restrict the structured listing to failed units.
-p <property[,property...]> -property <property[,property...]>	Restrict ‘show’ to specific fetched properties. ToSh still injects ‘Id’ internally so multiple-unit output stays structured.
-show <columns>	Use ToSh display-only column selection on structured unit rows or property sets.
-hide <columns>	Hide display columns while preserving the underlying typed objects.
-show-all	Expose every selectable structured display column for the current output shape.

Pipeline input: none — The structured ‘systemctl’ builtin is explicit-arg-first and does not currently consume pipeline input.

Output: Returns typed systemd unit rows for ‘list-units’, typed unit-file rows for ‘list-unit-files’, and structured unit property-set objects for supported ‘show’ and ‘status’ queries. Other subcommands currently fall back to the native ‘systemctl’ output.

```
systemctl # List units as typed rows
systemctl --type service | where _.Active == active # Filter active services
in the pipeline
systemctl list-unit-files --type service | where _.Enabled # Inspect
installed enabled service unit files
systemctl status sshd.service | get { Id, ActiveState, MainPID, RecentLogCount
} # Inspect structured unit status details
```

25.13.18 timespan

Library: Tosh.Stdlib.Time

timespan

Parses a duration into a CLR duration value.

Signature

```
timespan <duration>
```

Argument	Description
duration	Duration text such as 250ms, 5s, 2m, 1h, or 1d.

Output: A System.TimeSpan value parsed/constructed from the supplied components.

```
timespan 250ms # Parse milliseconds
timespan 1h30m # Parse a compound duration
```

25.13.19 uname

Library: Tosh.Stdlib.Sys

uname

Returns kernel and operating system information as a Tosh object.

Signature

```
uname [-a|-s|-n|-r|-v|-m|-o]
```

Flag / Option	Description
-s	System Name
-n	Node Name
-r	Kernel Release
-v	Kernel Version
-m	Machine
-o	Operating System
-a, -all	Return the full structured uname object.
-kernel-name	Return the system/kernel name.
-nodename	Return the network node name.
-kernel-release	Return the kernel release.
-kernel-version	Return the kernel version.
-machine	Return the machine hardware name.
-operating-system	Return the operating system name.

Output: A record describing the host OS: kernel name, machine, version, and release.

```
uname # Return structured system information
uname -sr # Return selected fields
```

25.13.20 uptime

Library: Tosh.Stdlib.Sys

uptime

Returns system uptime and load averages.

Signature

uptime

Output: A record describing system uptime: boot time, elapsed duration, and load averages where available.

```
uptime # Show uptime and load averages
```

25.13.21 vars

Library: Tosh.Stdlib.Sys

vars

Lists visible variables or opens an interactive variable browser.

Signature

vars [filter] | vars browse [filter]

Argument**Description**

filter

Optional case-insensitive name filter. (optional)

browse [filter]

Open the interactive variable browser, optionally filtered. (optional)

Output: Records describing each binding in scope: name, kind (var/const/import), and current value.

```
vars # List visible variables
vars path # Filter variable names
vars browse env # Browse variables interactively
```

25.13.22 whoami

Library: Tosh.Stdlib.Sys

whoami

Returns the current user principal.

Signature

whoami

Output: A single string containing the current user name.

```
whoami # Show the current user
```

25.14 Text

25.14.1 cut

Library: *Tosh.Stdlib.Text*

cut

Extracts character or delimited fields from text.

Signature

```
cut (-f fields [-d delimiter] | -c chars) [path ...]
```

Argument	Description
path ...	Optional files to read instead of pipeline input. (optional) (<i>path-like</i>)
Flag / Option	Description
-f <fields>	Select 1-based delimited fields. Supports comma-separated values and ranges such as 1,3-5.
-d <delimiter>	Field delimiter for -f mode. Defaults to tab.
-c <chars>	Select 1-based character positions. Supports comma-separated values and ranges such as 1,3-5.

Output: ShellTextLine values containing the selected character ranges or delimited fields, one per input line.

```
echo "alpha,beta,gamma" | cut -d , -f 2 # Extract a delimited field
echo "abcdef" | cut -c 2-4 # Extract character positions
```

25.14.2 echo

Library: *Tosh.Stdlib.Text*

echo

Emits its arguments as pipeline objects.

Signature

```
echo <value> [value...]
```

Argument	Description
value	One or more values to emit as pipeline objects.

Output: Each argument as a separate pipeline object.

```
echo hello world
echo 42 | where { $_ > 10 }
```

25.14.3 grep

Library: *Tosh.Stdlib.Text*

grep

Searches text input with a regular expression or literal pattern.

Signature

```
grep [-i] [-v] [-F] [-n] [-r] [-w] [-o] [-c] [-l] [-L] [-A num] [-B
num]
[-C num] <pattern|regex> [path ...]
```

Argument	Description
pattern regex	The text pattern or .NET regular expression to search for. (<i>string/regex</i>)
path	Optional file paths to search instead of consuming piped text. (optional) (<i>path-like</i>)

Flag / Option	Description
-i	Ignore case.
-v	Invert the match.
-F	Treat the pattern as a literal string instead of a regex.
-n	Include source line numbers in text-file results.
-m	Multiline mode.
-s	Singleline mode so ' ' matches newlines.
-x	Require a full-line match.
-r, -R, -recursive	Recurse into directories when path arguments are supplied.
-w, -word-regexp	Match only whole words.
-o, -only-matching	Print only the matching portion of each line.
-c, -count	Print only the count of matching lines per source.
-l, -files-with-matches	Print only the names of files that contain a match.
-L, -files-without-match	Print only the names of files that contain no match.
-A, -after-context	Print N lines of trailing context after each match.
-B, -before-context	Print N lines of leading context before each match.
-C, -context	Print N lines of context around each match.
-explicit-capture	Return structured capture results instead of plain matching lines.

Pipeline input: scalar — Consumes scalar text from the pipeline. When paths are supplied explicitly, grep reads file contents instead.

Output: Matching text lines, or structured regex capture objects with `-explicit-capture`.

```
echo one two three | grep tw # Pipe text into grep
echo "Alpha" | grep -i "^alpha$" # Use regex flags
grep -F literal README.md # Search a file literally
```

25.14.4 head

Library: *Tosh.Stdlib.Text*

head

Returns the first objects or first text lines.

Signature

```
head [-n count] [-c bytes] [path ...]
```

Argument	Description
path ...	Optional files to read instead of pipeline input. (optional) <i>(path-like)</i>
Flag / Option	Description
-n, -lines <count>	Return this many lines or pipeline items. Defaults to 10.
-c, -bytes <bytes>	Return this many bytes from each file. Requires file paths.

Output: The first N items of the input stream (default 10), preserving order.

```
ls | head -n 5 # Take the first five pipeline items
head -n 20 app.log # Read the first twenty lines of a file
head -c 128 payload.bin # Read leading bytes from a file
```

25.14.5 join-lines

Library: Tosh.Stdlib.Text

join-lines

Joins input values into a single text value.

Signature

```
join-lines [separator]
```

Argument	Description
separator	Separator inserted between values. Defaults to the platform newline. (optional)

Output: A single string formed by concatenating input lines with the configured separator.

```
echo alpha beta gamma | join-lines ", " # Join values with a comma
read-lines names.txt | join-lines # Join lines with newlines
```

25.14.6 lines

Library: Tosh.Stdlib.Text

lines

Splits text input into individual lines.

Signature

```
lines [text ...]
```

Argument	Description
text ...	Text values to split. When omitted, reads pipeline input. (optional)

Output: ShellTextLine values — one per line of the input text.

```
lines "alpha\nbeta" # Split an explicit string
read-file notes.txt | lines | where _.Contains("TODO") # Split file text into
lines
```

25.14.7 match

Library: Tosh.Stdlib.Text

match

Matches text with a regular expression and returns shell record objects.

Signature

```
match [-i] [-m] [-s] [-x] [-explicit-capture] <pattern|regex> [text
...]
```

Argument	Description
pattern regex	The .NET regular expression pattern or Regex object to apply. (<i>string/regex</i>)
text ...	Optional explicit text values. When omitted, reads pipeline text. (optional)

Flag / Option	Description
-i, -ignore-case	Use case-insensitive matching.
-m, -multiline	Enable multiline mode so ^ and \$ match line boundaries.
-s, -singleline	Enable singleline mode so . matches newlines.
-x,	Ignore unescaped whitespace and allow # comments
-ignore-pattern-whitespace	in the regex pattern.
-explicit-capture	Only capture explicitly named or numbered groups.

Output: Match records describing each capture: full match, group values, and 0-based positions.

```
echo "PID=42" | match "PID=(?<Pid>[0-9]+)" | get Pid
echo "Alpha" | match -i "^alpha$"
```

25.14.8 raw

Library: Tosh.Stdlib.Text

raw

Emits plain text without rich table or record rendering.

Signature

```
raw [value...]
```

Argument	Description
value ...	Optional explicit values to emit as plain text instead of rich object display. (optional)

Pipeline input: scalar, record, list, table — Consumes pipeline values and emits one plain-text line per input item.

Output: Returns ShellTextLine values so the final display stays plain text instead of tables or record views.

```
echo 1317 | raw # Show a scalar without rich boxing
echo System.DayOfWeek.Friday | raw # Show an enum's raw CLR string form
```

25.14.9 replace

Library: *Tosh.Stdlib.Text*

replace

Replaces text in each input value.

Signature

```
replace [-i] [-m] [-s] [-x] [-explicit-capture] [-r] <pattern|regex>
<replacement> [text ...]
```

Argument	Description
pattern regex	String pattern, regex pattern, or Regex object to replace.
replacement	Replacement text.
text ...	Optional explicit text values. When omitted, reads pipeline text. (optional)

Flag / Option	Description
-r, -regex	Treat the pattern as a regular expression.
-i, -ignore-case	Use case-insensitive matching.
-m, -multiline	Enable regex multiline mode.
-s, -singleline	Enable regex singleline mode so . matches newlines.
-x,	Ignore unescaped whitespace and allow # comments
-ignore-pattern-whitespace	in the regex pattern.
-explicit-capture	Only capture explicitly named or numbered groups in regex mode.

Output: ShellTextLine values containing the input lines with the configured replacements applied.

```
echo alpha-beta | replace beta BETA
echo "A1 B2" | replace -r "[0-9]" "#"
```

25.14.10 `split`

Library: Tosh.Stdlib.Text

`split`

Splits text values into smaller text values.

Signature

```
split [-r] [-i] [-m] [-s] [-x] [-explicit-capture] [delimiter|regex]
[text ...]
```

Argument	Description
<i>delimiter regex</i>	Delimiter string, regex pattern, or Regex object. Defaults to whitespace splitting when omitted. (optional)
<i>text ...</i>	Optional explicit text values. When omitted, reads pipeline text. (optional)

Flag / Option	Description
<code>-r, -regex</code>	Treat the delimiter as a regular expression.
<code>-i, -ignore-case</code>	Use case-insensitive regex matching.
<code>-m, -multiline</code>	Enable regex multiline mode.
<code>-s, -singleline</code>	Enable regex singleline mode so <code>.</code> matches newlines.
<code>-x,</code> <code>-ignore-pattern-whitespace</code>	Ignore unescaped whitespace and allow <code>#</code> comments in the regex pattern.
<code>-explicit-capture</code>	Only capture explicitly named or numbered groups in regex mode.

Output: ShellTextLine values — one per substring produced by the configured split.

```
echo "alpha,beta,gamma" | split ","
echo "alpha,beta;gamma" | split -r "[,;]"
```

25.14.11 `tail`

Library: Tosh.Stdlib.Text

`tail`

Returns the last objects or last text lines.

Signature

```
tail [-n count] [-c bytes] [-f] [path ...]
```

Argument	Description
<i>path ...</i>	Optional files to read instead of pipeline input. (optional) (<i>path-like</i>)

Flag / Option	Description
<code>-n, -lines <count></code>	Return this many lines or pipeline items. Defaults to 10.
<code>-c, -bytes <bytes></code>	Return this many trailing bytes from each file. Requires file paths.
<code>-f, -follow</code>	Continue following a single file and emit newly appended lines.

Output: The last N items of the input stream (default 10), preserving order.

```
ls | tail -n 5 # Take the last five pipeline items
tail -n 50 app.log # Read the last fifty lines of a file
tail -f app.log # Follow appended log lines
```

25.14.12 `template`

Library: `Tosh.Stdlib.Text`

`template`

Renders pipeline objects into text using `{{ member.path }}` placeholders.

Signature

```
template <text>
```

Argument	Description
text	A template string with <code>{{ member.path }}</code> placeholders.

Pipeline input: scalar, record — Applies the template to each piped object.

Output: Rendered text with placeholders replaced by each pipeline object's member values.

```
ls | template "{{Name}} is {{Length}} bytes" # Format file info
echo {|name="World"|} | template "Hello, {{name}}!" # Simple template
rendering
```

25.14.13 `tr`

Library: `Tosh.Stdlib.Text`

`tr`

Translates or deletes characters in text.

Signature

```
tr [-d] <set1> [set2] [path ...]
```

Argument	Description
set1	Characters to translate from (or delete with -d).
set2	Characters to translate to. Required unless -d is used. (optional)
path	Optional file path(s) to read instead of pipeline input. (optional)

Flag / Option	Description
-d	Delete characters in set1 instead of translating.

Pipeline input: scalar — Reads text lines from the pipeline when no file paths are given.

Output: Translated or filtered text lines.

```
echo HELLO | tr A-Z a-z # Convert to lowercase
echo "hello world" | tr -d ' ' # Delete spaces
```

25.14.14 uniq

Library: Tosh.Stdlib.Text

uniq

Collapses adjacent duplicate input values.

Signature

```
uniq [-c] [-i] [path ...]
```

Argument	Description
path	Optional file path(s) to read instead of pipeline input. (optional)

Flag / Option	Description
-c	Prefix each line with its count of consecutive occurrences.
-i	Compare lines case-insensitively.

Pipeline input: scalar — Reads text lines from the pipeline or from file arguments.

Output: Unique adjacent values, optionally prefixed with counts.

```
echo a a b b b c | uniq # Collapse adjacent duplicates
echo a a b b b c | uniq -c # Count consecutive occurrences
```

25.14.15 wc

Library: Tosh.Stdlib.Text

wc

Counts lines, words, bytes, characters, and longest-line length.

Signature

```
wc [-lwmcL] [path ...]
```

Argument	Description
path ...	Optional file paths to measure. When omitted, ‘wc’ measures the current text pipeline. (optional) <i>(path-like)</i>

Flag / Option	Description
-l	Show line counts.
-w	Show word counts.
-c	Show byte counts.
-m	Show character counts.
-L	Show the longest-line length.

Pipeline input: scalar, record — Consumes the current text pipeline when explicit file paths are omitted.

Output: Returns typed text-statistics objects, and appends a ‘total’ row when multiple files are counted.

```
wc README.md
wc -lwm README.md
echo one two three | wc
```

Note

Wc returns typed statistics objects instead of formatted text, so you can still ‘get’, ‘where’, or ‘summarize’ them later. Selector flags like ‘-l’ and ‘-w’ only change the visible columns, not the underlying objects.

25.14.16 write

Library: Tosh.Stdlib.Text

write

Writes rendered values without a trailing newline.

Signature

```
write [value...]
```

Argument	Description
value ...	Values to render. When omitted, renders pipeline input. (optional)

Output: Emits nothing; writes its arguments to stdout (without a trailing newline) as a side effect.

```
write "no newline" # Write text without a trailing newline
```



```
echo alpha beta | write # Write piped values
```

25.14.17 writeline

Library: Tosh.Stdlib.Text

writeline

Writes rendered values with a trailing newline.

Signature

```
writeline [value...]
```

Argument	Description
value ...	Values to render. When omitted, renders pipeline input. (optional)

Output: Emits nothing; writes its arguments to stdout (each terminated by a newline) as a side effect.

```
writeline "hello" # Write a line
echo alpha beta | writeline # Write piped values with a newline
```

Part IV

Cookbook

Chapter 26

Common Tasks

26.1 File System Operations

26.1.1 Find Large Files

```
# Find files larger than 100MB
find . -type f -size +100mb | sort Size | reverse

# Using ls with pipeline
ls -laR | where _.Type == file | where _.Size > 100mb | sort Size | reverse
```

26.1.2 Batch Rename

```
# Rename all .txt files to .md
ls *.txt | each {
    var newName = replace ".txt" ".md" _.Name
    mv _.FullName $newName
}
```

26.1.3 Disk Space Report

```
du -d 1 . | sort Size | reverse | first 10
echo $"Total: {du -s . | get Size}"
```

26.1.4 Watch for New Files

```
func onNewFile(event) handles DirectoryChanged {
    if ($event.ChangeType == "Created") {
        writeline $"New file: {$event.Name}"
    }
}
```

26.2 Text Processing

26.2.1 Log Analysis

```
# Count errors by type
read-lines /var/log/app.log
| grep "ERROR"
| match "ERROR\s+(\w+)"
| get 1
| frequencies
| sort Count
| reverse
```

26.2.2 Delimited Column Types

from csv and from tsv infer column types, so numeric columns arrive ready to compare rather than as text. Inference is deliberately narrow—integers, decimals, and true/false:

```
Id,Name,Amount,Ratio,Active
001,Alice,150,2.5,true

# Id      -> string (leading zero: an identifier, not a number)
# Name    -> string
# Amount  -> int
# Ratio   -> double
# Active  -> bool
```

The rules, and what each one protects:

Per column, not per cell A column is typed only when every non-empty cell agrees. One n/a in a numeric column leaves the whole column textual, so values within a column always compare with each other.

Integers and decimals reconcile A column of 1, 2, 2.5 is double. Integers narrow to int where they fit, widening to long only when they must.

Leading zeros stay text 007 and 01234 are identifiers; converting them destroys the zero irreversibly. 0 and 0.5 are ordinary numbers.

No thousands separators 1,234 is textual—the comma is also the delimiter.

No dates 01/02/26 is three different days depending on locale, so date-like columns stay text. Convert explicitly with cast.

Empty cells Not evidence either way, and become null in a typed column rather than a value that would not compare with its neighbours.

Pass `-raw` (or `-no-infer`) to switch inference off and receive every column as text.

from json needs none of this: JSON declares its own types.

26.2.3 CSV Processing

```
# Filter and transform CSV data
cat sales.csv
| from csv
| where _.Amount > 100
| sort Amount
| reverse
| select Date Customer Amount
| to csv
| write-file big-sales.csv
```

26.2.4 Extract Unique Values

```
read-lines data.txt | split "," | flatten | distinct | sort
```

26.3 System Administration

26.3.1 Process Monitor

```
# Top CPU consumers
func top-cpu(n: int = 10) {
  ps -ef | sort Cpu | reverse | first $n | select Pid Name Cpu Memory
}
top-cpu 5
```

26.3.2 Service Health Check

```
func check-services(services...) {
  for svc in $services {
    var status = systemctl status $svc
    var state = $status.ActiveState
    var color = if ($state == "active") { "green" } else { "red" }
    styled $" ${$svc}: ${$state}" --fg $color
  }
}
check-services nginx postgresql redis
```

26.3.3 Network Diagnostics

```
# Check connectivity to multiple hosts
["google.com", "github.com", "cloudflare.com"] | each {
  var result = ping -c 1 -W 1000 $_
  var status = if ($result | any) { "OK" } else { "FAIL" }
  echo $"${$_}: ${$status}"
}
```

26.4 Data Transformation

26.4.1 JSON API Processing

```
# Parse and filter JSON data
var data = read-file api-response.json | from json
$data.items
  | where _.status == "active"
  | map { { | name = _.name, created = _.created_at | } }
  | sort created
  | to json --compact
```

26.4.2 Build a Report

```
func report(title: string, items) {
  writeline $"=== ${$title} ==="
```

```
writeline $"Total items: {$items | count}"
writeline $" Sum: {$items | sum}"
writeline $" Avg: {$items | average}"
writeline $" Min: {$items | min}"
writeline $" Max: {$items | max}"
writeline ""
}

var sizes = ls | where _.Type == file | get Size
report "File Sizes" ($sizes | map { _.Bytes })
```

26.4.3 Pivot Data

```
# Group and summarize
ls | group-by Extension | each {
  echo {
    Extension = _.Key,
    Count = (_.Values | count),
    TotalSize = (_.Values | sum Size)
  }
} | sort TotalSize | reverse
```

26.5 Functional Patterns

26.5.1 Pipeline Composition

```
func large-files => ls | where { _.Type == file and _.Size > 1mb }
func by-size => sort Size | reverse
func top(n: int = 5) => first $n

large-files | by-size | top 10
```

26.5.2 Memoization with Records

```
var cache = {||}
func expensive-calc(key) {
  if (not ($cache | has-prop $key)) {
    set-prop $cache $key (some-slow-operation $key)
  }
  get-prop $cache $key
}
```

26.5.3 Fibonacci with Unfold

```
func fibonacci(n: int) {
  unfold [0, 1] { [$_[0], [$_[1], $_[0] + $_[1]]] } | first $n
}
fibonacci 20
# [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...]
```

26.5.4 Power Set

```
func power-set(items) {
    $items | reduce [[]] {
        var existing = $acc
        $existing + ($existing | map { $_ + [_] })
    }
}
power-set [1, 2, 3]
# [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]
```

26.6 Configuration Recipes

26.6.1 Custom Prompt

```
# Two-line prompt with git info
$tosh.Config.Prompt.HeaderLeftLayout = "Time, Directory, Git"
$tosh.Config.Prompt.HeaderRightLayout = "Jobs, Duration"

# Minimal prompt character
$tosh.Config.Prompt.NameText = ""

# Custom time format
$tosh.Config.Display.DateTime.ScalarFormat = "HH:mm"
```

profile.tosh

26.6.2 Project-Specific Setup

```
module Projects {
    export func dev() {
        cd ~/projects/myapp
        writeline "Dev environment loaded"
    }

    export func build() {
        cd ~/projects/myapp
        /usr/bin/dotnet build
    }

    export func test() {
        cd ~/projects/myapp
        /usr/bin/dotnet test
    }
}
```

autoload/projects.tosh

26.7 TUI Applications

26.7.1 Interactive File Browser

```
func browse(path?: string = ".") {
    var items = ls $path | get Name
    var choice = tui pick $"Contents of {$path}:" ...$items ".."
```

```

    if ($choice == "..") {
        browse (dirname $path)
    } else if ((is-dir "${path}/${choice}")) {
        browse "${path}/${choice}"
    } else {
        cat "${path}/${choice}"
    }
}
browse

```

26.7.2 Service Manager

```

func service-manager() {
    var services = systemctl list-units --type service
    | where _.Active == "active"
    | get Unit
    var choice = tui pick "Select service:" ...$services
    var action = tui pick "Action for {$choice}:" "status" "restart" "stop" "logs"

    switch ($action) {
        case "status" { systemctl status $choice }
        case "restart" { systemctl restart $choice }
        case "stop" { systemctl stop $choice }
        case "logs" { journalctl -n 50 -u $choice }
    }
}

```


Part V

Diagnostics

Chapter 27

D diagnostic Code Reference

Every diagnostic raised by TōSh carries a stable identifier of the form `tosh.namespace.name`. User code may pattern-match on these identifiers in **try/catch** blocks; tooling such as the language server, the MCP server, and editor extensions key off them. Non-error diagnostics can be suppressed with the `hush` builtin (scope-local) or by adding their codes to `$tosh.Config.Diagnostics.Hushed`.

This chapter lists all 534 diagnostic codes currently emitted by the implementation, grouped by namespace. It is generated from the source tree by `scripts/extract-diagnostic-codes.tosh` and should be regenerated whenever new diagnostics are added.

Namespace summary

Namespace	Count	Purpose
<code>tosh.bind.*</code>	2	—
<code>tosh.command.*</code>	2	—
<code>tosh.compile.*</code>	2	—
<code>tosh.config.*</code>	2	Raised by the configuration loader or by the ‘config’ command.
<code>tosh.edit.*</code>	2	—
<code>tosh.format.*</code>	3	—
<code>tosh.get.*</code>	1	Raised by the ‘get’ command and friends.
<code>tosh.help.*</code>	1	Raised by the ‘help’ command.
<code>tosh.history.*</code>	1	Raised by the history subsystem (for example, re-playing entries when no host is attached).
<code>tosh.naming.*</code>	2	—
<code>tosh.native.*</code>	3	—
<code>tosh.parser.*</code>	167	Raised by the lexer or parser before any code runs. Indicates malformed source text.
<code>tosh.row.*</code>	1	—
<code>tosh.runtime.*</code>	327	Raised by the engine while evaluating a script. The bulk of TōSh diagnostics live here.
<code>tosh.tui.*</code>	8	Raised by the ‘tui’ subsystem (terminal UI widgets, screens, providers).
<code>tosh.type.*</code>	9	—
<code>tosh.user.*</code>	1	—

27.1 tosh.bind.*

tosh.bind.unknown_command

Command '{name}' is not a registered builtin or function declared in this source.

tosh.bind.unknown_variable

Variable '\${name}' is not declared in any enclosing scope.

27.2 tosh.command.*

tosh.command.missing_subcommand

'prompt' requires a segment name.

tosh.command.unknown_subcommand

'prompt {subcommand}' is not a recognized segment.

27.3 tosh.compile.*

tosh.compile.implicit_dynamic

Variable '{decl.Symbol.Name}' has no type annotation and the inferer could not pin down a concrete type.

tosh.compile.missing_type_annotation

Function '{fn.Name}' is missing a return-type annotation.

27.4 tosh.config.*

Raised by the configuration loader or by the 'config' command.

tosh.config.missing_value

Configuration path '{path}' needs a value.

tosh.config.reload_unavailable

Configuration reload is not available in this session.

27.5 tosh.edit.*

tosh.edit.no_editor

No terminal editor is available.

tosh.edit.pipeline_unsupported

'edit' cannot run inside a pipeline.

27.6 tosh.format.*

tosh.format.file_not_found

format: file '{path}' does not exist.

tosh.format.no_paths

format requires at least one file path or '-' for stdin.

tosh.format.parse_error

Cannot format input: it does not parse.

27.7 tosh.get.*

Raised by the ‘get’ command and friends.

tosh.get.index_out_of_range

(see source)

27.8 tosh.help.*

Raised by the ‘help’ command.

tosh.help.no_inline_provider

Inline help (-cli) is not available in this environment.

27.9 tosh.history.*

Raised by the history subsystem (for example, replaying entries when no host is attached).

tosh.history.run_unavailable

History replay is not available in this session.

27.10 tosh.naming.*

tosh.naming.shadowed_builtin

Function ‘{commandName}’ shadows built-in command ‘{commandName}’.

tosh.naming.shadowed_underscore

Redeclaring ‘__’ shadows an existing binding.

27.11 tosh.native.*

tosh.native.count_width_suspect

(see source)

tosh.native.entry_point_not_found

Native library ‘{libraryPath}’ exports no symbol named ‘{symbolName}’.

tosh.native.library_not_found

Native library ‘{nativeTarget}’ was not found.

27.12 tosh.parser.*

Raised by the lexer or parser before any code runs. Indicates malformed source text.

tosh.parser.accidental_double_dot

Did you mean ‘?’ (member access) instead of ‘.’? (range)?

tosh.parser.assert_does_not_accept_message

Assert no longer accepts a trailing custom message.

tosh.parser.assignment_in_predicate

Use ‘==’ for equality comparisons, not ‘=’.

tosh.parser.const_requires_value

A ‘const’ declaration requires an initializer.

tosh.parser.duplicate_input_redirection

Only one input redirection is allowed per pipeline.

tosh.parser.duplicate_subcommand_modifier

Subcommand modifier '{modifierToken.Text}' is repeated.

tosh.parser.empty_type_refinement_block

Type refinement blocks require at least one clause.

tosh.parser.expected_anonymous_function_body

Anonymous functions require '=>' or a block body.

tosh.parser.expected_anonymous_function_expression

Anonymous '=>' functions require an expression body.

tosh.parser.expected_assignment_operator

(see source)

tosh.parser.expected_assignment_target

Assignments require a variable or member path target.

tosh.parser.expected_bind_body

Bind statements require a body.

tosh.parser.expected_bind_function

Bind blocks only support function bindings.

tosh.parser.expected_block

The '{owner}' statement requires a block.

tosh.parser.expected_calling_convention

A callback needs a calling convention name after 'callconv'.

tosh.parser.expected_catch_variable

Catch clauses require a variable name when parentheses are used.

tosh.parser.expected_class_body

Class definitions require a body.

tosh.parser.expected_class_member

Expected a member inside class '{className}'.

tosh.parser.expected_closing_paren

A closing ')' is required after the computed property name.

tosh.parser.expected_command_name

Expected a function name.

tosh.parser.expected_comprehension_for

Comprehensions require a 'for' clause after '<|'.

tosh.parser.expected_comprehension_in

Comprehensions require 'in' after the variable name.

tosh.parser.expected_comprehension_operator

Expected '<|' in generator comprehension.

tosh.parser.expected_constructor_parenthesis

Object construction uses C#-style parentheses.

tosh.parser.expected_destructuring_name

Expected a variable name in the destructuring pattern.

tosh.parser.expected_else_block

Else clauses require a block or nested if statement.

tosh.parser.expected_enum_body

Enum definitions require a body.

tosh.parser.expected_enum_member_value

Enum members require a value after '='.

tosh.parser.expected_equals_tuple_assign

Tuple assignment requires '=' after the variable list.

tosh.parser.expected_fat_arrow

Dict comprehension requires '=>' between key and value.

tosh.parser.expected_for_in

For loops require 'in' before the source pipeline.

tosh.parser.expected_function_parameter

Expected a function parameter name.

tosh.parser.expected_function_signature

Function definitions require a parameter list or '=>'.

tosh.parser.expected_here_string_value

A value is required after '«<'.

tosh.parser.expected_if_condition

If statements require a parenthesized condition.

tosh.parser.expected_if_expression_condition

If expressions require a parenthesized condition.

tosh.parser.expected_index_expression

Index access requires an expression inside '[' and ']'.

tosh.parser.expected_initializer

Expected an expression after '=' in the declaration of '{declaredName}'.

tosh.parser.expected_input_redirection_source

A file path is required after an input redirection operator.

tosh.parser.expected_interface_body

Interface definitions require a body.

tosh.parser.expected_let_equals

Let bindings require '=' between name and value.

tosh.parser.expected_match_arm_arrow

Match arms require '=>' between the pattern and result.

tosh.parser.expected_match_arm_underscore

Match arms must start with '_'

tosh.parser.expected_match_block

Match expressions require an arm block.

tosh.parser.expected_match_guard_condition

Match guards require a parenthesized condition.

tosh.parser.expected_match_value

Match expressions require a parenthesized value.

tosh.parser.expected_member_assignment_target

This assignment needs a member path like '\$person.Name'.

tosh.parser.expected_native_calling_convention

Native function bindings need a calling convention name after 'callconv'.

tosh.parser.expected_native_library

A raw function needs a library after 'from'.

tosh.parser.expected_native_symbol_name

Native function bindings need a symbol name after 'as'.

tosh.parser.expected_nested_type

Expected a type declaration inside class '{className}'.

tosh.parser.expected_operand

Expected an operand in this expression.

tosh.parser.expected_parenthesized_source

The '{owner}' statement requires a source.

tosh.parser.expected_postfix_condition

Postfix '{keyword.Text}' requires a condition.

tosh.parser.expected_priority_value

Expected an integer priority value.

tosh.parser.expected_projection_member_path

Projected fields must be member paths.

tosh.parser.expected_property_accessor

Property accessors must be 'get' or 'set'.

tosh.parser.expected_property_name

Expected a property name.

tosh.parser.expected_range_separator

Type alias ranges use '..' between lower and upper bounds.

tosh.parser.expected_raw_struct_body

Raw struct '{nameToken.Text}' requires a body.

tosh.parser.expected_raw_struct_field

Expected a field name inside the raw struct body.

tosh.parser.expected_raw_struct_field_default

Raw struct fields require a value after '='.

tosh.parser.expected_raw_struct_field_type

Field '{fieldName}' needs a type.

tosh.parser.expected_record_field_default

Record fields require a value after '='.

tosh.parser.expected_record_field_name

Record literals require a field name before '='.

tosh.parser.expected_record_field_separator

(see source)

tosh.parser.expected_record_fields

Record definitions require a field list.

tosh.parser.expected_redirection_target

A file path is required after a redirection operator.

tosh.parser.expected_refinement_coerce_after_if

Guarded refinement clauses require 'coerce' after the condition.

tosh.parser.expected_refinement_coercer

Refinement coercers require an expression after 'coerce'.

tosh.parser.expected_refinement_guard

Refinement coercion guards require an expression after 'if'.

tosh.parser.expected_refinement_predicate

Refinements require a predicate after 'where'.

tosh.parser.expected_require_from

Selective require statements need 'from' before the target path.

tosh.parser.expected_require_target

Require statements need a ToSh file, module, assembly, or project path.

tosh.parser.expected_script_input_list

Script input lists require '(...)'.

tosh.parser.expected_splat_target

Argument splatting requires a variable or collection reference.

tosh.parser.expected_subcommand_name

The '{keyword.Text}' keyword requires a subcommand name.

tosh.parser.expected_switch_block

Switch statements require a case block.

tosh.parser.expected_switch_case

Switch blocks may only contain 'case' and 'default' entries.

tosh.parser.expected_switch_value

Switch statements require a parenthesized value.

tosh.parser.expected_trait_body

Trait definitions require a body.

tosh.parser.expected_tuple_assign_name

Expected a variable name in tuple assignment.

tosh.parser.expected_type_name

Expected a native parameter type.

tosh.parser.expected_type_parameter

Expected a type-parameter name after 'where'.

tosh.parser.expected_union_body

Union definitions require a body.

tosh.parser.expected_using_target

Using statements require a namespace or type alias target.

tosh.parser.expected_variable_name

Expected a variable name.

tosh.parser.hollow_subcommand_must_be_empty

Hollow subcommand '{nameToken.Text}' may only contain nested subcommands.

tosh.parser.if_expression_requires_else

If expressions require an else block.

tosh.parser.incompatible_subcommand_modifiers

'eager' and 'hollow' cannot be combined on a subcommand.

tosh.parser.invalid_method_name

Method calls need a single method name after '..

tosh.parser.invalid_numeric_separator

Digit separators must sit between digits.

tosh.parser.invalid_splat_target

Argument splatting currently requires a variable-style reference.

tosh.parser.invalid_spread_target

Spread requires a variable reference.

tosh.parser.invalid_type_refinement_clause

Type refinement blocks only support 'where', 'coerce', and 'if ... coerce' clauses.

tosh.parser.invalid_unit_literal

A unit literal contains an unknown or invalid unit expression.

tosh.parser.invalid_unit_magnitude

A unit literal must begin with a valid number.

tosh.parser.match_default_keyword_required

Use 'default' instead of '___' for the wildcard arm.

tosh.parser.missing_argument_separator

Arguments must be separated by ','.

tosh.parser.missing_bind_member_separator

Bound functions must be separated by a newline or ';'.

tosh.parser.missing_block_separator

Block statements must be separated by a newline or ';'.

tosh.parser.missing_class_member_separator

Class members must be separated by a newline or ';'.

tosh.parser.missing_closing_angle

A closing '>' is required here.

tosh.parser.missing_closing_brace

A closing '}' is required here.

tosh.parser.missing_closing_bracket

A closing ']' is required after list comprehension.

tosh.parser.missing_closing_paren

A closing ')' is required here.

tosh.parser.missing_closing_paren_tuple_assign

A closing ')' is required for tuple assignment.

tosh.parser.missing_closing_parenthesis

A native function binding requires a parameter list.

tosh.parser.missing_command_after_pipe

(see source)

tosh.parser.missing_command_after_pipe_forward

(see source)

tosh.parser.missing_constructor_separator

Constructor arguments must be separated by ','.

tosh.parser.missing_dict_closing_delimiter

A closing '%}' is required after dict comprehension.

tosh.parser.missing_dict_entry_separator

Dict entries must be separated by ',' or a newline.

tosh.parser.missing_function_parameter_separator

Function parameters must be separated by ','.

tosh.parser.missing_list_separator

Array items must be separated by ','.

tosh.parser.missing_match_arm_separator

Match arms must be separated by a newline, ';', or ';;'.

tosh.parser.missing_pipeline_separator

Expression pipeline stages must be separated by '|'.

tosh.parser.missing_predicate_separator

Predicate expressions must be separated by ';' or a newline.

tosh.parser.missing_projection_closing_brace

A closing '}' is required here.

tosh.parser.missing_projection_separator

Projected member paths must be separated by ','.

tosh.parser.missing_raw_struct_field_separator

Raw struct fields must be separated by a newline or ';'.

tosh.parser.missing_record_closing_brace

(see source)

tosh.parser.missing_record_closing_delimiter

A closing '}' is required here.

tosh.parser.missing_record_field_separator

Record fields must be separated by ',' or a newline.

tosh.parser.missing_set_closing_delimiter

A closing ':' is required after set comprehension.

tosh.parser.missing_set_separator

Set items must be separated by ','.

tosh.parser.missing_statement_separator

Top-level statements must be separated by a newline or ';'.

tosh.parser.missing_ternary_colon

A ternary expression requires ':'.

tosh.parser.missing_tuple_separator

Tuple elements must be separated by ','.

tosh.parser.missing_type_argument_separator

Generic type arguments must be separated by ','.

tosh.parser.missing_type_parameter_separator

Type parameters must be separated by ','.

tosh.parser.nameof_expects_a_name

'{identifierToken.Text}' does not name anything.

tosh.parser.nameof_missing_close_paren

Expected ')' after nameof identifier.

tosh.parser.native_buffer_requires_length

'{typeName}' needs a positive capacity.

tosh.parser.numeric_literal_overflow

This {radix} literal is too large for a 64-bit integer.

tosh.parser.range_requires_integer

Range bounds and steps must be 32-bit integers.

tosh.parser.raw_func_requires_library

A top-level 'raw func' needs a library.

tosh.parser.raw_struct_array_requires_length

Field '{fieldName}' needs a positive array length.

tosh.parser.raw_struct_clause_requires_integer

'{clause}' requires a byte count.

tosh.parser.refinement_requires_expression

Refinement predicates use expression syntax.

tosh.parser.rest_parameter_must_be_last

A rest parameter must be the last parameter.

tosh.parser.spaced_literal_delimiter

'{delimiter}' must be written without a space.

tosh.parser.subcommand_params_require_arrow

Parameter lists on subcommands require a '=' body.

tosh.parser.try_requires_handler

Try statements require a catch block, a finally block, or both.

tosh.parser.unexpected_argument_separator

An argument is required between commas.

tosh.parser.unexpected_background_operator

Unexpected background operator.

tosh.parser.unexpected_constructor_separator

A constructor argument is required between commas.

tosh.parser.unexpected_current_item_expression_tokens

This current-item expression has extra tokens after it.

tosh.parser.unexpected_function_parameter_separator

A function parameter is required between commas.

tosh.parser.unexpected_get_expression_tokens

This get expression has extra tokens after it.

tosh.parser.unexpected_interface_member

Interface bodies can only contain method signatures (func name(params)).

tosh.parser.unexpected_list_separator

An array item is required between commas.

tosh.parser.unexpected_pipeline_separator

Unexpected pipeline separator.

tosh.parser.unexpected_projection_separator

A projected member path is required between commas.

tosh.parser.unexpected_token

Unexpected token '{Current.Text}'.

tosh.parser.unexpected_trait_member

Trait bodies can contain method signatures (func) and property declarations (prop).

tosh.parser.unknown_property_accessor

Unknown property accessor '{accessorName}'.

tosh.parser.unknown_subcommand_modifier

Unknown subcommand modifier '{text}'.

tosh.parser.unsupported_double_index_lookup

Index access supports '[value]', '[key,]', or '[,value]'.

tosh.parser.terminated_ansi_c_string

ANSI-C string literals must be terminated.

tosh.parser.terminated_block_comment

Block comments must be closed.

tosh.parser.terminated_interpolated_string

Interpolated string literals must be terminated.

tosh.parser.terminated_string

String literals must be terminated.

tosh.parser.terminated_triple_quoted_string

Triple-quoted string literals must be terminated.

tosh.parser.using_requires_namespace

'using' is reserved for CLR namespaces and aliases.

tosh.parser.variable_references_require_dollar

Variable assignments must use '\$' after declaration.

tosh.parser.yield_in_defer

A deferred block cannot yield.

27.13 tosh.row.*

tosh.row.index_out_of_range

(see source)

27.14 tosh.runtime.*

Raised by the engine while evaluating a script. The bulk of TōSh diagnostics live here.

tosh.runtime.alloc_negative_size

Allocated buffers cannot have a negative size.

tosh.runtime.alloc_requires_single_value

Allocated buffer declarations require exactly one size or type value.

tosh.runtime.annotation_conversion_failed

'{owner}' produced a value that could not be converted to '{typeName}'.

tosh.runtime.annotation_unknown_type

'{owner}' uses unknown type annotation '{typeName}'.

tosh.runtime.assert_requires_predicate

The 'assert' command requires a predicate block or callable.

tosh.runtime.assertion_failed

(see source)

tosh.runtime.await_requires_future

'await' expects a future or a CLR Task.

tosh.runtime.background_command_must_be_external

Background jobs currently require external command stages.

tosh.runtime.background_pipeline_not_supported

Background jobs currently support an optional input expression followed by external command stages only.

tosh.runtime.background_pipeline_requires_command

Background pipelines require at least one external command stage.

tosh.runtime.base_constructor_already_initialized

CLR base class '{ClrBaseType!.FullName}' has already been initialized for this instance.

tosh.runtime.base_type_argument_arity

Class '{@class.Name}' supplies '{@class.BaseTypeArguments.Count} type argument(s) to base class '{baseClassDef.Name}', which expects {baseClassDef.TypeParameterNames.Count}.

tosh.runtime.base_type_argument_missing

Class '{@class.Name}' extends generic class '{baseClassDef.Name}' without supplying type arguments.

tosh.runtime.bind_target_not_native_module

'{statement.ModuleName}' is not a native library module.

tosh.runtime.break_outside_loop

'break' can only be used inside 'for', 'while', or 'each' blocks.

tosh.runtime.callable_argument_count_mismatch

Callable '{callable.CallableName}' expects at least {callable.RequiredParameterCount} argument(s) but received {argumentCount}.

tosh.runtime.callable_invocation_requires_single_value

Callable invocation in expression context must produce exactly one value.

tosh.runtime.callable_named_argument

Callable '{CallableName}' has no parameter named '{named.Key}'.

tosh.runtime.cartesian_product_args

'cartesian-product' requires a second sequence and an optional combiner.

tosh.runtime.cast_failed

(see source)

tosh.runtime.chain_requires_sequence

'chain' requires at least one sequence argument.

tosh.runtime.channel_invalid_capacity

'channel' capacity must be a positive integer.

tosh.runtime.channel_recv_requires_channel

'channel-recv' first argument must be a ShellChannel.

tosh.runtime.channel_select_requires_channel

'channel-select' arguments must be ShellChannel values.

tosh.runtime.channel_send_requires_channel

'channel-send' requires a channel argument.

tosh.runtime.chunk_requires_positive_integer

'chunk' requires a positive integer size.

tosh.runtime.chunk_requires_size

'chunk' requires exactly one integer size argument.

tosh.runtime.combinations_k_non_negative

k must be non-negative.

tosh.runtime.combinations_requires_k

'combinations' requires exactly one integer argument (k).

tosh.runtime.command_failed

(see source)

tosh.runtime.command_windows_unavailable

'{Name}' is not available on Windows.

tosh.runtime.compose_requires_callable

Argument {i + 1} is not callable.

tosh.runtime.compose_requires_two_callables

The 'compose' command requires at least two callable values.

tosh.runtime.compound_assignment_requires_value

Variable '{assignment.Name}' does not have a value yet.

tosh.runtime.const_reassignment

Cannot reassign constant '{name}'.

tosh.runtime.const_redeclaration

Cannot redeclare constant '{name}'.

tosh.runtime.constructor_cycle

Constructor cycle detected while initializing class '{Name}'.

tosh.runtime.constructor_parameter_type_conversion_failed

Constructor argument '{parameter.Name}' could not be converted to '{parameter.TypeName}'.

tosh.runtime.continue_outside_loop

'continue' can only be used inside 'for', 'while', or 'each' blocks.

tosh.runtime.converge_requires_seed_and_callable

'converge' requires a seed value and a callable value or block.

tosh.runtime.curry_argument_count_mismatch

Curried callable '{__inner.CallableName}' accepts {_targetArity} total argument(s) but received {combinedArguments.Length}.

tosh.runtime.curry_requires_callable

The 'curry' command requires exactly one callable value.

tosh.runtime.curry_requires_fixed_arity_callable

The 'curry' command currently requires a fixed-arity callable.

tosh.runtime.curry_requires_nonzero_arity

The 'curry' command requires a callable that accepts at least one argument.

tosh.runtime.cycle_no_arguments

'cycle' takes no arguments. Pipe a sequence into it.

tosh.runtime.dedup_too_many_args

'dedup' accepts at most one member path argument.

tosh.runtime.defer_body_failed

Deferred scope body failed: {failure.Message}

tosh.runtime.defer_cleanup_failed

Deferred cleanup #{cleanupIndex} failed: {item.Title}

tosh.runtime.destructuring_requires_array

Array destructuring requires an array or list value.

tosh.runtime.destructuring_requires_record

Record destructuring requires a record or dictionary value.

tosh.runtime.duplicate_base_constructor_initializer

Constructor '{Name}()' calls '\$super(...)' more than once.

tosh.runtime.duplicate_class_property

Class '{@class.Name}' defines property '{duplicateProperties.Key}' more than once.

tosh.runtime.duplicate_constructor

(see source)

tosh.runtime.duplicate_function_parameter

Function '{name}' defines parameter '{duplicateParameters.Key}' more than once.

tosh.runtime.duplicate_named_argument

Named argument '{namedArgument.Name}' was supplied more than once.

tosh.runtime.duplicate_primary_constructor_chain

Constructor '{Name}()' calls '\$this(...)' more than once.

tosh.runtime.duplicate_record_field

Record '{record.Name}' defines field '{duplicateFields.Key}' more than once.

tosh.runtime.duplicate_rune_parameter

Rune '{rune.Name}' defines parameter '{duplicateParameters.Key}' more than once.

tosh.runtime.duplicate_script_flag

Script flag '-{optionName}' is inferred for more than one flag.

tosh.runtime.duplicate_script_input

Script input '{parameter.Name}' is declared more than once.

tosh.runtime.duplicate_subcommand

Subcommand '{childNode.Name}' is declared more than once at this level.

tosh.runtime.each_requires_callable_or_block

'each' requires exactly one callable value or block.

tosh.runtime.enum_member_conversion_failed

Enum member '{@enum.Name}.{member.Name}' could not be converted to '{underlying-Type.Name}'.

tosh.runtime.enum_member_requires_single_value

Enum member '{@enum.Name}.{member.Name}' must resolve to exactly one value.

tosh.runtime.enum_member_value_required

Enum member '{@enum.Name}.{member.Name}' requires an explicit value.

tosh.runtime.enumerate_too_many_args

'enumerate' accepts at most one argument (the starting index).

tosh.runtime.error

(see source)

tosh.runtime.exception

(see source)

tosh.runtime.exec_invalid_command

'exec' needs a non-empty command name.

tosh.runtime.exec_missing_command

'exec' needs a command to run.

tosh.runtime.exec_pipeline_unsupported

'exec' only works as a standalone command.

tosh.runtime.expression_failed

Argument '{parameter.Name}' could not be converted to '{parameter.TypeName}'.

tosh.runtime.extend_member_not_a_method

'extend {extend.TypeName}' can only declare methods.

tosh.runtime.extend_sealed_class

Class '{@class.Name}' cannot extend sealed class '{@class.BaseClassName}'.

tosh.runtime.external_command_is_directory

'{external.ResolvedPath ?? commandSyntax.Name}' is a directory, not an executable file.

tosh.runtime.external_command_not_executable

'{external.ResolvedPath ?? commandSyntax.Name}' is not executable.

tosh.runtime.fading_member

Property '{property.Name}' on class '{Name}' is fading (deprecated).

tosh.runtime.filter_requires_callable_or_block

'filter' requires exactly one callable value or block.

tosh.runtime.find_index_requires_callable_or_block

'find-index' requires exactly one callable value or block.

tosh.runtime.findmnt_command_failed

(see source)

tosh.runtime.findmnt_command_missing

The system 'findmnt' command was not found.

tosh.runtime.findmnt_command_start_failed

Failed to start the system 'findmnt' command.

tosh.runtime.findmnt_json_parse_failed

Could not parse structured 'findmnt' output. {exception.Message}

tosh.runtime.flat_map_requires_callable_or_block

'flat-map' requires exactly one callable value or block.

tosh.runtime.frequencies_too_many_args

'frequencies' accepts at most one member path argument.

tosh.runtime.function_argument_count_mismatch

Function '{definition.Name}' expects {expected} argument(s) but received {context.Arguments.Count}.

tosh.runtime.function_overload_ambiguous

Multiple overloads matched function '{Name}' with {arguments.Count} argument(s).

tosh.runtime.function_overload_not_found

No overload matched function '{Name}' with {arguments.Count} argument(s).

tosh.runtime.generic_argument_type_mismatch

'{target.OwnerLabel}' inferred type parameter '{typeParameterName}' as '{bound.Name}', but argument '{parameterName}' is '{clrType.Name}'.

tosh.runtime.generic_constraint_failed

'{target.OwnerLabel}' requires '{typeParameterName}' to satisfy '{constraintName}', but '{clrType.Name}' does not.

tosh.runtime.generic_return_type_mismatch

Function '{definition.Name}' inferred '{rawReturn}' as '{bound.Name}', but returned a '{value.GetType().Name}'.

tosh.runtime.generic_type_argument_count_mismatch

Function '{definition.Name}' has {typeParamsForSeed.Count} type parameter(s) but received {explicitList.Count} type argument(s).

tosh.runtime.group_by_requires_selector

'group-by' requires exactly one member path, callable, or block.

tosh.runtime.group_while_requires_callable_or_block

'group-while' requires exactly one callable value or block.

tosh.runtime.hermit_has_constructor

Hermit class '{@class.Name}' cannot have constructors.

tosh.runtime.hostnamectl_command_failed

(see source)

tosh.runtime.hostnamectl_command_missing

The system 'hostnamectl' command was not found.

tosh.runtime.hostnamectl_command_start_failed

Failed to start the system 'hostnamectl' command.

tosh.runtime.hostnamectl_json_parse_failed

Could not parse structured 'hostnamectl status' output. {exception.Message}

tosh.runtime.http_request_failed

The HTTP request failed. {exception.Message}

tosh.runtime.http_status_failed

HTTP request returned {(int)response.StatusCode} {response.ReasonPhrase ?? string.Empty}

tosh.runtime.index_assignment_failed*(see source)***tosh.runtime.infinite_eager_comprehension**

Cannot use an infinite source in a list, set, or dict comprehension. Use a generator comprehension (...) instead of [...] and pipe to '| first N'.

tosh.runtime.input_redirection_source_not_found

Input redirection source '{resolved}' does not exist.

tosh.runtime.input_redirection_source_null

Input redirection source cannot be null.

tosh.runtime.interface_type_argument_arity_mismatch

Generic interface '{ifaceDefinition.Name}' expects {ifaceArity} type argument(s) <{string.Join(

tosh.runtime.interface_type_argument_constraint_violation

Generic interface '{ifaceDefinition.Name}' requires type parameter '{clause.TypeParameter}' to satisfy '{constraintName}', but '{argText}' (CLR {bound.FullName ?? bound.Name}) does not.

tosh.runtime.interleave_requires_sequence

'interleave' requires exactly one array argument.

tosh.runtime.intersperse_requires_separator

'intersperse' requires exactly one separator argument.

tosh.runtime.invalid_member_assignment_target

Assignments to members require a member path target.

tosh.runtime.invalid_regex

The regular expression is invalid. {exception.Message}

tosh.runtime.invoke_requires_callable

The 'invoke' command requires a callable value.

tosh.runtime.ip_command_failed*(see source)***tosh.runtime.ip_command_missing**

The system 'ip' command is not available on Windows.

tosh.runtime.ip_command_start_failed

Failed to start the system 'ip' command.

tosh.runtime.ip_json_parse_failed

Could not parse structured 'ip {structuredRequest.Mode.ToString().ToLowerInvariant()}' output. {exception.Message}

tosh.runtime.ip_subcommand_unsupported_windows

'ip {windowsRequest.Mode.ToString().ToLowerInvariant()}' is not supported on Windows.

tosh.runtime.iterate_requires_seed_and_callable

'iterate' requires a seed value and a callable value or block.

tosh.runtime.journalctl_command_failed

The system 'journalctl' command failed.

tosh.runtime.journalctl_command_missing

The system 'journalctl' command was not found.

tosh.runtime.journalctl_command_start_failed

Failed to start the system 'journalctl' command.

tosh.runtime.journalctl_json_parse_failed

Could not parse structured journal output. {exception.Message}

tosh.runtime.list_materialization_failed

(see source)

tosh.runtime.loginctl_command_failed

(see source)

tosh.runtime.loginctl_command_missing

The system 'loginctl' command was not found.

tosh.runtime.loginctl_command_start_failed

Failed to start the system 'loginctl' command.

tosh.runtime.loginctl_json_parse_failed

Could not parse structured 'loginctl list-sessions' output. {exception.Message}

tosh.runtime.loginctl_show_parse_failed

Could not parse structured 'loginctl show' output. {exception.Message}

tosh.runtime.lsblk_command_failed

(see source)

tosh.runtime.lsblk_command_missing

The system 'lsblk' command was not found.

tosh.runtime.lsblk_command_start_failed

Failed to start the system 'lsblk' command.

tosh.runtime.lsblk_json_parse_failed

Could not parse structured 'lsblk' output. {exception.Message}

tosh.runtime.lscpu_command_failed

(see source)

tosh.runtime.lscpu_command_missing

The system 'lscpu' command was not found.

tosh.runtime.lscpu_command_start_failed

Failed to start the system 'lscpu' command.

tosh.runtime.lscpu_json_parse_failed

Could not parse structured 'lscpu' summary output. {exception.Message}

tosh.runtime.lsfd_columns_failed

(see source)

tosh.runtime.lsfd_command_failed

(see source)

tosh.runtime.lsfd_command_missing

The system 'lsfd' command was not found.

tosh.runtime.lsfd_command_start_failed

Failed to start the system 'lsfd' command while discovering columns.

tosh.runtime.lsfd_json_parse_failed

Could not parse structured 'lsfd' output. {exception.Message}

tosh.runtime.lsipc_command_failed

(see source)

tosh.runtime.lsipc_command_missing

The system 'lsipc' command was not found.

tosh.runtime.lsipc_command_start_failed

Failed to start the system 'lsipc' command.

tosh.runtime.lsipc_json_parse_failed

Could not parse structured 'lsipc' output. {exception.Message}

tosh.runtime.map_requires_callable_or_block

'map' requires exactly one callable value or block.

tosh.runtime.member_assignment_failed

(see source)

tosh.runtime.missing_base_constructor_initializer

Class '{requestedBy.Name}' must initialize base class '{Name}' with constructor arguments.

tosh.runtime.missing_hollow_methods

Class '{@class.Name}' must implement hollow methods from '{parentClass.Name}':
{string.Join(

tosh.runtime.missing_hollow_properties

Class '{@class.Name}' must implement hollow properties from '{parentClass.Name}':
{string.Join(

tosh.runtime.missing_interface_methods

Class '{@class.Name}' does not implement all methods of interface '{ifaceName}'. Missing:
{string.Join(

tosh.runtime.missing_interface_type_arguments

Class '{@class.Name}' fulfills generic interface '{ifaceDefinition.Name}' without type arguments.

tosh.runtime.missing_ouerrule

Method '{method.Name}' in class '{@class.Name}' shadows a parent method but is not marked 'ouerrule'.

tosh.runtime.missing_script_argument

Missing required script argument '{parameter.Name}'.

tosh.runtime.missing_script_flag

Missing required script flag '{parameter.Name}'.

tosh.runtime.missing_trait_methods

Class '{@class.Name}' does not implement required methods from trait '{traitName}'. Missing:
{string.Join(

tosh.runtime.missing_trait_properties

Class '{@class.Name}' does not implement required properties from trait '{traitName}'.
Missing: {string.Join(

tosh.runtime.nameof_requires_dollar

Variable references in nameof require '\$'. Use nameof({nameOf.Identifier}).

tosh.runtime.native_alloc_argument_count

native-alloc expects exactly one argument.

tosh.runtime.native_alloc_negative_size

native-alloc cannot allocate a negative number of bytes.

tosh.runtime.native_alloc_requires_size_or_type

A native allocation needs a byte count or a supported interop type name.

tosh.runtime.native_argument_count_mismatch

Native function '{ModuleName}.{Name}' expects {atLeast}{callableCount} argument(s) but received {context.Arguments.Count}.

tosh.runtime.native_argument_type_conversion_failed

Argument {argumentIndex + 1} for native function '{ModuleName}.{Name}' could not be converted to '{parameter.TypeName}'.

tosh.runtime.native_binding_requires_type

Native {owner} requires an explicit CLR type.

tosh.runtime.native_buffer_range

'{parameter.Name}' needs a {required}-byte buffer but received {nativeBuffer.ByteLength} bytes.

tosh.runtime.native_buffer_requires_out

Parameter '{parameter.Name}' must be declared 'out' to use a buffer.

tosh.runtime.native_callback_buffer_parameter

Callback '{rawCallback.Name}' cannot take a buffer parameter.

tosh.runtime.native_callback_by_reference

A callback cannot be passed by reference.

tosh.runtime.native_callback_by_reference_parameter

Callback '{rawCallback.Name}' cannot take a by-reference parameter.

tosh.runtime.native_callback_expects_function

Parameter '{parameter.Name}' of '{ModuleName}.{Name}' expects the callback '{callback-Definition.Name}'.

tosh.runtime.native_callback_return_convention

Callback '{rawCallback.Name}' cannot declare a success convention.

tosh.runtime.native_count_width_not_integer

'{widthName} count' needs an integer width.

tosh.runtime.native_free_requires_buffer

native-free needs at least one native buffer.

tosh.runtime.native_free_requires_native_buffer

native-free only accepts buffers created by native-alloc.

tosh.runtime.native_offsetof_argument_count

{Name} expects a type name and a field name.

tosh.runtime.native_offsetof_requires_field_name

{Name} requires a field name.

tosh.runtime.native_offsetof_requires_struct_layout_type

'{type.FullName} ?? type.Name}' is not a sequential or explicit-layout struct.

tosh.runtime.native_pointer_required

(see source)

tosh.runtime.native_read_requires_length

native-read bytes requires a byte count.

tosh.runtime.native_read_requires_mode

native-read needs a read mode or type name.

tosh.runtime.native_read_requires_native_type

native-read currently supports native scalar types, struct-layout types, 'cstring', or 'bytes', not '{mode}'.

tosh.runtime.native_read_requires_source

native-read needs a native buffer or pointer source.

tosh.runtime.native_sizeof_requires_type

native-sizeof requires at least one type name.

tosh.runtime.native_type_name_required

A native interop type name is required here.

tosh.runtime.native_variadic_not_last

'..' must be the last parameter.

tosh.runtime.native_write_argument_count

native-write expects a target, a value, and an optional offset.

tosh.runtime.native_write_offset_requires_int

native-write offsets must be integers.

tosh.runtime.native_write_requires_value

native-write requires a non-null value.

tosh.runtime.native_write_unsupported_value

native-write does not know how to write values of type '{runtimeType.Name}'.

tosh.runtime.nested_type_not_declared

Nested type '{member.Name}' in class '{@class.Name}' did not produce a type.

tosh.runtime.networkctl_command_failed

(see source)

tosh.runtime.networkctl_command_missing

The system 'networkctl' command was not found.

tosh.runtime.networkctl_command_start_failed

Failed to start the system 'networkctl' command.

tosh.runtime.networkctl_parse_failed

Could not parse structured 'networkctl list' output. {exception.Message}

tosh.runtime.non_exhaustive_match

This match expression did not match any arm.

tosh.runtime.nonzero_exit_code

Command exited with code {exitCode}.

tosh.runtime.not_callable

Value of type '{(target?.GetType()).Name ??

tosh.runtime.null_dict_key

Dict keys cannot be null.

tosh.runtime.override_no_base_method

Method '{method.Name}' in class '{@class.Name}' is marked 'override' but no parent class defines '{method.Name}'.

tosh.runtime.parallel_requires_callable_or_block

'parallel' requires a callable value or block.

tosh.runtime.parameter_default_conversion_failed

The default value for parameter '{parameter.Name}' could not be converted to '{parameter.TypeName}'.

tosh.runtime.parameter_type_conversion_failed

Argument '{parameter.Name}' could not be converted to '{parameter.TypeName}'.

tosh.runtime.partial_argument_count_mismatch

Callable '{_inner.CallableName}' accepts at most {maximum} argument(s) but received {combinedArguments.Length}.

tosh.runtime.partial_merge_non_partial_record

Cannot merge partial record '{record.Name}' with existing non-partial record.

tosh.runtime.partial_merge_non_partial_struct

Cannot merge partial struct '{@struct.Name}' with existing non-partial struct.

tosh.runtime.partial_mismatch

Cannot extend class '{@class.Name}' as partial: the original class was not declared as partial.

tosh.runtime.partial_requires_callable

The 'partial' command requires a callable value.

tosh.runtime.partition_requires_callable_or_block

'partition' requires exactly one callable value or block.

tosh.runtime.permutations_args

'permutations' accepts at most one integer argument (k).

tosh.runtime.permutations_k_non_negative

k must be non-negative.

tosh.runtime.predicate_expression_required

'skip-until' requires a predicate expression.

tosh.runtime.primary_chain_arguments

'\$this(...)' does not match the primary constructor of '{Name}'.

tosh.runtime.primary_chain_must_be_first

'\$this(...)' must be the first executable statement in a constructor.

tosh.runtime.primary_chain_without_primary

Class '{Name}' cannot call '\$this(...)' because it has no primary constructor.

tosh.runtime.race_requires_multiple_operations

'race' requires at least two callable values or blocks.

tosh.runtime.raw_func_requires_library

Raw function '{statement.Binding.Name}' needs a library.

tosh.runtime.raw_struct_bool_field

Raw struct '{structName}' field '{field.Name}' cannot be 'bool'.

tosh.runtime.raw_struct_cstring_requires_length

Raw struct '{structName}' field '{field.Name}' needs an inline buffer length.

tosh.runtime.raw_struct_duplicate_field

Raw struct '{syntax.Name}' declares '{field.Name}' more than once.

tosh.runtime.raw_struct_not_blittable

Raw struct '{plan.Name}' does not have a valid native layout.

tosh.runtime.raw_struct_requires_fields

Raw struct '{rawStruct.Name}' has no fields.

tosh.runtime.raw_struct_size_mismatch

Raw struct '{plan.Name}' is {actualSize} bytes, but 'size {declared}' was declared.

tosh.runtime.raw_struct_unsupported_array_element

Raw struct '{structName}' field '{field.Name}' cannot be an array of '{typeName}'.

tosh.runtime.raw_struct_unsupported_field_type

Raw struct '{structName}' field '{field.Name}' has an unsupported type '{typeName}'.

tosh.runtime.recur_empty_seeds

'recur' requires at least one seed value.

tosh.runtime.recur_requires_seeds_and_callable

'recur' requires seed values and a callable.

tosh.runtime.recursion_limit_exceeded

Maximum ToastScript recursion depth was exceeded.

tosh.runtime.redirection_target_not_single_path

Redirection targets must resolve to exactly one path.

tosh.runtime.redirection_target_null

Redirection target cannot be null.

tosh.runtime.redirection_target_unavailable

Cannot open '{redirection.Path}' for redirection: {exception.Message}

tosh.runtime.reduce_requires_seed_and_callable

'reduce' requires a seed value and a callable value or block.

tosh.runtime.refinement_failed

Value for '{owner}' does not satisfy its refinement.

tosh.runtime.refinement_requires_boolean

{title} must evaluate to boolean values.

tosh.runtime.refinement_specialization_failed

Refinement alias '{genericDefinition.Name}' could not be specialized for '{closedTypeName}'.

tosh.runtime.regex_flags_not_applicable

Regex option flags only apply to string patterns.

tosh.runtime.repeat_negative_count

Repeat count cannot be negative.

tosh.runtime.repeat_requires_value

'repeat' requires a value and an optional count.

tosh.runtime.repeatedly_requires_callable

'repeatedly' requires a callable or block.

tosh.runtime.require_exports_nothing

'{statement.Target}' declares no exports, so this require imports nothing.

tosh.runtime.require_failed

(see source)

tosh.runtime.reserved_variable_name

'{name}' is a {titleSuffix}.

tosh.runtime.return_type_conversion_failed

Function '{definition.Name}' returned a value that could not be converted to '{definition.ReturnTypeName}'.

tosh.runtime.scan_requires_seed_and_callable

'scan' requires a seed value and a callable value or block.

tosh.runtime.scope_requires_block

'scope' requires a block argument.

tosh.runtime.script_flag_cannot_be_rest

Script flags cannot use rest parameters.

tosh.runtime.script_inputs_must_be_top_level

Script input declarations must be top-level statements.

tosh.runtime.script_option_requires_value

Option '{optionName}' requires a value.

tosh.runtime.script_rest_argument_must_be_last

A script rest argument must be the last argument.

tosh.runtime.self_unavailable_in_constructor_default

The default for parameter '{parameter.Name}' cannot use '\$this'.

tosh.runtime.serialization_depth_exceeded

Value nests deeper than {MaxDepth} levels and cannot be serialized.

tosh.runtime.settle_requires_operation

'settle' requires at least one callable value or block.

tosh.runtime.shy_using_requires_scope

Shy using statements require a function, block, or module scope.

tosh.runtime.sort_requires_selector

'sort' selectors must be a member path, callable value, or block.

tosh.runtime.spawn_requires_command

'spawn' requires an external command name or path.

tosh.runtime.spawn_target_is_directory

'{commandName}' resolves to a directory, not an executable.

tosh.runtime.spawn_target_not_executable

'{commandName}' exists but is not executable.

tosh.runtime.spawn_target_not_found

External command '{commandName}' was not found.

tosh.runtime.splat_infinite_range

Cannot splat an infinite range.

tosh.runtime.splat_requires_collection

Argument splatting requires a non-null collection value.

tosh.runtime.spread_requires_record

Spread in a record literal requires a record or dictionary value.

tosh.runtime.step_by_positive

'step-by' requires $n \geq 1$.

tosh.runtime.step_by_requires_n

'step-by' requires exactly one integer argument.

tosh.runtime.subcommand_must_be_script_scoped

Subcommand '{statement.Name}' must be declared at script or parent-subcommand scope.

tosh.runtime.subcommand_required

(see source)

tosh.runtime.subexpression_requires_single_value

Subexpressions used as arguments must produce exactly one value.

tosh.runtime.super_initializer_must_be_first

'\$super(...)' must be the first executable statement in a constructor.

tosh.runtime.super_without_base_class

Class '{Name}' cannot call '\$super(...)' because it has no base class.

tosh.runtime.systemctl_command_failed

(see source)

tosh.runtime.systemctl_command_missing

The system 'systemctl' command was not found.

tosh.runtime.systemctl_command_start_failed

Failed to start the system 'systemctl' command.

tosh.runtime.systemctl_json_parse_failed

Could not parse structured 'systemctl list-units' output. {exception.Message}

tosh.runtime.systemctl_show_parse_failed

Could not parse structured 'systemctl show' output. {exception.Message}

tosh.runtime.throw

(see source)

tosh.runtime.time_invalid_target

'time' requires a block, callable, or command as its first argument.

tosh.runtime.time_requires_argument

'time' requires a command or block to measure.

tosh.runtime.timeout_elapsed

Operation timed out after {timeoutDuration.TotalSeconds:0.###} seconds.

tosh.runtime.timeout_invalid_duration

'timeout' duration must be greater than zero.

tosh.runtime.timeout_requires_seconds_and_operation

'timeout' requires a duration and an operation.

tosh.runtime.to_requires_input

'to {formatName}' has nothing to serialize.

tosh.runtime.transpose_requires_records

'transpose' requires record/object pipeline items.

tosh.runtime.tree_command_failed

(see source)

tosh.runtime.tree_command_missing

The system 'tree' command was not found.

tosh.runtime.tree_command_start_failed

Failed to start the system 'tree' command.

tosh.runtime.tree_json_parse_failed

Could not parse structured 'tree' output. {exception.Message}

tosh.runtime.tuple_assignment_arity_mismatch

This destructuring has {targetCount} targets but the tuple has {valueCount} elements.

tosh.runtime.type_name_conflict

{declaredKind} '{name}' conflicts with an existing refinement alias.

tosh.runtime.ulimit_getrlimit_failed

Failed to read current limit for '{resourceName}'.

tosh.runtime.ulimit_invalid_value

Invalid limit value '{newValue}'.

tosh.runtime.ulimit_setrlimit_failed

Failed to set limit for '{resourceName}' (errno {errno}).

tosh.runtime.ulimit_unknown_resource

Unknown resource '{resourceName}'.

tosh.runtime.ulimit_unsupported

'ulimit' is not supported on Windows.

tosh.runtime.umask_invalid

Invalid umask value '{arg}'.

tosh.runtime.umask_unsupported

'umask' is not supported on Windows.

tosh.runtime.unexpected_exception

(see source)

tosh.runtime.unexpected_interface_type_arguments

Interface '{ifaceDefinition.Name}' is not generic and does not accept type arguments.

tosh.runtime.unexpected_script_argument

Unexpected {(node.Name is null ?

tosh.runtime.unfold_requires_pair_or_null

'unfold' callable must return a [value, next-state] pair or null to stop.

tosh.runtime.unfold_requires_seed_and_callable

'unfold' requires a seed value and a callable value or block.

tosh.runtime.uninitialized_variable

Variable '{variableReference.Name}' has been declared but not assigned yet.

tosh.runtime.unknown_base_class

Class '{@class.Name}' extends unknown class '{@class.BaseClassName}'.

tosh.runtime.unknown_cast_target

(see source)

tosh.runtime.unknown_command

Command '{commandSyntax.Name}' was not found.

tosh.runtime.unknown_enum_underlying_type

Enum '{@enum.Name}' uses unknown underlying type '{@enum.UnderlyingTypeName}'.

tosh.runtime.unknown_function_reference

Function '{funcRef.Name}' was not found.

tosh.runtime.unknown_interface

Class '{@class.Name}' fulfills unknown interface '{ifaceName}'.

tosh.runtime.unknown_named_argument

There is no parameter named '{namedArgument.Name}'.

tosh.runtime.unknown_script_flag

Unknown script flag '-{optionName}'.

tosh.runtime.unknown_static_assignment_target

'{path}' does not name a static member, because '{head}' names no type.

tosh.runtime.unknown_tilde_target

'~{expansion.Name}' names neither a directory alias nor a user.

tosh.runtime.unknown_trait

Class '{@class.Name}' uses unknown trait '{traitName}'.

tosh.runtime.unknown_type

Type '{name}' was not found.

tosh.runtime.unknown_type_argument

Type '{name}' could not be resolved as a type argument for '{methodName}'.

tosh.runtime.unknown_type_name

Type '{typeName}' could not be resolved for type parameter '{typeParamsForSeed[i]}' of function '{definition.Name}'.

tosh.runtime.unknown_variable

'\${variableReference.Name}' is a constructor parameter of

tosh.runtime.unsupported_native_allocation_type

'{type.FullName ?? type.Name}' is not a supported native allocation type.

tosh.runtime.unsupported_native_byref_string

By-ref native string parameters need an explicit pointer type.

tosh.runtime.unsupported_native_calling_convention

Native interop does not support calling convention '{name}'.

tosh.runtime.unsupported_native_command_type

Native interop does not support '{typeName}'.

tosh.runtime.unsupported_native_interop_type

Native interop does not currently support '{typeName}'.

tosh.runtime.unsupported_native_string_return

Native string returns need an explicit interop string type.

tosh.runtime.using_export_not_supported

'using' cannot be exported.

tosh.runtime.using_not_supported

This runtime does not support 'using' statements.

tosh.runtime.value_not_callable

The provided value is not callable.

tosh.runtime.variable_reference_requires_dollar

Variable '{variableName}' exists, but variable references must start with '\$'.

tosh.runtime.window_requires_positive_integer

'window' requires a positive integer size.

tosh.runtime.window_requires_size

'window' requires a size and an optional callable or block.

tosh.runtime.zip_requires_sequence

'zip' requires a second sequence and an optional combiner block.

27.15 tosh.tui.*

Raised by the 'tui' subsystem (terminal UI widgets, screens, providers).

tosh.tui.filter.no_items

No items provided for 'tui filter'.

tosh.tui.layout.missing_orientation

The 'tui layout' subcommand requires an orientation.

tosh.tui.missing_subcommand

The 'tui' command requires a subcommand.

tosh.tui.no_inline_provider

Inline prompts (-cli) are not available in this environment.

tosh.tui.no_screen

Expected a TuiScreen from pipeline input.

tosh.tui.pick.no_items

No items provided for 'tui pick'.

tosh.tui.run.no_screen

No TuiScreen provided to 'tui run'.

tosh.tui.unknown_subcommand

Unknown tui subcommand '{subcommand}'.

27.16 tosh.type.*

tosh.type.arity

Function '{call.Name}' expects {DescribeArity(required, maxAccepted)} but received {positionals.Count}.

tosh.type.command_argument

Command '{call.Name}' argument {i + 1} expects '{expected.DisplayName}' but received

'{actual.DisplayName}'.

tosh.type.command_arity

Command '{call.Name}' expects {DescribeArity(required, maxAccepted)} but received {provided}.

tosh.type.index

Dictionary is keyed by '{BoundType.FromClr(keyType).DisplayName}' but received '{indexType.DisplayName}'.

tosh.type.member_not_found

Method '{call.MethodName}' was not found on type '{targetType.DisplayName}'.

tosh.type.mismatch

Argument {i + 1} of '{call.Name}' expects '{declared.DisplayName}' but received '{actual.DisplayName}'.

tosh.type.operator

Operator '{op}' is not compatible with operand types '{left.DisplayName}' and '{right.DisplayName}'.

tosh.type.pipeline_input

Command '{call.Name}' does not accept pipeline input of type '{previousOutput.DisplayName}'.

tosh.type.unknown_option

Command '{call.Name}' has no option '{text}'.

27.17 tosh.user.*

tosh.user.type

(see source)

Index

Symbols

* (operator) 24
** (operator) 24
+ (operator) 24
- (operator) 24
.. (operator) 76
... (operator) 31
/ (operator) 24
// (operator) 24
< (operator) 25
<= (operator) 25
= (operator) 27
== (operator) 25
~= (operator) 25
> (operator) 25
>= (operator) 25
?. (operator) 27
?: (operator) 27
?? (operator) 27
% (operator) 24
&& (operator) 26

A

abstract (keyword) . 65
all (command) 180
and (operator) 26
any (command) 180
append-file (command)
138
apropos (command) 219
arg (keyword) 87
as (operator) 27
as-file (command) 134
assert (command) . 217
async (command) .. 129
average (command) 180
await (command) .. 129

B

back (command) ... 139
basename (command)
139

benchmark (command)
220
bg (command) 206
bind (keyword) 54
break (keyword) 38

C

call (command) ... 116
call-method (command)
117
callback (keyword) . 58
callconv (keyword) . 54
cartesian-product
(command) . 181
case (keyword) 35
cast (command) ... 117
cat (command) 140
catch (keyword) 37
cd (command) 140
chain (command) .. 181
channel (command) 130
channel-close
(command) . 130
channel-recv (command)
131
channel-select
(command) . 131
channel-send (command)
132
chmod (command) .. 141
chown (command) .. 141
chunk (command) .. 182
class (keyword) 62
clear (command) .. 220
clone (command) .. 118
close (command) .. 142
coerce (keyword) 21
collect (command) 182
combinations (command)
183
comment (keyword) ... 4

Complex (keyword) .. 15
complex (command) 135
complex (keyword) .. 15
compose (command) 169
config (command) . 220
const (keyword) 29
constructors (command)
118
contains (operator) . 26
continue (keyword) . 38
converge (command)
169
copy-to (command) 142
count (command) .. 183
cp (command) 143
curry (command) .. 170
cut (command) 247
cycle (command) .. 170

D

date (command) ... 232
dbg (command) 221
debug (command) .. 221
dedup (command) .. 183
default (keyword) .. 35
defer (keyword) 37
del-prop (command)
118
describe (command)
184
describe-type
(command) . 119
df (command) 144
dirname (command) 145
dirs (command) ... 145
distinct (command)
184
du (command) 146

E

each (command) ... 185

eager (keyword) 87
 echo (command) ... 247
 edit (command) ... 222
 else (keyword) 34
 ends-with (operator) 26
 enum (keyword) 73
 enumerate (command)
 185
 env (command) 233
 eval (command) ... 222
 event (keyword) 74
 events (command) . 223
 exec (command) ... 223
 exists (command) . 147
 exit (command) ... 224
 export (command) . 233
 export (keyword) ... 32
 extend (keyword) ... 74
 extends (keyword) .. 69

F

fading (keyword) ... 65
 false (keyword) 8
 fg (command) 207
 filter (command) . 186
 finally (keyword) .. 37
 find (command) ... 147
 find-index (command)
 186
 findmnt (command) 148
 first (command) .. 187
 fixed (keyword) 65
 flag (keyword) 87
 flags (keyword) 73
 flat-map (command)
 187
 flatten (command) 187
 fluid (keyword) 71
 flush (command) .. 149
 for (keyword) 34
 forget (command) . 234
 format (command) . 218
 forward (command) 150
 free (command) ... 234
 frequencies (command)
 188
 from (command) ... 135
 from (keyword) 54
 fulfills (keyword) 19f.,
 69
 func (keyword) 41
 funcs (command) .. 120

G

get (command) 188
 get-methods (command)
 120
 get-prop (command)
 120
 get-props (command)
 121
 glob (command) ... 150
 glob (keyword) 80
 global (keyword) ... 32
 grep (command) ... 247
 group-by (command)
 189
 group-while (command)
 189
 guarded (keyword) .. 65
 guid (command) ... 235

H

handles (keyword) .. 48
 has-method (command)
 121
 has-prop (command)
 122
 hash (command) ... 136
 head (command) ... 248
 help (command) ... 224
 hermit (keyword) ... 65
 hidden (keyword) ... 87
 history (command) 225
 history-search
 (command) . 226
 hollow (keyword) ... 65
 hostname (command)
 235
 hostnctl (command)
 235
 http (command) ... 175
 hush (command) ... 227

I

id (command) 236
 if (keyword) 34, 39
 ignore (command) . 189
 implements (keyword)
 69
 in (operator) 26
 inspect (command) 190
 interface (keyword) 70
 interleave (command)
 190

interpolation (keyword)
 7
 intersperse (command)
 191
 invoke (command) . 170
 ip (command) 177
 is (keyword) 36
 is (operator) 27
 is in (operator) 26
 is not (operator) ... 27
 is not in (operator) 26
 is-dir (command) . 151
 is-file (command) 151
 is-in (operator) 26
 is-link (command) 151
 is-not-in (operator) 26
 iterate (command) 171

J

jobs (command) ... 207
 join (command) ... 191
 join-lines (command)
 249
 journalctl (command)
 236

K

kill (command) ... 208

L

last (command) ... 192
 lazy (keyword) .. 14, 65
 leaky (keyword) 51
 length (command) . 152
 let (keyword) 77
 lines (command) .. 249
 ln (command) 152
 load-assembly
 (command) . 122
 local (keyword) 65
 loginctl (command)
 237
 ls (command) 153
 lsblk (command) .. 154
 lscpu (command) .. 238
 lsfd (command) ... 208
 lsipc (command) .. 240

M

map (command) 192
 mat (command) 136
 mat (keyword) 14
 match (command) .. 250

match (keyword) 35
 Matrix (keyword) ... 14
 max (command) 193
 median (command) . 193
 members (command) 123
 methods (command) 123
 min (command) 193
 mkdir (command) .. 156
 mkdir-temp (command)
 156
 module (keyword) ... 59
 mv (command) 156

N

name-of (keyword) .. 81
 nameof (keyword) 81
 native-alloc (command)
 124
 native-free (command)
 124
 native-offsetof
 (command) . 124
 native-read (command)
 125
 native-sizeof
 (command) . 125
 native-write (command)
 126
 networkctl (command)
 241
 new (command) 126
 new (keyword) 81
 none (command) ... 194
 not (operator) 26
 not in (operator) ... 26
 not-in (operator) ... 26
 null (keyword) 8

O

obsolete (keyword) . 65
 open-file (command)
 157
 or (operator) 26
 override (keyword) . 65
 overrule (keyword) . 65

P

pack (keyword) 57
 parallel (command)
 194
 parse (command) .. 137
 partial (command) 172

partial (keyword) .. 61,
 65
 partition (command)
 195
 percentile (command)
 195
 permutations (command)
 196
 ping (command) ... 179
 position (command)
 158
 private (keyword) .. 65
 prompt (command) . 211
 prompt-dir (command)
 212
 prompt-duration
 (command) . 212
 prompt-exit (command)
 213
 prompt-git (command)
 214
 prompt-history
 (command) . 214
 prompt-jobs (command)
 214
 prompt-newline
 (command) . 215
 prompt-text (command)
 215
 prompt-time (command)
 216
 prompt-userhost
 (command) . 216
 prop (keyword) 62
 props (command) .. 127
 protected (keyword) 65
 proud (keyword) 65
 ps (command) 209
 public (keyword) ... 65
 pwd (command) 159

Q

quote (keyword) 52

R

race (command) ... 132
 raise (command) .. 218
 raise (keyword) 74
 range (keyword) 13
 raw (command) 250
 raw (keyword) . 54, 57f.,
 65

read-bytes (command)
 159
 read-file (command)
 160
 read-from (command)
 160
 read-line (command)
 227
 read-line-from
 (command) . 161
 read-lines (command)
 161
 read-to-end (command)
 162
 readlink (command)
 162
 readonly (keyword) . 65
 realpath (command)
 163
 record (keyword) ... 72
 recur (command) .. 172
 reduce (command) . 196
 rename (command) . 197
 repeat (command) . 173
 repeatedly (command)
 173
 replace (command) 251
 require (keyword) .. 53
 required (keyword) . 65,
 74
 return (keyword) ... 38
 reverse (command) 197
 rm (command) 163
 round (command) .. 174
 row (command) 197
 rune (keyword) 51

S

scan (command) ... 198
 scope (command) .. 132
 sealed (keyword) 51, 65
 seek (command) ... 164
 seq (command) 242
 set-prop (command)
 127
 settle (command) . 133
 shared (keyword) ... 65
 shy (keyword) ... 32, 65
 signal (command) . 210
 size (keyword) 57
 skip (command) ... 198

skip-until (command) 199
 skip-while (command) 199
 sleep (command) .. 243
 sort (command) ... 199
 source (command) . 227
 spawn (command) .. 133
 split (command) .. 252
 starts-with (operator) 26
 stat (command) ... 164
 static (keyword) ... 65
 stdev (command) .. 200
 step-by (command) 200
 strict (keyword) ... 65
 struct (keyword) 57, 71
 styled (command) . 217
 subcommand (keyword) 87
 sum (command) 201
 summarize (command) 201
 switch (keyword) ... 35
 systemctl (command) 243

T

tail (command) ... 252
 take-until (command) 202
 take-while (command) 203
 tee (command) 203
 tempfile (command) 165
 template (command) 253

this (keyword) 62f.
 throw (keyword) 38
 time (command) ... 228
 timeout (command) 134
 timespan (command) 244
 to (command) 137
 touch (command) .. 165
 tr (command) 253
 trait (keyword) 70
 transpose (command) 203
 tree (command) ... 166
 true (keyword) 8
 try (keyword) 37
 tui (command) 228
 type-of (command) 128
 types (command) .. 128

U

ulimit (command) . 229
 umask (command) .. 230
 uname (command) .. 245
 undef (command) .. 219
 unfold (command) . 174
 union (keyword) . 57, 71
 uniq (command) ... 254
 unless (command) . 231
 unless (keyword) ... 39
 until (keyword) 35
 uptime (command) . 245
 uses (keyword) .. 19, 69
 using (keyword) 53

V

var (keyword) 29
 variance (command) 204

vars (command) ... 246
 vec (command) 138
 vec (keyword) 14
 Vector (keyword) ... 14
 view (command) ... 231
 vital (keyword) 65

W

wait-for (command) 211
 wc (command) 254
 where (command) .. 204
 where (keyword) . 54, 77
 which (command) .. 231
 while (keyword) 35
 whoami (command) . 246
 window (command) . 205
 with-retry (command) 232
 write (command) .. 255
 write-bytes (command) 166
 write-file (command) 167
 write-line-to (command) . 168
 write-to (command) 168
 writeline (command) 256

X

xargs (command) .. 205

Y

yield (keyword) 45

Z

zip (command) 206